



Technical Documentation Version 9.6

Optimization



Center for Advanced Decision Support
for Water and Environmental Systems
UNIVERSITY OF COLORADO **BOULDER**

These documents are copyrighted by the Regents of the University of Colorado. No part of this document may be reproduced, stored in a retrieval system, or transmitted in any form or by any means electronic, mechanical, recording or otherwise without the prior written consent of The University of Colorado. All rights are reserved by The University of Colorado.

The University of Colorado makes no warranty of any kind with respect to the completeness or accuracy of this document. The University of Colorado may make improvements and/or changes in the product(s) and/or programs described within this document at any time and without notice.

Contents

1. Optimization Overview	1
About Optimization	1
System-wide Approach	2
Preemptive Linear Goal Programming	2
Optimization Variables	3
Physical Constraints	3
Prioritized Policy	4
Advantages of Optimization	6
Optimization License	6
2. Mathematics of Preemptive Linear Goal Programming	7
About Preemptive Linear Goal Programming	7
Linear Programming	7
Goal Programming	8
Preemptive Goal Programming	8
Soft Constraints and Derived Objectives	12
Prevent Infeasibilities	13
Two-dimensional Example	13
Goal Programming With Many Dimensions	15
Water Resource Management Example	16
Satisfaction Variables in Soft Constraints	17
Derived Objectives	19
Summation	19
Single Maximin	20
Repeated Maximin	20
Summation with Reward Table	21
Freezing Constraints and Variables	24
Freezing Constraints	25
Freezing Variables	26
Shrinking Constraints	27
3. Combining Optimization With Simulation	31
Standard Controller Sequence	31
Simulation	31
Optimization	32
Post-optimization Rulebased Simulation	32
Post-optimization Ruleset	33
Automatically Generated Ruleset	34
Custom Post-optimization Ruleset	35

Contents

Initialization Rules	35
Usage	36
Enable or Disable Initialization Rules based on Controller	36
Timing	37
Restore Point	37
Create a Restore Point	39
Use a Restore Point	40
Specify a Restore Point from the Run Control dialog	40
Specify a Restore Point from a script action	42
Optimization Restore Point Manager	43
Export a Restore Point	44
Export from the Restore Point Manager	44
Automate Restore Point export via script	45
Import a Restore Point	45
Import from the Restore Point Manager	45
Automate Restore Point import via script	47
Delete Restore Points	47
Restore Point Informational Diagnostics	47
Restrictions on Restore Point Use	48
Limitations of Optimization	49
Optimization License Required	49
Linear Relationships	49
Linear Constraint Expressions	49
Approximation Error	52
Solution Time	52
Perfect Foresight	52
Limited Objects and Method Selections	53
Debugging	53
4. Variables, Constraints, and Objectives	55
RPL Optimization Goal Set	55
Prioritized Policy	55
Goal Types	58
Hard Constraints	58
Soft Constraints	59
Objectives	59
Statements in Goal Sets	60
Objective	61
Soft Constraint Set	62
Constraint	64
Freeze	65
With Reward Table	65
For	69
With	69
If	69
If Else	70

Else If Branch	70
Else Branch	70
Print	71
Notice	71
Warning	71
Alert	71
Stop Run	71
Timestep References in Optimization	72
Optimization Variables	72
Slots and Variables	72
Slots as CPLEX Variables	72
Linearized Slots	73
Variables Unrelated to Slots	73
Slots Unused by Optimization	73
Constraints Define Variables	73
Physical Constraints	74
Variable Bounds	75
User-defined Variables	76
Linearization, Approximation, and Replacement	79
Link Substitution	80
Constraint Substitution	80
Expression Substitution	80
Variable Substitution	80
Direct Substitution	82
Tangent Approximation	83
Two-point Line Approximation	83
Piecewise Approximation	84
Lambda Method	85
5. Optimization Objects and Methods	87
How Objects Are Described	87
Canal	87
General Slots	88
User Methods	90
Canal Flow Numerical Approximation	90
Modeling a Canal	90
Confluence	90
Data Objects	90
Inline Power Plant	91
Level Power Reservoir	91
General Slots	91
User Methods	96
Avoidance Zones	96
None	97
Unit Head Based Avoidance Zones	97

Contents

Bank Storage	97
None	97
CRSS Bank Storage	97
Cavitation	97
None	97
Unit Head and Tailwater Based Regions	97
Diversion from Reservoir	98
None	98
Available Flow Based Diversion	98
Energy in Storage	98
None	98
EIS Table Lookup	98
Future Value	99
None	100
Cumulative Storage Value Table	100
Head Loss	101
None	101
Shared Penstock Head Loss method	101
Hydrologic Inflow	101
Optimization Evaporation	102
None	102
Opt Input Evaporation	102
Optimization Future Value	105
None	105
Opt Cumulative Storage Value Table	105
Optimization Power	105
None	105
Independent Linearizations	105
Power Coefficient	106
Power Surface Approximation	107
Optimization Reserves	116
None	116
Constraint Based Single Timestep	116
Optimization Spill	128
None	130
Opt Monthly Spill	130
Opt Unregulated	130
Opt Regulated	131
Opt Regulated and Unregulated	131
Opt Regulated and Bypass	131
Opt Regulated, Bypass and Unregulated	131
Opt Bypass, Regulated and Unregulated	131
Optimization Tailwater	131
Opt Linked or Input	132
Opt Base Value Only	132
Opt Base Value Plus Lookup Table	132
Opt Stage Flow Lookup Table	134

Opt Coefficients Table	135
Power	136
Plant Power Coefficient	136
Plant Efficiency Curve	139
Peak Power	139
Peak and Base	139
Unit Power Table	139
Turbine Capacity	140
Startup	141
None	141
Unit Lumped Cost	141
Modeling a Level Power Reservoir	142
Pumped Storage Reservoir	142
General Slots	142
User Methods	147
Avoidance Zones	148
None	148
Unit Head Based Avoidance Zones	148
Bank Storage	148
None	148
CRSS Bank Storage	148
Cavitation	148
None	149
Unit Head and Tailwater Based Regions	149
Diversion from Reservoir	149
None	149
Available Flow Based Diversion	149
Energy in Storage	149
None	149
EIS Table Lookup	149
Future Value	151
None	151
Cumulative Storage Value Table	151
Head Loss	152
None	152
Shared Penstock Head Loss	153
Hydrologic Inflow	153
Optimization Evaporation	153
None	153
Opt Input Evaporation	153
Optimization Future Value	156
None	156
Opt Cumulative Storage Value Table	156
Optimization Head Computation	157
Optimization Power	157
None	157
Independent Linearizations	157

Contents

Power Coefficient	158
Power Surface Approximation	158
Optimization Pump Power	168
None	168
Pump Independent Linearizations	168
Optimization Reserves	170
None	170
Constraint Based Single Timestep	170
Optimization Spill	182
None	184
Opt Monthly Spill	184
Opt Unregulated	185
Opt Regulated	185
Opt Regulated and Unregulated	185
Opt Regulated and Bypass	185
Opt Regulated, Bypass and Unregulated	185
Opt Bypass, Regulated and Unregulated	185
Optimization Tailwater	185
Opt Linked or Input	186
Opt Base Value Only	186
Opt Base Value Plus Lookup Table	187
Opt Stage Flow Lookup Table	188
Opt Coefficients Table	189
Pump Power Numerical Approximation	190
Pool Elevation Linearization Automation	191
Power	191
Plant Power Coefficient	192
Plant Efficiency Curve	194
Peak Power	194
Peak and Base	194
Unit Power Table	194
Turbine Capacity	196
Startup	197
None	197
Unit Lumped Cost	197
Modeling a Pumped Storage Reservoir	197
Reach	198
General Slots	198
User Methods	199
Diversion from Reach	199
None	199
Available Flow Based Diversion	199
Gain Loss	200
None	200
Constant Gain Loss	200
Local Inflow and Solution Direction	200
Min Diversion Bypass	200

Routing	201
No Routing	201
Time Lag	201
Impulse Response	202
Numerical Approximation	202
Modeling a Reach	203
Slope Power Reservoir	203
General Slots	203
User Methods	206
Avoidance Zones	207
None	207
Unit Head Based Avoidance Zones	207
Bank Storage	208
None	208
CRSS Bank Storage	208
Cavitation	208
None	208
Unit Head and Tailwater Based Regions	208
Diversion from Reservoir	208
None	209
Available Flow Based Diversion	209
Energy in Storage	209
None	209
EIS Table Lookup	209
Future Value	210
None	210
Cumulative Storage Value Table	210
Head Loss	212
None	212
Shared Penstock Head Loss method	212
Hydrologic Inflow	212
Optimization Backwater	212
None	212
Lambda	213
Independent Linearizations	213
Optimization Evaporation	213
None	213
Opt Input Evaporation	213
Optimization Future Value	216
None	216
Opt Cumulative Storage Value Table	216
Optimization Power	216
None	216
Independent Linearizations	217
Power Coefficient	217
Power Surface Approximation	218
Optimization Reserves	227

Contents

None	227
Constraint Based Single Timestep	227
Optimization Spill	228
None	230
Opt Monthly Spill	230
Opt Unregulated	230
Opt Regulated	230
Opt Regulated and Unregulated	230
Opt Regulated and Bypass	230
Opt Regulated, Bypass and Unregulated	230
Opt Bypass, Regulated and Unregulated	231
Optimization Tailwater	231
Opt Linked or Input	231
Opt Base Value Only	231
Opt Base Value Plus Lookup Table	232
Opt Stage Flow Lookup Table	233
Opt Coefficients Table	234
Power	235
Plant Power Coefficient	236
Plant Efficiency Curve	238
Peak Power	238
Peak and Base	239
Unit Power Table	239
Turbine Capacity	240
Slope Storage	241
Slope Storage Coefficients	241
Startup	241
None	241
Unit Lumped Cost	242
Modeling a Slope Power Reservoir	242
Storage Reservoir	242
General Slots	243
User Methods	246
Bank Storage	246
None	246
CRSS Bank Storage	246
Diversion From Reservoir	247
None	247
Available Flow Based Diversion	247
Energy in Storage	247
None	247
EIS Table Lookup	247
Evaporation and Precipitation	248
Future Value	249
None	249
Cumulative Storage Value Table	249
Hydrologic Inflow	250

Optimization Evaporation	250
None	250
Opt Input Evaporation	250
Optimization Future Value	253
None	253
Opt Cumulative Storage Value Table	253
Optimization Spill	254
None	256
Opt Monthly Spill	256
Opt Unregulated	256
Opt Regulated	256
Opt Regulated and Unregulated	257
Opt Regulated and Bypass	257
Opt Regulated, Bypass and Unregulated	257
Opt Bypass, Regulated and Unregulated	257
Release Capacity Numerical Approximation	257
Modeling a Storage Reservoir	257
Thermal	258
User Methods	258
Load	258
Modified Load	258
None	258
Calculate Modified Load	258
Preferred Customer	259
Regulation	259
Economic Valuation User Methods	260
Block Economic Value	261
None	261
Calculate Block Economic Value	261
Linear Economic Value	262
None	262
Calculate Linear Economic Value	262
Thermal Unit Replacement Value	263
None	263
Calculate Thermal Unit Replacement Value	263
Modeling a Thermal Object	264
6. Setting Up and Running an Optimization Model	267
Select Timestep and Run Range	267
Set Up and Test a Simulation Model	268
Select Optimization Methods	268
Add a Thermal Object	270
Enter Lower and Upper Bounds	271
Bounds on Variables	271
Setting Bounds	272
Optimization Limits for Data Verification	273

Contents

Provide Approximation Points	275
Data Table Slots	275
LP Param Table Slots	275
Additional Guidelines	277
Turbine Capacity Approximation	277
Power Approximation	278
Initialization Rules to Set Approximation Points	278
Create an Optimization Goal Set	279
Create a Post-optimization Ruleset	280
Run Parameters	282
Run the Model	282
7. Debugging and Solution Analysis	285
Optimization Debugging Tools	285
Priority-oriented Optimization Solution Analysis Tool	285
Accessing the Analysis Tool	287
Save/Load Problem	287
Solution Information Panel	288
Solutions Without a Freeze Statement	289
Frozen Constraints Tabs	289
Frozen Variables Tabs	290
Export as CSV	291
Tooltips on Variables	292
Saving Optimization Files	293
A. Technical Appendix	295
Optimization Units and Scaling	295
Mixed-integer Programming	296
Variables That Bring Integer Programming Into the Problem	297
How Integer Programs Are Solved	298
Timestep Approach	298
Defining Constraints	298
Additional Parameters for Controlling This Solution	299
Setting Parameters	299
Lower Cutoff Parameter	299
Limiting the Number of Solutions	300
Time Limit	300
Optimality Tolerance	300
Unit Power	301
Mathematical Formulation of MIP for Unit Power	301
Triangulation	304
Lambda Variables for Convex Combinations of Discrete Operating Points (Triangles)	306
Defining Constraints	307

Chapter 1

Optimization Overview

Topics

- [About Optimization](#), page 1
- [Advantages of Optimization](#), page 6
- [Optimization License](#), page 6

About Optimization

The RiverWare Optimization solver provides a different solution approach. Instead of solving a model on a timestep-by-timestep basis, Optimization provides a single global solution over all objects and all timesteps. Typically, Optimization is used to determine the *best* solution in terms of a stated objective, such as maximizing the value of hydropower over the run period, while also satisfying higher-priority policy constraints.

This document provides the following information about the RiverWare Optimization solver.

- Overview of the Optimization solver
- Details about the preemptive linear goal programming approach used by RiverWare
- How Optimization interacts with Simulation and Rulebased Simulation
- How variables and constraints (policy) are formulated for RiverWare Optimization
- Methods and slots on each object that apply for Optimization
- Creating and analyzing an Optimization model
- Debugging an Optimization model
- Technical information about units, scale, and mixed-integer programming

Following are key characteristics of Optimization:

- [System-wide Approach](#), page 2
- [Preemptive Linear Goal Programming](#), page 2
- [Optimization Variables](#), page 3
- [Physical Constraints](#), page 3
- [Prioritized Policy](#), page 4

System-wide Approach

Simulation and Rulebased Simulation solve each object individually, one timestep at a time. Optimization, on the other hand, provides a global solution across all objects and across all timesteps. This allows the solution to trade off objectives both spatially and over time.

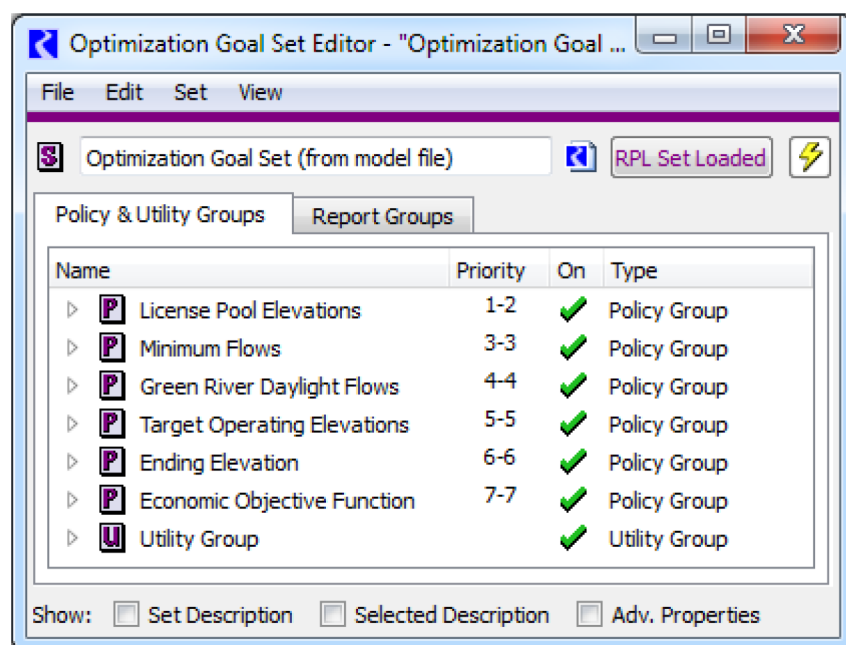
For example, using Optimization, the model can *look ahead* to a peak price period and send water from upstream reservoirs to a downstream reservoir with high generating capacity early in the run, so that it can generate at maximum capacity during peak prices. At the same time, it ensures that the upstream reservoirs do not release so much water that they will violate their elevation targets by the end of the run based on inflow forecasts.

Hydropower objectives are a common focus of Optimization models. RiverWare includes some built-in functionality that assists in formulating common hydropower objectives; however, there is nothing that limits RiverWare Optimization objectives to be related to hydropower, nor is hydropower required to be included in an Optimization model. Optimization models can just as easily optimize a water management objective unrelated to hydropower.

Preemptive Linear Goal Programming

The Optimization solution in RiverWare is a preemptive linear goal program. The user expresses the policy of the system in a set of prioritized goals. These goals contain either constraints or objectives. A linear program (LP) is solved at each priority level, beginning at the highest priority. If the goal contains a set of soft constraints, the objective function for the LP is to maximize the satisfaction of the constraints. The objective value (level of satisfaction) will not be reduced when the LPs are solved for lower priority goals. In other words, once a constraint has been satisfied at a high priority, it will not be violated to meet a lower priority constraint or objective. In this way, as the solution progresses through the priorities, it shrinks the solution space or removes degrees of freedom from the solution. Typically the final priority is a maximize or minimize objective, such as maximizing the total value of hydropower over the run period. The objective will be maximized/minimized within the reduced solution space after maximizing the satisfaction of all higher priority constraints.

Figure 1.1



RiverWare provides three solution approaches in the case that not all soft constraints within a goal cannot be satisfied due to hydrologic conditions and/or higher priority constraints. The first approach, Summation, minimizes the total deviation from the constraint values. The second approach, Maximin, minimizes the single largest deviation (technically it maximizes the lowest satisfaction). The third approach, Repeated Maximin, minimizes the single largest deviation. Then it freezes that value and minimizes the next largest deviation, and so on until it has either minimized all deviations or satisfied all remaining constraints. In practice, the Repeated Minimax approach is most-often used.

Optimization Variables

An optimization problem consists of a set of equations that include linear combinations of optimization variables. In RiverWare, some Optimization variables are directly related to slots. For example, Outflow and Storage on a reservoir object are common variables. Each of these slots at each timestep corresponds directly to an individual variable in the Optimization problem. In some cases slots correspond to a linear combination of variables. For example, Power is typically defined by a piecewise linear function of Turbine Release. Each individual variable, or *piece*, is multiplied by the slope of that segment of the power curve. The sum of the pieces is equal to the total Turbine Release. Some variables have no relation to a slot in RiverWare. For example, the satisfaction level of each Repeated Maximin iteration is added to the Optimization problem as a variable. RiverWare decides automatically which variables to add to the Optimization problem based on what is referenced in the Optimization Goal Set.

Physical Constraints

RiverWare automatically adds all relevant physical constraints to the Optimization problem. For example, the mass balance constraint for a reservoir is

$$Storage_t = Storage_{t-1} + Inflow_t - Outflow_t + \sum Gains_t - \sum Losses_t$$

This is the defining constraint for the Storage, Inflow and Outflow variables at time t . RiverWare determines which individual variables to include in Gains and Losses depending on the method selection on the reservoir. For example, Evaporation could be an additional loss variable in the mass balance constraint. For a reach, the defining physical constraints are determined by the selected routing method.

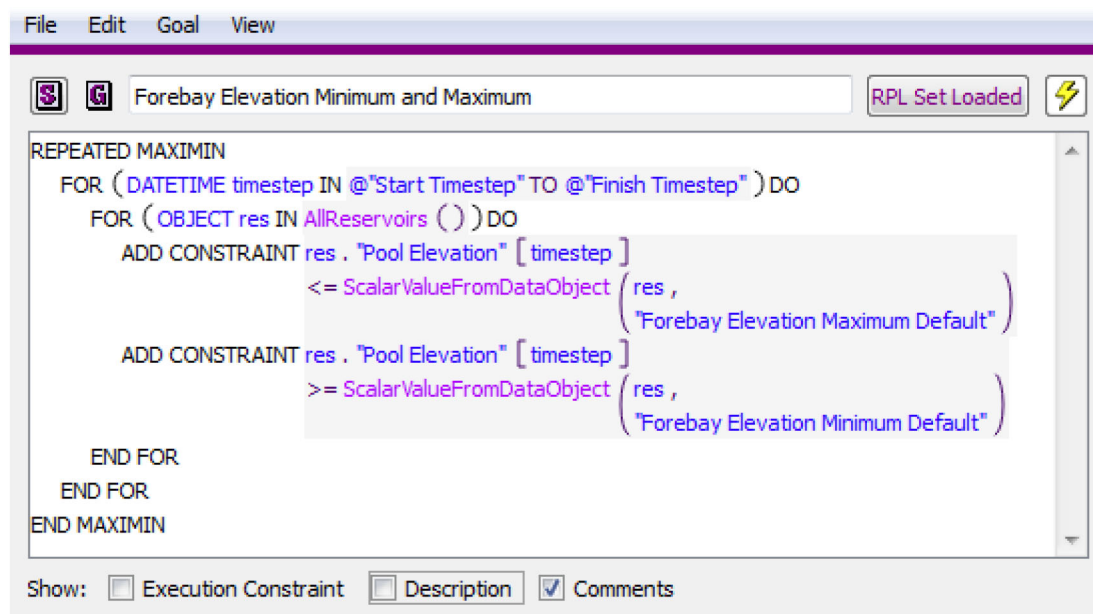
RiverWare only adds physical constraints as necessary based on the policy in the Optimization Goal Set. In this way it minimizes the size of the Optimization problem in order to achieve maximum computational efficiency. Physical constraints are always treated as hard constraints.

Prioritized Policy

The policy of a river basin is expressed by the user in an Optimization Goal Set. Like rulesets in Rulebased Simulation, the Goal Set is formulated in RiverWare Policy Language (RPL). Also similarly to a ruleset, the goals in a Goal Set are expressed in priority order. For example, license minimum and maximum pool elevation constraints for reservoirs might be the highest priority goal. These are goals that would essentially never be violated. Lower priority goals might include target operating elevation range constraints. These would represent elevations that should be maintained under normal operating conditions but might be exceeded in extreme scenarios such as flood control operations. Goals with a Minimize or Maximize objective tend to be at the lowest priority, after all policy constraints have been satisfied to the greatest extent possible.

A user has the flexibility to formulate any type of constraint or objective they desire as long as the expressions are linear combinations of variables. Constraints can include average values over multiple timesteps or summations over multiple slots on multiple objects. Similar constraints for multiple objects can be grouped into a single constraint statement. For example, a single maximum elevation constraint can be added for all reservoirs in a model by creating a loop over a list of reservoirs and referencing the maximum elevations from a data object slot corresponding to each reservoir.

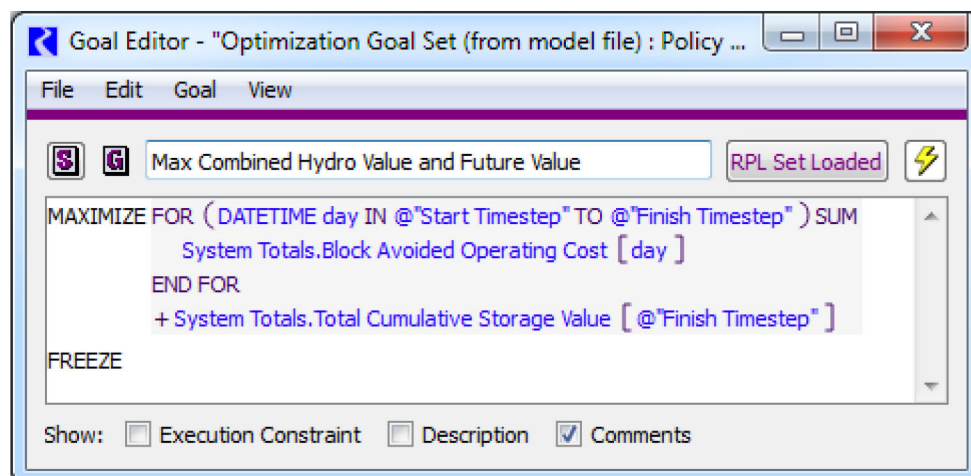
Figure 1.2



Conditional statements can be added for the inclusion of constraints in the form of If-Then statements. For example, seasonal constraints might apply only if the run period is within specified dates, or Special Operations constraints might only apply if data are present in a certain slot.

Objectives allow for a trade-off between present value and future value. An objective might include the combined value of the hydropower generated during the run and the value of energy in storage (future potential value) at the end of the run.

Figure 1.3



Advantages of Optimization

Optimization is valuable when trying to determine a *best* solution, as opposed to just evaluating a solution. Also, because the Optimization solution is global in space and time, it is useful for planning current operations in a manner that leaves the system in a good position to meet upcoming conditions.

Complex systems can require much trial and error to approach a *best* solution, and it might not be obvious how changes to one part of the system affect other parts of the system or operations in the future. Optimization provides an efficient means to find the best operations for near-term objectives without jeopardizing the ability to meet constraints in the future.

Optimization License

To use the RiverWare optimization solution, you must have the RiverWare With Optimizer version of the RiverWare license. Go to RiverWare.org for licensing information.

The license allows to you access the RiverWare Optimization module, which embeds the third-party solver IBM ILOG CPLEX. Additional information about CPLEX can be found in the About RiverWare dialog—from the main RiverWare workspace, select **Help**, then **About RiverWare**.

Chapter 2

Mathematics of Preemptive Linear Goal Programming

This section provides technical details about the mathematics in the Preemptive Linear Goal Programming solution used by RiverWare Optimization. This includes details about how constraints and objectives are applied and how they get formulated internally in the RiverWare Optimization solution. It also describes the use of priorities to handle conflicting objectives.

Topics

- [About Preemptive Linear Goal Programming, page 7](#)
- [Soft Constraints and Derived Objectives, page 12](#)
- [Freezing Constraints and Variables, page 24](#)
- [Shrinking Constraints, page 27](#)

About Preemptive Linear Goal Programming

This section provides a conceptual overview to the Preemptive Linear Goal Programming solution used by RiverWare Optimization. It begins with brief, general descriptions of linear programming, goal programming and preemptive goal programming. These are followed by some simple examples to illustrate the use of preemptive goal programming to handle conflicting objectives and conflicting constraints.

Linear Programming

The RiverWare Optimization solver uses linear programming at its core. Linear programming is a common optimization approach where linear objective function is maximized or minimized subject to a set of linear constraints.

$$\text{Maximize } z = c_1x_1 + c_2x_2 + \dots + c_{n-1}x_{n-1} + c_nx_n$$

Subject to:

$$a_{1,1}x_1 + a_{1,2}x_2 + \dots + a_{1,n-1}x_{n-1} + a_{1,n}x_n = b_1$$

$$a_{2,1}x_1 + a_{2,2}x_2 + \dots + a_{2,n-1}x_{n-1} + a_{2,n}x_n \leq b_2$$

$$a_{3,1}x_1 + a_{3,2}x_2 + \dots + a_{3,n-1}x_{n-1} + a_{3,n}x_n \geq b_3$$

...

$$a_{m,1}x_1 + a_{m,2}x_2 + \dots + a_{m,n-1}x_{n-1} + a_{m,n}x_n = a_m$$

where each x is a variable in the problem. Each c is a numeric coefficient in the objective function. Each a is a numeric coefficient in a constraint. Each b is the numeric right-hand-side value of each constraint. Each constraint can be an “equal-to,” “less-than-or-equal-to,” or “greater-than-or-equal-to” constraint. The objective function and the left-hand-side of every constraint must be a linear combination of one or more variables. Additional general information about linear programming, including approaches for solving a linear program, are covered in any text on the basics of linear programming optimization and thus are not covered here.

In RiverWare, an optimization model ultimately gets formulated as a linear program. The details about that formulation are covered in following sections.

Goal Programming

One limitation of standard linear programming is that all constraints are *hard* constraints. If there is not a solution that can simultaneously satisfy all constraints, then the problem is infeasible. Also, there is only a single objective function.

Often in water resources contexts, a system will have multiple objectives that need to be traded off against each other. For example, there might be an objective to maximize the value of hydropower generation, but that must also be traded off with environmental objectives and an objective to carry sufficient reserve capacity. If environmental objectives are expressed as constraints (e.g. minimum and maximum flows), cases can result in which the problem becomes infeasible. For example, low inflow inputs can produce a case in which it is not possible to simultaneously satisfy minimum reservoir level constraints and minimum outflow requirements.

One approach to handle the case of conflicting objectives or conflicting constraints is goal programming. In traditional goal programming, a penalty function is applied to any deviation from a constraint value, and the penalties are minimized as part of the objective function. Typically penalty functions are convex. Most often the penalty function is linear, piecewise linear or quadratic because of the available solution mechanisms. For example, some objectives have a term that minimizes the sum of squared violations. In order to combine terms with non-commensurate units in a single objective, the deviations are typically scaled. Additionally the penalty functions and multiple objectives in the objective function might be weighted to express the relative importance of some policies over others. One challenge of this approach is that it can be difficult to interpret the meaning of the weights applied. Also the solution can be sensitive the weights that are used, and it can be difficult to anticipate the effects of modifying weights without significant trial and error.

Rather than combining weighted penalty terms and weighted objectives in a single objective function, RiverWare Optimization uses an alternative form of goal programming referred to as preemptive goal programming, which is described in the following section.

Preemptive Goal Programming

RiverWare Optimization uses a form of goal programming referred to as preemptive goal programming. Constraints and objectives are expressed at distinct priorities. At each priority an optimization problem is solved either to maximize or minimize the stated objective at that priority. If the priority contains constraints, the objective is to maximize the satisfaction of the constraints. (In RiverWare, this *derived* objective to maximize constraint satisfaction is automatically formulated based on the constraints that the user specifies.) Before moving on to the

next priority, the objective value is *frozen*. This is the equivalent of adding a constraint to the problem that sets the objective function expression equal to the objective value. See “Freezing Constraints and Variables” on page 24 for a detailed explanation and example of the concept of *freezing*. This freezing guarantees that lower priority constraints and objectives will not degrade higher priority constraints and objectives. After the higher priority solutions, there are typically many alternative optima, multiple feasible solutions that would equally maximize the objective function. Any solution for a lower priority must come from the set of alternative optima from the preceding solution. As the solution proceeds through the priorities, the solution space is reduced by the constraints added at each priority. Another way to say this is that the set of alternative optima is reduced, or degrees of freedom are removed from the problem.

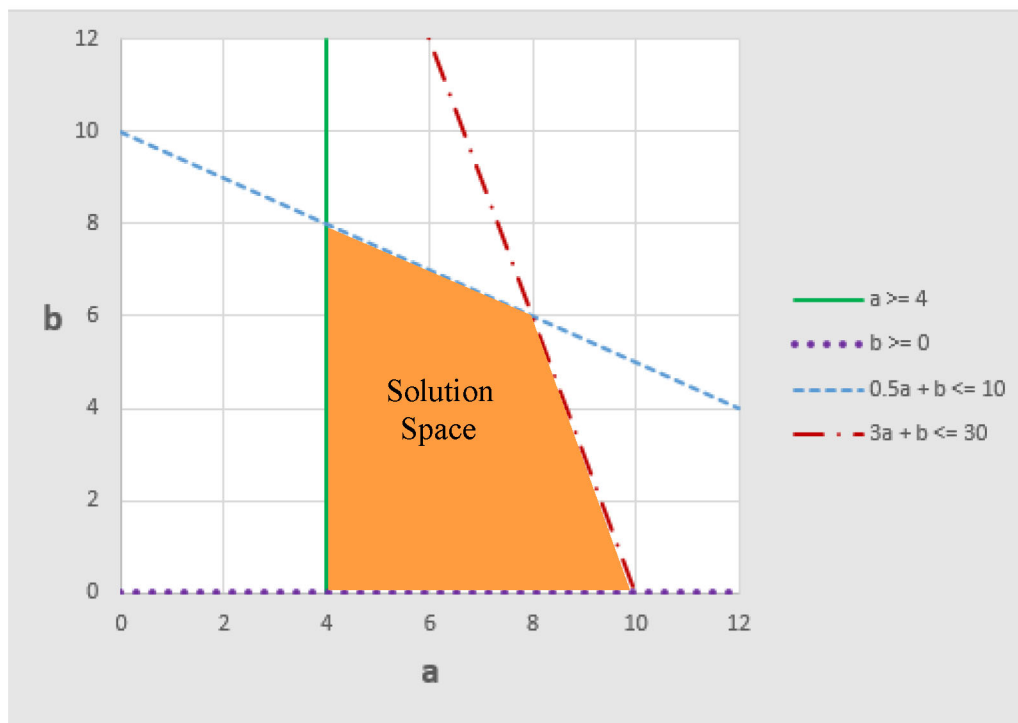
The following simple example illustrates the basic concepts of preemptive goal programming, including prioritized objectives, the shrinking solution space, alternative optima and freezing of variables and constraints.

Assume we have two variables, a and b . The problem has only two dimensions. The variables are constrained by the following (hard constraints):

$$\begin{aligned} a &\geq 4 \\ b &\geq 0 \\ 0.5a + b &\leq 10 \\ 3a + b &\leq 30 \end{aligned}$$

These constraints limit the solution space, as illustrated in Figure 2.1. In other words, they have removed degrees of freedom from the solution.

Figure 2.1



Any feasible solution must be within the orange region (including the edges that bound the region).

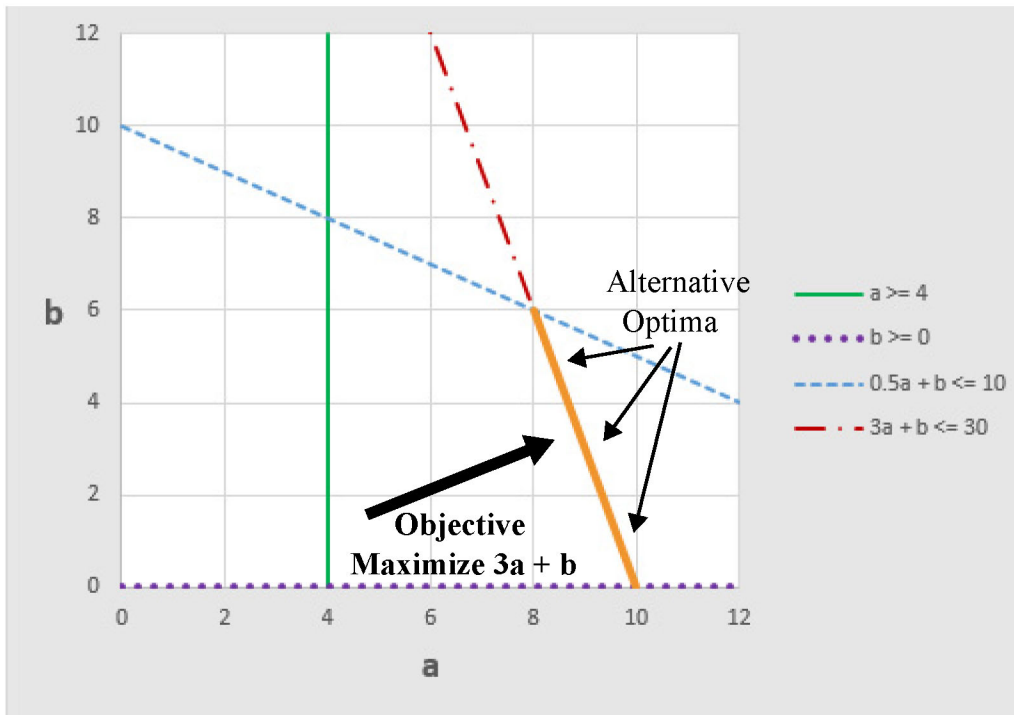
Chapter 2

Mathematics of Preemptive Linear Goal Programming

The problem has two prioritized objectives.

- Objective 1 (highest priority): Maximize $z_1 = 3a + b$
- Objective 2: (lower priority): Maximize $z_2 = b$

First, we will solve for the highest priority objective, Maximize $z_1 = 3a + b$.



The figure above shows the direction that the objective is pushing the solution. In this case, there are many possible solutions that will equally maximize the objective. Another way to state this is that there are alternative optima. Any point that is on the line $3a + b = 30$ with a between 8 and 10 (equivalently, b between 0 and 6) satisfies all constraints and equally maximizes the objective. The alternative optima are illustrated by the orange line segment in the figure above. Given the current set of constraints and objectives we have included in the problem (we have not yet added Objective 2), there is no preference for one point over another from this set of alternative optima. If we increase a , then we must decrease b , but in all cases the objective value, z_1 , is 30. The simplex method for solving linear programs tends to generate *extreme point* solutions. In this case it would generate a solution at one of the end points of the line segment.

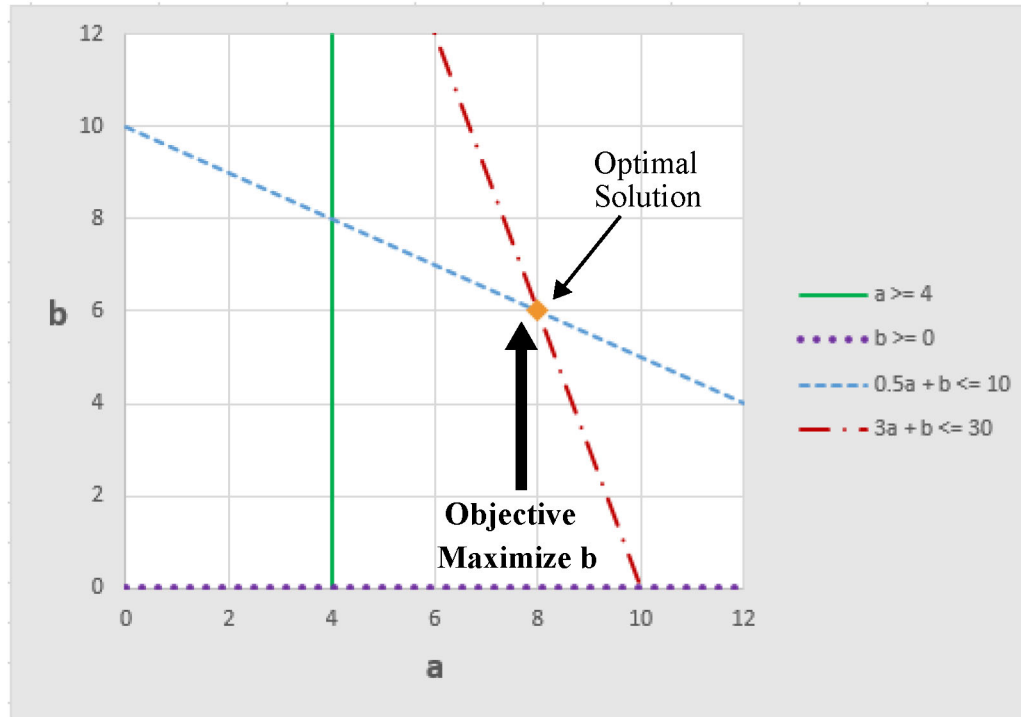
Now as we move on to Objective 2, we will not degrade the objective value of the higher priority Objective 1. In order to guarantee that Objective 2 does not degrade Objective 1, we will only use the solutions from the set of alternative optima from Objective 1. This is done by *freezing* the objective value with an “equal-to” constraint:

$$3a + b = 30$$

Note: We have frozen the constraint $3a + b \leq 30$ by changing it to an “equal to” constraint.

The new constraint reduces the solution space to the orange line segment in the figure above (i.e. remove degrees of freedom).

Now we apply Objective 2: Maximize $z_2 = b$. This is illustrated in the next figure. Once again the large arrow shows the direction that the objective is driving the solution.



In this case, the objective evaluates to a single optimal solution with the following values for all variables:

$$\begin{aligned} a &= 8 \\ b &= 6 \\ z_1 &= 30 \\ z_2 &= 6 \end{aligned}$$

All degrees of freedom have been removed from the solution. There are no longer any alternative optima, only a single optimal solution.

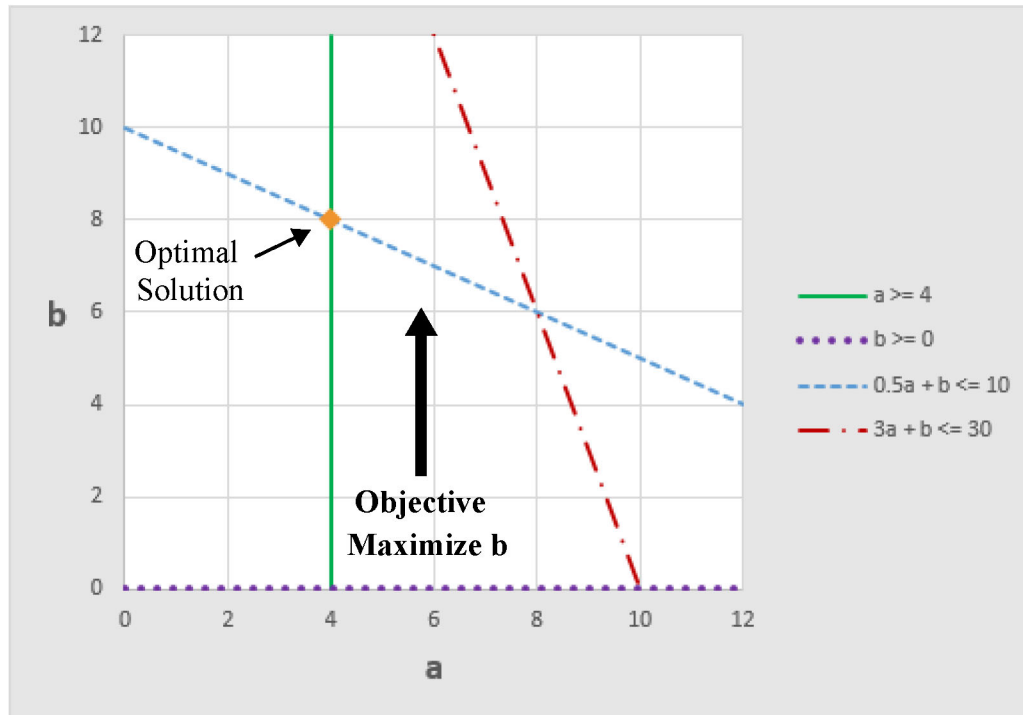
Now let's assume we have the same problem, but we reverse the priorities of the two objectives.

Objective 1 (highest priority): Maximize $z_1 = b$

Objective 2: (lower priority): Maximize $z_2 = 3a + b$

In this case, Objective 1 would remove all degrees of freedom, evaluating to a single optimal solution, as illustrated in [Figure 2.2](#).

Figure 2.2



The objective value gets frozen as.

$$z_1 = 8$$

The constraint $0.5a + b \leq 10$ also gets frozen, changing it to an “equal to” constraint.

$$0.5a + b = 10$$

In addition the constraint $a \geq 4$ gets frozen.

$$a = 4$$

There are no alternative optima, so there are no degrees of freedom left for Objective 2 to affect the solution. The final solution is determined after Objective 1.

$$a = 4$$

$$b = 8$$

$$z_1 = 8$$

$$z_2 = 20$$

Soft Constraints and Derived Objectives

In RiverWare, operational policy constraints created by the user are typically added to the optimization problem as prioritized soft constraints. (Constraints defining the physical characteristics of the system are automatically added to the optimization problem by RiverWare as hard constraints; see “[Physical Constraints](#)” on page 74 for details.)

RiverWare automatically converts the soft constraints at each priority into a *derived objective* to maximize the satisfaction of the soft constraints (minimize violations). This is done to prevent an infeasible problem in the case that it is not possible to satisfy all constraints. If it is not possible to fully satisfy all constraints at a given priority, the solution will get as close as possible, rather than return an infeasible solution. The next section provides a simple example with two variables to illustrate the conceptual use of prioritized soft constraints. This is followed by a simple example from a water resources management context. See [“Satisfaction Variables in Soft Constraints”](#) on page 17 for details about how soft constraints are defined. See [“Derived Objectives”](#) on page 19 for details about how derived objectives are formulated.

Prevent Infeasibilities

Two-dimensional Example

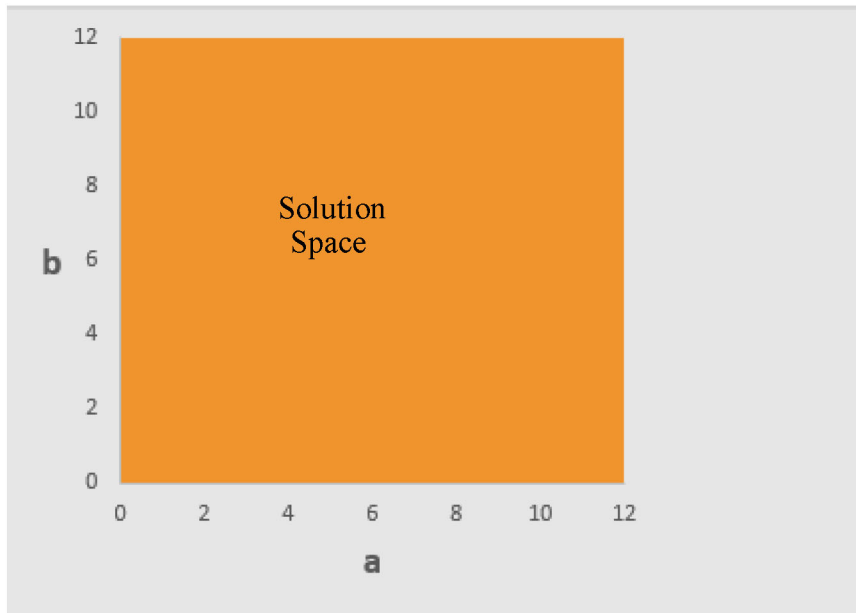
To illustrate the conceptual use of prioritized soft constraints to prevent infeasibilities, assume an optimization problem with two variables, a and b . The variables have the following formal bounds.

$$0 \leq a \leq 12$$

$$0 \leq b \leq 12$$

[Figure 2.3](#) illustrates the solution space given these bounds.

Figure 2.3



The problem has the following soft constraints and objectives in order from highest priority to lowest priority.

1. Soft Constraint: $a + b \leq 10$
2. Soft Constraint: $a \geq 5$
3. Soft Constraint: $b \geq 6$

Chapter 2

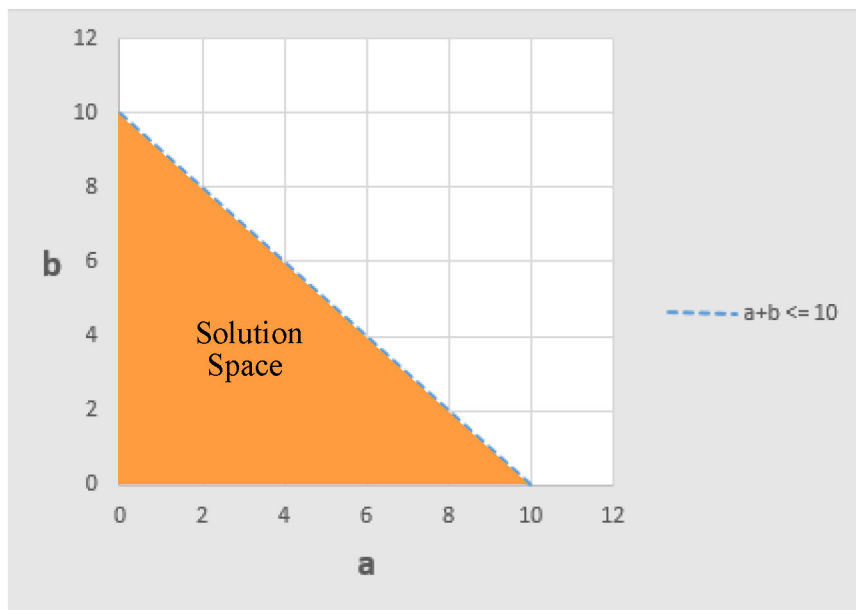
Mathematics of Preemptive Linear Goal Programming

4. Objective: Maximize $z_1 = a + b$

5. Objective: Maximize $z_2 = b$

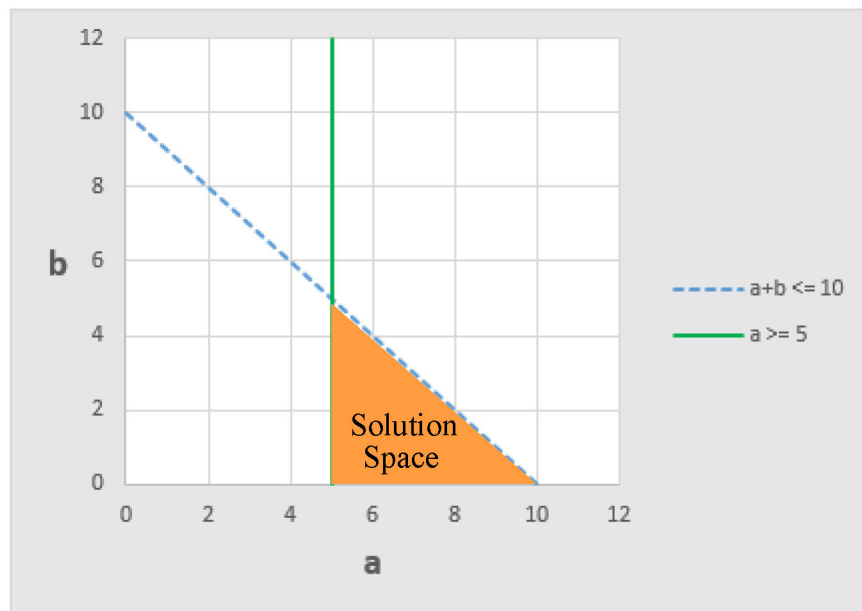
At priority 1, adding the constraint $a + b \leq 10$ reduces the solution space; see [Figure 2.4](#).

Figure 2.4



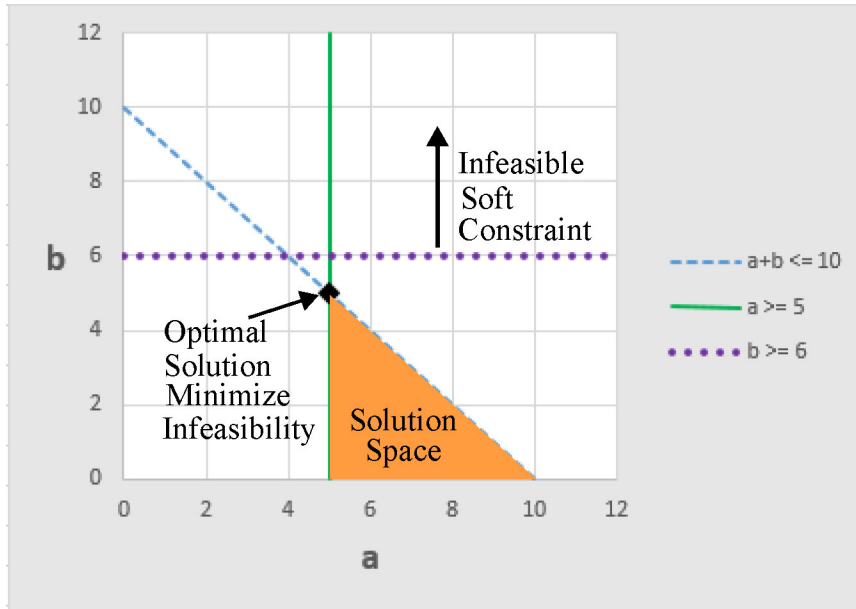
At priority 2, the constraint $a \geq 5$ further reduces the solution space; see [Figure 2.5](#).

Figure 2.5



Now when we get to priority 3, we can see that the constraint $b \geq 6$ cannot be satisfied without violating one of the two higher priority constraints. If the constraints were all hard constraints, the problem would be infeasible, and there would be no solution. Using soft constraints, the solution gets as close as possible to satisfying the constraint. It does this in a way that minimizes the violation of the priority 3 constraint in a manner that does not compromise the satisfaction of the two higher priority constraints; see Figure 2.6.

Figure 2.6



In this case, priority 3 removes all degrees of freedom from the solution. All of the constraints are frozen, and there is only one optimal solution.

$$a + b = 10$$

$$a = 5$$

$$b = 5$$

With no degrees of freedom remaining, the two objectives can do nothing to affect the solution.

Goal Programming With Many Dimensions

In the two dimensional examples above, we saw that a soft constraint may have one of several possible effects on the solution space:

- Cut away a portion of the 2-d solution space, leaving a smaller 2-d area as the new solution space
- Limit the solution to a 1-d line segment
- Pick a 0-d solution, a single point, which is a vertex

A soft constraint could also have no effect because it does not reduce space of feasible solutions.

If we extend the goal programming solution to a problem with three variables (three dimensions), the initial solution space is represented by a 3-d polyhedron. A soft constraint may have one of several possible outcomes:

Chapter 2

Mathematics of Preemptive Linear Goal Programming

- Cut away a portion of the 3-d polyhedron, leaving a smaller 3-d polyhedron as the new solution space
- Limit the solution to a face of the polyhedron, which is a 2-d polyhedron
- Limit the solution to a 1-d line segment on the polyhedron
- Pick a 0-d solution, a single point, which is a vertex of the polyhedron
- No effect because it does not reduce the polyhedron of feasible solutions

Goal programming in higher dimensions is similar. In n dimensions, if a constraint has an effect, it may result in a smaller polyhedron with n or fewer dimensions.

Goal programs used in a water resource management context typically have many dimensions. Each physical component (e.g. Reservoir Storage, Reservoir Outflow, Reach Outflow) at each timestep is an individual variable in the problem. Additional decision variables that do not represent physical components of the system may also be included at each timestep (e.g. Energy Value). This results in a solution space represented by a polyhedron in many dimensions.

Note: RiverWare only adds the variables to the problem that are necessary based on the specified policy. See “[Constraints Define Variables](#)” on page 73 and “[Physical Constraints](#)” on page 74 for details. In addition, any quantities that are known inputs are included in the problem as numeric values, not as variables. For example, the Inflow to the upstream reservoir in a basin might be provided as an input. In this case, the Inflow would not be a variable. The numeric inputs would get incorporated into problem as appropriate, in mass balance constraints for example.)

Water Resource Management Example

To illustrate how goal programming applies in a water management context, consider the following simple example.

A reservoir is modeled for a single timestep with a length of one day. The initial storage of the reservoir is 50,000 acre-ft, and the inflow over the timestep is an input at 2,000 acre-ft/day. (Assume a single inflow, a single outflow and no other gains or losses.) The reservoir has the following operational policy in priority order, highest priority to lowest.

1. Minimum Storage Constraint: $Storage_t \geq 45,000$ acre-ft
2. Minimum Outflow Constraint: $Outflow_t \geq 10,000$ acre-ft/day
3. Maximize Storage Objective: Maximize $Storage_t$

The priority 1 constraint can be fully satisfied and guarantees that the Storage will not go below 45,000 acre-ft when solving for the later priorities.

The priority 2 constraint cannot be fully satisfied without drafting the reservoir to 42,000 acre-ft, but priority 1 already guaranteed that the reservoir would not go below 45,000 acre-ft. So priority 2 will get as close as possible to meeting the Minimum Outflow by setting the Outflow to 7,000 acre-ft/day. This will freeze both constraints as follows:

$$Storage_t = 45,000 \text{ acre-ft}$$

$$Outflow_t = 7,000 \text{ acre-ft/day}$$

In this case there are no degrees of freedom remaining for the priority 3 objective, so it will do nothing to the solution. If the constraints at priorities 1 and 2 were added as hard constraints, the problem would be infeasible, and there would be no solution. Using soft constraints provides a solution that gets as close to the constraints as possible based on their priorities.

Now assume that the inflow over the timestep is 7,000 acre-ft/day. Both constraints could be fully satisfied, and the objective at priority 3 would result in the following solution.

$$Storage_t = 47,000 \text{ acre-ft}$$

$$Outflow_t = 10,000 \text{ acre-ft/day}$$

In practice, optimization problems for a water management context are far more complex, spanning many timesteps, multiple objects (physical features of the basin) and many operational policy constraints and objectives. The initial solution space is a polyhedron of many dimensions. In those contexts, the general conceptual approach is the same as for this example: at each priority get as close to satisfying the constraints as is possible (maximize satisfaction) without degrading any of the higher priority constraints or objectives. However there are multiple possible approaches to *maximize satisfaction*. The use of soft constraints and derived objectives to maximize soft constraint satisfaction in the RiverWare Optimization solver are described in more technical detail in the following sections.

Satisfaction Variables in Soft Constraints

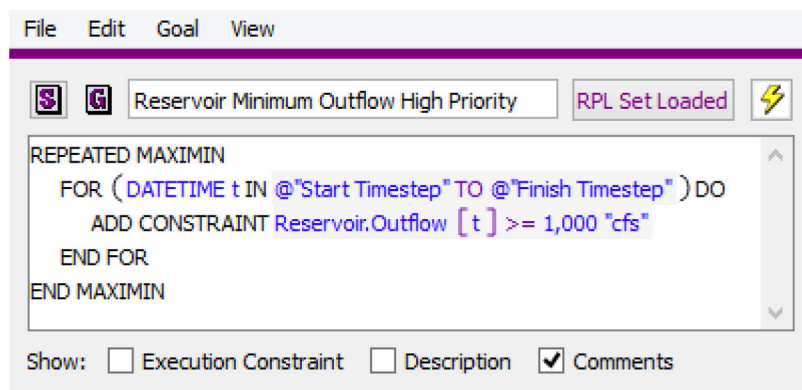
This section describes how satisfaction variables are introduced to formulate soft constraints. The following section describes how those satisfaction variables are included in a derived objective to maximize the soft constraint satisfaction (minimize violations).

In RiverWare, soft constraint violations, v_i , are scaled from 0 to 1 where 1 corresponds to the maximum violation from a higher priority constraint or bound. For performance reasons, the RiverWare Optimization solution actually uses a formulation based on satisfaction rather than violations.

$$s_i = 1 - v_i$$

It then maximizes satisfaction, where 1 corresponds to fully satisfied, and 0 corresponds to doing no better than the previous bound.

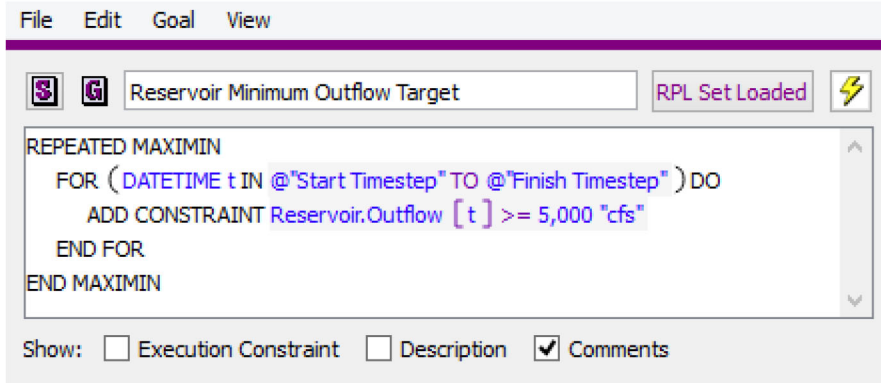
As an example, assume a reservoir has the following high priority minimum flow constraint.



Chapter 2

Mathematics of Preemptive Linear Goal Programming

(Technically this would translate to multiple constraints, one for each timestep.) Then assume that at a lower priority a more restrictive target minimum outflow is added.



Now assume that the high priority minimum flow constraint of 1,000 cfs was fully satisfied on all time steps, but on one time step it was only possible to get an outflow 4,000 cfs due to some other high priority constraints and low inflows. The lower priority target minimum outflow would not be fully satisfied. The satisfaction, s_i , for that particular constraint would be 0.75. 4,000 cfs is 75% of the way from the old lower bound of 1,000 cfs to the new constraint value of 5,000 cfs.

For a greater-than-or-equal-to constraint containing only a single variable, this can be expressed as:

$$s_i \leq \frac{VariableValue - OldLowerBound}{NewConstraintValue - OldLowerBound}$$

$$0 \leq s_i \leq 1$$

In the above example,

$$s_i = \frac{4000cfs - 1000cfs}{5000cfs - 1000cfs} = 0.75$$

Now assume that the high priority constraint setting a minimum of 1,000 cfs did not exist, and thus the constraint using 5,000 cfs was the first constraint with this left-hand-side formulation. In this case, the *OldLowerBound* would be taken from the Lower Bound set on the variable (slot configuration); see "Variable Bounds" on page 75 for details. Assume that the slot Lower Bound was 0 cfs. Then the satisfaction would be

$$s_s = \frac{4000cfs - 0cfs}{5000cfs - 0cfs} = 0.8$$

More generally, a soft constraint is defined as

$$a_1x_1 + a_2x_2 + \dots + a_{n-1}x_{n-1} + a_nx_n \leq s_i(b - OldUpperBound) + OldUpperBound$$

or

$$a_1x_1 + a_2x_2 + \dots + a_{n-1}x_{n-1} + a_nx_n \geq s_i(b - OldLowerBound) + OldLowerBound$$

$$0 \leq s_i \leq 1$$

where each x_n is an optimization variable in the constraint. Each a_n is a numeric coefficient on the corresponding variable, b is the right-hand-side numeric constraint value, and OldBound is the previous limit for the right-

hand-side either from a higher priority constraint of the same form (same left-hand-side) or based on the slot bounds if there is not a higher priority constraint with the same LHS.

Note: Internally, RiverWare always rearranges all constraints to have this same *canonical* form, with the sum of variables and their coefficients on the LHS and all constant numeric values combined in a single RHS value.

For every priority that contains soft constraints, RiverWare adds the constraints in this form, using satisfaction variables, to Optimization problem. It will then solve the Optimization problem at that priority with a derived objective to maximize the satisfaction. The derived objective that RiverWare automatically creates to maximize the value of the satisfaction variables is described in the following section.

Derived Objectives

For all priorities that contain soft constraints, RiverWare automatically generates an objective to maximize the satisfaction of the soft constraints and then solves the Optimization problem at that priority to maximize the derived objective with the new soft constraints added. The manner in which satisfaction is maximized depends on which type of derived objective is selected. Each of the derived objectives are described in the following sections. See “[Soft Constraint Set](#)” on page 62 for details about how to select the derived objective to use.

There are three general types of derived objectives in RiverWare: Summation, Single Maximin and Repeated Maximin. In addition Summation with Reward Table (Reward Function) is described here as a fourth type of derived objective, though technically it is implemented by adding a “With Reward Table” statement within a Summation derived objective. See “[With Reward Table](#)” on page 65 for details.

Summation

The Summation derived objective includes a separate satisfaction variable in each constraint added to the Optimization problem at the current priority p . The objective is then to maximize the sum of all of the satisfaction variables added at that priority.

$$\text{Maximize } \sum_{\text{all } i} s_{p,i}$$

where there is one satisfaction variable, $s_{p,i}$, for each constraint i added at the current priority p .

After maximizing the derived objective, the objective value is frozen so that the objective (satisfaction) is not degraded by lower priority objectives. Conceptually the following constraint is added to the Optimization problem

$$\sum_{\text{all } i} s_{p,i} = z_p$$

where z_p is the value from maximizing the derived objective. Technically this constraint does not get included, but the equivalent of adding this constraint is carried out by freezing select constraints and variables. See “[Freezing Constraints and Variables](#)” on page 24 for details about freezing constraints and variables.

The potential downside of the Summation approach is that it can tend to place all of the deviation on one or just a few constraints. For example, if a minimum flow constraint cannot be met for every timestep, it might minimize

Chapter 2

Mathematics of Preemptive Linear Goal Programming

the total violation by satisfying the constraint at all but one timestep and putting a very large violation on that one timestep. Typically, this is not the preferred solution in a water resources context.

Single Maximin

The Single Maximin derived objective applies the same satisfaction variable, s_p , for all constraints added at priority p . The objective is then to maximize the value of that single satisfaction variable.

Maximize s_p

Instead of minimizing the total violation, this approach will minimize the single largest violation (formally it maximizes the minimum satisfaction). This prevents a single, very large violation by spreading the violation out over multiple variables or timesteps.

Conceptually, the following constraint is added to the Optimization problem to freeze the satisfaction.

$$s_p = z_p$$

where z_p is the value from maximizing the derived objective. Technically this constraint does not get included, but the equivalent of adding this constraint is carried out by freezing select constraints and variables. The solution will freeze all constraints that are limiting the satisfaction if s_p is less than 1. These are the constraints with the largest violation. See “Freezing Constraints and Variables” on page 24 for details about freezing constraints and variables.

The potential downside of the Single Maximin approach is that it does nothing to improve the satisfaction of the remaining constraints that are not frozen. There might be other constraints introduced at priority p that could have a higher satisfaction than the limiting constraints, but this approach does nothing to enforce that higher satisfaction when moving on to solve at lower priorities. It only guarantees that they will have the same satisfaction as the least satisfied constraint. This issue is addressed by the next type of derived objective, Repeated Maximin.

Repeated Maximin

The Repeated Maximin approach begins with the single maximin. A single satisfaction variable, $s_{p,1}$, is applied for the first iteration for all constraints added at the current priority p , and it solves the Optimization problem with an objective to maximize the satisfaction variable (minimize the largest violations). It then conceptually freezes the satisfaction variable (satisfaction of the constraints with the largest violation). It also freezes all constraints that are limiting the satisfaction if $s_{p,1}$ is less than 1. These are the constraints with the largest violation. (See “Freezing Constraints and Variables” on page 24 for details about freezing constraints.) It then adds a new satisfaction variable, $s_{p,2}$, for all remaining constraints and solves to minimize the next largest violation (technically maximize the next smallest satisfaction). It repeats this process until either all violations have been minimized (all constraints added at the current priority p have been frozen) or all remaining constraints can be fully satisfied ($s_{p,i}$ equals 1).

while $s_{p,i-1} < 1$

Maximize $s_{p,i}$

$$s_{p,i} \geq s_{p,i-1} \quad \text{for } i > 1$$

where $s_{p,i}$ is a single satisfaction variable applied to all constraints added at the current priority p that have not yet been frozen prior to the i th Repeated Maximin iteration.

If it is possible to fully satisfy all constraints added at the current priority p (i.e. $s_{p,i} = 1$), there will only be a single iteration. In this case there may not be any constraints frozen, dependent on whether the goal forced any constraints to their limits.

When it is not possible to fully satisfy all constraints at a given priority, the Repeated Maximin approach tends to spread out deviations as evenly as possible, which is often the desired solution in a water resources context. We recommend Repeated Maximin as the default approach for most soft constraints. It is relatively easy to change to one of the other types of derived objectives later if this is desirable.

The potential downside of the Repeated Maximin approach is that in some cases it can require numerous iterations when it is not possible to satisfy all constraints. Depending on the size of the model this can cause a significant increase run time. The Summation approach always requires only a single solution at each priority, so it tends to be faster, but it does not, in general, provide the same solution quality as Repeated Maximin. In some cases, a reasonable balance between solution quality and run time can be provided by applying the Summation with Reward Table approach as described in the next section.

Summation with Reward Table

Summation with Reward Table is a special case of the Summation derived objective. Rather than summing all satisfaction variables at a given priority equally, this approach applies a reward function to each of the satisfaction variables. Typically this approach is used to penalize larger violations more than smaller violations (greater reward for higher satisfaction). For example, a common approach is to minimize the sum of squared violations. This tends to *smooth out* large violations by spreading them out over multiple time steps rather than concentrating large violations on a few time steps. Applying the appropriate reward function can often produce a result that compares to the Repeated Maximin solution in terms of solution quality but has the benefit of only requiring a single solution at the given priority and can therefore be much faster.

Note: Technically, the Summation with Reward Table approach is implemented by adding a With Reward Table statement within a Summation derived objective. See “[With Reward Table](#)” on page 65 for details on implementing Summation with Reward Table.

A *reward table* is used to apply a piecewise function to the satisfaction variables within a Summation objective. Instead of using the standard Summation derived objective for maximizing satisfaction, the derived objective becomes

$$\text{Maximize } \sum_{\text{all } i} R_{p,i}$$

where R is the reward function for the satisfaction variables.

$$R_{p,i} = f(S_{p,i})$$

The reward table slot has two columns. The first column contains satisfaction values. The second column contains the corresponding reward values. The values in both columns must be between 0 and 1. The reward table must be concave. (Reward must be a concave function of Satisfaction.)

Details of the general formulation of the Summation with Reward Table derived objective are given below, followed by an example that minimizes the sum of squared violations.

The following constraint is added that defines the satisfaction $S_{p,i}$ of constraint i introduced at priority p as the sum of all piecewise segments.

Chapter 2

Mathematics of Preemptive Linear Goal Programming

$$S_{p,i} = \sum_{\text{all } j} x_{p,i,j}$$

where $x_{p,i,j}$ is the length (Δx) of each segment j . The number of segments is defined by the Reward Table (# segments = # rows - 1). The maximum length of each segment is constrained by the satisfaction values $s_{p,i,j}$ entered in the first column of the Reward Table.

$$0 \leq x_{p,i,j} \leq s_{p,i,j} - s_{p,i,j-1}$$

The first row in the table is indexed as row 0. The slope for each segment $m_{p,i,j}$ is calculated as

$$m_{p,i,j} = \frac{r_{p,i,j} - r_{p,i,j-1}}{s_{p,i,j} - s_{p,i,j-1}}$$

where each $r_{p,i,j}$ is the value in the Reward column of the Reward Table corresponding to the satisfaction $s_{p,i,j}$.

The objective maximizes the total reward R_p , which is defined as follows.

$$\text{Maximize } R_p = \sum_{\text{all } i} \sum_{\text{all } j} m_{p,i,j} x_{p,i,j}$$

An example is provided below for a reward table that corresponds to minimizing the sum of squared violations

Example 2.1 Example Reward Table: Minimize the Sum of Squared Violations

Conceptually the desired derived objective is to minimize the sum of squared violations of all constraints introduced at the current priority p .

$$\text{Minimize } \sum_{\text{all } i} v_{p,i}^2$$

where $v_{p,i}$ is the violation of constraint i introduced at the current priority p . Violations are scaled from 0 to 1, where 1 corresponds to the maximum violation based on a higher priority constraint or variable bound. Thus an equivalent objective to minimize the sum of squared violations is

$$\text{Maximize } \sum_{\text{all } i} (1 - v_{p,i}^2)$$

For performance reasons, the RiverWare Optimization solution maximizes satisfaction variables rather than minimizing violations. See [“Satisfaction Variables in Soft Constraints”](#) on page 17 for details about how the satisfaction variables are defined.

$$S_{p,i} = 1 - v_{p,i}^2$$

The maximize objective then becomes

$$\text{Maximize } \sum_{\text{all } i} (1 - (1 - S_{p,i})^2)$$

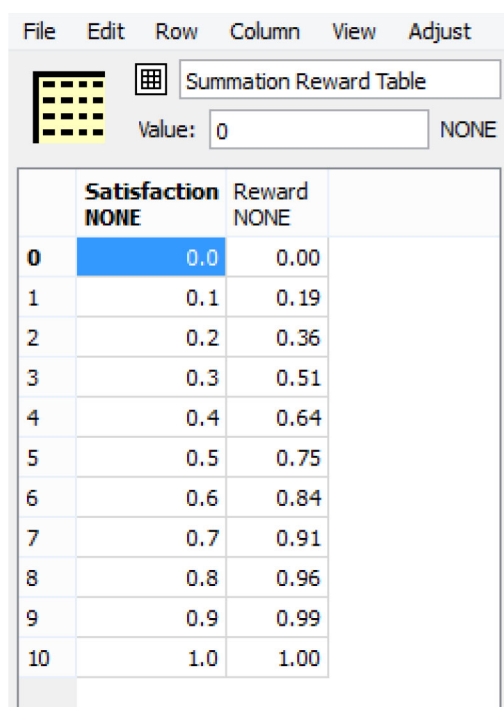
[Table 2.1](#) shows how this can be approximated by a piecewise linear function that linearizes the function in 10 discrete segments.

Table 2.1 Reward Table to Minimize Squared Violations

v_i	v_i^2	s_i	$1 - v_i^2$
1.0	1.00	0.0	0.00
0.9	0.81	0.1	0.19
0.8	0.64	0.2	0.36
0.7	0.49	0.3	0.51
0.6	0.36	0.4	0.64
0.5	0.25	0.5	0.75
0.4	0.16	0.6	0.84
0.3	0.09	0.7	0.91
0.2	0.04	0.8	0.96
0.1	0.01	0.9	0.99
0.0	0.00	1.0	1.00

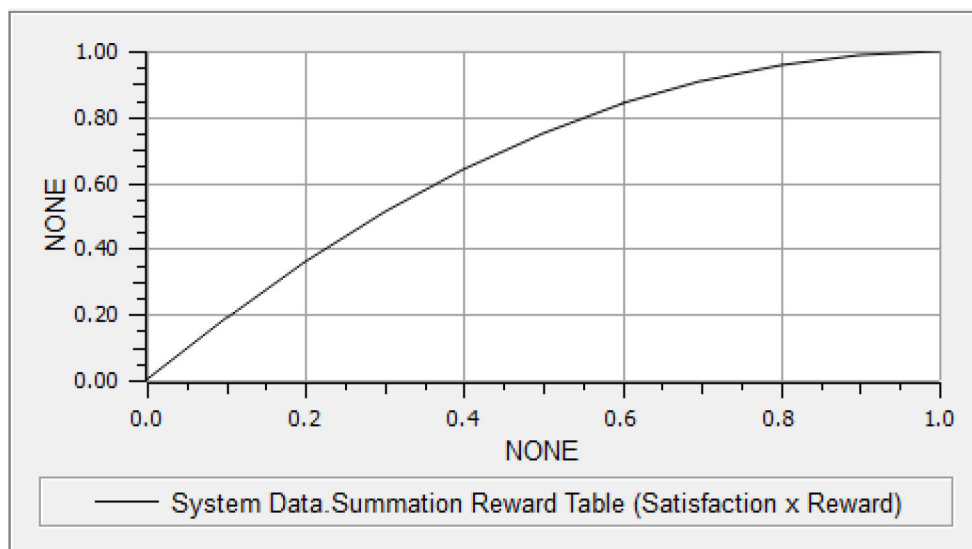
The reward table entered in RiverWare is taken from the last two columns, as shown in [Figure 2.7](#) and [Figure 2.8](#).

Figure 2.7



	Satisfaction	Reward
0	0.0	0.00
1	0.1	0.19
2	0.2	0.36
3	0.3	0.51
4	0.4	0.64
5	0.5	0.75
6	0.6	0.84
7	0.7	0.91
8	0.8	0.96
9	0.9	0.99
10	1.0	1.00

Figure 2.8



More generally, any concave table may be used. For example, a table can be created for minimizing cubed violations. Tables with more rows may degrade performance (increase run time) and numerical stability. There is a tendency for optimal solutions to be at the discrete values in the table. This may be more noticeable when tables with fewer rows (larger segments) are used. For example, if the reward table only had three rows corresponding to satisfaction values of 0, 0.5 and 1, it might be common to see the solution at a satisfaction of either 0.5 or 1.0. Some experimentation may be required to determine the best table size for a given goal.

Freezing Constraints and Variables

After solving at each priority, the RiverWare Optimization solution guarantees that later priorities will not degrade the objective from the current priority by locking in or *freezing* portions of the solution. See [“Preemptive Goal Programming”](#) on page 8 for conceptual examples of this. For numeric stability reasons, RiverWare does not technically freeze the objective function itself but rather freezes constraints and variables. These are constraints that are forced to their limits by the objective and variables that are forced to their upper or lower bounds. Freezing the appropriate constraints and variables guarantees that the objective will not be degraded by lower priorities. Violations of soft constraints always results in freezing that removes degrees of freedom from the solution. If there are no violations, freezing usually reduces the size of the feasible polyhedron without removing degrees of freedom. This section provides details about the freezing of constraints and variables.

Information about frozen constraints and objectives is helpful in Optimization solution analysis. Constraints that get frozen at the priority that they are introduced, which are typically soft constraints that cannot be fully satisfied, are constraints that are driving the solution at that priority. Constraints that were introduced at an earlier priority and get frozen at a lower priority are limiting the objective function at the lower priority. In the case of unsatisfied soft constraints at a given priority, the frozen constraints that were introduced at an earlier priority are preventing the constraints from being fully satisfied. See [“Priority-oriented Optimization Solution Analysis Tool”](#) on page 285 for details about frozen constraint information in the Optimization Solution Analysis Tool.

Freezing always occurs automatically after each iteration of a Repeated Maximin derived objective. This is necessary given the nature of the solution. A second Repeated Maximin iteration at a given priority only makes sense when constraints and variables are frozen after the first iteration. Otherwise it would simply be solving the same problem over and over. For all other objective types (Maximize, Minimize, Summation, Single Maximin), an explicit Freeze statement must be added to the goal after the objective statement. See “Freeze” on page 65 for details about how to apply a Freeze statement to goals. A common mistake is forgetting to add the Freeze statement after these objectives. If the Freeze statement is omitted, the solution will look the same as if the goal was never included in the optimization policy. The reason for this design is to retain the ability to have *test* objectives that provide information for a subsequent goal without using it to freeze the solution. This approach is not common, but when needed, it is very useful.

Freezing Constraints

Assume that at priority 1 a reservoir has a Minimum Storage constraint for every timestep.

$$\text{Reservoir.Storage}[t] \geq 10,000 \text{ acre-ft}$$

At priority 2, the reservoir has a Minimum Outflow constraint.

$$\text{Reservoir.Outflow}[t] \geq 5,000 \text{ cfs}$$

Assume that, given the initial Storage and Inflow over the first three timesteps (t1, t2, t3), it is not possible to meet both the Minimum Outflow and the Minimum Storage. The Minimum Storage is at a higher priority, so the solution would satisfy those constraints, and it would get the Outflow as close to the Minimum of 5,000 cfs as possible. In doing so, it would take the Storage down to the minimum by the third timestep. Using as much water as is available from the reservoir is the means by which it would get as close to the Minimum Outflow as possible. In other words, the Minimum Outflow constraint at priority 2 forces the Minimum Storage from priority 1 to be *tight* on t3, and the Minimum Storage constraint on t3 is limiting the objective for priority 2.

For all priorities below priority 2, we already know that the Storage at t3 must be at the Minimum Storage in order to prevent degrading the priority 2 objective to maximize the satisfaction of the Minimum Outflow constraint. So RiverWare *freezes* the Minimum Storage constraint on t3 by changing it to an “equal-to” constraint.

$$\text{Reservoir.Storage}[t3] = 10,000 \text{ acre-ft}$$

This would remove a degree of freedom from the solution for all subsequent priorities. The Storage on t1 and t2 is still constrained to be greater-than-or-equal-to 10,000 acre-ft, but at this point they have not been forced to be equal to 10,000 acre-ft, so they are not frozen here.

Technically, the constraint would be frozen in its canonical form including the satisfaction variable; see “Satisfaction Variables in Soft Constraints” on page 17 for details. Assume that the initial lower bound on Reservoir Storage before priority 1 was 0 acre-ft, then this would look like the following:

$$1.0 \times \text{Reservoir.Storage}[t3] = s_1(10,000 \text{ acre-ft} - 0 \text{ acre-ft}) + 0 \text{ acre-ft}$$

where 1.0 is the coefficient on the Reservoir Storage variable, s_1 is the satisfaction variable from priority 1, which would have been frozen at a value of 1.0 when solving at priority 1, and 0 acre-ft was the previous bound on Reservoir Storage.

Note: Technically, all evaluation is done internally in RiverWare Optimization units, which are typically scaled SI units, not in user display units as shown here.

Chapter 2

Mathematics of Preemptive Linear Goal Programming

The Minimum Outflow constraints would also get frozen at priority 2 but with a satisfaction variable value less than 1. Assume that the previous bound on Outflow was 0 cfs, and the priority 2 solution was only able to get the outflow to 4,000 cfs on all three timesteps. The satisfaction would be 0.8 (80% from the old bound of 0 cfs to the new constraint value of 5,000 cfs). The Minimum Outflow constraint would be frozen as follows:

$$1.0 \times \text{Reservoir.Outflow}[t1] = s_2(5,000 \text{ cfs} - 0 \text{ cfs}) + 0 \text{ cfs}$$

where s_2 is the satisfaction variable for priority 2 and will be frozen at a value of 0.8. The Minimum Outflow constraints at t2 and t3 would be frozen in a similar manner.

RiverWare determines which constraints to freeze by evaluating the dual price for each constraint. The dual price represents the unit improvement in the objective function that would result from a unit change in the constraint right-hand-side value while holding all else constant. If a constraint's dual price is greater than 0, it is an indication that the constraint is limiting the objective (i.e. the constraint is binding) so that constraint gets frozen. For example, assume that reducing the Minimum Storage limit on t3 from 10,000 acre-ft to 9,999 acre-ft would allow an Outflow of 4,000.2 cfs on all three timesteps. This would improve the objective value from 80% to 80.004%. The dual price of the Minimum Storage constraint on t3 would be 0.004%/acre-ft. This is greater than zero, so the Minimum Storage constraint on t3 gets frozen. If the Minimum Storage value on t1 or t2 were relaxed, the objective would still be limited by the Minimum Storage on t3. The the objective value would not be improved by changing the t1 or t2 constraint value. Thus the dual price for these constraints is zero, and the constraints do not get frozen. Internally this evaluation is always carried out in RiverWare Optimization units, and the dual price must actually be greater than a freezing tolerance of 10^{-6} in RiverWare Optimization units in order to freeze the constraint.

When looking at the Optimization Solution Analysis Tool for priority 2, you would see the Minimum Outflow constraints as constraints from the current priority that were frozen. See [“Priority-oriented Optimization Solution Analysis Tool”](#) on page 285 for details. These constraints were driving the solution at priority 2. You would also see the Minimum Storage constraint from t3 as a constraint that was introduced earlier that was frozen at priority 2, indicating that it limited the satisfaction of the priority 2 Minimum Outflow constraints.

Freezing Variables

Variables that are binding on the solution get frozen as well. This occurs when variables are forced to their upper or lower bound by the objective. (For variables that correspond to slots, the upper and lower bounds are set in their slot configuration by the user. See [“Variable Bounds”](#) on page 75 for information about setting the required bounds on variables. For variables that are added to the problem automatically by RiverWare and do not correspond to slots, RiverWare sets the bounds automatically. One example of the latter is the satisfaction variable for a soft constraint.)

For example, Regulated Spill typically has a lower bound of zero. Assume that there are no policy constraints that require Regulated Spill to be greater-than-or-equal-to a non-zero value. An objective to maximize Power generation will often drive Regulated Spill to zero in order to use as much water as possible for generation. The variable will get frozen at its lower bound of zero.

$$\text{Reservoir.Regulated Spill}[t] = 0 \text{ cfs}$$

Another common case in which variables are frozen is the freezing of satisfaction variables for soft constraints. Satisfaction variables automatically have upper and lower bounds of 0 and 1. For example, assume that the goal at

priority i contains a Repeated Maximin derived objective. RiverWare will add a new satisfaction variable, s_i , to the problem.

$$0 \leq s_i \leq 1$$

If the solution is able to fully satisfy all of the constraints in goal i , then it will freeze the satisfaction constraint at its upper bound of 1.

$$s_i = 1$$

Similar to the use of dual prices for freezing constraints, reduced costs are used in the freezing of variables. RiverWare freezes all variables with a reduced cost greater than zero. The reduced cost represents the unit decrease in the objective value for a unit change in the bound on the variable (unit increase for a lower bound or unit decrease for an upper bound). In the Regulated Spill example, increasing the lower bound on Regulated Spill to be greater than zero would decrease the amount of power that could be generated. Some water would have to be used for spill rather than generation. Thus increasing the lower bound would reduce the objective value, so Regulated Spill would have a non-zero reduced cost. If the upper bound on the satisfaction variable in the second example were less than one, it would reduce the objective to maximize satisfaction, so the reduced cost on the satisfaction variable is non-zero.

In this case that the satisfaction variable is frozen at a value of 1, it is as if the objective is frozen directly. This is not the case in general, however. The satisfaction variable only gets frozen if it is at its bound. (Typically this is when the satisfaction variable is at its upper bound of 1, but it can also occur if a satisfaction variable on an individual constraint in a Summation derived objective is at its lower bound of zero.) If the satisfaction is somewhere between 0 and 1, the satisfaction will not get frozen directly. Freezing all constraints with a non-zero dual price and all variables with a non-zero reduced cost has the equivalent effect of freezing the objective value but performs much better in terms of numeric stability.

Typically information about frozen variables is not as useful as information about frozen constraints when trying to understand the Optimization solution. Often this is due to the fact that when a variable gets frozen at a bound, this is an *obvious* limit that the user considers *intuitively*. Thus it does not tend to provide the user with more information than they already had. In some cases, however, identifying unexpected frozen variables can assist in debugging an Optimization model, particularly if incorrect upper or lower bounds were set on a variable unintentionally.

Shrinking Constraints

The concept of *shrinking constraints* can be important for understanding some optimization solution analysis information provided by RiverWare and for understanding how the solution behaves in terms of adding new constraints to the optimization problem.

In water resource management policies, it is common to have constraints at different priorities that have the same form (same left-hand-side) and differ only in the right-hand-side constraint value. For example, assume that a reservoir Storage variable has a formal upper bound of 10,000 acre-ft. In the policy constraints, there is a high priority Maximum Storage constraint to not exceed 9,000 acre-ft. At a lower priority there is a Target Operating Range constraint that applies a more restrictive max Storage of 8,000 acre-ft. Typically it is desirable to be below this elevation, but it might not always be possible due to high inflows. Then at a lower priority there is a Target Operating Storage Point constraint that constrains the Storage equal to 7,000 acre-ft, if possible.

Chapter 2

Mathematics of Preemptive Linear Goal Programming

Note: Internally, all “equal-to” constraints are converted to an equivalent pair of constraints.

$$\text{Reservoir.Storage}[t] \geq 7,000 \text{ acre-ft}$$

$$\text{Reservoir.Stroage}[t] \leq 7,000 \text{ acre-ft}$$

Together, in addition to the upper bound, there would be three constraints with the same left-hand-side (LHS), in order from highest priority (1) to lowest (3):

- Priority 1: $\text{Reservoir.Storage}[t] \leq 9,000 \text{ acre-ft}$
- Priority 2: $\text{Reservoir.Storage}[t] \leq 8,000 \text{ acre-ft}$
- Priority 3: $\text{Reservoir.Storage}[t] \leq 7,000 \text{ acre-ft}$

Note: In practice, there would typically be constraints on other variables at intervening priorities between the constraints with the same LHS. For simplicity here we are only looking at the constraints with common LHS.

Rather than add a new constraint to the optimization problem each time, RiverWare minimizes the size of the optimization problem by *shrinking* the lower priority constraints into the higher priority constraint of the same form.

The priority 1 constraint, when written as a soft constraint with a satisfaction variable would have the following form:

$$\text{Reservoir.Storage}[t] \leq s_1(9,000 \text{ acre-ft} - 10,000 \text{ acre-ft}) + 10,000 \text{ acre-ft}$$

Assuming that the priority 1 constraint is fully satisfied, the priority 2 constraint would conceptually have the following form:

$$\text{Reservoir.Storage}[t] \leq s_2(8,000 \text{ acre-ft} - 9,000 \text{ acre-ft}) + 9,000 \text{ acre-ft}$$

However, instead of adding a new constraint, RiverWare notices that the new constraint has the same LHS and only differs in the right-hand-side value, so it shrinks the priority 2 constraint into the priority 1 constraint in the following form.

$$\text{Reservoir.Storage}[t] \leq s_2(8,000 \text{ acre-ft} - 9,000 \text{ acre-ft}) + s_1(9,000 \text{ acre-ft} - 10,000 \text{ acre-ft}) + 10,000 \text{ acre-ft}$$

The less-than-or-equal-to portion of the priority 3 constraint would then shrink into this constraint in the same manner.

$$\begin{aligned} &\text{Reservoir.Storage}[t] \\ &\leq s_3(7,000 \text{ acre-ft} - 8,000 \text{ acre-ft}) + s_2(8,000 \text{ acre-ft} - 9,000 \text{ acre-ft}) + s_1(9,000 \text{ acre-ft} - 10,000 \text{ acre-ft}) + 10,000 \text{ acre-ft} \end{aligned}$$

Note: All evaluation is done internally in RiverWare Optimization units, which are typically scaled SI units, not in user display units as shown here.

More generally, constraints with shrinking have the following form:

$$\begin{aligned} &a_1x_1 + a_2x_2 + \dots + a_{m-1}x_{m-1} + a_mx_m \\ &\leq s_n(b_n - b_{n-1}) + s_{n-1}(b_{n-1} - b_{n-2}) + \dots + s_2(b_2 - b_1) + s_1(b_1 - \text{UpperBound}) + \text{UpperBound} \end{aligned}$$

where each x_m is a variable in the LHS of each of the constraints in the shrinking sequence, each a_m is the corresponding coefficient on the variable, each s_n is the satisfaction variable associated with the constraint at the n th priority that contains a constraint with this common LHS, and each b_n is the right-hand-side constraint value at

the n th priority that contains a constraint with this LHS. *UpperBound* is the formal upper bound on the LHS. It is calculated by evaluating the LHS with each variable with a positive coefficient replaced by the variable's upper bound and each variable with a negative coefficient replaced by the variable's lower bound. For variables that correspond directly to a slot, the variable bounds are taken directly from the Upper and Lower Bounds set in the slot configuration. See [“Variable Bounds”](#) on page 75 for details.

The form is the same for a “greater-than-or-equal-to constraint”, except that the *LowerBound* is used. The *LowerBound* is calculated by evaluating the LHS with each variable with a positive coefficient replaced by the variable's lower bound and each variable with a negative coefficient replaced by the variable's upper bound.

$$a_1x_1 + a_2x_2 + \dots + a_{m-1}x_{m-1} + a_mx_m \\ \geq s_n(b_n - b_{n-1}) + s_{n-1}(b_{n-1} - b_{n-2}) + \dots + s_2(b_2 - b_1) + s_1(b_1 - \text{LowerBound}) + \text{LowerBound}$$

In the example above, if the priority 2 constraint were not fully satisfied, then that constraint would be frozen. (See [“Freezing Constraints”](#) on page 25 for details about freezing constraints.) In that case, the priority 3 constraint (or any subsequent constraints with the same LHS) would never get added to problem. Because the LHS value was already *locked in* at priority 2, no lower priority constraints can change its value. Thus, there is no need to add any new constraints with the same LHS to the problem. In other words, RiverWare identifies if a new constraint would shrink to a constraint that is already frozen. If that is the case, then the new constraint does not get added to the problem. This can be important to understand when analyzing the solution. No solution analysis information will be provided for a constraint that would shrink to a constraint that was already frozen because the new constraint never gets added to the problem. If all of the constraints at one priority would shrink to constraints that are already frozen, then no new constraints get added to the problem at that priority, and thus there is no need to solve the problem at that priority. As a result, that priority will not show up in the standard diagnostic output with an objective value (percent satisfaction), nor will that goal appear in the Optimization Solution Analysis Tool. See [“Priority-oriented Optimization Solution Analysis Tool”](#) on page 285 for details.

If some constraints from a given priority would shrink to frozen constraints, but others do not (i.e. there is still a solution at that priority, but some constraints introduced at that priority are omitted because they would not change the solution), the reporting of the satisfaction for the soft constraint derived objective depends on the type of derived objective that is selected. If the derived objective is Single Maximin or Repeated Maximin, then the satisfaction only reflects the satisfaction of the constraints that were formally added to the problem. It does not reflect the satisfaction of the constraints that were omitted (and would possibly have been violated). A tilde (~) is added to the reported satisfaction value to indicate that it is greater than the actual satisfaction from the user's perspective because it does not include the omitted constraints. For a Summation derived objective, the reported average satisfaction is calculated as if the omitted constraints had been added to the problem. Thus it reflects the satisfaction from the user's perspective, and no special notation is required.

If a frozen constraint shrinks to a higher priority constraint, the Optimization Solution Analysis Tool lists which constraint it shrinks to as part of the frozen constraint information (see [“Priority-oriented Optimization Solution Analysis Tool”](#) on page 285). Also, if a constraint is frozen, the solution analysis tool will report that all of its *shrink to* constraints were frozen as well. Without understanding how constraints shrink to earlier constraints, this can be confusing because it appears that a higher priority constraint was frozen even though the solution was not at the limit set by the constraint. Really it is the lowest priority constraint that shrank to that higher priority constraint that was forced to its limit and was frozen.

Chapter 2

Mathematics of Preemptive Linear Goal Programming

Chapter 3

Combining Optimization With Simulation

A complete RiverWare Optimization run makes use of a combination of the Simulation, Optimization and Rulebased Simulation. The motivation for this approach along with the details of each of the components are described in the following sections.

Topics

- [Standard Controller Sequence, page 31](#)
- [Post-optimization Ruleset, page 33](#)
- [Initialization Rules, page 35](#)
- [Restore Point, page 37](#)
- [Limitations of Optimization, page 49](#)

Standard Controller Sequence

A standard RiverWare Optimization run uses the Simulation - Optimization - Rulebased Simulation controller approach as follows: a Simulation run is made to clear any output values, check data, and propagate input values, then the Optimization is performed, and finally, a Post-optimization Rulebased Simulation is run to set the optimal values on the objects. Every run should be made as follows:

- Switch the run controller to **Simulation**, and select **Start**. Let the model run to completion.
- Switch the run controller to **Optimization**, load a Goal Set (if not already loaded), and select **Start**. Let the model run to completion.
- Switch the run controller to **Rulebased Simulation**, load a Ruleset (if not already loaded), and select **Start**. Let the model run to completion.

The use of the three controllers is described in more detail in the following sections.

Simulation

The user starts with a model set up to run with the Simulation controller. The Simulation run serves five purposes described below.

Clear Output Values

This guarantees that leftover output data from a previous solution does not inadvertently get used in the current run.

Check for Required Inputs

Many Simulation methods check for required input data at the start of the Simulation run.

Preprocessing of Data

If the model contains initialization rules or expression slots that evaluate at the beginning of the run to preprocess data, these get evaluated during the simulation.

Propagate Input Values

Input values on linked slots get propagated across the links in the Simulation run so that they are properly registered as values for the corresponding variables.

Simulate where Possible

The Simulation run can also be used to simulate all aspects of the basin for which the user does not want to optimize. For example, the user might know that one reservoir may only release minimum flows during the run. Then the Outflow for that reservoir can be set as input to the minimum flow value for all timesteps, and the Simulation run will propagate the effect of this. Then in the optimization run, this reservoir is not considered.

Note: A minimum flow requirement could alternatively be set by a high priority constraint in the Optimization Goal Set.

In most cases, after the Simulation run most slots of interest will still display NaN.

Optimization

The solution of the linear goal program takes place during the Optimization run. During this run RiverWare takes the combination of the prioritized Optimization Goal Set and the model data (topology, physical characteristic data, time series data) and uses them to formulate a series of linear programs which it sends to CPLEX. CPLEX solves each linear program and returns the optimal values of the variables to RiverWare. See [“Mathematics of Preemptive Linear Goal Programming”](#) on page 7 for details of the preemptive linear goal programming solution.

At the end of the Optimization run, RiverWare stores the final optimal value for every variable. At this point, no values have been set in slots on the workspace. That is to say that all slots which displayed NaN after the Simulation run will still display NaN after the Optimization run. Values will not be set in slots until the Post-optimization Rulebased Simulation, which is described in the following section.

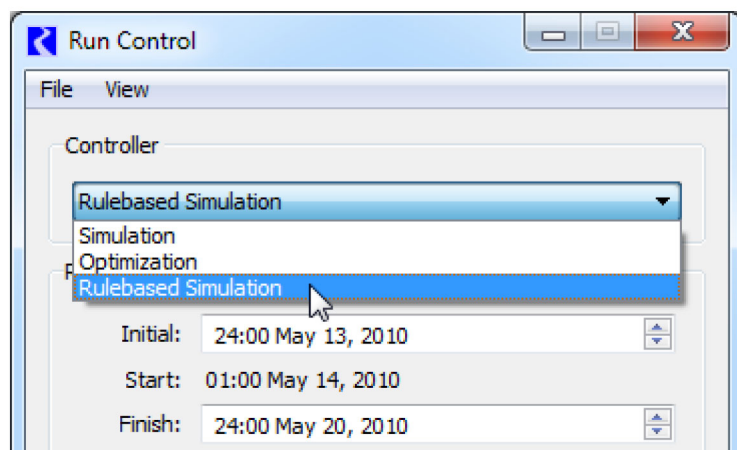
Post-optimization Rulebased Simulation

After the run with the Optimization controller is completed, the Optimization solution is stored internally in RiverWare. Values from the solution are not yet returned to the workspace. All slots that displayed NaN at the end of the Simulation run will continue to display NaN at the end of the Optimization run. The solution is returned to the workspace by running a Post-optimization Rulebased Simulation. This is a special use of the RiverWare Rulebased Simulation (RBS) controller. The Ruleset for a Post-optimization RBS does not typically reflect the

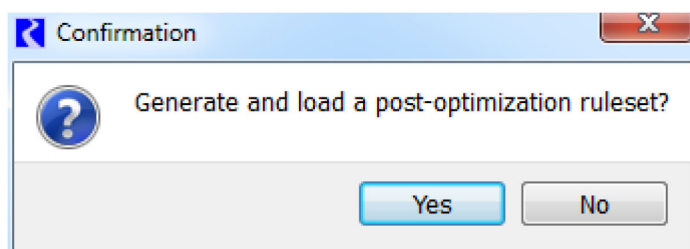
operating policy of the basin as it would for a standard RBS. The operating policy is expressed in the Optimization Goal Set (though there is technically nothing to prevent formulating rules in the Post-optimization ruleset to override results from the Optimization solution with additional policies). Rather the rules in a Post-optimization ruleset select values on the simulation objects, typically reservoir outflows, based on the values from the Optimization solution. With these values set, the simulation can solve fully for all remaining variables.

A second purpose of the Post-optimization RBS is to remove approximation error introduced by linear approximations in the Optimization solution. For example, Power must be linearized in the Optimization solution, introducing approximation error into the solution for Power. In the Post-optimization RBS, the reservoirs solve for Power using the full non-linear simulation method given the Outflow specified by the Optimization solution. This removes the approximation error introduced into Power from the linearization employed for Optimization.

After the Optimization run is complete, switch the Controller to **Rulebased Simulation**.



If no Ruleset is loaded when the controller is changed from Optimization to Rulebased Simulation, a message will appear asking if the user wants RiverWare to generate and load a ruleset. Either select **Yes** to use this automatically generated Ruleset for the Post-optimization RBS, or create your own custom set. See [“Post-optimization Ruleset”](#) on page 33 for descriptions of these two options.



Post-optimization Ruleset

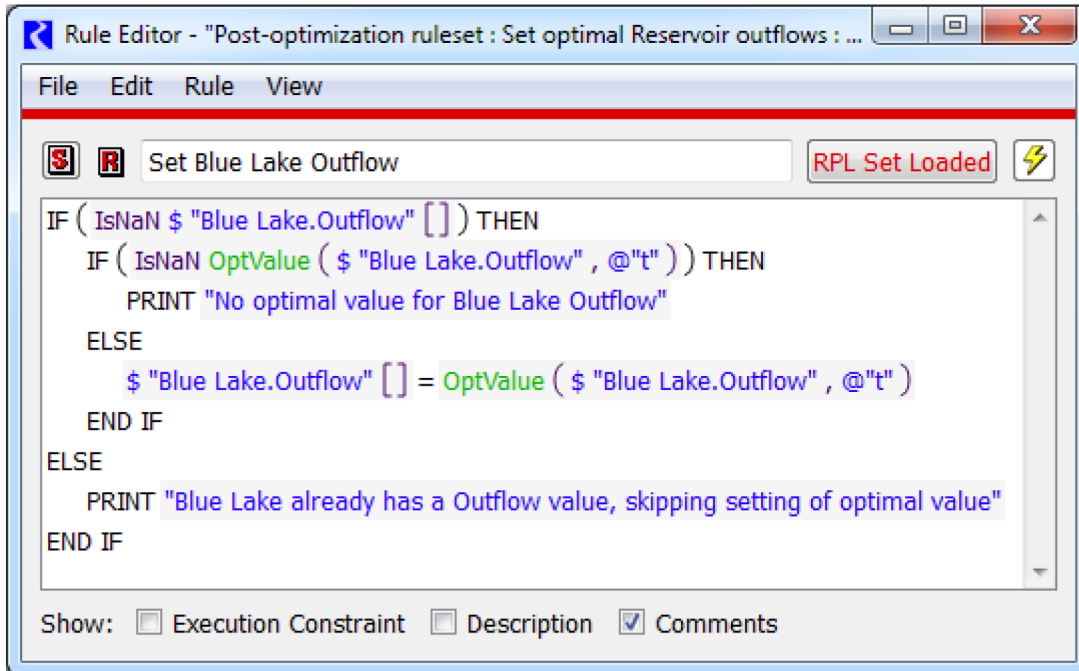
The final step of a complete Optimization run in RiverWare is a Post-optimization Rulebased Simulation (see [“Post-optimization Rulebased Simulation”](#) on page 32). The ruleset used for this special case of RBS can either be generated automatically by RiverWare, or a custom set can be developed by the user. These two options are described in the following sections.

Automatically Generated Ruleset

The automatically generated ruleset contains two policy groups. The first group (higher priority) is called “Set optimal Reservoir outflows.” The rules in this group set Reservoir.Outflow to the value from the Optimization solution by calling the OptValue() function. There is one rule for each reservoir, and they are ordered topologically. Specifically, the low to high priority order of rules corresponds to an upstream to downstream ordering of the associated reservoirs. Thus when using the default agenda order (...3,2,1), this causes the reservoirs to solve upstream to downstream.

Note: If two objects are on different tributaries or in unconnected basins, then there is no constraint on the relative order of their associated rules. Reservoirs in a No Optimization subbasin are excluded from the automatically generated ruleset. Figure 3.1 is an example of an automatically generated Post-optimization Rule.

Figure 3.1



The second policy group (lower priority) is called “Set optimal user defined variables values.” This policy group will only contain rules if the user has specified any user-defined variables (see “[User-defined Variables](#)” on page 76). The form of the rules is the same as those for reservoir outflows, but they set values on user defined variable custom slots. These user defined variable slots have no effect on the simulation when using the automatically generated Post-optimization Ruleset. Their values are set only for user reference.

If no ruleset is loaded when the user switches from the Optimization controller to the Rulebased Simulation controller after an Optimization run, then the user will be prompted regarding whether they would like RiverWare to generate an automatic Post-optimization Ruleset.

Custom Post-optimization Ruleset

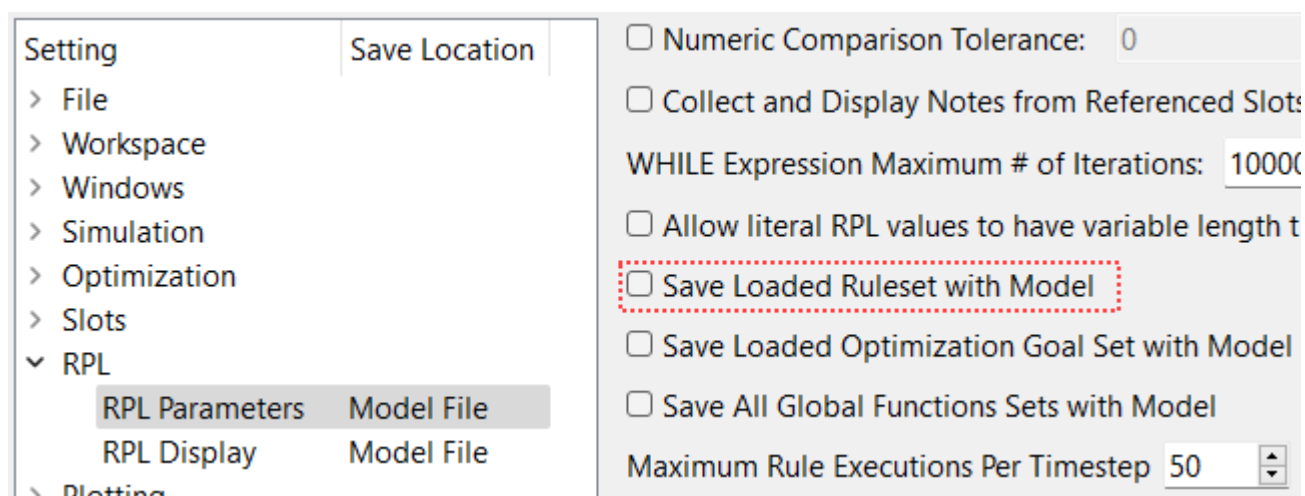
In some cases, setting only reservoir outflows might not be sufficient for a Post-opt RBS. When Outflow is set on a Reservoir in RBS, the object will solve by putting as much of the Outflow through the turbines as is possible based on physical parameters. It will only use Spill if the specified Outflow is greater than the Turbine Capacity or if the Pool Elevation is above the crest of an Unregulated Spillway. This might be sufficient in a system where it is always desirable to prevent Spill, when possible. In some systems, however, a specified Spill amount might be required by an environmental policy. The Optimization Goal Set would likely contain one or more goals corresponding to this policy. In order to get the specified Spill into the Post-optimization RBS, a rule must set Turbine Release and Spill individually, rather than setting total Outflow (or alternatively set Outflow and Spill and let the model calculate Turbine Release). Other uses for a custom Ruleset are as follows:

- If other water uses, such as diversions, need to be specified in the RBS based on the Optimization solution.
- To refine the Optimization solution further in the Post-optimization RBS, possibly to make small adjustments to the flows once outputs have been calculated with the approximation errors removed.
- To add post-processing calculations to the Post-optimization rules.

A useful approach for creating a custom Post-optimization Ruleset is to begin with the automatically generated set. Then revise the rules as needed using the basic structure of the automatically generated set.

Once a custom Post-optimization Ruleset has been created, it can optionally be saved with the model file. Then the next time the model is opened, the Ruleset will automatically be opened and loaded. To save the Ruleset with the model file it must first be loaded. Then on the workspace select **Policy**, then **RPL Parameters** to access the Settings Manager. Check the box for **Save Loaded Ruleset with Model** as shown in Figure 3.2. Select OK or Apply and save the model with the Ruleset now included.

Figure 3.2 RPL Parameters in the Settings Manager



Initialization Rules

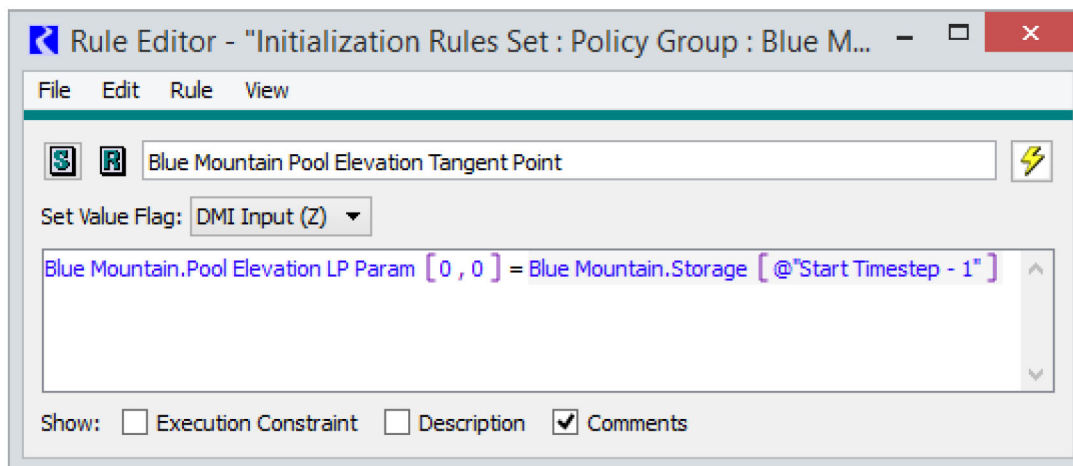
Initialization Rules can be used for preprocessing of data for Optimization runs. See “[Initialization Rules Set](#)” in *RiverWare Policy Language (RPL)* for general information on initialization rules.

Usage

One use of Initialization Rules that is particular to Optimization is setting approximation points. Approximation points allow the user to control how non-linear processes are linearized. See [“Linearization, Approximation, and Replacement”](#) on page 79 for general information on linearization. See [“Provide Approximation Points”](#) on page 275 for guidelines on setting approximation points.

In some cases, it might make sense to set the approximation points based on the specific run conditions. For example, for a reservoir that has a large seasonal variation in Pool Elevation, it might not be possible to set Tangent and Line approximation points in the Pool Elevation LP Param table or an Operating Head point in the Power LP Param table (when using the Independent Linearizations method in the Optimization Power category) that will result in sufficiently small approximation error across all conditions. In these cases, initialization rules can be used to set the approximation points such that the approximation error will be small for the expected operating range of the given run but would be large if operating in a different range in another season. [Figure 3.3](#) is an example initialization rule that sets the Pool Elevation LP Param Tangent point equal to the initial Storage for the run.

Figure 3.3 Sample Initialization Rules setting LP Param approximation points



A second rule could set the two Line points to the Tangent point plus or minus some expected delta. In this manner, the approximations will be updated automatically when the model runs, rather than needing to adjust the values manually at the start of each run.

Enable or Disable Initialization Rules based on Controller

Initialization rules can execute at the beginning of the Simulation controller run, the Optimization run, and/or at the beginning of the post-optimization Rulebased Simulation. The execution is controlled by the separate toggles/settings on the Run Control as shown in [Figure 3.4](#) and [Figure 3.5](#).

Figure 3.4 Run Control showing Execute Initialization Rules for Sim/Rule controller

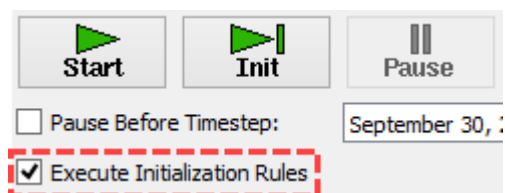
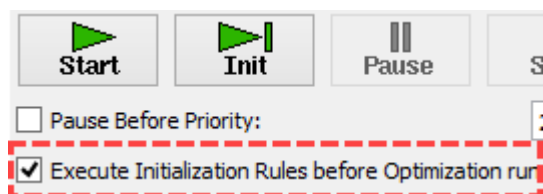


Figure 3.5 Run Control showing Execute Initialization Rules before Optimization run



You can use scripts to turn on or off these toggles/flags. See [“Set Init. Rules Exec. Flag”](#) in *Automation Tools*.

Timing

Initialization rules execute *before* prerun expression slots. These execution times should be kept in mind when formulating the logic of initialization rules used for Optimization and which slot values they can reference.

Restore Point

In some contexts it is useful to conduct a sequence of optimization runs, each of which involves an optimization policy whose initial portion is the same for each run. A Restore Point allows RiverWare to perform the computations for the initial, shared portion of the policy once, save the state of the optimization problem after the last common goal, then use that saved state as an advanced start for subsequent runs in the sequence. Thus a Restore Point can be used to reduce the overall solution time for the sequence of runs.

Example 3.1 Restore Point Use Case

You want to execute three optimization runs that all use the same input data and have the same high priority constraints. They differ only in a lowest priority objective function.

- Run 1 (Baseline) solves for expected operations using an objective to maximize economic value.
- Run 2 determines the upper bound on hydropower generation with an objective to maximize generation.
- Run 3 determines the lower bound on hydropower generation with an objective to minimize generation.

Assume the following goal set:

1. Soft constraints: Pool Elevation Max
2. Soft constraints: Pool Elevation Min
3. Soft constraints: Outflow Max
4. Soft constraints: Outflow Min
5. Soft constraints: Target Ending Elevations
6. Create Restore Point
7. Objective: Maximize Economic Value
8. Objective: Maximize Hydropower Generation
9. Objective: Minimize Hydropower Generation

The Restore Point could be used to reduce the run time for the three runs in the following manner.

- Run 1: Turn off goals 8 and 9. The optimization will solve goals 1-5 as normal. In goal 6, it will create a Restore Point to save the state of the optimization problem after goal 5 (see [“Create a Restore Point”](#) on page 39.) Then it will continue on and solve through the objective in goal 7 as normal.
- Run 2: Turn off goal 7, and turn on goal 8. Run using the Restore Point (see [“Use a Restore Point”](#) on page 40). The optimization will restore the optimization problem at goal 6 then advance directly to solving goal 8.
- Run 3: Turn off goal 8, and turn on goal 9. Run using the Restore Point. The optimization will restore the optimization problem at goal 6 then advance directly to solving goal 9.

For runs 2 and 3, the optimization has to solve a single goal only. For large models with extensive goal sets, the ability to skip over many of the goals can significantly reduce the solution time.

Optimization runs that use a Restore Point still follow the complete Simulation, Optimization, Rulebased Simulation controller sequence (see [“Standard Controller Sequence”](#) on page 31). The Restore Point only affects the run with the Optimization controller.

Optimization Restore Points can also be used in combination with Multiple Run Management when the various runs in the concurrent multiple runs differ only in lower priorities. See [“About Multiple Runs” in *Solution Approaches*](#).

Note: An Optimization Restore Point would not be used with MRM when the differences across the multiple runs are different input traces, such as different inflow time series for each run. The use of a Restore Point would only be appropriate when the differences across the multiple runs affect only

lower priorities in the optimization. See [“Restrictions on Restore Point Use”](#) on page 48 for more information on the limitations when using a Restore Point.

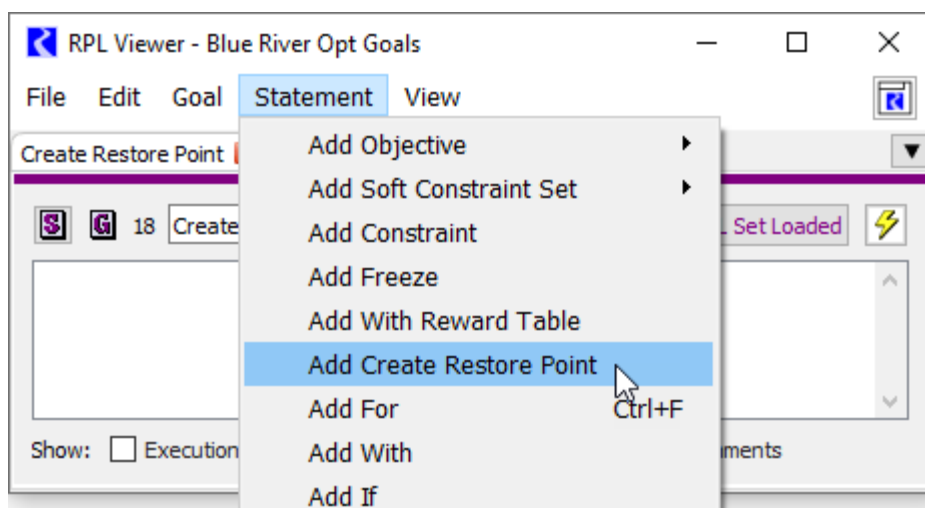
The Optimization Restore Point preserves all information about the optimization problem at the time the Restore Point is created, including solution analysis information. A run that starts by using a Restore Point will report all information in the Priority-oriented Optimization Solution Analysis Tool (POSAT) as for a normal optimization run that solves all priorities. (See [“Priority-oriented Optimization Solution Analysis Tool”](#) on page 285.)

Create a Restore Point

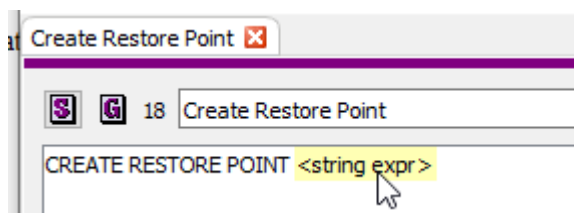
An Optimization Restore Point is created via a Create Restore Point statement in an optimization goal.

To create a Restore Point

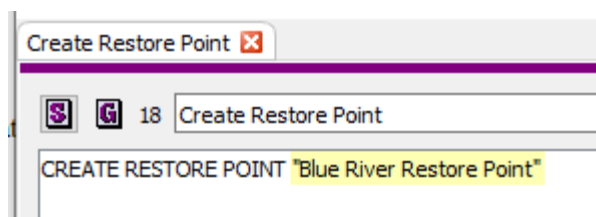
1. Add a goal to the goal set at the desired priority for the Restore Point.
2. Open the new goal, and from the **Statement** menu select **Add Create Restore Point**.



3. Double-click the **<string expr>** to edit it.



4. Type a name for the Restore Point, then Enter.



Chapter 3

Combining Optimization With Simulation

5. Run the optimization. When the goal with the Create Restore Point statement executes, it will save a representation of the optimization problem at the end of the previous goal internally in memory with the name specified. Otherwise the optimization will execute as normal.

Tip: Like all RPL statements, a Create Restore Point statement can be included within other statements. For example, you could add the Create Restore Point statement inside an If statement if you only want it to execute if certain conditions are true. An Execution Constraint can also be used in the goal with the Create Restore Point statement to control its execution (see [“Execution Constraint / Execute Block Only When”](#) in *RiverWare Policy Language (RPL)*).

Note: When a Restore Point is created from a RPL statement, it automatically overwrites any existing Restore Point with the same name.

Note: A Create Restore Point statement should not be included in a goal with a Soft Constraint set or Minimize or Maximize Objective. The Create Restore Point statement should be in its own goal.

Once a Restore Point has been created, the details can be viewed in the [Optimization Restore Point Manager](#).

Restore Points are not saved in the model file. When a model is loaded, there will be no Restore Points present. To save a Restore Point for use after reloading the model or in a different RiverWare session, see [“Export a Restore Point”](#) on page 44 and [“Import a Restore Point”](#) on page 45.

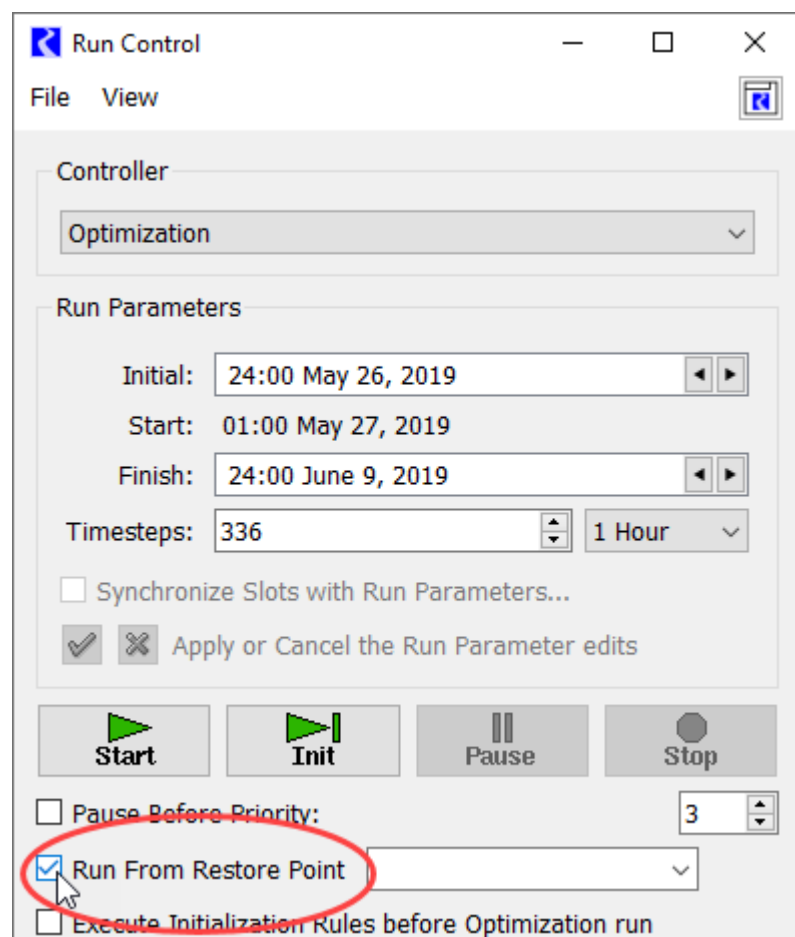
Use a Restore Point

Use of a Restore Point as an advanced start for an optimization run can be specified either from the Run Control dialog or from the Set Controller script action.

Specify a Restore Point from the Run Control dialog

The use of a Restore Point can be specified interactively from the Run Control dialog.

1. When the Optimization controller is selected, select the box for **Run From Restore Point**.



2. The menu at the right will list all Restore Points currently available in the [Optimization Restore Point Manager](#). Select the desired Restore Point.



Alternatively, type the name of the desired Restore Point. This approach can be used to specify a Restore Point that does not yet exist in the Restore Point Manager. Specification of the Restore Point to use is saved in the model file.

Note: Although the Restore Point does not have to be present in the [Optimization Restore Point Manager](#) in order to specify it in the Run Control dialog, the specified Restore Point must be present in the Restore Point Manager when the Optimization controller is started. See [“Import a Restore](#)

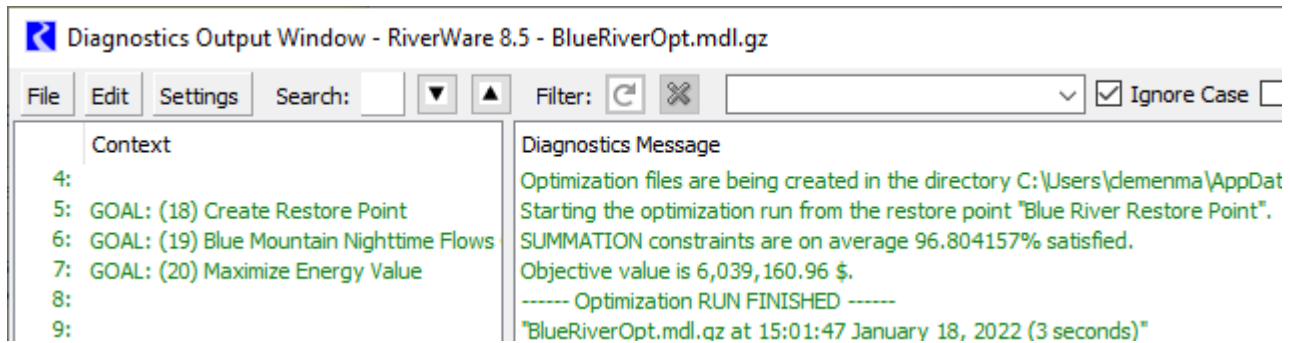
Chapter 3

Combining Optimization With Simulation

[Point](#)” on page 45 for instructions to import a Restore Point previously saved to a file. The import of the Restore Point must occur prior to starting the Optimization controller.

3. Start the Optimization run sequence.

When the Optimization controller starts, it will skip all goals prior to the goal with the specified Restore Point. At the Restore Point goal, it will re-instate the optimization problem using the Restore Point then solve all remaining goals as normal. The diagnostic output will include a message indicating that the run started with the Restore Point. In the image below, the diagnostic message indicates, that goals 1-17 were skipped, and the run started by re-instating the Blue River Restore Point at priority 18. It then solved priorities 19 and 20 as normal.

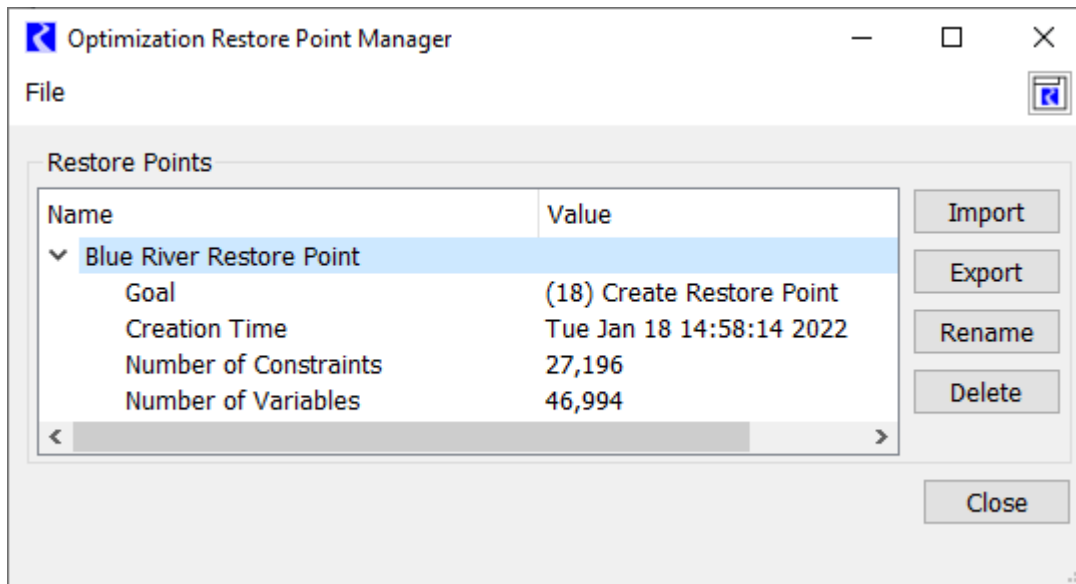


Specify a Restore Point from a script action

The use of a Restore Point can also be specified from the Set Controller script action so that it can be automated. See [“Set Controller” in Automation Tools](#) for details of how to configure the script action. The script action changes the state of the controller. Then the optimization run can be started either manually or via the Execute Run script action. Once the run is started, the behavior is the same as when the Restore Point is specified via the Run Control dialog.

Optimization Restore Point Manager

The Optimization Restore Point Manager provides information about all Restore Points currently available.



The Optimization Restore Point Manager can be accessed by selecting **Optimization Restore Point Manager...** from one of the following three menus.

- The **Workspace** menu on the main workspace
- The **Set** menu from the Optimization Goal Set Editor
- The **View** menu on the Run Control dialog box

Restore Points are not saved in the model file. When model is loaded, the Restore Point Manager will be empty until either a Restore Point is created in an optimization run (see [“Create a Restore Point”](#) on page 39) or a Restore Point is imported (see [“Import a Restore Point”](#) on page 45).

For each Restore Point, the Restore Point Manager provides the following pieces of information.

- Name of the Restore Point
- The priority and name of the goal that created the Restore Point
- The time at which the Restore Point was created
- The number of constraints in the optimization problem represented by the Restore Point
- The number of variables in the optimization problem represented by the Restore Point

The Restore Point Manager supports the following actions for Restore Points

- **Import** a Restore Point previously saved to a file (see [“Import a Restore Point”](#) on page 45)
- **Export** a selected Restore Point to a file (see [“Export a Restore Point”](#) on page 44)
- **Rename** a selected Restore Point
- **Delete** a selected Restore Point or delete all Restore Points (see [“Delete Restore Points”](#) on page 47)

These actions can be selected from the **File** menu of the Optimization Restore Point Manager or from buttons on the right side of the Optimization Restore Point Manager dialog box.

Export a Restore Point

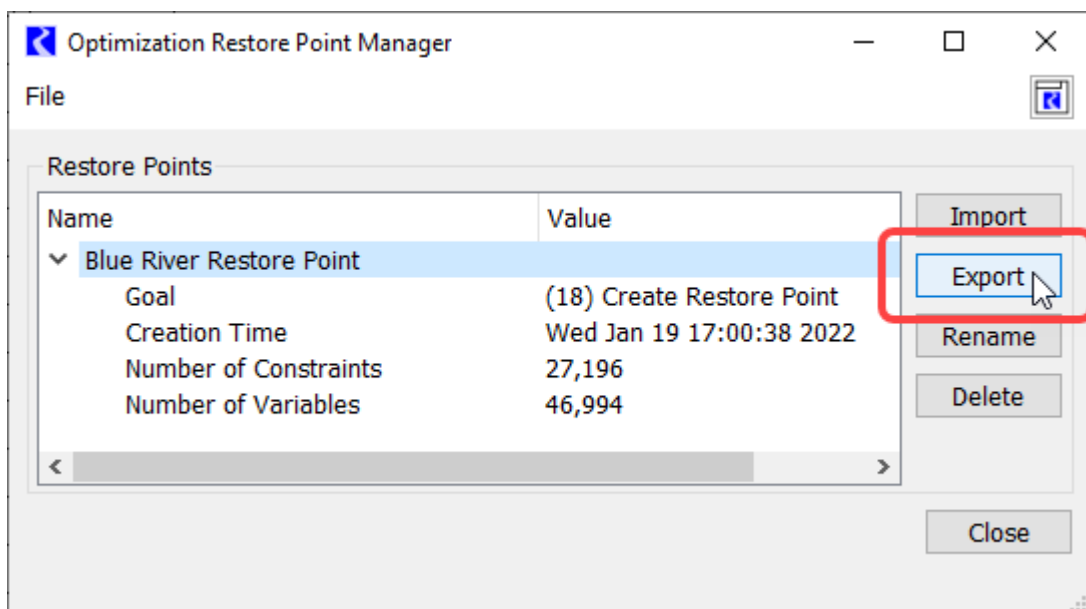
Optimization Restore Points are not saved in the model file. To preserve a Restore Point for use when the model is reloaded or in another RiverWare session, it must be exported to a file and then imported into the model in which it will be used. (For instructions to import a Restore Point, see [“Import a Restore Point”](#) on page 45.) Restore Points can be exported to a file in three ways.

- Export a Restore Point via the Restore Point Manager
- Export a Restore Point with a script action
- Export a Restore Point with a RCL batch script command

Export from the Restore Point Manager

To export a Restore Point via the [Optimization Restore Point Manager](#), use the following steps.

1. Open the Optimization Restore Point Manager from one of the following locations.
 - The **Workspace** menu on the main workspace
 - The **Set** menu on the Optimization Goal Set Editor
 - The **View** menu on the Run Control dialog box
2. Select the name of the desired Restore Point. (If the desired Restore Point has not yet been created and thus is not shown in the Restore Point Manager, see [“Create a Restore Point”](#) on page 39.)
3. Select the **Export** button on the right side of the Restore Point Manager (or select **Export Selected Restore Point...** from the **File** menu).



4. In the resulting file chooser, navigate to the desired folder and type the desired file name. Then select **Save**.

Optimization Restore Point files are saved with the .orp extension by default.

Note: The exporting of a Restore Point to a file can take a noticeable amount of time, particularly for large optimization models with numerous timesteps and a large number of optimization goals, in other words, when the optimization problem has a large number of constraints and variables. However, the time to export and import a Restore Point is generally much less than the time required to repeat the portion of the optimization run represented by the Restore Point.

Automate Restore Point export via script

A Restore Point can be exported to a file by the [Export Optimization Restore Point](#) script action. See “[Export Optimization Restore Point](#)” in *Automation Tools* for details.

When running RiverWare in batch mode, a Restore Point can be exported by using the [ExportOptRestorePoint](#) command in the RCL script. See “[ExportOptRestorePoint](#)” in *Automation Tools* for details.

Import a Restore Point

Optimization Restore Points are not saved in the model file. To use a Restore Point that was created before the model is loaded, it must be exported to a file. (For instructions to export a Restore Point, see “[Export a Restore Point](#)” on page 44.) The Restore Point can then be imported for use when the model is reloaded or in another RiverWare session. Restore Points can be imported from a file in three ways.

- Import a Restore Point via the Restore Point Manager
- Import a Restore Point with a script action
- Import a Restore Point with a RCL batch script command

Note: In order to use a Restore Point in an optimization run, the Restore Point must be imported before starting the Optimization controller.

Import from the Restore Point Manager

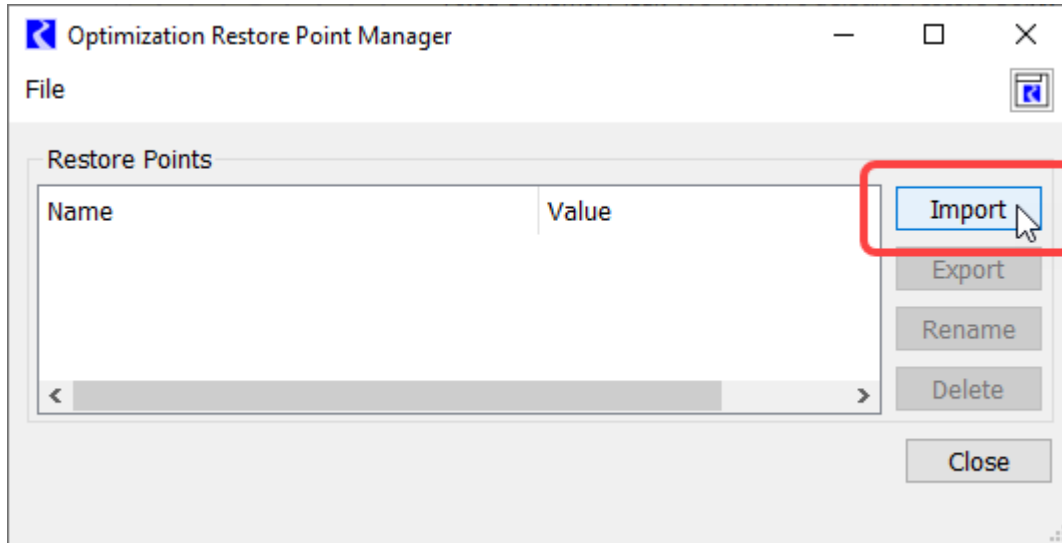
To use the [Optimization Restore Point Manager](#) to import a Restore Point that was previously saved to a file, use the following steps.

1. Open the Optimization Restore Point Manager from one of the following locations.
 - The **Workspace** menu on the main workspace
 - The **Set** menu on the Optimization Goal Set Editor
 - The **View** menu on the Run Control dialog box

Chapter 3

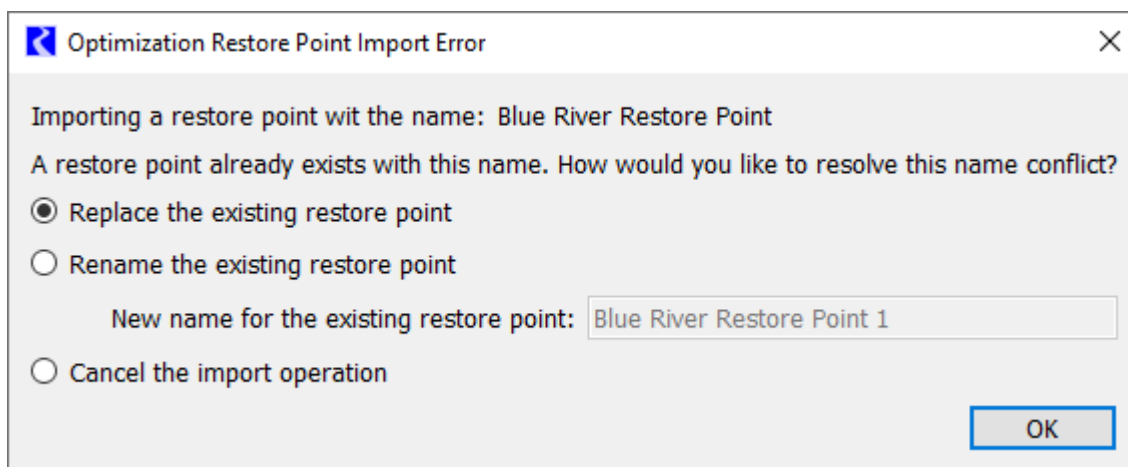
Combining Optimization With Simulation

2. Select the **Import** button on the right side of the Restore Point Manager (or select **Import Restore Point...** from the **File** menu).



3. In the resulting file chooser, navigate to the desired Restore Point file, and select it. Then select **Open**.
4. If there is already an existing Restore Point with the same name as the Restore Point being imported from a file, you will be presented with three options.
 - Replace the existing Restore Point
 - Rename the existing Restore Point so that it can be preserved - RiverWare will give the existing Restore Point a new, default name; the Restore Point being imported will maintain its name from the file.
 - Cancel the import operation

Select the appropriate option.



At this point, the Restore Point is ready to use in an Optimization run (see [“Use a Restore Point”](#) on page 40).

Note: The importing of a Restore Point from a file can take a noticeable amount of time, particularly for large optimization models with numerous timesteps and a large number of optimization goals, in

other words, when the optimization problem represented by the Restore Point has a large number of constraints and variables. However, the time to export and import a Restore Point is generally much less than the time required to repeat the portion of the optimization run represented by the Restore Point.

Automate Restore Point import via script

A Restore Point can be imported from a file by the [Import Optimization Restore Point](#) script action. See “[Import Optimization Restore Point](#)” in *Automation Tools* for details. The script action provides options to rename the Restore Point on import and options for handling name conflicts similar to when importing via the Restore Point Manager.

When running RiverWare in batch mode, a Restore Point can be imported by using the [ImportOptRestorePoint](#) command in the RCL script. See “[ImportOptRestorePoint](#)” in *Automation Tools* for details.

Delete Restore Points

Restore Points currently stored in the [Optimization Restore Point Manager](#) can be deleted in three manners

- Delete Restore Points via the Restore Point Manager

In the [Optimization Restore Point Manager](#) select the desired Restore Point. Then select the **Delete** button on the right of the Restore Point Manager. Alternatively select **Delete Selected Restore Point** from the **File** menu, or select the option to **Delete All Restore Points** from the **File** menu.

- Use a script action to delete all Restore Points

The Delete Optimization Restore Points script action can be used to delete all Restore Points. This script action deletes all Restore Points; it does not support deleting individual Restore Points. See “[Delete Optimization Restore Points](#)” in *Automation Tools* for details.

- Use a RCL command to delete Restore Points in a batch run.

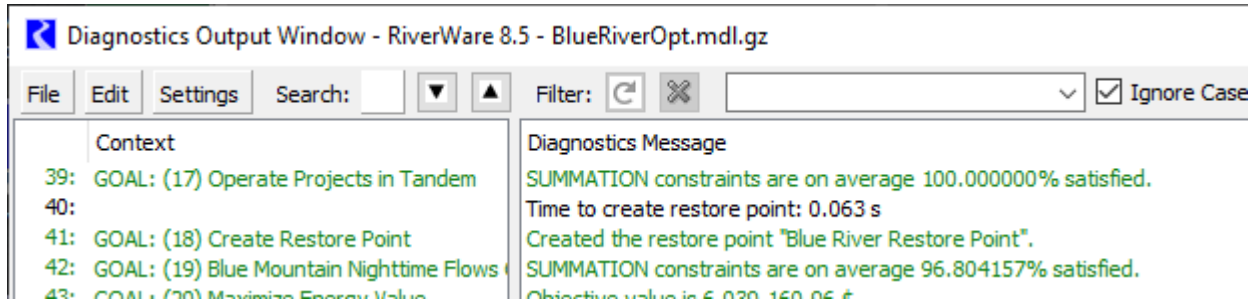
When running RiverWare in batch mode, the [DeleteAllOptRestorePoints](#) command can be added to the RiverWare Command Language (RCL) script to delete all Restore Points. See “[DeleteAllOptRestorePoints](#)” in *Automation Tools* for details.

Restore Points are not saved in the model file, so closing RiverWare, loading a new model or reloading the same model will also delete all Restore Points.

Restore Point Informational Diagnostics

Informational diagnostics related to Optimization Restore Points can be issued from the Optimization diagnostics and Workspace diagnostics. (See “[Diagnostics Manager](#)” in *Debugging and Analysis* for instructions to access the Optimization and Workspace Diagnostic Configuration.) In either Diagnostic Configuration dialog, select the **Opt. Restore Point Management** diagnostic category.

Optimization diagnostics for Restore Points are issued during a run with the Optimization controller that creates a Restore Point and include information about the time to create the Restore Point.



Workspace diagnostics for Restore Points are issued during Restore Point import and export and include the file imported/exported and information on the timing of the steps in the import/export.

Restrictions on Restore Point Use

An Optimization Restore Point preserves the state of the complete optimization problem at the time the Restore Point is created. As such, when the Restore Point is used as an advanced start for an optimization run, it assumes that nothing has changed in the model or optimization goal set that would change the optimization for any priorities prior to the Restore Point. This means that the following conditions must be true.

- The model configuration when using the Restore Point in a run must not have changed from the run in which the Restore Point was created. This includes objects, method selections, links, and slot configurations, including lower and upper bounds on slots that are optimization variables.
- The Run Range and timestep size must be the same as the run in which the Restore Point was created.
- The optimization goal set for all priorities prior to and including the priority with the Create Restore Point statement must be identical to the goal set used for the run that created the restore point. This includes goal names, RPL logic, and goal activation. (Goals at priorities after the goal that created the restore point can differ when using the Restore Point.)
- All input slot data that impacts priorities prior to the creation of the Restore Point must be identical to the run in which the Restore Point was created. This includes input data on series slots used in physical process methods, static parameter data on table, scalar, and periodic slots used in physical process methods, and all slot values referenced by optimization goal RPL logic. (Slot values that are referenced only by RPL logic in goals after the Restore Point can differ from the values in the run in which the Restore Point was created and are often the components of the model that change for alternative runs that start with the Restore Point.)

If any of these conditions is not true, the run using the Restore Point could result in an infeasible solution, could fail with other errors, or could produce unintended results.

In addition, a Restore Point can be used only when running with a RiverWare version that is in the same minor release series as the RiverWare version that created the restore point. For example, a Restore Point created with RiverWare 8.5 can be used with the RiverWare 8.5.2 Patch Release, but attempting to use it with RiverWare 8.6.2 or RiverWare 9.0 would result in an error.

Limitations of Optimization

The Optimization solution is based on a linear programming solution, and as such, has some inherent limitations as described below.

Optimization License Required

To use the RiverWare Optimization solution, you must have the RiverWare With Optimizer version of the RiverWare license. See riverware.org for licensing information.

The license allows to you access the RiverWare Optimization module, which embeds the third-party solver IBM ILOG CPLEX. Additional information about CPLEX can be found in the About RiverWare dialog—from the main RiverWare workspace, select **Help**, then **About RiverWare**.

Linear Relationships

All relationships must be linear or piecewise linear. This means that all means that all relationships that are non-linear must be converted to linear or piecewise linear approximations. RiverWare carries out this linear approximation automatically based on approximation points specified by the user. See “[Linearization, Approximation, and Replacement](#)” on page 79 for details.

Some common non-linear physical processes that must be linearized include the following:

- Power: A function of Operating Head and Turbine Release
- Turbine Capacity: A function of Operating Head
- Tailwater: A function of Outflow and (optionally) Tailwater Base Value (often downstream Pool Elevation)
- Operating Head: A function of Pool Elevation and Tailwater
- Unregulated Spill: A function of Pool Elevation

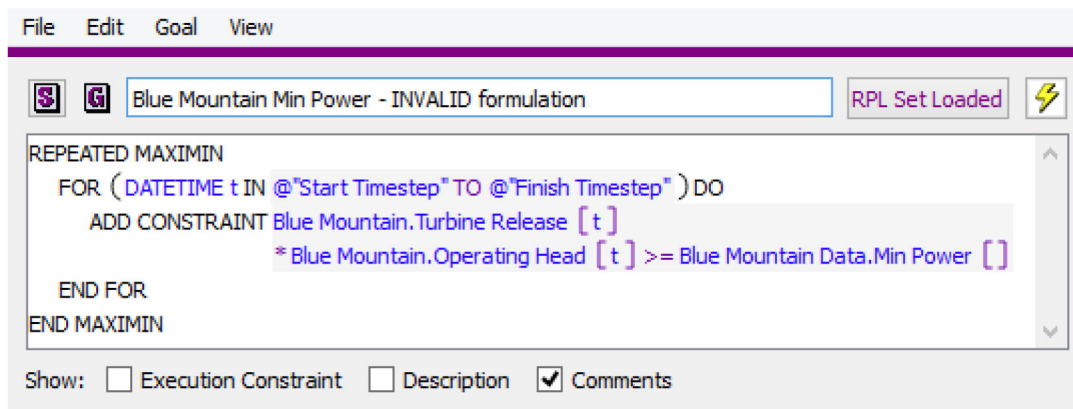
Linear Constraint Expressions

A corollary to the requirement that all relationship must be linear is that constraint statements can only contain linear expressions. This means that a constraint cannot contain a non-linear combination of variables. Also, to prevent the potential formulation of nonlinear expressions of variables in a constraint, RiverWare does not allow the use of IF/THEN expressions within a constraint statement (even if the IF/THEN expression does not contain a reference to a variable), nor the use of any RPL predefined functions within a constraint statement. Shown below are examples of constraint formulations that would not be valid.

Chapter 3

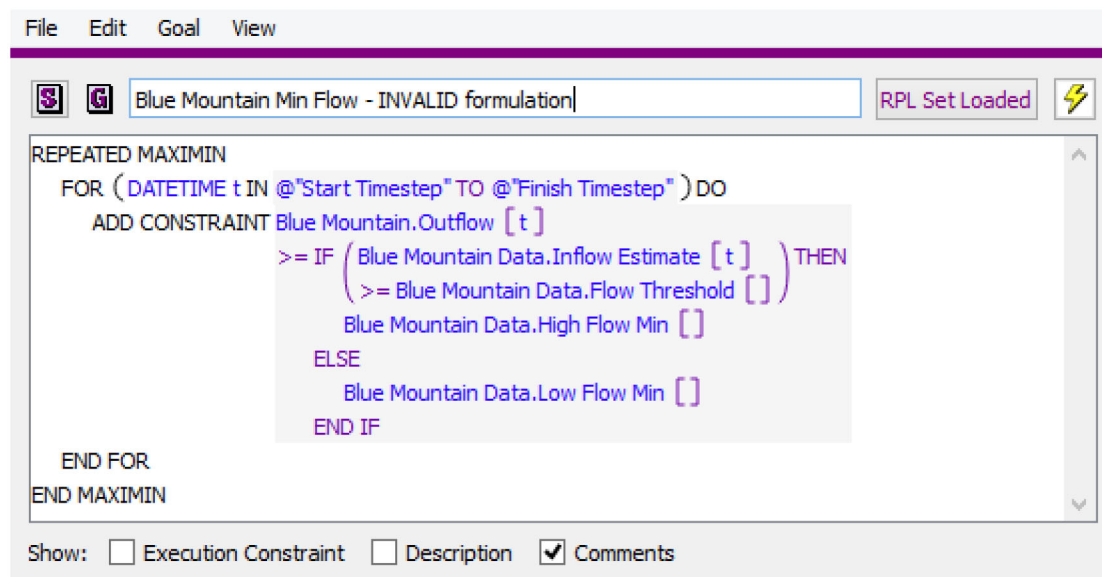
Combining Optimization With Simulation

Figure 3.6



The constraint shown in [Figure 3.6](#) contains a nonlinear combination of two variables, Turbine Release and Operating Head, and would cause the Optimization run to terminate.

Figure 3.7



The constraint shown in [Figure 3.7](#) contains an IF/THEN expression within the constraint statement and would cause the run to terminate. This type of logic can be achieved by putting the constraint statement within a WITH statement that carries out the IF/THEN logic. [Figure 3.8](#) is an example of a valid goal.

Figure 3.8

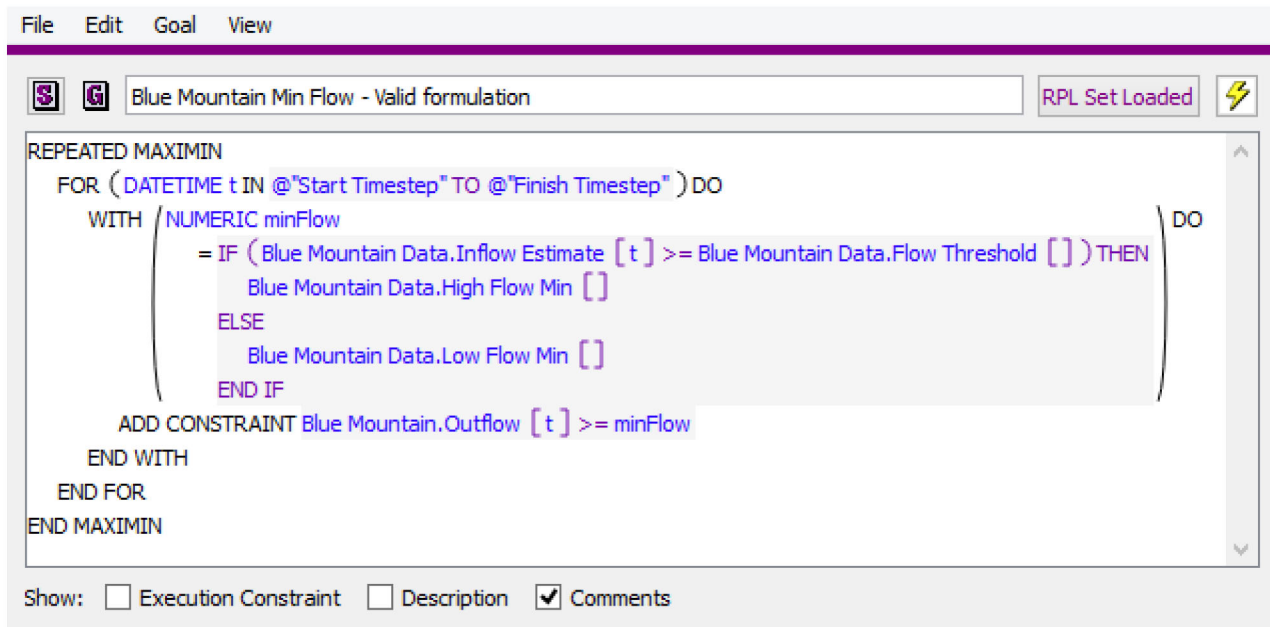


Figure 3.9 is an example of a valid use of IF/THEN logic in an Optimization goal outside of a constraint statement.

Note: In this type of use, the expression in the WITH statement cannot contain a reference to an Optimization variable. Otherwise the expression would return an invalid value, which would cause the goal to terminate without writing any constraints.

Figure 3.9

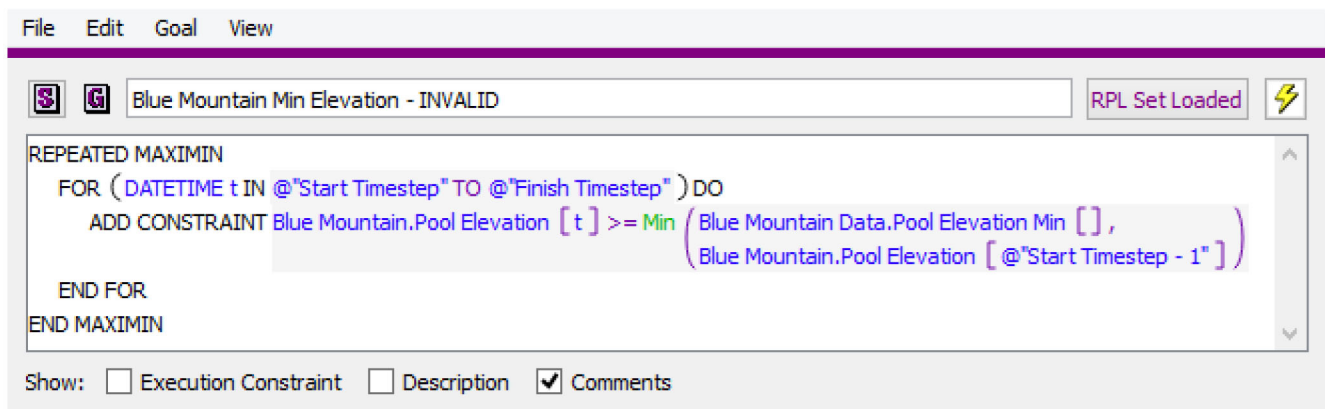


Figure 3.9 contains a RPL predefined function, **Min**, within the constraint statement and would cause the run to terminate. User-defined functions can be used within constraint statements as long as the functions do not contain nonlinear functions of variables.

Approximation Error

A side effect of the requirement for linear relationships is approximation error. All variables that are linearized will have approximation error in the Optimization solution. Typically the largest and most significant approximation error shows up in the linearization of Power. In the Post-optimization Rulebased Simulation, the rules typically set flow values based on the values for those flow variables from the Optimization solution. Then the objects dispatch and calculate Power and other quantities based on the nonlinear Simulation methods. Therefore the calculation of Power in the Post-optimization RBS, once the approximation error from linearization is removed, will differ from the Power value in the Optimization solution. The Power approximation error tends to be most noticeable when the Optimization policy contains constraints that generation must equal some value (load). In these cases, the Optimization solution will often say that the constraint is 100% satisfied, but the Power calculated in the Post-optimization RBS will not match the constraint value exactly.

Solution Time

The computational requirements of the preemptive linear goal programming solution limit the size of the Optimization problem that can be solved in a *reasonable* amount of time. For large models (many objects and/or many goals) keeping the run length reasonable typically means limiting the number of timesteps. The definition of *reasonable* solution time is dependent on the uses of the model, and the user will need to determine what is an acceptable solution time. When determining the upper limit on the number of timesteps, one important point to consider is that the Optimization solution time for a given model does not increase linearly with the number of timesteps, as it generally does for RBS. The relationship between solution time and number of timesteps is typically somewhere between quadratic and cubic. The total solution time is a combination of a number factors, including but not necessarily limited to the following:

- Number of objects (roughly correlates to the number of variables)
- Number of timesteps
- Number of constraints
- Number of goals
- Difficulty of the solution (Objectives that force a lot constraints to be frozen or that have soft constraint violations can take longer to solve.)
- Hardware: Number of available processors, processor speed, memory

Perfect Foresight

The ability of the Optimization solution to *look ahead* in both time and space (global solution) is one of the benefits of Optimization. Optimization can be used to make sure you do not compromise your ability to satisfy constraints in the future when optimizing for today. However, if not managed appropriately, perfect foresight can be a potential weakness. Because the solution does not inherently put a higher value on one time period over another, in some cases it can exploit the solution to maximize an objective farther into the future, which has a result of degrading the objective in the near term. A common example would be if the value of Energy is somewhat higher later in the run. The solution might save as much water as possible until this higher value period and then generate at max capacity over the high value period. This might be considered a *good* solution, but in some cases there is more uncertainty about input data farther out in the forecast horizon, and it is not desirable for the solution to *over-optimize* these

future periods at the expense of near term operations. This potential consequence of Optimization's *perfect foresight* is important to keep in mind when considering the appropriate run horizon for an Optimization model.

Limited Objects and Method Selections

Not all object types are supported by Optimization. Additionally, only a subset of Simulation user methods can be used in conjunction with Optimization. See [“Optimization Objects and Methods”](#) on page 87 for a complete listing of the available objects and methods. When creating a model that will be used for Optimization, it is important to make Simulation method selections that are compatible with Optimization.

Debugging

The Optimization solution is generally more difficult to debug than Rulebased Simulation. Because it is a global solution, there is typically not a single cause for a bad output or *odd* result. It can be the combination of multiple constraints and input data at multiple locations and multiple timesteps. Often the solution can be exploited to improve some objective function if it is not properly constrained, resulting in unrealistic or undesirable operations. In these cases it can be difficult to determine what exactly is contributing to the undesirable result. See [“Debugging and Solution Analysis”](#) on page 285 for descriptions of tools that can assist in debugging the Optimization solution.

Chapter 3

Combining Optimization With Simulation

Chapter 4

Variables, Constraints, and Objectives

Topics

- [RPL Optimization Goal Set, page 55](#)
- [Optimization Variables, page 72](#)
- [Linearization, Approximation, and Replacement, page 79](#)

RPL Optimization Goal Set

The operating policy of a basin is expressed in a RPL Optimization Goal Set. This section describes the components of the Optimization Goal Set. An Optimization Goal Set uses much of the same syntax as a Rulebased Simulation ruleset. See “[Managing RPL Sets](#)” in *RiverWare Policy Language (RPL)* for details.

Key differences between the formulation of rules and Optimization goals are described in the following sections.

Prioritized Policy

In Optimization, each aspect of the basin operating policy is expressed in a goal, and each goal is given a unique priority. In this way, goals for Optimization are analogous to rules for Rulebased Simulation. The contents of each goal are either constraints or objectives. See “[Goal Types](#)” on page 58 and “[Statements in Goal Sets](#)” on page 60 for details about the contents of goals. This section focuses on the top level structure of the goal set.

RBS rules execute from lowest priority to highest priority. Optimization goals on the other hand, execute from highest priority to lowest priority (lowest number to highest number). Each priority that contains an objective, either a Maximize or Minimize objective or a Soft Constraint Set derived objective, will trigger a solution of the Optimization problem. The problem at each priority will include the constraints from all higher priority goals, along with their maximized satisfaction, and any new constraints added at the current priority. Thus at each priority, degrees of freedom are removed from the problem reducing the solution space. If the goal contains only hard constraints, or if the logic in the goal results in no new constraints getting added to the Optimization problem, then there will not be a new solution of the Optimization problem at that priority.

RBS rules are normally ordered to execute in an upstream to downstream order, object-by-object. Generally there is no need to order Optimization goals in an upstream to downstream order. The priorities of the goals should correspond to their *true* priorities in operations. The highest priority goals should correspond to the most important operating constraints, those that should essentially never be violated.

Chapter 4

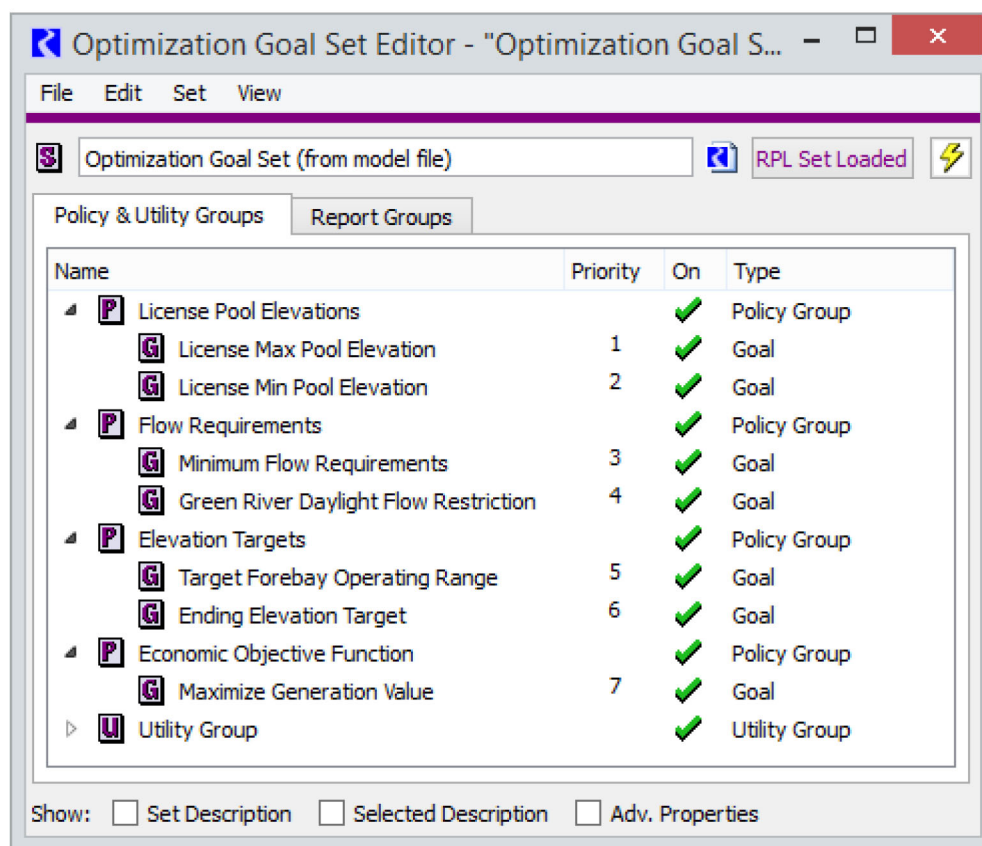
Variables, Constraints, and Objectives

A single goal can contain constraints for multiple objects, but it is generally recommended that a single goal correspond to a single operating policy. For example, the highest priority goal might contain the license maximum Pool Elevation constraints for all reservoirs at all timesteps. The second goal might contain the license minimum Pool Elevation constraints for all reservoirs at all timesteps. This is generally better practice than placing the minimum and maximum elevation constraints in the same goal or placing the minimum elevation constraint in the same goal (same priority) as a minimum flow constraint. When multiple policies are combined in a single goal, it is not always obvious how the policies will trade off with one another if it is not possible to satisfy all constraints at that priority.

Medium priority goals generally contain constraints that you expect to get satisfied most of the time but that may not be possible to satisfy under some operating conditions, for example under extreme high or low flows. Lower priority goals typically contain target operating constraints, targets that you would like to meet under ideal conditions but are expected to be frequently violated due to higher priority constraints and input data (e.g. inflows). For example, you might have a low priority goal to keep flows equal across all reservoirs on each timestep, but most of the time you cannot keep flows exactly equal due to other requirements on the system. Typically the lowest priority contains an objective function to optimize with the remaining degrees of freedom given all higher priority constraints. For example, the objective might maximize the total value of hydropower generation during the run within the operational limits specified by higher priority constraints.

Figure 4.1 is a simple example of an Optimization Goal Set.

Figure 4.1



In this goal set, the License Max and Min Pool Elevations are at the highest priorities. You would therefore expect that these elevation limits are almost never violated. The Minimum Flow Requirement is at the next priority, so you would also expect it to be satisfied most of the time. However, if there were a case with low flows and it was not possible to meet both the minimum elevation requirement and the minimum flow requirement, then the solution would meet the minimum elevation and would violate the minimum flow requirement. It would get the flow as close to the minimum as possible, and in doing so, it would take the reservoirs to the minimum elevation.

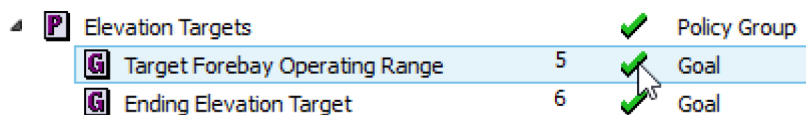
The Target Forebay Operating Range goal is at a lower priority. This suggests that ideally you would like to keep the forebays (Pool Elevation) within some specified range, but you are not going to violate either of the flow requirements in goals 3 and 4 to do so.

The final goal is maximizing the value of generation. Normally an objective such as this would use as much water as possible to produce as much generation as possible. The Ending Elevation Target goal at priority 6 is likely in place to prevent the maximum value goal from draining all reservoirs to the bottom of their operating range. The solution will only maximize the economic value of generation within the limits specified by the higher priority goals. It can only use the degrees of freedom that remain after solving to meet all of the higher priority constraints. The solution would go outside of the target forebay operating range (priority 5) if necessary to meet a flow requirement (priority 3 or 4), but it would not go outside of the target operating range just to increase the generation value (priority 7).

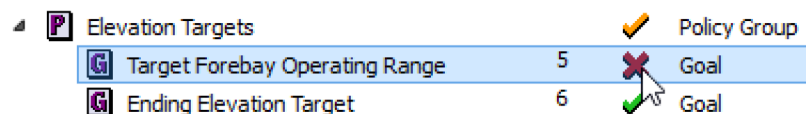
Note: In practice, most Optimization Goal Sets contain many more goals than the example shown here in order to model all of the operational requirements the system.

Note: All the goals correspond to operational policies. There are no goals added to express the physical constraints that define the system (e.g. mass balance and routing). All physical constraints get included in the Optimization problem automatically by RiverWare (see “Physical Constraints” on page 74). There is no need for the user to formulate the physical constraints.

Like rulesets, Optimization Goal Sets can be organized into policy groups and utility groups, as seen in the example above. The policy groups contain goals and the utility groups contain functions used within the goals. As with rules, the policy groups are for organizational purposes only. The solution evaluates each goal individually in priority order. It does not solve by policy group. Priorities of individual goals and policy groups can be shifted by clicking and dragging the goal/group name to the new priority location. Individual goals and policy groups can be activated and deactivated by clicking on the green check mark or red **X** next to the goal or policy group name.



Clicking on the green check mark will change it to a red **X**.



Clicking on the red **X** will reactivate the goal or policy group and change it back to a green check mark.

Goal Types

Technically there is only a single type of goal that is added to a goal set. However, goals can be thought of as falling in one of three general categories: *hard constraints*, *soft constraints*, and *objectives*. Like rules, Optimization goals consist of statements. The goal *type* is determined by the type of statements that are added to it. See “[Statements in Goal Sets](#)” on page 60 for descriptions of the types of statements that can be added to a goal. The goal types are described in the following sections.

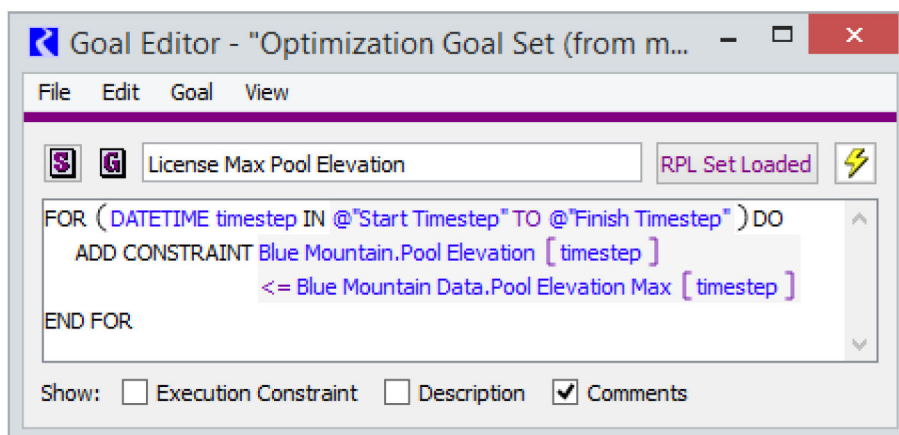
Hard Constraints

A hard constraint goal includes Constraint statements (see “[Constraint](#)” on page 64) that are not within a Soft Constraint Set derived objective (see “[Soft Constraint Set](#)” on page 62). A goal with only hard constraints does not trigger a solution of the Optimization problem. It only adds new constraints to the problem. The constraints will not have an effect on the solution until the next priority that includes either a Soft Constraint Set derived objective or a Maximize or Minimize objective.

Hard constraints should be used with caution. If any hard constraint cannot be fully satisfied at the next priority with a solution, then an infeasible solution (i.e. no solution) will be returned. These infeasibilities can often be difficult to debug. It can be tempting to consider the highest priority operating constraints as *hard* constraints that are never violated, for example, a license Pool Elevation limit. However it is better practice to formulate these highest priority policies as soft constraints at the highest priorities in the goal set. Once the soft constraints are satisfied at a high priority, they will effectively be treated as hard constraints when solving at all remaining priorities. In the rare case that the high priority constraints cannot be fully satisfied, the soft constraint approach will still return a solution rather than an infeasibility. The solution returned will provide the user with more information about where, when and why the violation occurred than if the model simply returns an infeasible solution.

[Figure 4.2](#) is an example of a technically valid hard-constraint formulation.

Figure 4.2



However, it is recommended instead that the contents of goal such as this with operational policy be placed within a Soft Constraint Set derived objective statement.

One common use of hard constraints is to formulate the defining constraints for user-defined variables (see “[User-defined Variables](#)” on page 76). In fact, defining constraints are typically the only recommended use of hard

constraints. Again, these defining, hard constraints should be formulated in a manner that they are guaranteed to always be feasible. They are typically placed at the highest priorities in the goal set.

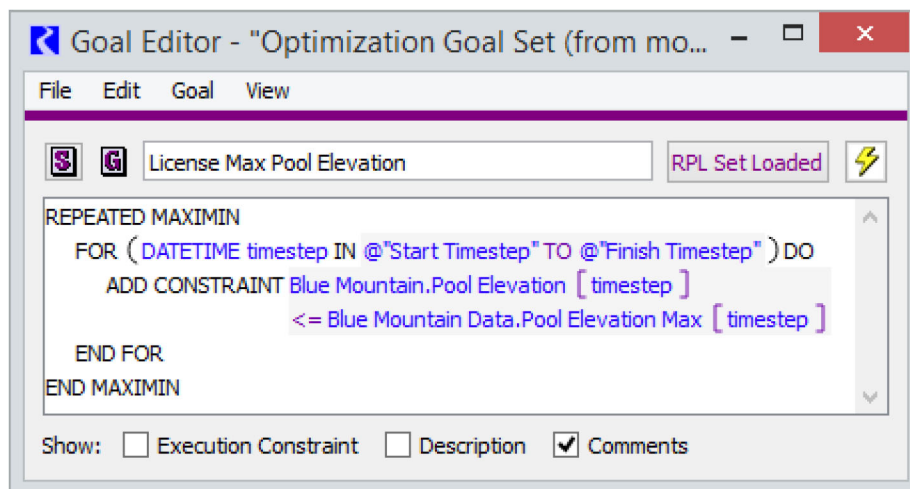
Soft Constraints

The large majority of goals in an Optimization Goal Set are typically soft constraint goals. These goals have a Soft Constraint Set derived objective statement (see [“Soft Constraint Set”](#) on page 62) at the top level within the goal, and within the Soft Constraint Set statement are one or more Constraint statements. The Soft Constraint Set triggers a solution of the optimization problem as long as the goal actually adds new constraints to the problem. (Some goals may contain logic to only add constraints under certain conditions, for example seasonal constraints. If the logic within the goal results in no new constraints being added, then there is no need for a solution after the goal is processed because there is no change to the Optimization problem.) RiverWare automatically converts the goal into a derived objective to maximize the satisfaction of the constraints in the goal given the objective value (satisfaction level) of all higher priority goals. RiverWare provides four different approaches for maximizing the satisfaction of soft constraints (see [“Derived Objectives”](#) on page 19). See [“Soft Constraint Set”](#) on page 62 for details on implementing soft constraints.

Soft constraint goals should be used to formulate all operational constraints (e.g. Pool Elevation limits, minimum Outflow requirements, minimum Power generation requirements, etc.). They are also often used for lower priority *target* operations and *solution quality* constraints, constraints that prevent the Optimization solution from exploiting remaining degrees of freedom to produce operations that would never be carried out in practice. These lower priority target constraints are usually not expected to be satisfied under all conditions.

[Figure 4.3](#) is an example of a maximum Pool Elevation policy formulated as a soft constraint goal using the Repeated Maximin derived objective.

Figure 4.3



Objectives

Objective goals are used to maximize or minimize a specified quantity. Technically, and objective maximizes a linear expression of variables. For example, the objective might maximize the total value of hydropower generation over the entire run period.

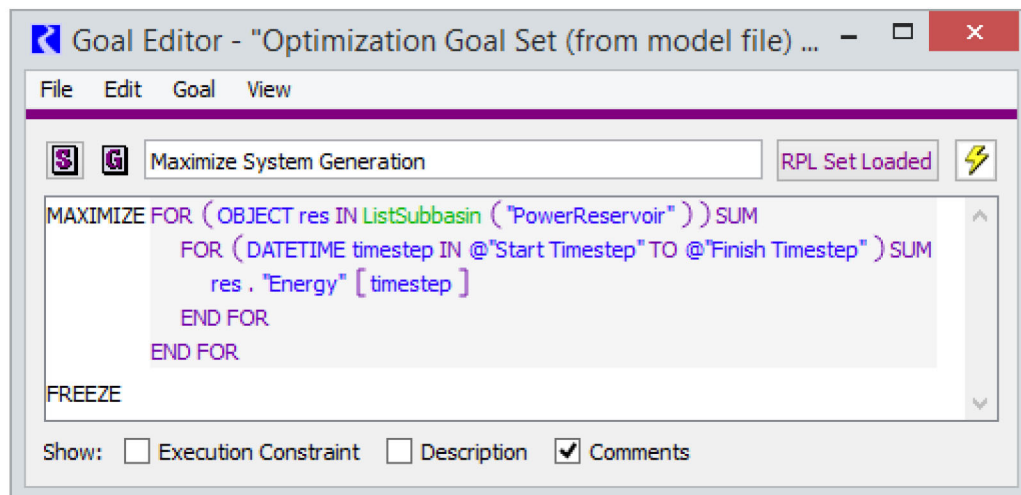
Chapter 4

Variables, Constraints, and Objectives

Typically objectives are at or near the lowest priorities in the goal set. They maximize/minimize the objective given any remaining degrees of freedom after all higher priority constraints. The solution from an objective goal will often use up most remaining degrees of freedom in the Optimization problem. See “[Objective](#)” on page 61 for details on implementing an objective.

[Figure 4.4](#) is an example of an objective to maximize the total system hydropower generation over the entire run.

Figure 4.4

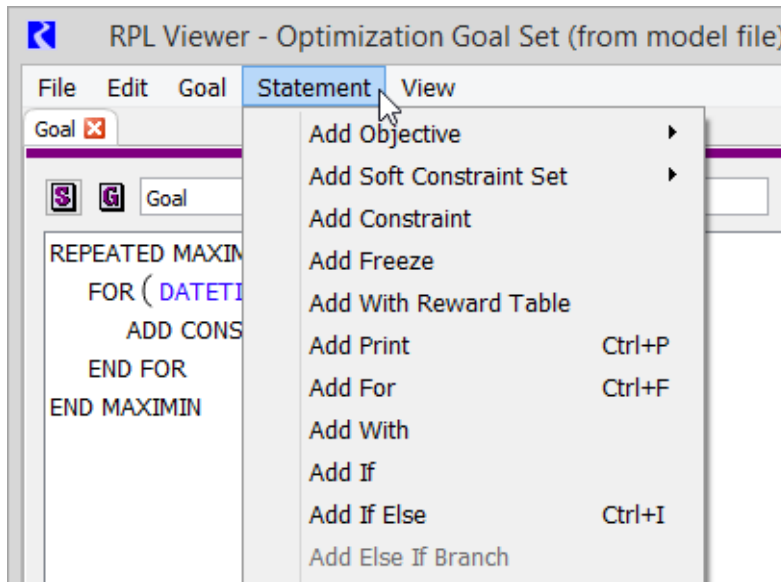


This goal loops over all power reservoirs in the model and over all timesteps and creates an expression that is the sum of the Energy variable (slot) for those reservoirs and timesteps. The objective maximizes that sum.

Statements in Goal Sets

This section describes the pieces of an Optimization Goal. At the top level, an Optimization Goal consists of statements, which are available from the **Statement** menu in the RPL Viewer or Goal Editor dialog (see [Figure 4.5](#)). The list of available statements is slightly different than the list available for rules (see “[RPL Statements](#)” in *RiverWare Policy Language (RPL)*). A description of each is given below.

Figure 4.5



Objective

An Objective statement can be a minimize or maximize objective and has the form:

MAXIMIZE <expr>

or

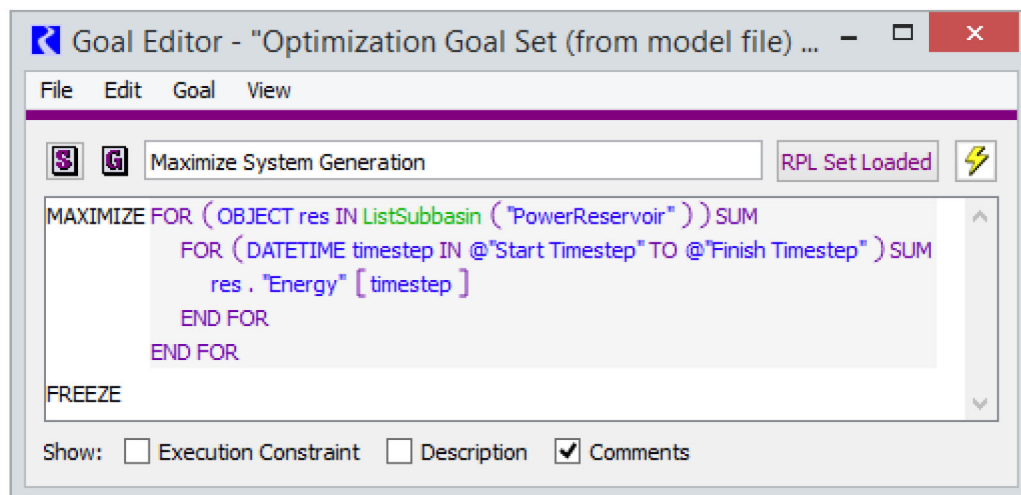
MINIMIZE <expr>

The user then defines the expression to minimize or maximize. The expression within the objective statement may contain multiple terms and summations over multiple slots and/or timesteps as long as it is a linear combination of variables.

Note: A Freeze statement must be added at the end of an Objective statement in order to freeze the objective value. Otherwise the objective value from the goal will not necessarily be maintained at lower priorities.

Figure 4.6 is an example of an objective to maximize the total system hydropower generation over the entire run.

Figure 4.6



This goal loops over all power reservoirs in the model and over all timesteps and creates an expression that is the sum of the Energy variable (slot) for those reservoirs and timesteps. The objective maximizes that sum.

Soft Constraint Set

When a Soft Constraint Set is added, the Optimization solution will attempt to satisfy all of the constraints in the set completely. If it is not possible to fully satisfy all constraints introduced at the current priority due to physical conditions and/or higher priority goals, the solution will attempt to satisfy the constraints as fully as possible based on the one of the three available solution methods (derived objectives) described below. The satisfaction level of the constraint set is frozen once the Optimization problem has been solved at that priority (that is, once a constraint has been satisfied at a high priority, it will not be violated to meet a lower priority goal or objective).

Note: Only one Soft Constraint Set can be added to a goal, but a single Soft Constraint Set can contain multiple individual Constraint statements.

See “[Satisfaction Variables in Soft Constraints](#)” on page 17 for the mathematical details about how constraint satisfaction is defined.

When you add a Soft Constraint Set, you select one of the following solution approaches to maximize satisfaction.

- [Summation](#)
- [Single Maximin](#)
- [Repeated Maximin](#)

Summation

This approach will minimize the total deviation from the constraint values within the Soft Constraint Set (formally it maximizes the total satisfaction).

$$\text{Maximize } \sum_{\text{all } i} S_{p,i}$$

where there is one satisfaction variable $S_{p,i}$ for each constraint i added at the current priority p .

The potential downside of this approach is that it can tend to place all of the deviation on one or just a few variables. For example, if a minimum flow constraint cannot be met for every timestep, it might minimize the total violation by satisfying the constraint at all but one timestep and putting a very large violation on that one timestep. Typically, this is not the preferred solution in a water resources context. The concentration of large violations when using a Summation Soft Constraint Set can be prevented by applying a With Reward Table statement within the Summation. See [“With Reward Table”](#) on page 65 for details about using the With Reward Table statement. See [“Summation”](#) on page 19 for details about the Summation derived objective.

Note: A Freeze statement must be added at the end of a Summation derived objective statement in order to freeze the objective value (satisfaction). Otherwise the satisfaction of the soft constraints from the Summation goal will not necessarily be maintained at lower priorities.

Single Maximin

Instead of minimizing the total violation, this approach will minimize the single largest violation (formally it maximizes the minimum satisfaction). This prevents a single, very large violation by spreading the violation out over multiple variables or timesteps.

Maximize S_p

where S_p is a single satisfaction variable applied to all constraints added at the current priority p .

The potential downside of this approach is that it does nothing to improve the satisfaction of the remaining constraints that are not contributing to the largest violation. This issue is addressed by the next type of derived objective, Repeated Maximin. See [“Single Maximin”](#) on page 20 for details about the Single Maximin derived objective.

Note: A Freeze statement must be added at the end of a Single Maximin derived objective statement in order to freeze the objective value (satisfaction). Otherwise the satisfaction of the soft constraints from the Single Maximin goal will not necessarily be maintained at lower priorities.

Repeated Maximin

This approach begins with the single maximin. It then freezes the satisfaction of the largest violation and resolves the problem by minimizing the next largest violation. It repeats this process until either all violations have been minimized or all remaining constraints are fully satisfied.

while $S_{p,i-1} < 1$

Maximize $S_{p,i}$

$S_{p,i} \geq S_{p,i-1}$ for $i > 1$

where $S_{p,i}$ is a single satisfaction variable applied to all constraints added at the current priority p that have not yet been frozen prior to the i th Repeated Maximin iteration.

The Repeated Maximin approach tends to spread out deviations as evenly as possible, which is often the desired solution in a water resources context. The potential downside of the Repeated Maximin approach is that in some cases it can require numerous iterations when it is not possible to satisfy all constraints. Depending on the size of the model this can cause a significant increase run time. The Summation approach always requires only a single solution at the given priority, so it tends to be faster, but it does not, in general, provide the same solution quality as Repeated Maximin. In some cases, a reasonable balance between solution quality and run time can be provided by applying a With Reward Table statement within a Summation. See [“With Reward Table”](#)

Chapter 4

Variables, Constraints, and Objectives

on page 65 for details about the Summation With Reward Table. See “[Repeated Maximin](#)” on page 20 for details about the Repeated Maximin derived objective.

Constraint

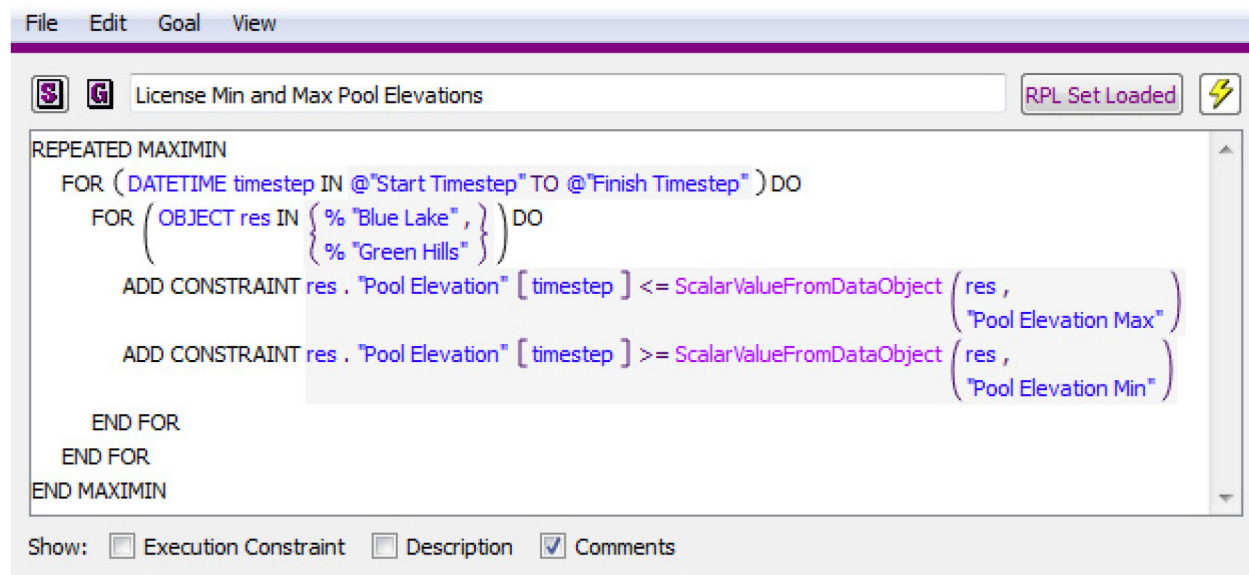
Constraints are formulated as either $\leq$, $\geq$ or $=$ expressions. Constraint statements can be added within a Soft Constraint Set, a For statement, an If statement or a With statement (or within a nested combination of these statement types). If a Constraint statement is added that is not within a Soft Constraint Set, it will be treated as a hard constraint.

Note: It is not necessary to add physical constraints in Optimization Goals. Physical constraints are added automatically by RiverWare as needed based on the model topology and user-selected methods; see “[Physical Constraints](#)” on page 74 for details.

The logical operators are added from the RPL Palette. The user can formulate a constraint with variables and/or scalar values on either side of the logical operator as long as all expressions within the constraint are linear combinations of variables and maintain dimensional consistency. RiverWare will automatically convert all constraints to a canonical form with variables (and coefficients, where applicable) on the left-hand-side and scalar values on the right-hand side when the Optimization problem is processed internally.

[Figure 4.7](#) shows a goal containing basic minimum and maximum Pool Elevation constraints that are being added as Repeated Maximin soft constraints for two reservoirs for all timesteps in the run.

Figure 4.7



The minimum and maximum Pool Elevation values are being retrieved from slots on a data object corresponding to each reservoir by a user-defined function called `ScalarValueFromDataObject`.

Note: If and With expressions cannot be added within a Constraint statement. Also RPL predefined functions cannot be used within Constraint statements; however user-defined functions can be used within a Constraint statement as long as they result in linear constraint expressions. RPL predefined functions can be used in the outer level Optimization Goal statements that contain Constraint

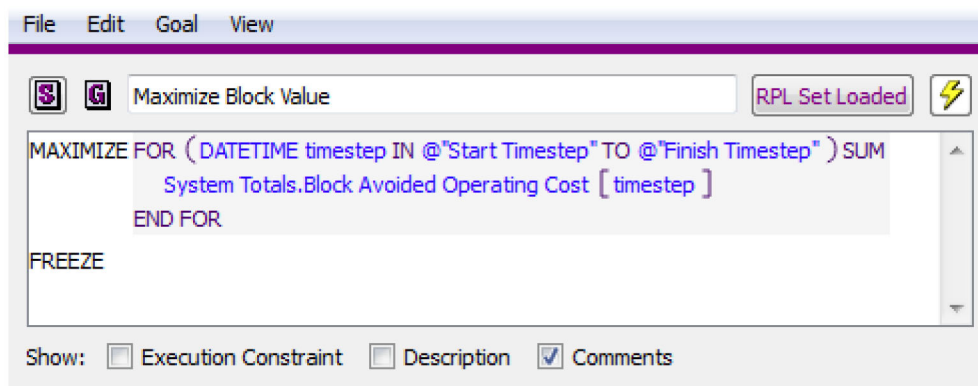
expressions, for example within the boolean expression of an If statement that contains a Constraint statement.

Freeze

A Freeze statement locks in the value of an objective after solving a minimize or maximize objective. A Freeze statement should be added after an objective statement when it is desired to actually preserve the solution resulting from solving the objective function (i.e. for standard optimization objectives). In some cases it might be necessary to maximize or minimize an objective to calculate values for ‘temporary’ use in the solution before solving the ‘true’ objective at a later priority. In this case, it is not desired to freeze value of the objective function for the final solution, and a Freeze statement would not be added.

Figure 4.8 shows a Freeze statement added after a typical objective to maximize the value of hydropower over the entire run. Freeze statements must also be added after Summation and Single Maximin Soft Constraint sets in order to lock in the derived objective value (preserve the constraint satisfaction). It is not necessary to add a Freeze statement after a Repeated Maximin Soft Constraint Set. The satisfaction variables for Repeated Maximin are frozen automatically.

Figure 4.8



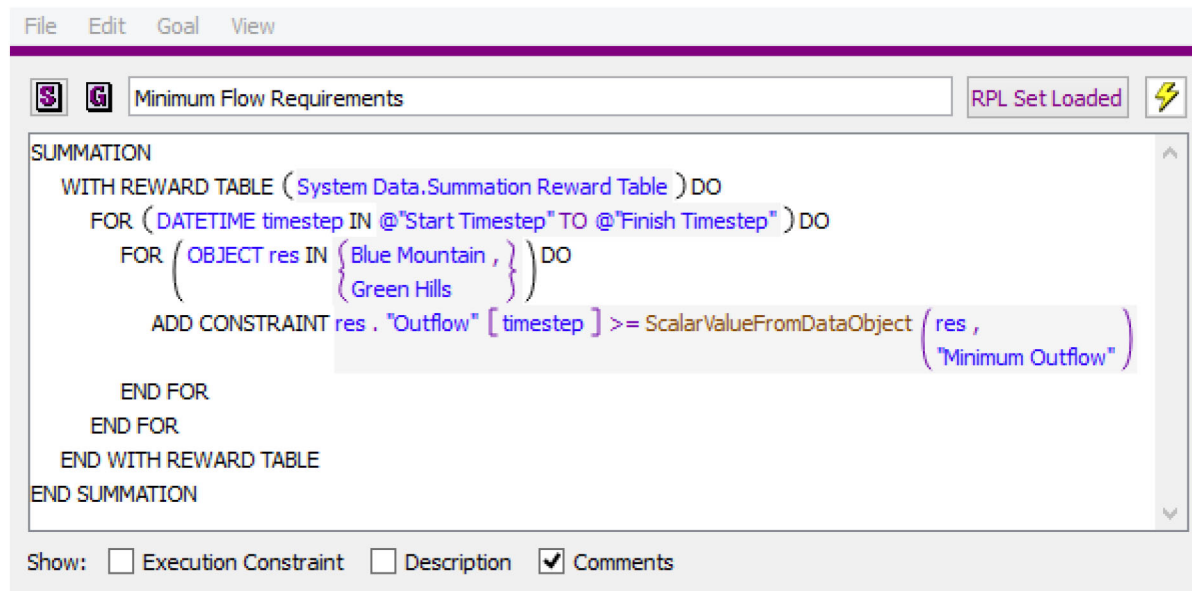
With Reward Table

A With Reward Table statement allows a reward function to be applied to the satisfaction variables within a Summation objective for a soft constraint set. The With Reward Table statement can only be used within a Summation soft constraint set. Using the With Reward Table statement in any other context will result in an error. See Figure 4.9 for an example.

Chapter 4

Variables, Constraints, and Objectives

Figure 4.9



The reward table allows a piecewise function to be applied to the satisfaction variables within Summation objective. Instead of using the standard Summation derived objective for maximizing satisfaction, the derived objective becomes as follows:

$$\text{Maximize } \sum_{\text{all } i} R_i$$

where R is the reward function for the satisfaction variables.

$$R_i = f(S_i)$$

Typically this approach is used to penalize larger violations more than smaller violations. For example, a common approach is to minimize the sum of squared violations. This tends to *smooth out* large violations by spreading them out over multiple time steps rather than concentrating large violations on a few time steps. Applying the appropriate reward function within a Summation soft constraint set can often produce a result that compares to the Repeated Maximin solution in terms of solution quality but has the benefit of only requiring a single solution at the given priority and can therefore be much faster.

The piecewise reward function must be described in a user created table slot. The table slot is given as an argument to the With Reward Table statement.

WITH REWARD TABLE <slot expr>

The reward table slot must have two columns with units of NONE. The first column contains satisfaction values. The second column contains the corresponding reward values. The values in both columns must be between 0 and 1. If the user does not include rows for a satisfaction of 0 and 1, RiverWare will add these rows internally when processing the piecewise reward function with values of 0 and 1 used for the reward. The reward table must be concave. (Reward must be a concave function of Satisfaction.)

An example is provided below for a reward table that corresponds to minimizing the sum of squared violations

Example 4.1 Example Reward Table: Minimize the Sum of Squared Violations

Conceptually the desired derived objective is to minimize the sum of squared violations.

$$\text{Minimize } \sum_{\text{all } i} v_i^2$$

Violations are scaled from 0 to 1, where 1 corresponds to the maximum violation based on a higher priority constraint or bound. Thus an equivalent objective to minimize the sum of squared violations is

$$\text{Maximize } \sum_{\text{all } i} (1 - v_i^2)$$

For performance reasons, the RiverWare Optimization solution maximizes satisfaction variables rather than minimizing violations. See “[Soft Constraint Set](#)” on page 62 for details about how the satisfaction variables are defined.

$$S_i = 1 - v_i$$

The maximize objective then becomes

$$\text{Maximize } \sum_{\text{all } i} (1 - (1 - S_i)^2)$$

[Table 4.1](#) shows how this can be described by a piecewise linear function that linearizes the function in 10 discrete segments.

Table 4.1 Reward Table to Minimize Squared Violations

v_i	v_i^2	s_i	$1 - v_i^2$
1.0	1.00	0.0	0.00
0.9	0.81	0.1	0.19
0.8	0.64	0.2	0.36
0.7	0.49	0.3	0.51
0.6	0.36	0.4	0.64
0.5	0.25	0.5	0.75
0.4	0.16	0.6	0.84
0.3	0.09	0.7	0.91
0.2	0.04	0.8	0.96
0.1	0.01	0.9	0.99
0.0	0.00	1.0	1.00

The table entered in RiverWare is taken from the last two columns. See [Figure 4.10](#) and [Figure 4.11](#) for examples.

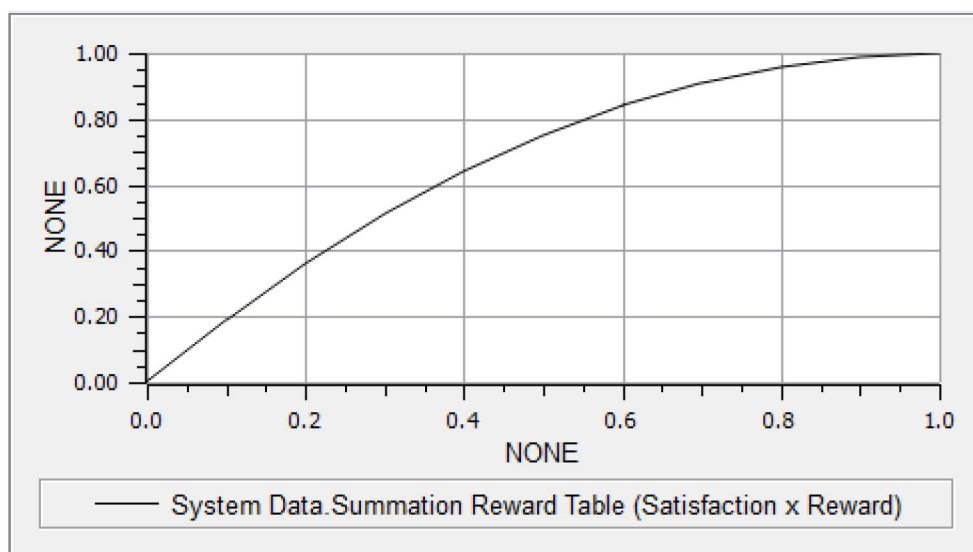
Chapter 4

Variables, Constraints, and Objectives

Figure 4.10

File Edit Row Column View Adjust			
		Summation Reward Table	
		Value: 0	NONE
	Satisfaction NONE	Reward NONE	
0	0.0	0.00	
1	0.1	0.19	
2	0.2	0.36	
3	0.3	0.51	
4	0.4	0.64	
5	0.5	0.75	
6	0.6	0.84	
7	0.7	0.91	
8	0.8	0.96	
9	0.9	0.99	
10	1.0	1.00	

Figure 4.11



More generally, any concave table may be used. For example, a table can be created for minimizing cubed violations. Tables with more rows may degrade performance (increase run time) and numerical stability. There is a tendency for optimal solutions to be at the discrete values in the table. This may be more noticeable when tables with fewer rows (larger segments) are used. For example, if the reward table only had three rows corresponding to

satisfaction values of 0, 0.5 and 1, it might be common to see the solution at a satisfaction of either 0.5 or 1.0. Some experimentation may be required to determine the best table size for a given goal.

It is permissible to use the same reward table for many goals, and this is probably the easiest way to get started. If the results suggest that it may be beneficial to use different tables for some goals, then this can easily be changed. It is also possible to use different reward tables within the same Summation, if desired, by including multiple With Reward Table statements within a single Summation Soft Constraint Set, each referencing a different table slot.

For

Iterative loops can be very useful for adding similar constraints for multiple objects or over multiple timesteps. These iterative loops are added through a For statement. An index variable is assigned to a new value for each iteration of the loop. Inside the loop is one or more Constraint statements which should use the index variable to write a different constraint for each iteration of the loop. The default For loop is as follows:

```
FOR (NUMERIC index IN <list expr>) DO
  ADD CONSTRAINT <boolean expr>
END FOR
```

where the number of loops is determined by the number of elements in the <list expr>, the NUMERIC label indicates the expression data type of the elements in the <list expr>, and the index is the variable name which will take on the value of each element for use inside the loop. All of these parts of the For statement may be modified. It is also possible to nest a For statement inside another For statement (by replacing the ADD CONSTRAINT statement with another FOR statement) allowing the loops to iterate over two or more variables.

Because Optimization provides a global solution in time, there is no valid notion of *Current Timestep*; therefore, an important use of FOR loops is to write constraints over all timesteps. Such a FOR loop would have the following form:

```
FOR (DATETIME index IN @"Start Timestep" TO @"Finish Timestep") DO
  ADD CONSTRAINT Object.Slot[index] == <numeric expr>
END FOR
```

The time range for the constraint can be modified by changing the expressions in the list expression.

With

A value can be set on a local variable using a With statement. A With statement evaluates an expression and sets the result to a local variable with a given name and type. The statements contained within the With statement may then reference the variable. The default With statement is as follows:

```
WITH (NUMERIC val = <numeric expr>) DO
  ADD CONSTRAINT <boolean expr>
END WITH
```

If

The addition of a constraint can be made conditional on a boolean expression using an If statement. For example, a constraint might only be applied if the timestep is within a specified season or if a valid value is present within a specified slot on a data object. Without an Else, the statement will do nothing if the boolean expression evaluates to false.

Chapter 4

Variables, Constraints, and Objectives

```
IF (<boolean expr>) THEN
    <statement>
END IF
```

One important difference in the use of an If statement in an Optimization Goal compared to RBS rules is that an If statement in Optimization cannot contain expressions with Optimization variables. For example,

```
IF (Reservoir.Pool Elevation["@Start Timestep"] >= DataObject.Max Elevation[]) THEN
```

would cause the Optimization run to terminate with an error message because Pool Elevation is a variable in the Optimization problem and thus cannot be known when the If statement is evaluated (unless Pool Elevation at the Start Timestep happened to be specified as an input).

Also, If expressions cannot be added within a Constraint statement; however, Constraint statements can be added within an If statement.

If Else

This is similar to the If statement, but an alternative statement will be executed if the boolean expression evaluates to false.

```
IF (<boolean expr>) THEN
    <statement>
ELSE
    <statement>
END IF
```

Else If Branch

Although not a statement by itself, you can add one or more ELSE IF branches to an IF or IF ELSE statement. The ELSE IF is available when the boolean condition or a consequence statement is selected (except when an ELSE consequence statement is selected). An ELSE IF branch is added after the selected branch.

```
IF(<boolean expr>) THEN
    <statement>
ELSE IF(<boolean expr>) THEN
    <statement>
END IF
```

Else Branch

Although not a statement by itself, you can add a single ELSE branch to an IF Statement or ELSE IF statement. This part of the statement will be evaluated when none of the other IF or ELSE IF boolean conditions are true.

```
IF(<boolean expr>) THEN
    <statement>
ELSE
    <statement>
END IF
```

Print

A Print statement evaluates its expression and formats the result into a message. The blue message is displayed in the Diagnostics Output window only when the Print Statements diagnostics group is enabled.

```
PRINT <expr>
```

where the <expr> is any expression or concatenated expressions which can be fully evaluated and represented as a string. A Print statement can be useful within an If statement to notify the user that system is within a specified condition. See [“Print Statements” in *Debugging and Analysis*](#) for details about the Print statement.

Notice

A Notice statement posts an orange message to the Diagnostics Output Window regardless of diagnostics settings.

```
NOTICE <expr>
```

where the <expr> is any expression or concatenated expressions which can be fully evaluated and represented as a string. See [“Notice, Warning, and Alert Messages” in *Debugging and Analysis*](#) for details.

Warning

A Warning statement posts a message with a pink background to the Diagnostics Output Window regardless of diagnostics settings.

```
WARNING <expr>
```

where the <expr> is any expression or concatenated expressions which can be fully evaluated and represented as a string. See [“Notice, Warning, and Alert Messages” in *Debugging and Analysis*](#) for details.

Alert

An Alert statement posts a message with an orange background to the Diagnostics Output Window regardless of diagnostics settings.

```
ALERT <expr>
```

where the <expr> is any expression or concatenated expressions which can be fully evaluated and represented as a string. See [“Notice, Warning, and Alert Messages” in *Debugging and Analysis*](#) for details.

Note: The Notice, Warning and Alert statements all have the same behavior. They simply post messages with different colors. This allows you to define messages with three different levels of severity.

Stop Run

A Stop Run statement terminates the run and posts the provided message as a diagnostic error message in red text.

```
STOP RUN <expr>
```

A Stop Run might be used if it is known that a certain condition (that can be evaluated prior to the execution of the current goal) will produce an undesirable result, and thus a full Optimization run is not desired. A Stop Run would typically be contained within an If statement.

Timestep References in Optimization

Optimization provides a global solution in time and space (i.e. it solves all timesteps and all objects simultaneously); therefore there is not a valid notion of *Current Timestep* in Optimization. As such, DateTime expressions such as @"Current Timestep", @"t" and empty brackets [] (representing current controller timestep on series slots) are not valid in an Optimization Goal Set. Use of these DateTime values will cause the Optimization run to terminate. In order to evaluate an expression or apply a constraint over all timesteps (or a specified range of timesteps) a For loop is used where the index variable is a DateTime set to the values of a list of timesteps in the run. For example, adding a constraint on a reservoir's Pool Elevation for every timestep could have the following form:

```
FOR (DATETIME index IN @"Start Timestep" TO @"Finish Timestep") DO
    ADD CONSTRAINT Reservoir.Pool Elevation[index] <= 2,350 "ft"
END FOR
```

Optimization Variables

An optimization problem technically consists of a set of equations that include linear combinations of optimization variables. The following sections describe how those variables are defined.

Slots and Variables

In RiverWare, some Optimization variables are directly related to slots. Other variables are related to slots by a linearized expression. Still other variables have no relation to slots, and some slots have no relation to variables. RiverWare decides automatically which variables to add to the Optimization problem based on what is referenced in the Optimization Goal Set.

All slots that are related to Optimization variables, either directly or by linearized expressions, are series slots. These could be multislots or agg series slots, which are forms of series slots. In the case of agg series slots, each column corresponds to a separate variable.

Note: While it is common to refer to a slot and a variable synonymously, technically each individual variable within the Optimization problem, is associated with a single slot *at a single timestep*.

Slots as CPLEX Variables

Some slots are related directly to variables. For example, Outflow and Storage on a reservoir object are common variables. Each of these slots at each timestep corresponds directly to an individual variable in the Optimization problem. Some additional examples of slots that are directly related to variables are provided below for various objects.

- Canal: Flow1
- Confluence: Outflow
- Reach: Outflow
- Reservoir: Outflow, Spill, Storage, Release, Diversion, Cumulative Storage Value

- Power Reservoir: Turbine Release, Pumped Storage Outflow
- Slope Power Reservoir: $Qc1$, Level Storage, Pool Elevation

Linearized Slots

In some cases slots correspond to a linear combination of variables. One example is Power. The manner in which Power is linearized depends on the method selected in the Optimization Power category. With the commonly used Independent Linearizations method, Power is defined by a piecewise linear function of Turbine Release. Each individual variable, or *piece*, is multiplied by the slope of that segment of the power curve. The sum of the pieces is equal to the total Turbine Release. Examples of slots that correspond to linearized expressions of variables are provided below.

- Canal: Delta Elevation, Elevation 1, Elevation2, Flow 2
- Confluence: Inflow 1, Inflow 2
- Reach: Inflow
- Reservoir: Canal Flow, Energy In Storage, Inflow, Pool Elevation, Spill Cost, Spilled Energy, Spilled Power
- Power Reservoir: Best Turbine Flow, Energy, Future Value Used Energy, Hydro Capacity, Operating Head, Power, Pumped Storage Inflow, Spill Capacities, Tailwater Base Value, Tailwater Elevation, Turbine Capacity
- Pumped Storage Reservoir: Pump Capacity, Pump Energy, Pump Power, Pump Q Capacity
- Slope Power Reservoir: Backwater Elevation, Inflow 2, Wedge

Variables Unrelated to Slots

Some variables have no relation to a slot in RiverWare. For example, the satisfaction level of each Repeated Maximin iteration or the satisfaction of each constraint in a Summation derived objective is added to the Optimization problem as a variable. See [“Satisfaction Variables in Soft Constraints”](#) on page 17 for details about how satisfaction variables are defined.

Slots Unused by Optimization

Some slots apply to Simulation only and do not get used by the Optimization solution. For example, the Total Inflows and Inflow Sum slots on a Reservoir object do not get used by the Optimization solution. The components that make up Total Inflow and Inflow Sum in Simulation are expressed as individual variables and are incorporated into the mass balance constraint for the Reservoir, but there is not a separate Total Inflows or Inflow Sum variable that is defined. (Total Inflow and Inflow Sum will get calculated in the final results from the Post-optimization Rulebased Simulation.)

Constraints Define Variables

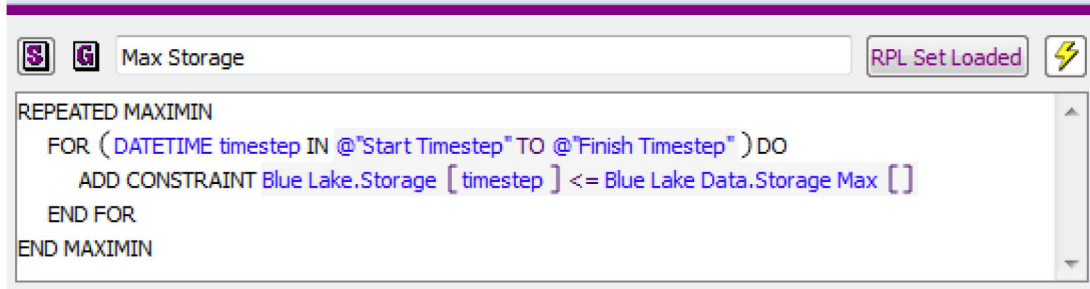
In an optimization run, the controller executes the user’s policy which consists of a series of constraints and objectives. RPL expressions add constraints to the current optimization problem, cause the problem to be solved with a given objective, or freeze some aspects of the current problem solution. When the policy refers to the value of a slot at a given timestep, RiverWare automatically adds constraints to the problem which define the variable

Chapter 4

Variables, Constraints, and Objectives

associated with the slot value at that timestep. As an example, assume the following goal is at priority 1 in the Goal Set. See [Figure 4.12](#).

Figure 4.12



RiverWare will add the constraint shown in the goal to Optimization problem for every timestep (as a soft constraint), replacing the scalar slot Blue Lake Data.Storage Max with the actual value in the slot. Also, because this constraint references the Blue Lake Storage for every timestep, it will add the defining constraint for Blue Lake Storage, which is the mass balance constraint, for every timestep.

$$\text{Blue Lake.Storage}_t = \text{Blue Lake.Storage}_{t-1} + \text{TimestepLength} \times (\text{Blue Lake.Inflow}_t - \text{Blue Lake.Outflow}_t)$$

Notice that the mass balance constraint has also defined the variables for Inflow and Outflow at every timestep. Other variables could be included in the mass balance constraint as well, depending on the selected user methods on the reservoir (e.g. Hydrologic Inflow, Evaporation). RiverWare will also automatically add the upper and lower bound constraints for each of these three variables at each timestep based on the upper and lower bounds in the Slot Configuration for each; see [“Variable Bounds”](#) on page 75 for details. If Inflow and Outflow are linked to slots on other objects, additional constraints will be added automatically which define the link relationship. In addition, a defining constraint will be added for Outflow setting it equal to the sum of Turbine Release and Spill, along with the upper and lower bound constraints for each of these new variables. So adding a single policy constraint can actually add numerous defining constraints for variables to the Optimization problem.

Physical Constraints

RiverWare automatically adds all relevant physical constraints to the Optimization problem. For example, the mass balance constraint for a reservoir is

$$\text{Storage}_t = \text{Storage}_{t-1} + \text{TimestepLength} \times \left(\text{Inflow}_t - \text{Outflow}_t + \sum \text{Gains}_t - \sum \text{Losses}_t \right)$$

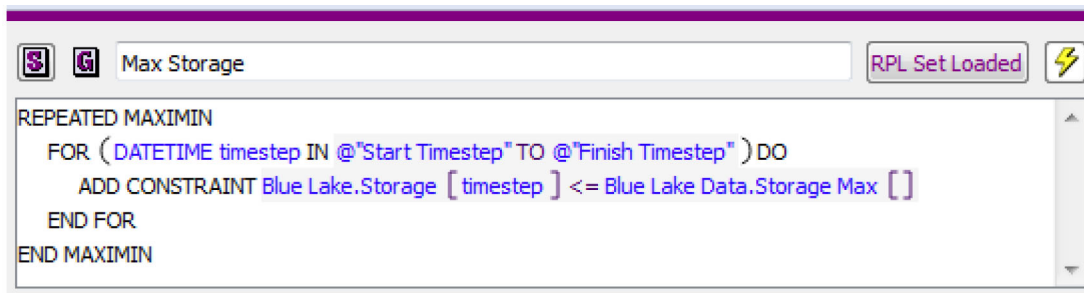
This is the defining constraint for the Storage, Inflow and Outflow variables at time t . RiverWare determines which individual variables to include in Gains and Losses depending on the method selection on the reservoir. For example, Evaporation could be an additional loss variable in the mass balance constraint. For a reach, the defining physical constraints are determined by the selected routing method.

RiverWare only adds physical constraints as necessary based on the policy in the Optimization Goal Set. In this way it minimizes the size of the Optimization problem in order to achieve maximum computational efficiency. Physical constraints are always treated as hard constraints.

Variable Bounds

Upper and lower bounds are required for all decision variables. The bounds represent the absolute minimum and maximum for the slot (variable). The variable bounds remain constant and are automatically converted by RiverWare into hard constraints on the variable for every time step in the Optimization solution. They get used in defining constraint satisfaction for soft constraints. Soft constraint satisfaction is always scaled based on the difference between the constraint value (right-hand-side) and the previous bound, the RHS value from an earlier (higher priority) constraint with the same left-hand-side. If there is no earlier constraint with the same left-hand-side, then the previous bound is taken from the variable bounds. In other words, if a constraint in the Optimization goal set is the first constraint of its form, then the satisfaction of the constraint is calculated relative to the slot (variable) bound. For example, assume the following max storage goal is the first goal in the Optimization Goal Set (priority 1). See [Figure 4.13](#).

Figure 4.13



The satisfaction S_t for the constraint on a single timestep would be as follows:

$$S_t \leq \frac{\text{Blue Lake.Storage}[t] - \text{StorageUpperBound}}{\text{Blue Lake Data.Storage Max} - \text{StorageUpperBound}}$$

$$0 \leq S_t \leq 1$$

See “[Satisfaction Variables in Soft Constraints](#)” on page 17 for details about how satisfaction variables in soft constraints are defined.

Often the variable bounds are based on the actual physical minimum and maximum. For example, the Storage Lower and Upper Bounds should be the smallest and largest Storage values in the Elevation Volume table. Turbine Release should be limited to zero as the Lower Bound and Turbine Capacity for the Upper Bound. In other cases, the absolute physical minimum or maximum may not be well-defined, such as maximum Inflow (or if economic variables are included in the model).

In these cases of variables without an obviously defined minimum or maximum, the slot bounds should not unnecessarily constrain the problem. These types of variables will typically be limited, either directly or indirectly, through the constraints in the Optimization Goal Set. It is also important, however, that the slot bounds not be set arbitrarily large or small (i.e. orders of magnitude different than the realistic limits) as this can result in numerical instability due to scaling issues in the optimization problem.

It is important that slot bounds not be confused with policy constraints, nor be used to apply policy constraints. For example, a hydropower plant might have a non-zero minimum generation requirement that it must meet, but the Power slot Lower Bound should still be zero. The minimum generation requirement would be set through a constraint in the Optimization Goal Set. The slot bounds become hard constraints in the optimization problem and

Chapter 4

Variables, Constraints, and Objectives

essentially remain constant in the model, whereas policy constraint values (usually stored in custom slots) are typically incorporated as soft constraints and may change for different runs.

The slot bounds do not affect the Simulation solution. They are only applied in the Optimization solution. The Simulation solution is limited only by the physical characteristics described in table slots such as the Elevation Volume Table and Plant Power Table.

The variable bounds are displayed on the Configure Slot dialog. From the slot dialog select **View**, then **Configure**.

The screenshot shows the 'Configure Slot' dialog box. The 'Object' is 'Blue Mountain' and the 'Slot' is 'Storage'. Under 'Units', 'Unit Type' is 'Volume', 'Unit Scheme' is 'Scheme 1', and 'Use Volume settings' is selected. The 'Units' dropdown is 'acre-ft' and 'Scale' is '1000'. 'Precision' is '2' and 'Format' is 'Float'. The 'Repair incorrect units (within type only)' checkbox is unchecked. The 'Bounds' section is circled in red, showing 'Lower: 0.01' and 'Upper: 115.30' both with a unit of '1000 acre-ft'. The 'Convergence' section shows 'Type: Percent' and 'Value: 0.01 %'. Buttons at the bottom are 'OK', 'Apply', 'Reset', and 'Cancel'.

See [“Bounds on Variables”](#) on page 271 for details about setting slot bounds in a model.

User-defined Variables

In addition to the set of standard variables that are automatically included in the Optimization problem, it is also possible to add customized user-defined variables. User-defined variables correspond to custom slots that the user adds to the model. The user must define the variables in a constraint that they add to the Goal Set. Then they can use the variable in a later policy constraint or objective.

There are five steps for implementing a user defined variable.

1. Add a series slot or agg series slot to an object in the model specifying appropriate units.
2. Specify that the slot (or column of the agg series slot) is a user-defined variable.

From the slot dialog select **View**, then **Configure**. In the resulting Configure Slot dialog, select the **Is User Defined Variable** check box, near the bottom of the dialog.

Configure Slot

Object : System Data
Slot : Energy Sales

☐ Values are integers

Units

Unit Type: Value
Unit Scheme: Scheme 1
☒ Use Value settings
☐ Use settings common to slots with name "Energy Sales"
☐ Use custom settings below
Units: \$ Scale: 1
Precision: 2 Format: Float
☐ Repair incorrect units (within type only)

Bounds

Lower: NaN NONE
Upper: NaN NONE

Convergence

Type: Percent Value: 0.01 %

Optimization

☒ Is User Defined Variable

OK Apply Reset Cancel

Chapter 4

Variables, Constraints, and Objectives

3. Set appropriate Lower and Upper Bounds for the variable in the **Configure Slot** dialog. See [“Variable Bounds”](#) on page 75 for details about variable bounds.

Configure Slot

Object : System Data

Slot : Energy Sales

☐ Values are integers

Units

Unit Type: Value

Unit Scheme: Scheme 1

☒ Use Value settings

☐ Use settings common to slots with name "Energy Sales"

☐ Use custom settings below

Units: \$ Scale: 1

Precision: 2 Format: Float

☐ Repair incorrect units (within type only)

Bounds

Lower: 0.00 \$

Upper: 1500000.00 \$

Convergence

Type: Percent Value: 0.01 %

Optimization

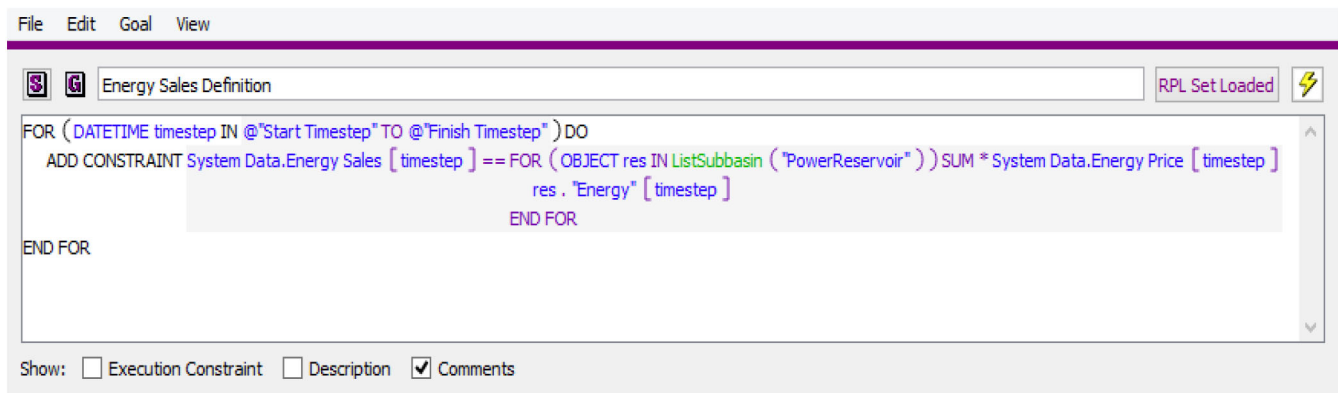
☒ Is User Defined Variable

OK Apply Reset Cancel

4. Define the variable in a constraint in the Goal Set.

[Figure 4.14](#) is an example that defines the variable Energy Sales at each time step as the total energy generation from all power reservoirs multiplied by the Energy Price. (This assumes that Energy Price is an input.) Variables are typically defined using hard constraints.

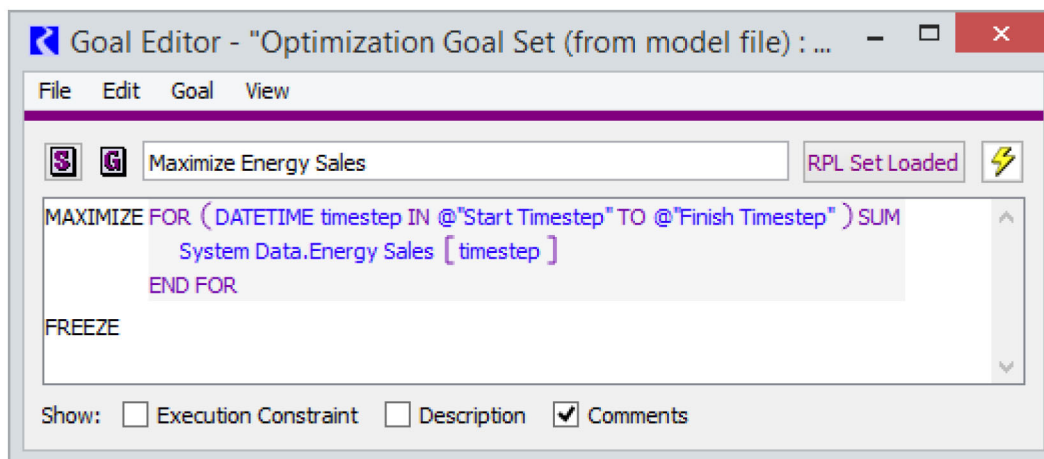
Figure 4.14



5. Use the variable in a policy constraint or objective.

Figure 4.15 maximizes the sum over the entire run of the Energy Sales variable defined in the previous step.

Figure 4.15



User-defined variables provide much flexibility in extending the existing RiverWare methods for Optimization. The one restriction on user-defined variables is that, as with all Optimization constraints, the defining constraints must be linear expressions of other variables.

Linearization, Approximation, and Replacement

Some slots, such as Storage and Outflow, correspond directly to decision variables in the Optimization problem. Other slots, such as Power, are translated into linear combinations of other decision variables. RiverWare automatically selects which substitution methods to use based on the slots referenced and the context in which they are referenced in the policy (goal). All substitution is done automatically behind the scenes in RiverWare. For all substitution methods, if the resulting expression is not a function of only decision variables, substitution continues using whatever method is appropriate until only decision variables remain.

Details of the substitution methods are given below.

Link Substitution

Link substitution replaces a linearized slot with the slots that it is linked to. An example is the substitution of a reach Inflow that can be a combination of Outflows from one or more other objects.

Constraint Substitution

Constraint substitution replaces expressions containing one linearized slot by equivalent expressions containing only one decision variable slot. For example, Pool Elevation is a linearized slot that has a direct relationship to Storage, a decision variable slot, defined by the Elevation Volume Table. A constraint such as Pool Elevation greater than or equal to some value can be replaced by an equivalent constraint on Storage.

Direct substitution will not work for constraints with multiple terms. For example, Operating Head cannot be replaced by a constant directly because Operating Head is Pool Elevation minus Tailwater Elevation, and thus there are two linearized slots on the left hand side of the constraint. When it is available, direct substitution is the preferred method for linearization because the only linearization error is the error in the table.

Expression Substitution

Expression substitution replaces a linearized slot with a linear expression of other slots. An example for Precipitation Volume is shown below.

$$\text{Precipitation Volume}_t = \text{TimestepLength} \times \text{Precipitation Rate}_t \times \frac{\text{Surface Area}_t + \text{Surface Area}_{t-1}}{2}$$

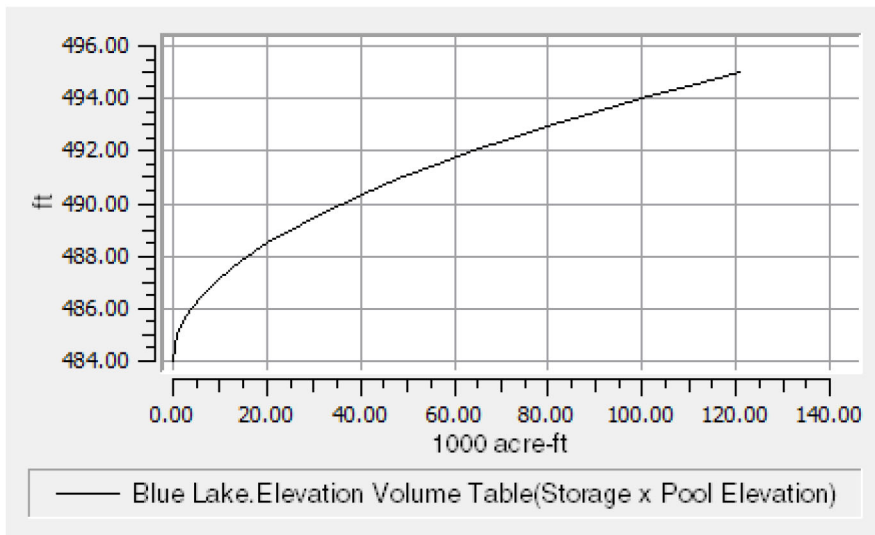
In the above expression, Surface Area is itself a linearized variable that is a function of Storage. So Surface Area will be substituted by an expression in terms of Storage.

Variable Substitution

Variable substitution replaces a linearized slot with a known linear expression of other slots. Variable substitution can use direct substitution, a two dimensional linearization (tangent, two-point line or piecewise) or three-dimensional linearization (constant and multiple approximation). These methods are described below. RiverWare automatically chooses the correct linearization method to use based on the slot and the context of the policy expression.

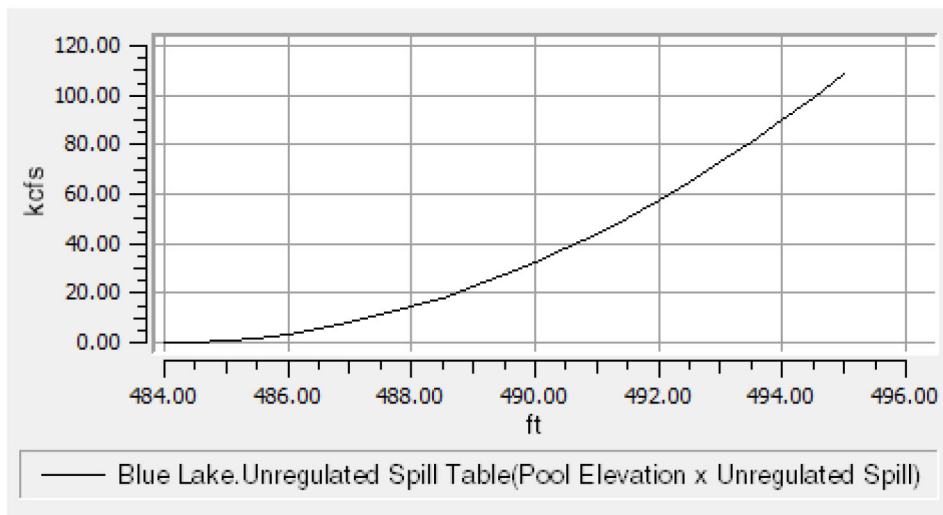
The mathematical functions used by RiverWare to model the relationships between two slots can be classified as convex, concave or nonconvex, nonconcave. Which linearization methods can be used to approximate a functional relationship between two slots and the accuracy of the approximation depends on the convexity of the function. Riverware expects a specific convexity for each functional relationship. If a user input function defined by a table slot does not match the expected convexity, an error is returned. An example of a concave function is the relationship between Pool Elevation and Storage. See [Figure 4.16](#).

Figure 4.16



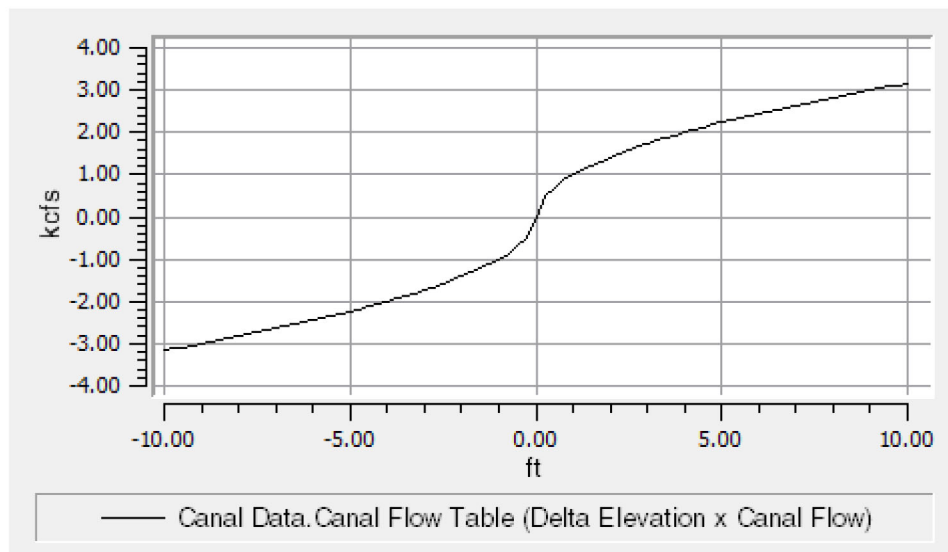
A convex function is the relationship between Unregulated Spill and Pool Elevation. See [Figure 4.17](#).

Figure 4.17



A nonconvex, nonconcave function is the relationship between Canal Flow and Delta Elevation. See [Figure 4.18](#).

Figure 4.18



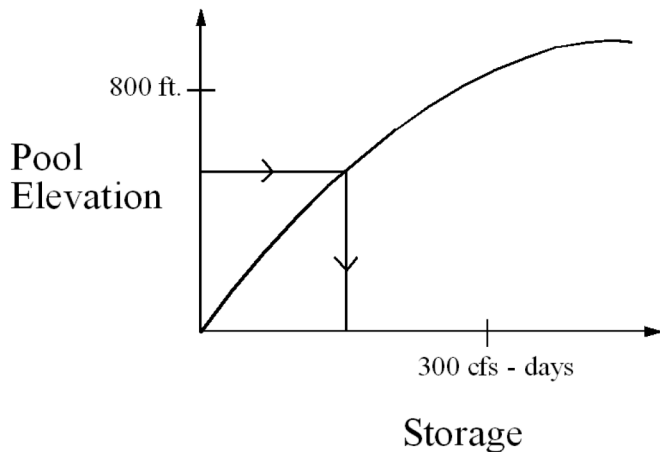
A tangent approximation will over-approximate a concave function and under-approximate a convex function. A two-point (secant) approximation will under-approximate a concave function and over-estimate a convex function.

The points used to define the tangent, two-point line and piecewise approximations are specified in linearization parameter tables. These are slots with LP Param in the name. See [“Optimization Objects and Methods”](#) on page 87 for details about LP Param table slots within individual slot and method descriptions.

Direct Substitution

Substitution replaces the a variable with the value of another variable from the linearization function table. For example, Pool Elevation is replaced by the equivalent Storage (see [Figure 4.19](#)). Values that are not listed directly in the table are approximated by linear interpolation. Substitution is only allowed for single term constraints. Substitution is generally the preferred linearization method when it is available because it has the least approximation error. RiverWare automatically uses this approach whenever there is a constraint with a single term.

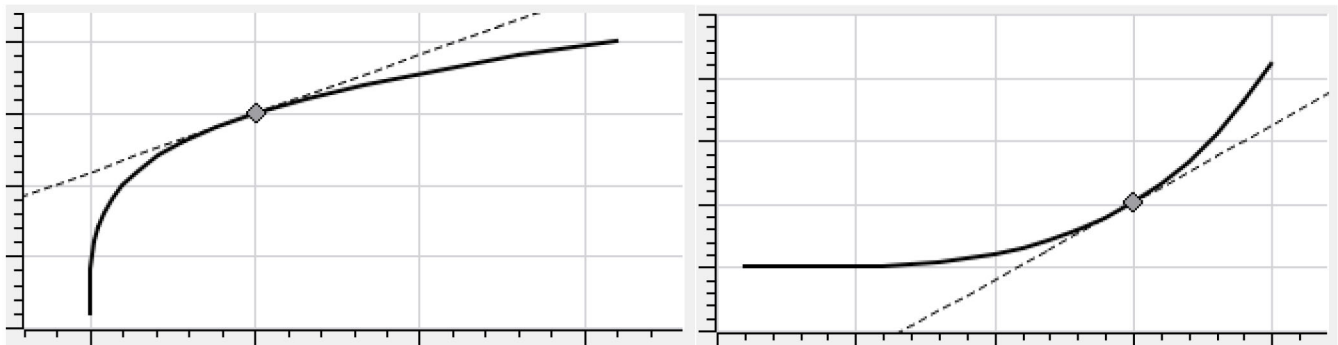
Figure 4.19



Tangent Approximation

For the tangent approximation a nonlinear function is approximated as the line tangent to the function at a specified point. The same approximation point is used for every time step. Tangent is preferred when an over estimation is acceptable for a concave function or an under approximation is acceptable for a convex function. Tangent approximations are good when the value of a function will not vary greatly from the initial value (approximation point). Tangent approximations are not always available because of the errors they can introduce for certain types of functions. For example, tangent approximations are not used for Spill Capacities because it could result in a non-zero value for a function with a zero input. The best approximation point for a tangent to be taken at is the median value expected during the run.

Figure 4.20 Tangent approximations of a nonlinear function



Two-point Line Approximation

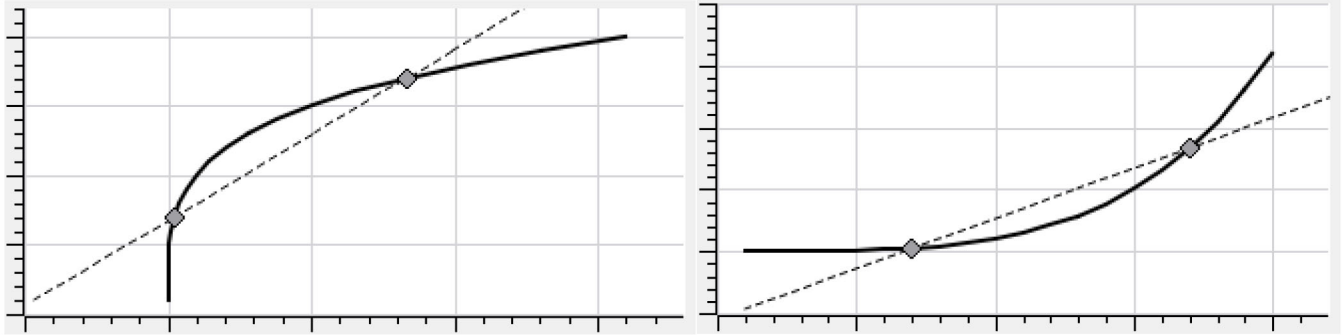
For the two-point line (secant) approximation a nonlinear function is approximated by a line through two points on the function. The two points are fixed and remain the same for every time step. Unlike the tangent approximation, the two point line approximation sometimes underestimates a function and sometimes overestimates a function.

Chapter 4

Variables, Constraints, and Objectives

The best approximation points for a line are points that will define a line close to the expected values during the run.

Figure 4.21 Two-point line (secant) approximation



Piecewise Approximation

For a piecewise approximation a nonlinear function is approximated as a piecewise linear function. The x values for an arbitrary number of points are input into the third column of the LP Param table. Lines are generated between adjacent points. Several constraints are added to the LP model that make sure that the approximation will work.

As an example, Power (P) is a function of Turbine Release (Q). A piecewise approximation would be

$$P = aQ_1 + bQ_2$$

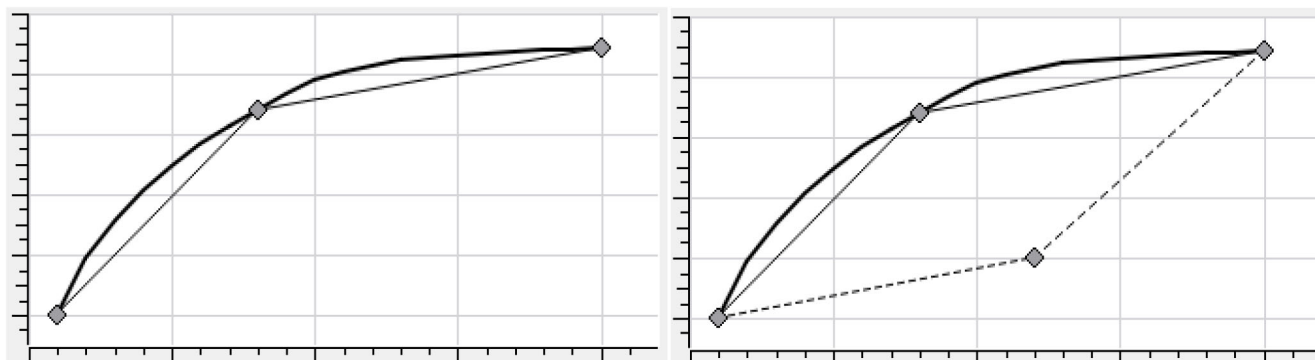
with the constraints

$$Q = Q_1 + Q_2$$

$$0 \leq Q_i \leq u_i$$

Where a and b are the slopes of the approximation lines and u_i is the length of segment i . The optimization process can *cheat* when dealing with less than or equal constraints on concave functions and greater than or equal to constraints on convex functions. The figure below shows a piecewise approximation of a concave function using two lines (three points). If a minimization is done to this function or it is used with a less than or equal to constraint, the solution would try to pick values that correspond to the dashed line rather than the solid line. That is, it would use the pieces out of order. RiverWare prevents this use of pieces out of order by automatically using the two-line approximation in these cases. If substitution is not available, piecewise approximations are the next preferred linearization method because they can fit the curve closely and minimize the approximation error better than line or tangent generally can. With piecewise approximations it is best to cover the range of values of the function, especially in range of the expected values. Additional points should be included where there are breaks in the curve. A minimum of two points are required. As many points as desired may be used to fit the curve, but a larger number of points will increase the size of the optimization problem and could impact performance (run time).

Figure 4.22 Piecewise approximation of a concave function



Lambda Method

Note: This section is under development.

Chapter 4

Variables, Constraints, and Objectives

Chapter 5

Optimization Objects and Methods

Topics

- [How Objects Are Described, page 87](#)
- [Canal, page 87](#)
- [Confluence, page 90](#)
- [Data Objects, page 90](#)
- [Inline Power Plant, page 91](#)
- [Level Power Reservoir, page 91](#)
- [Pumped Storage Reservoir, page 142](#)
- [Reach, page 198](#)
- [Slope Power Reservoir, page 203](#)
- [Storage Reservoir, page 242](#)
- [Thermal, page 258](#)

How Objects Are Described

This section describes slots, methods, and constraints used by the following objects in Optimization. The following information is provided for each object.

- **General Slots**—These slots always exist on the object when the Optimization controller is selected, regardless of selected methods. See [“General Slots” in *Objects and Methods*](#) for details about general Simulation slots.
- **User Methods**—Used to specify how a feature is to be represented.
- **Numerical Approximations**—Describe how variables are replaced or approximated.
- **Modeling the object**—Instructions for building a model of each object.

Canal

The Canal object models gravity flow through a canal connected to two reservoirs.

General Slots

• Elevation 1

Type:	Agg Series Slot
Units:	Length
Description:	Pool Elevation of reservoir 1
Information:	
Defined by:	Replacement by the linked reservoir Pool Elevation slot (which in turn may be Numerically Approximated).

• Elevation 2

Type:	Agg Series Slot
Units:	Length
Description:	Pool Elevation of reservoir 2
Information:	
Defined by:	Replacement by the linked reservoir Pool Elevation slot (which in turn may be Numerically Approximated).

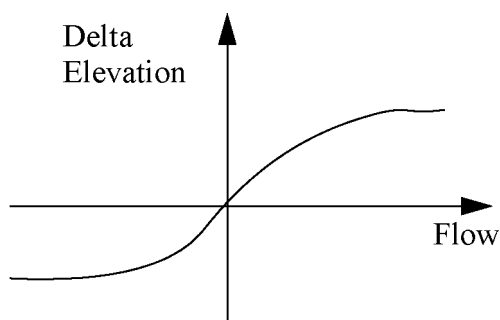
• Flow 1

Type:	Agg Series Slot
Units:	Flow
Description:	flow to or from the canal end connected to reservoir 1
Information:	This slot must be linked to the Canal Flow slot on reservoir 1.
Defined by:	Explicit Optimization variable as $0.0 = \text{Flow 1} + \text{Flow 2}$

• Flow 2

Type:	Agg Series Slot
Units:	Flow
Description:	flow to or from the canal end connected to reservoir 2
Information:	This slot must be linked to the Canal Flow slot on reservoir 2.
Defined by:	Numerical 3-D Approximation in terms of Lower Elevation and Delta Elevation. Approximation is based on the Canal Flow Table. The Flow 2 LP Param table contains a value for Lower Elevation used to index the Lower Elevation column of the Canal Flow Table. This approximated value, therefore, reduces Flow 2 to a function of Delta Elevation. The length values in the Flow 2 LP Param table are then used as approximation points indexing the Delta Elevation column of the Canal Flow Table. The Canal Flow Table should have increasing values of Lower Elevation and Delta Elevation. The function is expected to be non-convex. The preferred order of approximation is substitution, piece-wise, two-point line, tangent.

Non-concave, non-convex function. Not drawn to scale.



• Canal Flow Table

Type:	Table Slot
Units:	Length, Length, Flow
Description:	3-D table used to find canal flow by interpolation given a Lower Elevation, Delta Elevation between the lower and higher pool elevations and resulting Flow 2.
Information:	
Defined by:	User input

• Delta Elevation

Type:	Agg Series Slot
Units:	Length
Description:	The average elevation difference between the two reservoirs and across the canal at the current timestep
Information:	
Defined by:	

• Flow 2 LP Param

Type:	Table Slot
Units:	Length, Length, Length, Length
Description:	specifies the Lower Elevation and the Delta Elevation points used to take the tangent, line, and piecewise approximations for numerical approximation of Flow 2 as specified in the Canal Flow Table. The best Lower Elevation to choose should be close to the expected elevation for the whole run. For Delta Elevation the suggested points for tangent approximation is the estimated value for the whole run and for line approximation are 0 flow and some likely value. piecewise linearization is not used.
Information:	
Defined by:	User input

• Lower Elevation

Type:	Series Slot
Units:	Length

Description: The elevation of the lower of the two reservoirs for the current timestep
Information:
Defined by:

User Methods

On the Canal object, there are currently no categories available in RPL Optimization.

What exists as the default arrangement, however, reflects the Canal Flow Table simulation method of the Canal Flow Calculation category; see “[Canal Flow Table](#)” in *Objects and Methods* for details.

Canal Flow Numerical Approximation

Any time the Canal object is included in the Optimization problem, then Flow 2 is added to the problem, first in terms of a constraint, and then as a function of Lower Elevation and Delta Elevation (Numerical 3-D Approximation). The relationship between Lower Elevation, Delta Elevation and Flow 2 comes from the user-input Canal Flow Table. The table will be queried using the points defined in the Flow 2 LP Param table.

Modeling a Canal

Steps to follow in setting up a canal for optimization are as follows:

1. Set up a running simulation model with appropriate links to and from the canal. Use the Canal Flow Table method.
2. When RPL Optimization control is selected, fill in the Flow 2 LP Param table.

Confluence

The Confluence object has no selectable methods in RPL Optimization. A constraint is included in the optimization problem to ensure mass balance for all timesteps:

$$\text{Inflow1} + \text{Inflow 2} = \text{Outflow} \quad (5.1)$$

See “[General Slots](#)” in *Objects and Methods* for a full description of slots.

Data Objects

Data Objects hold slots that can be used in your policy either as reference information or as user-defined optimization variables.

Reference data is just values that are known before the optimization run like minimum flows, guide curves, power targets, and so on. You can write policy that uses the values in these slots.

User-defined optimization variables are slots that you want to compute in optimization. They must be series slots or a column of an agg series slot on a data object.

To specify that a series slot on a data object is a user-defined optimization variable, on the Configuration dialog, select the **Is User Defined Variable** check box. The slot can then participate in an optimization problem, but you must still add policy that defines the equation for this variable to pull it into the problem.

The automatically generated post-opt RBS ruleset includes rules to set the user-defined variable slots to their optimal values.

Inline Power Plant

The Inline Power Plant object models a run-of-the-river power plant where power facilities rely on the natural slope of the river channel to provide necessary head to generate power. Additionally, this object provides very little or no storage. See [“General Slots” in *Objects and Methods*](#) for details.

Note: The remainder of this section is under development.

Level Power Reservoir

The Level Power Reservoir object is similar to a Storage Reservoir but with added functionality to model power production facilities on the reservoir. The assumption that the water surface is level is inherent in this object.

See [“General Slots” in *Objects and Methods*](#) for details.

General Slots

• Canal Flow

Type:	Agg Series
Units:	Flow
Description:	Flow into (out of) the reservoir from (to) a canal
Information:	
Defined by:	Explicit Optimization variable in the mass balance constraint

• Diversion

Type:	Series Slot
Units:	Flow
Description:	Flow from the reservoir to a diverting object
Information:	
Defined by:	Explicit Optimization variable in the mass balance constraint

• Elevation Volume Table

Type:	Table
--------------	-------

Chapter 5

Optimization Objects and Methods

Units:	Length vs Volume
Description:	Table relating elevation of the reservoir to volume stored in the reservoir
Information:	
I/O:	Input only
Defined by:	Input only

• Energy

Type:	Agg Series Slot
Units:	Energy
Description:	Product of the power generated by flow through the turbines and the length of the timestep.
Information:	
Defined by:	Replaced by Power * Timestep Length

• Flow FROM Pumped Storage

Type:	Agg Series Slot
Units:	Flow
Description:	Flow into the reservoir from a pumped storage reservoir
Information:	
Defined by:	Explicit Optimization variable in the mass balance constraint. This slot should be linked to Outflow on a Pumped Storage object. The Pumped Storage object constrains its Outflow.

• Flow TO Pumped Storage

Type:	Agg Series Slot
Units:	Flow
Description:	Flow out of the reservoir into a pumped storage reservoir
Information:	
Defined by:	Explicit Optimization variable in the mass balance constraint. This slot should be linked to Pumped Flow on a Pumped Storage object.

• Inflow

Type:	MultiSlot
Units:	Flow
Description:	Inflow into the reservoir from upstream
Information:	
Defined by:	Explicit Optimization variable in the mass balance constraint

• Operating Head

Type:	Agg Series Slot
Units:	Length
Description:	Elevation difference between the average Pool Elevation and the average Tailwater Elevation during a timestep

Information:
Defined by: Replacement by $(\text{Pool Elevation}(t) + \text{Pool Elevation}(t-1)) / 2 - \text{Tailwater Elevation}$

• Outflow

Type: Agg Series Slot
Units: Flow
Description: Outflow from the reservoir
Information:
Defined by: Explicit Optimization variable as $\text{Outflow} = \text{Turbine Release} + \text{Spill}$

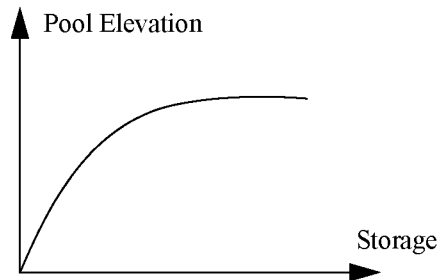
• Pool Elevation

Type: Series Slot
Units: Length
Description: Elevation of the water surface of the Reservoir
Information: When Pool Elevation is a part of the optimization problem, as it is in all conceivable RiverWare Optimization applications, this slot is numerically approximated as a function of Storage (Numerical 2-D Approximation). The relationship between Pool Elevation and Storage will come from the user-input Elevation Volume Table. The table will be queried either using user-input points defined in the Pool Elevation LP Param table.
Defined by: Numerical 2-D Approximation in terms of Storage, based upon the Elevation Volume Table. The Pool Elevation LP Param table values are used as approximation points indexing the Elevation Volume Table. The Elevation Volume Table should have increasing values of Pool Elevation and Storage. Storage is required to be a concave function of Pool Elevation. The preferred order of approximation is substitution, piece-wise, tangent, two-point line.

• Pool Elevation LP Param

Type: Table Slot
Units: Volume
Description: Specifies the Storage points used to take the tangent, line and piecewise approximations for Pool Elevation linearization
Information: This table is used for linearization unless Pool Elevation Linearization Automation category has selected Plant Automation. The best Storage point to choose:
For tangent approximation, the expected storage expected during the run
For the line approximation, the expected maximum and minimum Storage
For piecewise approximation, use points that cover the full range of expected Storage during the run with intermediate points, such that a piecewise linear curve reasonably approximates the actual curve.
Defined by: User input

Pool elevation storage relationship. Not drawn to scale.



• Power

Type:	Series Slot
Units:	Power
Description:	Power generated by flow through the turbines
Information:	
Defined by:	Numerical 3-D Approximation in terms of Operating Head and Turbine Release. Approximation is based on the Plant Power Table. The Power LP Param table contains a value for Operating Head used to index the Operating Head column of the Plant Power Table. This approximated value, therefore, reduces the Power to a function of Turbine Release at the given Operating Head. The flow values in the Power LP Param table are then used as approximation points indexing the Turbine Release column of the Plant Power Table. The Plant Power Table should have increasing values of Operating Head and Turbine Release. Power should be a concave function of Operating Head, but concavity is not strictly enforced; mild non-concave regions are permissible to allow for round-off error, and so on. The preferred order of approximation is substitution, piece-wise, two-point line, tangent.

• Return Flow

Type:	MultiSlot
Units:	Flow
Description:	Flow returning from a diversion object
Information:	
Defined by:	Explicit Optimization variable in the mass balance constraint (see Storage)

• Spill

Type:	Series Slot
Units:	Flow
Description:	Sum of the Bypass, Regulated Spill and Unregulated Spill
Information:	
Defined by:	Explicit Optimization variable as $\text{Spill} = \text{Bypass} + \text{Regulated Spill} + \text{Unregulated Spill}$

• Storage

Type:	Series Slot
Units:	Volume

Description: Volume of water stored in the reservoir
Information:
Defined by: Explicit Optimization variable as $\text{Storage} = \text{Storage}(t-1) + \text{Precipitation Volume} - \text{Evaporation} - \text{Change in Bank Storage} + \text{timestep} * (\text{Inflow} + \text{Canal Flow} + \text{Flow TO Pumped Storage} + \text{Hydrologic Inflow Net} + \text{Return Flow} - (\text{Outflow} + \text{Diversion} + \text{Flow FROM Pumped Storage}))$

• Tailwater Base Value

Type: Series Slot
Units: Length
Description: Elevation of tailwater or base elevation used to compute elevation of tailwater
Information:
Defined by: Explicit Optimization variable should be input or linked. See [“Optimization Reserves”](#) on page 116 for details about related constraints or about other objects to which the slot is linked.

• Tailwater Elevation

Type: Series Slot
Units: Length
Description: Water surface elevation on the downstream side of the dam
Information:
Defined by: Various approaches dependent on the method selected in the Optimization Tailwater category.

• Turbine Capacity LP Param

Type: Table
Units: Length, Length, Length
Description: LP Param table for turbine capacity
Information: see power methods for details

• Turbine Capacity

Type: Series Slot
Units: Flow
Description: Flow capacity of the entire power plant’s turbines
Information:
Defined by: Numerical 2-D Approximation in terms of Operating Head, based upon a maximum turbine capacity table. This capacity table is determined in various ways according to the Power method.

• Turbine Release

Type: Agg Series Slot
Units: Flow
Description: Flow through the turbines of a power reservoir
Information:

Defined by: Explicit Optimization variable as:
$$\text{Turbine Release} \leq \text{Power Plant Cap Fraction} * \text{Turbine Capacity}$$

User Methods

The following categories and methods are available for use in the Level Power Reservoir object in Optimization.

Because of dependency relationships, you may not be able to see all the methods in your model until you change other methods. In the discussion below, you will see the other methods that need to be selected to enable a given method to be used in your model. When building a model, you need to review this to ensure that required dependencies are satisfied so that the desired methods are available for use.

User method categories are as follows.

- [Avoidance Zones](#), page 96
- [Bank Storage](#), page 97
- [Cavitation](#), page 97
- [Diversion from Reservoir](#), page 98
- [Energy in Storage](#), page 98
- [,](#) page 99
- [Future Value](#), page 99
- [Head Loss](#), page 101
- [Hydrologic Inflow](#), page 101
- [Optimization Evaporation](#), page 102
- [Optimization Future Value](#), page 105
- [Optimization Power](#), page 105
- [Optimization Reserves](#), page 116
- [Optimization Spill](#), page 128
- [Optimization Tailwater](#), page 131
- [Power](#), page 136
- [Startup](#), page 141

Avoidance Zones

This category depends on selecting the Unit Power Table method, (see “[Unit Power Table](#)” on page 139), and contains methods for modeling the existence of undesirable regions of operation for turbines. There are two methods in this category, one which does not model avoidance zones at all, and one which

■ None

This the default, no action method; avoidance zones are not considered.

■ Unit Head Based Avoidance Zones

This method allows the user to specify a table that defines the conditions in which the turbines should not be operated.

See [“Unit-level Constraints”](#) on page 309 for details about added constraints.

Slots Specific to This Method

See [“Unit Head Based Avoidance Zones”](#) in *Objects and Methods* for slot descriptions.

Bank Storage

Not all methods are functional in RPL optimization. Of the available methods, None and CRSS Bank Storage are supported.

■ None

See [“None”](#) in *Objects and Methods* for details.

■ CRSS Bank Storage

See [“CRSS Bank Storage”](#) in *Objects and Methods* for details.

Cavitation

This category depends on selecting the Unit Power Table method, (see [“Unit Power Table”](#) on page 139), and contains methods for dealing with the problem of cavitation on turbines. Cavitation is the sudden formation and collapse of low-pressure bubbles in liquids by means of mechanical forces, and this process can cause damage to turbines under certain operating conditions.

■ None

This is the default, no action method.

■ Unit Head and Tailwater Based Regions

This method allows the user to specify the regions of operation in which cavitation does *not* occur, so these regions can be avoided. These regions can be dependent on both operating head and tailwater.

See [“Unit-level Constraints”](#) on page 309 for details about added constraints.

Slots Specific to This Method

See “[Unit Head and Tailwater Based Regions](#)” in *Objects and Methods* for slot descriptions.

Diversion from Reservoir

Not all methods in this category are supported in RPL optimization. Of the available methods, None and Available Flow Based Diversion are supported; selection of any other method will result in an error during begin run.

■ None

See “[None](#)” in *Objects and Methods* for details.

■ Available Flow Based Diversion

See “[Available Flow Based Diversion](#)” in *Objects and Methods* for details.

Energy in Storage

In Optimization, currently None and EIS Table Lookup are supported.

■ None

No Energy in storage is considered.

■ EIS Table Lookup

With this method selected, Energy in Storage is considered as a function of Pool Elevation.

If the optimization problem uses Energy In Storage, either directly or indirectly, this slot is added to the problem. Before being passed to the optimization solver, this slot is numerically approximated as a function of Pool Elevation (Numerical 2-D Approximation).

The relationship between Pool Elevation and Energy In Storage comes from the user-input Energy In Storage Table. The table is queried either using user-input points defined in the Energy In Storage LP Param table or using automatically calculated points, depending on method selection in the Pool Elevation Linearization Automation category.

If the selected method in the Pool Elevation Linearization Automation is None, the user-input Energy In Storage LP Param table values are used.

For other Pool Elevation Linearization Automation methods (Initial Target Input, Min Difference or Range Input, Min Difference) the same points automatically developed for the Pool Elevation linearization are used.

See “[EIS Table Lookup](#)” in *Objects and Methods* for details on the Simulation method.

Slots Specific to This Method

• Energy In Storage

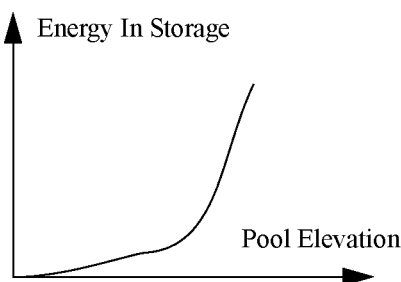
Type:	Series Slot
Units:	Energy vs Power
Description:	Energy in Storage in the reservoir
Information:	
Defined by:	Numerical 2-D Approximation in terms of Pool Elevation, based upon the Energy In Storage Table. The Energy In Storage LP Param table values are used as approximation points indexing the Energy In Storage Table. The Energy In Storage Table should have increasing values of Pool Elevation and Energy In Storage. Energy In Storage is required to be a convex function of Pool Elevation. The preferred order of approximation is substitution, piece-wise, tangent, two-point line.

• Energy in Storage LP Param

Type:	Table Slot
Units:	Length
Description:	Specifies the Pool Elevation points used to take the tangent, line and piecewise approximations for Energy In Storage linearization.
Information:	This table is used for linearization unless the Pool Elevation Linearization Automation category has a method selected other than None.
Defined by:	User input

• Energy in Storage Table

Type:	Table Slot
Units:	Length vs Energy
Description:	Table defining the relationship between Energy In Storage and Pool Elevation.
Information:	
Defined by:	User input Energy in storage pool elevation relationship. Not drawn to scale.



Future Value

This category and its methods are not dependent on other method selections.

■ None

See “None” in *Objects and Methods* for details.

■ Cumulative Storage Value Table

In RPL-Optimization, the Cumulative Storage Value is Numerically Approximated as described below.

See “Cumulative Storage Value Table” in *Objects and Methods* for details.

Slots Specific to This Method

• Cumulative Storage Value

Type:	Agg Series Slot
Units:	\$
Description:	Represents the future energy value of the current storage
Information:	
Defined by:	2-D approximation in terms of Anticipated Storage, based upon the Cumul Stor Val Table. The Cumul Stor Val LP Param table values (Storage) are used as approximation points indexing the Cumul Stor Val Table. The Cumul Stor Val Table should have increasing values of Storage and Cumulative Value. Cumulative Storage Value is required to be a concave function of Anticipated Storage. The preferred order of approximation is substitution, piece-wise, tangent, two-point line. The Cumul Stor Val Linearization Automation category’s Marginal Value to Table and Lin method can automate creation of the Cumul Stor Val LP Param table and the Cumul Stor Val Table.

• Anticipated Storage

Type:	Agg Series Slot
Units:	Volume
Description:	The combination of the actual storage plus water that would be expected to enter the reservoir after the Current Timestep but has not yet, due to lagging.
Information:	
Defined by:	

• Cumul Stor Val Table

Type:	Table Slot
Units:	Volume vs Value
Description:	The estimated total economic value of water stored in the reservoir for discrete storage values.
Information:	
Defined by:	User input or by automated procedure if Cumul Stor Val Linearization Automation category has selected the Marginal Value to Table and Lin method.

• Marginal Storage Value Table

Type:	Table Slot
--------------	------------

Units:	Storage vs \$ Value
Description:	Anticipated Storage versus Cumulative Storage Value per unit energy
Information:	This table should be increasing in storage, and logically decreasing in marginal value
Defined by:	Required input

- **Spill Cost**

Type:	Agg Series Slot
Units:	\$
Description:	
Information:	
Defined by:	

Head Loss

This category depends on the Unit Power Table method (see [“Unit Power Table”](#) on page 139), and contains methods for modeling additional head loss that occurs. This head loss may come from the configuration of the penstocks for bringing water to the turbines.

None

In this method, there is no additional head loss to be used in the power calculation. In terms of penstock head loss, this method should be selected if the penstocks for the units are independent and the penstock losses are typically incorporated in the power data. Thus the power data is specified in terms of operating head.

Shared Penstock Head Loss method

In this method, there is additional head loss that results because units share a common penstock. The operating head losses in the penstock depend on the total turbine release and are shared for all units. The net head is calculated by subtracting penstock losses from the operating head. The unit data and power must be specified in terms of unit Net Heads instead of Operating Head.

See [“Unit-level Constraints”](#) on page 309 for details about added constraints.

Slots Specific to This Method

See [“Shared Penstock Head Loss method”](#) in *Objects and Methods* for slot descriptions.

Hydrologic Inflow

If any method in this category is selected, hydrologic inflow is included in the reservoir mass balance. Optimization assumes hydrologic inflow to be known (data) and it is not solved for by the reservoir regardless of the method selected.

See [“Hydrologic Inflow”](#) in *Objects and Methods* for details.

Optimization Evaporation

This category can be used to model evaporation and precipitation.

None

This is the default method for the Optimization Evaporation category. It performs no calculations, and requires that None also be selected for the Evaporation and Precipitation category.

Opt Input Evaporation

This method is analogous to the Input Evaporation method in simulation and requires the Input Evaporation method to be selected for the Evaporation and Precipitation category. Evaporation Rate and Precipitation Rate are entered as a time series. Evaporation is calculated as a product of Evaporation Rate, Average Surface Area over the timestep and Timestep length. Similarly Precipitation Volume is calculated as the product of Precipitation Rate, Average Surface Area and Timestep length.

Caution: The linearization of the Surface Area variable can result in a small approximation error in optimization for Evaporation and Precipitation Volume. This means there can be a small difference between the mass balances in the optimization solution and the post-optimization rulebased simulation when using this method. It is important to use care when setting the approximation points for Surface Area in order to reduce this approximation error. Refer to the information on the Surface Area LP Param slot below. Also caution should be used if applying this method at a 1 Month timestep. All rates in the optimization mass balance are converted to monthly volumes based on a 30-day month, regardless of the month. This will also produce a difference between the mass balances in the optimization solution and the post-optimization rulebased simulation for a 1 Month timestep.

Slots Specific to This Method

Elevation Area Table

Type:	Table Slot
Units:	Length vs Area
Description:	Represents the Elevation-Surface Area relationship
Information:	This table must be input. It is used to derive the Volume Area Table.
Defined by:	User input

Evaporation

Type:	Series Slot
Units:	Volume
Description:	The volume of water lost to evaporation over the timestep
Information:	If this slot contains user input, it is added directly to the mass balance constraint, otherwise it is defined by the expression below.
Defined by:	Either user input or the following constraint: $\text{Evaporation} = \text{EvaporationRate} \times \frac{\text{SurfaceArea}(t-1) + \text{SurfaceArea}}{2} \times \text{Timestep}$

• Evaporation Rate

Type:	Series Slot
Units:	Velocity
Description:	The rate, in length per time, at which water is lost to evaporation at each timestep
Information:	This slot can be set as user input. If it is not set as an input, and if Evaporation is not an input, this slot defaults to zero. If Evaporation is an input, this slot is not used.
Defined by:	User input or defaults to zero

• Precipitation Rate

Type:	Series Slot
Units:	Velocity
Description:	The rate, in length per time, at which water is gained from precipitation at each timestep
Information:	This slot can be set as user input. If it is not set as an input, it defaults to zero.
Defined by:	User input or defaults to zero

• Precipitation Volume

Type:	Series Slot
Units:	Volume
Description:	The volume of water gained from precipitation over the timestep
Information:	The Input Evaporation method will not allow this slot to be set as an input.
Defined by:	Explicit constraint as follows: $\text{Precipitation Volume} = \text{PrecipitationRate} \times \frac{\text{SurfaceArea}(t-1) + \text{SurfaceArea}}{2} \times \text{Timestep}$

• Surface Area

Type:	Series Slot
Units:	Area
Description:	The area of the water surface at the end of the timestep
Information:	This slot is numerically approximated as a function of Storage (Numerical 2-D Approximation). The relationship between Pool Elevation and Storage comes from the automatically generated Volume Area Table. The table is queried using the user-input points defined in the Surface Area LP Param table.
Defined by:	Numerical 2-D Approximation in terms of Storage, based upon the Volume Area Table. The Surface Area LP Param table values are used as approximation points indexing the Volume Area Table. The preferred order of approximation is substitution, piecewise, two-point line, tangent. Most often the two-point line (secant) approximation will be used.

• Surface Area LP Param

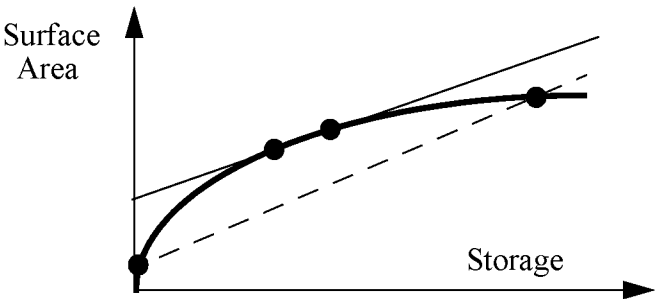
Type:	Table Slot
Units:	Volume
Description:	Specifies the Storage points used to take the tangent, line and piecewise approximations for Surface Area linearization

Information: The best Storage point to choose for tangent approximation would be the expected storage during the run; for line approximation, the expected maximum and minimum Storage; for piecewise approximation, use points that cover the full range of expected Storage during the run with intermediate points such that a piecewise linear curve reasonably approximates the actual curve. In most cases, the line (secant) approximation will be used. It is important to set these points carefully to minimize approximation error.

Defined by: User input

The following figure represents two alternative selections of points for the line approximation in the Surface Area LP Param table. The linear approximation represented by the dashed line corresponds to the selection of points near the extremes of the Volume Area table. This approximation will tend to result in an under-estimation of Surface Area, and thus an under-estimation of Evaporation and Precipitation Volume. Evaporation losses in the post-optimization rulebased simulation would be greater than the losses approximated in the optimization solution. The linear approximation represented by the solid line corresponds to the selection of points closer together in the Volume Area Table, and is more similar to the Tangent approximation. This approximation would tend to result in an over estimation of Surface Area, and the losses due to Evaporation in the post-optimization rulebased simulation would be less than the approximation in the optimization solution.

Surface area vs storage relationship with two alternative line approximations. Not drawn to scale.



• **Volume Area Table**

Type: Table Slot

Units: Volume vs Area

Description: Represents the Storage Volume-Surface Area relationship

Information: This table is read-only and is automatically generated at the start of the run from the Elevation Area Table and the Elevation Volume table. The method starts by copying the Elevation Area Table and then replaces the Pool Elevation Values with the corresponding Storage values. The Storage values are linearly interpolated based on the Elevation Volume Table

Defined by: Automatically generated

Optimization Future Value

This category allows the optimization to provide slots relating to the future value of water. Its appearance is dependent on selecting the Cumulative Storage Value Table method for the Future Value category.

■ None

If this method is selected, the future value slots will not be visible, and no linearization will be attempted.

■ Opt Cumulative Storage Value Table

If this method is selected, linearization are allowed.

Slots Specific to This Method

- **Cumul Stor Val LP Param**

Type:	Table Slot
Units:	Volume, Volume, Volume
Description:	Specifies the storage points used to take the tangent, line and piecewise approximations for Cumul Stor Val Table linearization
Information:	
Defined by:	User input or by automated procedure if Cumul Stor Val Linearization Automation category has selected the Marginal Value to Table and Lin method.

Optimization Power

The Optimization Power category allows the user to select which type of linearizations will be used for linearizing various slots. For Independent Linearizations all slots are linearized separately according to the user-specified methods. This category has no dependencies.

■ None

This is the default method. No slots are added for this method.

Note: It is an error to have this method selected for Optimization.

■ Independent Linearizations

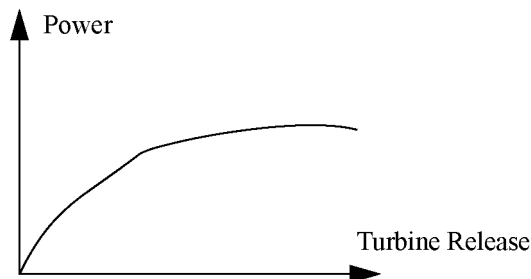
The Independent Linearizations method linearizes all the variables separately according to the user-selected methods. The variables that need to be linearized vary greatly depending on the other methods selected.

See the various Categories and Linearization Approaches for details.

Slots Specific to This Method

• Power LP Param

Type:	Table Slot
Units:	Length and Flow
Description:	Specifies the Operating Head and the flow points used to take the tangent, line and piecewise approximations for Power linearization
Information:	This table must be user input when the Independent Linearizations method is selected. The best Operating Head to choose should be close to the expected head during optimized period. Tangent approximation is generally not used, nor helpful as it often results in non-zero power for zero flow. The suggested points for a line approximation are 0 flow and best turbine flow for the entire plant. The suggested points for piecewise linearization are 0 flow, 1 unit best turbine flow, 2 units best turbine flow ... n units best turbine flow, and maximum turbine flow.
Defined by:	Input Power turbine release relationship for a fixed operating head valve. Not drawn to scale.



Method Details

If the Optimization problem uses Power, either directly or indirectly, the Power slot is added to the problem.

Before being passed to the optimization solver, the Power slot is numerically approximated as a function of Operating Head and Turbine Release (Numerical 3-D Approximation).

The relationship between Operating Head and Turbine Release comes from the Plant Power Table. This table must be user input. The table is queried using the points defined in the Power LP Param table.

■ Power Coefficient

The Power Coefficient method models Power at each time step as follows:

$$Power = \text{Turbine Release} \times \text{Power Coefficient Estimate}$$

Slots Specific to This Method

• Power Coefficient Estimate

Type:	Series Slot
Units:	PowerPerFlow

Description:	This represents the estimated Power Coefficient that gets used in the Optimization definition of Power.
Information:	The Power Coefficient Estimate is a required input for the run. It can either be input directly (manually or by DMI), or it could be set by an initialization rule. For example, an initialization rule could set the Power Coefficient Estimate based on a <i>Seed</i> (Slot Cache) value. If the Power Coefficient method is selected, and Power Coefficient Estimate is not an input for all time steps, then the run terminates with an error message.
Defined by:	Input

■ Power Surface Approximation

This method allows for the modeling of dynamic Operating Head to be incorporated into Optimization Power modeling. It provides a significant improvement in the Power approximation error over the linearization using the Power LP Param table, which assumes a constant Operating Head over the entire run period. The improvement is especially significant for projects that exhibit a wide range of Operating Head over the course of the run, particularly if the optimization policy is trying to maximize Power or set Power greater than or equal to a value.

The method introduces a new variable, PSA Head Factor, which represents the weighted contributions of Storage, Spill and Tailwater Base Value to Operating Head. The method generates a set of planes, the Power Surface, to constrain power as a function of Turbine Release and the PSA Head Factor.

The Power Surface can be thought of as pavement over a warped bridge that is level on one end and sloped and higher on the other end. The bridge ends represent zero Turbine Release and maximum Turbine Release respectively. In addition, the bridge is bowed concave across the center line everywhere except the level (zero Turbine Release) end. This bowing reflects Power as a function of the Head Factor for a fixed value of Turbine Release. The edges of the bridge represent Power as a function of Turbine Release for the low and high values of the Head Factor. Additional points defining a grid on the surface correspond to intermediate values of Turbine Release and the Head Factor. The user can define dimensions and points used for this grid. A piecewise linear surface is composed of cutting planes with each plane defining one triangle on the surface. A surface underneath the bridge composed of two planes defines a lower bound on Power given Turbine Release and Head Factor values.

Because only two planes can be defined for the lower bounds on power, additional approximation error can be introduced if the optimization policy has an incentive to minimize power. In these cases the optimization solution can have an incentive to under-approximate Power for a given Turbine Release, and thus the Post-optimization Rulebased Simulation will calculate a larger Power value than the optimization value for Power.

Slots Specific to This Method

- **PSA Sample Input**

Type:	Table Slot
Units:	Flow, Volume, Flow, Length
Description:	This table contains user-specified sample values for Turbine Release, Storage, Total Spill (if applicable) and Tailwater Base Value (if applicable) that are used by RiverWare to compute the PSA Head Factor Weights. The four parameters in this table represent all of the parameters that contribute to Operating Head. In other words, if a value is known for each of these parameters, it is possible to calculate the corresponding Operating Head.
Information:	The values in this table should span the full possible range for each parameter for the given run. For example, the Turbine Release column should contain a value

of 0, the highest possible turbine release from the reservoir, and any other intermediate values specified by the user. Specifying tighter upper and lower bounds for each parameter will improve the approximation, but it is important that all possible values for each variable within the run are spanned by the range for that parameter in the table. The user can determine how many rows to include in the table. It is expected that typically, three to four values are specified for each parameter. An example is shown below. The Spill column should only contain values if a method other than None has been selected in the Spill category. Otherwise the Spill column should be left empty (displays “NaN”). The Tailwater Base Value column should only contain values if the reservoir’s Tailwater Base Value slot is linked to the Pool Elevation slot of a downstream reservoir (that is, if the Downstream Reservoir Pool Elevation contributes to the calculation of Operating Head). Otherwise the Tailwater Base Value column should be left empty.

Defined by: User input

Turbine Release	Storage	Spill (Total)	Tailwater Base Value
0	0	0	500
100	1000	50	510
200	2000	100	520

• PSA Head Factor Grid Lines

Type: Scalar Slot

Units: None

Description: The number of PSA Head Factor values for each Turbine Release value used when calculating the PSA Grid Points

Information: This must be an integer greater than or equal to 2. The value should typically range from 2 to 4. A larger number will improve the approximation but will increase run time.

Defined by: User input

• PSA Turbine Release Grid Lines

Type: Scalar Slot

Units: None

Description: The number of Turbine Release values for each PSA Head Factor value used when calculating the PSA Grid Points

Information: This must be an integer greater than or equal to 2. The value should typically range from 2 to 4. A larger number will improve the approximation but will increase run time.

Defined by: User input

• PSA Sample Output

Type: Table Slot

Units: Flow, Volume, Flow, Volume, Power

Description: A table containing all permutations of the values entered in the PSA Sample Input table slot along with the Power calculated for each combination of Sample Input values. This sample output is used to calculate the PSA Head Factor Weights.

Information: RiverWare will populate this table at the beginning of the run. For the PSA Sample Input table shown above, with three values in each of the four columns, this table would contain 81 rows, one for each permutation. For a given combination of Turbine Release, Storage, Spill and Tailwater Base Value, RiverWare can calculate the corresponding Power. The third column will contain the Downstream Storage values corresponding to the Tailwater Base Value entries in the PSA Sample Input slot.

Defined by: Populated by RiverWare at the beginning of the run.

For each row i in the table:

$$SamplePower_i = f(SampleTurbineRelease_i, SampleStorage_i, SampleSpill_i, SampleTailwaterBaseValue_i)$$

The calculation of SamplePower depends on the selected Power and Tailwater methods.

• PSA Head Factor Weights

Type: Table Slot

Units: PerTime, None, PerTime

Description: The weights for Storage, Spill and Downstream Storage used when calculating PSA Head Factor

Information: This is a 1 x 3 table with a single weight for Storage, Spill and Downstream Storage. The weights are calculated by RiverWare by performing a regression on the values in the PSA Sample Outputs table slot. If one of the parameters is not used (contains NaN for all rows in the PSA Sample Input table), the corresponding weight in this table slot will be zero.

Defined by: Calculated by regression at the beginning of the run

• PSA Head Factor

Type: Series Slot

Units: Flow

Description: This slot is a variable in the optimization solution and represents the weighted contribution of Storage, Spill and Downstream Storage to Operating Head and thus power. It can be thought of as a storage analog of Operating Head.

Information: This variable is used in the automatically generated constraints that define Power. It will not, generally display any values, but the user can write a post-optimization rule that calculates the value of this variable at each time step using the equation shown below and the values in the PSA Head Factor Weights table slot.

Defined by: $PSA\ Head\ Factor = \alpha \cdot Storage + \beta \cdot Spill + \gamma \cdot Downstream\ Storage$

The coefficients α , β and γ come from the PSA Head Factor Weights table slot.

• PSA Grid Points

Type: Table Slot

Units: Flow, Volume, Flow, Volume, Power, Flow

Description: Points used to calculate the planes for the PSA Max Constraints and PSA Min Constraints, one row for each point with columns for Turbine Release, Storage, Spill, Downstream Storage, Power and PSA Head Factor.

Turbine Release Column—Smallest and largest values from PSA Sample Input and evenly spaced intermediate values corresponding to the number of lines in the PSA Turbine Release Grid Lines scalar slot.

Storage, Spill and Downstream Storage Columns—Smallest and largest values from PSA Sample Input and evenly spaced intermediate values corresponding to the number of lines in the PSA Head Factor Grid Lines scalar slot.

Power Column—Determined by selected Power method, for each row i in the table.

$$Power_i = f(TurbineRelease_i, Storage_i, Spill_i, DownstreamStorage_i)$$

Head Factor Column—The coefficients α , β and γ come from the PSA Head Factor Weights table slot.

$$HeadFactor_i = \alpha \cdot Storage_i + \beta \cdot Spill_i + \gamma \cdot DownstreamStorage_i$$

Information: The number of rows (points) in this table equals the product of the number of lines in the PSA Head Factor Grid Lines and PSA Turbine Release Grid Lines scalar slots. For each grid point, RiverWare calculates the value of Power and the PSA Head Factor corresponding to the parameter values in that row.

Defined by: Populated by RiverWare at the beginning of the run

• PSA Max Constraints

Type: Table Slot

Units: Flow, Flow, Flow, Flow, Flow, FLOW, PowerPerFlow, PowerPerFlow, Power

Description: This table contains the planes that define the upper bounds for power as a function of Turbine Release and PSA Head Factor, one row for each plane. Power is constrained to be less than or equal to the planes defined in this table. The first six columns contain three pairs of Turbine Release and PSA Head Factor values that define three points on the plane. The remaining three columns contain the corresponding Turbine Release Coefficient, Head Factor Coefficient and Constant Term that define the same plane and are used in the actual constraint expressions.

Information: The points used to define the planes come from the Turbine Release and Head Factor points in the PSA Grid Points table slot. RiverWare will not use all of the possible combinations of Turbine Release and Head Factor points but rather will only use the combinations necessary to define the Power Surface.

Defined by: Populated by RiverWare at the beginning of the run

For each row i in the PSA Max Constraints table slot, a constraint is added to the optimization problem, as follows:

$$Power \leq u_i \cdot \text{Turbine Release} + v_i \cdot \text{PSA Head Factor} + w_i$$

The coefficients u_i , v_i and w_i are the Turbine Release Coefficient, Head Factor Coefficient and Constant Term from row i of the PSA Max Constraints table. Turbine Release and PSA Head Factor are the actual variables in the optimization problem.

• PSA Min Constraints

Type: Table Slot

Units: Flow, Flow, Flow, Flow, Flow, FLOW, PowerPerFlow, PowerPerFlow, Power

Description: This table contains the planes that define the lower bounds for power as a function of Turbine Release and PSA Head Factor, one row for each plane. Power is constrained to be greater than or equal to the planes defined in this table. The first

	six columns contain three pairs of Turbine Release and PSA Head Factor values that define three points on the plane. The remaining three columns contain the corresponding Turbine Release Coefficient, Head Factor Coefficient and Constant Term that define the same plane and are used in the actual constraint expressions
Information:	This table will always contain two rows. Only two planes can be defined for the lower bounds on power.
Defined by:	Populated by RiverWare at the beginning of the run. For each row i in the PSA Min Constraints table slot, a constraint is added to the optimization problem, as follows: $Power \geq u_i \cdot \text{Turbine Release} + v_i \cdot \text{PSA Head Factor} + w_i$ where the coefficients u_i, v_i and w_i are the Turbine Release Coefficient, Head Factor Coefficient, and Constant Term from row i of the PSA Min Constraints table. Turbine Release and PSA Head Factor are the actual variables in the optimization problem.

Method Details

The following slots require input from the user:

- [PSA Sample Input](#), page 107
- [PSA Head Factor Grid Lines](#), page 108
- [PSA Turbine Release Grid Lines](#), page 108

RiverWare carries out the following steps to populate the remaining table slots associated with this method.

1. Populate the PSA Sample Output Table with all permutations of parameters in the PSA Sample Input table slot.
2. Calculate the PSA Head Factor Weights using a regression based on values in the PSA Sample Output table slot.
3. Add defining constraints for PSA Head Factor using the PSA Head Factor Weights.
4. Populate the PSA Grid Points table slot based on the number of grid lines specified by the user.
5. Generate upper bound planes using the PSA Grid Points and add the corresponding constraints on Power to the Optimization problem.
6. Generate lower bound planes using the PSA Grid Points and add the corresponding constraints on Power to the Optimization problem.

Power Surface Definition Details

This section provides details of the calculations carried out by RiverWare to generate the constraints that define the Power Surface. This material is included for reference only.

1. RiverWare populates the PSA Sample Output table slot. All permutations of values in the PSA Sample Input table slot are determined, and a row for each is added to the PSA Sample Output table slot. RiverWare may make some adjustments to prevent infeasible combinations. For example, the highest Turbine Release value may not be possible for all combinations of Storage, Spill and Tailwater Base Value. RiverWare will adjust the Turbine Release to use the maximum turbine release for the resulting Operating Head. The table is organized in blocks of constant Turbine Release, with the lowest Turbine Release block first and then increasing Turbine Release. The values in the remaining columns are ordered corresponding to increasing Operating Head. Storage

Chapter 5

Optimization Objects and Methods

is increasing. Spill and Tailwater Base Value (or Downstream Storage) are decreasing. For each combination of values (each row), RiverWare calculates the corresponding Operating Head and Power. The Power value is added to each row in the PSA Sample Output table. The details of these calculations are shown here for a given row i in the PSA Sample Output table.

Note: $Power_i$ is the only calculated quantity displayed in the table. Values from the intermediate calculations shown below are not displayed in the table.

$$PoolElevation_i = f(Storage_i)$$

Pool Elevation is calculated by interpolation on the reservoir Elevation Volume Table.

If Tailwater Base Value is linked to Pool Elevation of a downstream reservoir, as follows:

$$TailwaterBaseValue_i = DownstreamPoolElevation_i = f(DownstreamStorage_i)$$

Tailwater Base Value is calculated by interpolation on the downstream reservoir Elevation Volume Table. Tailwater Base Value is only calculated from the Downstream Storage if either Base Value Plus Lookup Table, Stage Flow Lookup Table or Coefficients Table is the selected Tailwater method *and* Tailwater Base Value is linked to the Pool Elevation of the downstream reservoir. Otherwise Tailwater Base Value is unused, and thus Operating Head and Power are not dependent on Downstream Storage.

$$Outflow_i = TurbineRelease_i + Spill_i$$

Spill will only be non-zero if a method other than None is selected in the Spill category.

$$TailwaterElevation_i = f(Outflow_i, TailwaterBaseValue_i)$$

The tailwater calculation is dependent on the selected method in the Tailwater category.

$$OperatingHead_i = PoolElevation_i - TailwaterElevation_i$$

$$Power_i = f(TurbineRelease_i, OperatingHead_i)$$

The Power calculation is dependent on the selected method in the Power category.

- RiverWare calculates the PSA Head Factor Weights using a combination of linear regression and averages.

FOR EACH Turbine Release value i

FOR EACH combination of Spill and Tailwater Base Value (or Downstream Storage), k , perform a linear regression to calculate a temporary Storage coefficient g_{ik} :

$$g_{ik} = \frac{n \sum (Power_j \times Storage_j) - \left(\left(\sum Power_j \right) \left(\sum Storage_j \right) \right)}{n \left(\sum Storage_j^2 \right) - \left(\sum Storage_j \right)^2}$$

where n is the number of Storage values.

Calculate the temporary Storage coefficient for the given Turbine Release, g_i :

$$g_i = \frac{\sum g_{ik}}{n}$$

where n is the number of Spill and Downstream Storage combinations.

FOR EACH combination of Storage and Downstream Storage, k , perform a linear regression to calculate a temporary Spill coefficient e_{ik} :

$$e_{ik} = \frac{n \sum (Power_j \times Spill_j) - \left(\left(\sum Power_j \right) \left(\sum Spill_j \right) \right)}{n \left(\sum Spill_j^2 \right) - \left(\sum Spill_j \right)^2}$$

where n is the number of Spill values.

Calculate the temporary Spill coefficient for the given Turbine Release, e_i :

$$e_i = \frac{\sum e_{ik}}{n}$$

where n is the number of Storage and Downstream Storage combinations.

FOR EACH combination of Storage and Spill, k , perform a linear regression to calculate a temporary Downstream Storage coefficient f_{ik} :

$$f_{ik} = \frac{n \sum (Power_j \times DownstreamStorage_j) - \left(\left(\sum Power_j \right) \left(\sum DownstreamStorage_j \right) \right)}{n \left(\sum DownstreamStorage_j^2 \right) - \left(\sum DownstreamStorage_j \right)^2}$$

where n is the number of Downstream Storage values.

Calculate the temporary Downstream Storage coefficient for the given Turbine Release, f_i :

$$f_i = \frac{\sum f_{ik}}{n}$$

where n is the number of Storage and Spill combinations.

Calculate average ratios over all Turbine Release values:

$$c = \frac{\sum (e_i / g_i)}{n}$$

$$d = \frac{\sum (f_i / g_i)}{n}$$

where n is the number of Turbine Release values.

Calculate the final Head Factor Weights:

$$\text{Storage Weight} = \frac{1}{c}$$

$$\text{Spill Weight} = -1$$

If Tailwater Base Value is linked to the downstream Pool Elevation:

$$\text{Downstream Storage Weight} = \frac{d}{c}$$

Chapter 5

Optimization Objects and Methods

These three weight values are displayed in the three columns of the PSA Head Factor Weights table slot.

In the optimization solution the PSA Head Factor is defined at a given time step by

If Tailwater Base Value is linked to the downstream Pool Elevation:

$$\text{PSA Head Factor} = \text{StorageWeight} \times \text{Storage} + \text{SpillWeight} \times \text{Spill} + \text{DownstreamStorageWeight} \times \text{Downstream Storage}$$

If Tailwater Base Value is not linked:

$$\text{PSA Head Factor} = \text{StorageWeight} \times \text{Storage} + \text{SpillWeight} \times \text{Spill}$$

- RiverWare calculates the grid points for the PSA Grid Points table slot. The number of rows (points) in this table equals the product of the number of lines in the PSA Head Factor Grid Lines and PSA Turbine Release Grid Lines scalar slots. The smallest and largest values in the PSA Sample Input slot lead to the smallest and largest values for each of the parameters that affect power (Turbine Release, Storage, Spill and Downstream Storage). One row in this table will correspond to the smallest Turbine Release, smallest Storage, largest Spill and largest Downstream Storage, and thus smallest power. Another row will correspond to the largest Turbine Release, largest Storage, smallest Spill and smallest Downstream Storage, and thus the largest Power. RiverWare generates evenly spaced values between the extremes for each variable. Within the PSA Grid Points table, for a given Turbine Release value, Storage is increasing and Spill and Downstream Storage are decreasing. For each grid point, RiverWare calculates the value of Power and the PSA Head Factor corresponding to the parameter values in each row and adds these to the final two columns in the table.

For each row i in the table:

$$\text{Power}_i = f(\text{TurbineRelease}_i, \text{Storage}_i, \text{Spill}_i, \text{DownstreamStorage}_i)$$

The Power calculation depends on the method selected in the Power category.

$$\text{HeadFactor}_i = \alpha \cdot \text{Storage}_i + \beta \cdot \text{Spill}_i + \gamma \cdot \text{DownstreamStorage}_i$$

The coefficients α , β and γ come from the PSA Head Factor Weights table slot.

- RiverWare calculates the upper bound planes. These are entered in the PSA Max Constraints table slot, one row for each plane, where the planes represent Power as a function of Turbine Release and PSA Head Factor. For each combination of three points in the PSA Grid Points Table, RiverWare calculates the corresponding plane. For any three points, the plane is defined by three linear equations, which can be represented in matrix form.

$$\bar{P} = A\bar{u}$$

$$\bar{P} = \begin{bmatrix} P_1 \\ P_2 \\ P_3 \end{bmatrix} \quad A = \begin{bmatrix} QT_1 & HF_1 & 1 \\ QT_2 & HF_2 & 1 \\ QT_3 & HF_3 & 1 \end{bmatrix} \quad \bar{u} = \begin{bmatrix} u \\ v \\ w \end{bmatrix}$$

The coefficients that define the plane are solved for by

$$\bar{u} = A^{-1}\bar{P}$$

RiverWare only uses the planes that are necessary to define the Power Surface. Each plane is checked against all remaining points in the PSA Grid Points table. If the Power value for any of the remaining points lies above the given plane, then that plane is rejected (that is, it would over-constrain Power, resulting in an under-estimation

of Power). For each plane i that is used, RiverWare adds the following constraint to the optimization problem for each time step:

$$Power \leq u_i \cdot \text{Turbine Release} + v_i \cdot \text{PSA Head Factor} + w_i$$

5. RiverWare calculates the lower bound planes. These are entered in the PSA Min Constraints table slot, one row for each plane. Only two planes can be calculated for the lower bounds. The points used for the two planes are as follows:

Plane 1:

- Point A1 - Min Turbine Release, Max Head Factor
- Point B1 - Max Turbine Release, Max Head Factor
- Point C1 - Min Turbine Release, Min Head Factor

Plane 2:

- Point A2 - Min Turbine Release, Max Head Factor
- Point B2 - Max Turbine Release, Max Head Factor
- Point C2 - Max Turbine Release, Max Head Factor

The coefficients defining the planes are calculated using the same matrix calculation described for the upper bound planes. For each lower bound plane i , RiverWare adds the following constraint to the optimization problem for each time step:

$$Power \geq u_i \cdot \text{Turbine Release} + v_i \cdot \text{PSA Head Factor} + w_i$$

Note: Regarding the lower bound approximation, because only two planes can be defined for the lower bounds on power, additional approximation error can be introduced if the optimization policy has an incentive to minimize power. This can occur if Power is included in a Minimize objective. It can also occur if the optimization problem contains constraints that require Power to be less than or equal to value.

Note: These types of constraints can be introduced through multiple forms of RPL optimization goals. The first is basic less than or equal to constraint.

ADD CONSTRAINT Res.Power[t] ≤ Value

Policy that requires Power to be equal to a value also adds a less than or equal to constraint. The following RPL statement:

ADD CONSTRAINT Res.Power[t] = Value

actually adds the following two constraints to the optimization problem:

$$\text{Res.Power}[t] \leq \text{Value}$$

$$\text{Res.Power}[t] \geq \text{Value}$$

A constraint to require the sum of Power from multiple reservoirs to be less than or equal to a value will have a similar effect. A final manner in which this can occur is through policy that minimizes Power indirectly through user-defined variables. For example, assume a user-defined variable is defined by the following RPL constraint:

ADD CONSTRAINT Res.Power[t] + UserVariable[t] = Value1

Then later the variable is constrained by the following:

```
ADD CONSTRAINT UserVariable[t] ≥ Value2
```

An equivalent constraint would be as follows:

```
ADD CONSTRAINT Res.Power[t] ≤ Value1 – Value2
```

It is possible to reduce the approximation error in these cases by adding constraints to the RPL Optimization Goal Set that essentially create more restrictive planes for the lower bound. These types of constraints are model-specific. Contact CADSWES if assistance is required to add more restrictive lower bounds to the Power Surface Approximation.

Optimization Reserves

This category depends on selecting either Plant Power Coefficient or Plant Efficiency Curve in the Power category. The methods in this category can be used to account for power reserve requirements in the optimization policy. There is no analogous simulation method associated with this category.

■ None

This is the default, no action method. No new slots will be added, and no new variables or constraints will be added to the optimization problem.

■ Constraint Based Single Timestep

This method introduces a set of duplicate variables that represent the value of standard variables assuming full deployment of either upward or downward reserves. In the RPL Optimization Goal Set, users can then write parallel constraints using the duplicate reserve variables for any constraint that cannot be violated while deploying reserves. The purpose of the new variables and their parallel constraints is so that the only reserves that are credited to a hydropower project are those that can actually be reasonably deployed. In other words, it will not *count* reserves that could not be deployed without violating specified constraints. The new method also introduces new variables for upward and downward reserves, which allow the user to write policy on the reserves themselves, for example, project minimum reserve requirements or total system reserve requirements.

One significant assumption is made that the deployment of these reserves are temporary and only meant to cover a single timestep. It is assumed that in the event reserves are deployed in actual operations, adjustments would be made for later time periods, including reestablishing reserves. With this assumption, the consequences of deploying reserves at a reservoir are limited to that reservoir and that timestep; constraints on downstream reservoirs and later timesteps are not considered when calculating the reserves that can be credited at a given timestep.

The deployment variables are constrained similarly to the original variables with two exceptions. First, the mass balance constraints with other reservoirs or intermediate reaches are omitted (within RiverWare). Second, if there are any constraints that can be violated during deployment then these constraints can also be omitted (within the Optimization Goal Set). The deployment variables will typically use the same linear approximations as the original variables. For example, Pool Elevation with Up Reserve will be approximated using the Pool Elevation LP Param table slot.

Note: This method has no analogous simulation method. The slots introduced by this method can be referenced in the RPL optimization policy to constrain the solution, but they will not, generally, display values after the post-optimization rulebased simulation like the standard variables, such as Outflow or Pool Elevation. The user does have the option of writing customized post-optimization rules to set values in these slots based on user-specified logic or using the OptValue or OptValuePiecewise predefined functions.

Slots Specific to This Method

• Up Reserve

Type:	Series Slot
Units:	Power
Description:	The upward reserves available; the amount the reservoir could increase generation on the given timestep
Information:	Introducing this variable in the optimization policy forces a two-point line approximation to be used for both Power and Power with Up Reserve if Independent Linearizations is selected in the Optimization Power category. This may introduce additional approximation error.
Defined by:	$Up\ Reserve = Power\ with\ Up\ Reserve - Power$

• Down Reserve

Type:	Series Slot
Units:	Power
Description:	The downward reserves available; the amount the reservoir could decrease generation on the given timestep
Information:	Introducing this variable forces a two-point line approximation to be used for both Power and Power with Down Reserve if Independent Linearizations is selected in the Optimization Power category. This may introduce additional approximation error.
Defined by:	$Down\ Reserve = Power - Power\ with\ Down\ Reserve$

• Power with Up Reserve

Type:	Series Slot
Units:	Power
Description:	The average Power generated if full upward reserves were deployed
Information:	This variable is defined by the Power approximation as a function of Turbine Release with Up Reserve and a combination of other variables. The set of other variables depends on which Power approximation is being applied. The Power approximation that is applied is dependent on the selected method in the Optimization Power category.
Defined by:	$Power\ with\ Up\ Reserve = f(Turbine\ Release\ with\ Up\ Reserve, \dots)$

• Power with Down Reserve

Type:	Series Slot
Units:	Power

Description: The average Power generated if full downward reserves were deployed
Information: This variable is defined by the Power linear approximation as a function of Turbine Release with Down Reserve and a combination of other variables. The set of other variables depends on which Power approximation is being applied. The Power approximation that is applied is dependent on the selected method in the Optimization Power category.
Defined by: $\text{Power with Down Reserve} = f(\text{Turbine Release with Down Reserve}, \dots)$

- **Turbine Release with Up Reserve**

Type: Series Slot
Units: Flow
Description: The Turbine Release at full deployment of upward reserves
Information: This variable is constrained to be less than or equal to Turbine Capacity with Up Reserve.
Defined by: $\text{Turbine Release with Up Reserve} = \text{Turbine Release} + \text{Turbine Change with Up Reserve}$
and
 $\text{Turbine Release with Up Reserve} \leq \text{Turbine Capacity with Up Reserve}$

- **Turbine Change with Up Reserve**

Type: Series Slot
Units: Flow
Description: The amount Turbine Release would change to fully deploy all upward reserves
Information: This variable is expected to always be non-negative, and thus the slot configuration lower bound should be set to 0.
Defined by: $\text{Turbine Release with Up Reserve} = \text{Turbine Release} + \text{Turbine Change with Up Reserve}$

- **Turbine Capacity with Up Reserve**

Type: Series Slot
Units: Flow
Description: The maximum Turbine Release possible given the full deployment of upward reserves
Information: This variable is defined by the Turbine Capacity approximation as a function of Operating Head with Up Reserve.
Defined by: $\text{Turbine Capacity with Up Reserve} = f(\text{Operating Head with Up Reserve})$

- **Operating Head with Up Reserve**

Type: Series Slot
Units: Flow
Description: The Operating Head at full deployment of upward reserves.
Information:

Defined by: Operating Head with Up Reserve =
$$\frac{\text{Pool Elevation with Up Reserve} + \text{Pool Elevation}[t - 1]}{2} - \text{Tailwater Elevation with Up Reserve}$$

• Spill with Up Reserve

Type: Series Slot
Units: Flow
Description: The amount of total Spill at full deployment of upward reserves
Information: This could be equal to Spill, or it could be less than Spill if flow is allowed to be shifted from Spill to Turbine Release during deployment. This depends on how the RPL policy is formulated.
Defined by: Spill with Up Reserve = Unregulated Spill with Up Reserve + Regulated Spill with Up Reserve + Bypass with Up Reserve

• Unregulated Spill with Up Reserve

Type: Series Slot
Units: Flow
Description: The amount of Unregulated Spill at full deployment of upward reserves
Information: This quantity is defined by the Unregulated Spill linear approximation as a function of Storage with Up Reserve. This slot will get added regardless of which Spill method is selected. If the selected Spill method does not include Unregulated Spill, then the value of this variable will automatically be set to zero.
Defined by: Unregulated Spill with Up Reserve = $f(\text{Storage with Up Reserve})$

• Regulated Spill with Up Reserve

Type: Series Slot
Units: Flow
Description: The amount of Regulated Spill at full deployment of upward reserves
Information: This could be equal to Regulated Spill, or it could be less than Regulated Spill if flow is allowed to be shifted from Spill to Turbine Release during deployment, or it could be greater than Regulated Spill if the reservoir has a minimum spill requirement as a percentage of total outflow. This depends on how the RPL policy is formulated. This slot will get added regardless of which Spill method is selected. If the selected Spill method does not include Regulated Spill, then the value of this variable will automatically be set to zero, and the slot should not be referenced in the RPL optimization policy.
Defined by: Regulated Spill with Up Reserve = Regulated Spill + Regulated Spill Change with Up Reserve
and
Regulated Spill with Up Reserve $\leq$ Regulated Spill Capacity with Up Reserve

• Regulated Spill Change with Up Reserve

Type: Series Slot
Units: Flow
Description: The amount Regulated Spill would change to fully deploy all upward reserves

Information: Often this variable will be negative (or 0, and thus the slot Upper Bound will often be set to 0); however it could be positive if the reservoir has a minimum spill requirement as a percentage of total outflow. It could be constrained to be 0, directly or indirectly, by RPL policy, or it could be allowed to be greater than or less than zero. This slot will get added regardless of which Spill method is selected. If the selected Spill method does not include Regulated Spill, then the slot should not be referenced in the RPL optimization policy.

Defined by: $\text{Regulated Spill with Up Reserve} = \text{Regulated Spill} + \text{Regulated Spill Change with Up Reserve}$

• Regulated Spill Capacity with Up Reserve

Type: Series Slot

Units: Flow

Description: The maximum Regulated Spill given full deployment of upward reserves

Information: This variable is defined by the Regulated Spill Capacity approximation as a function of Storage with Up Reserve. This slot will get added regardless of which Spill method is selected. Users will not typically need to do anything with this slot.

Defined by: $\text{Regulated Spill Capacity with Up Reserve} = f(\text{Storage with Up Reserve})$

• Bypass with Up Reserve

Type: Series Slot

Units: Flow

Description: The amount of Bypass at full deployment of upward reserves

Information: This could be equal to Bypass, or it could be less than Bypass if flow is allowed to be shifted from Bypass to Turbine Release during deployment, or it could be greater than Bypass if the reservoir has a minimum bypass requirement as a percentage of total outflow. This depends on how the RPL policy is formulated. This slot will get added regardless of which Spill method is selected. If the selected Spill method does not include Bypass, then the value of this variable will automatically be set to zero, and the slot should not be referenced in the RPL optimization policy.

Defined by: $\text{Bypass with Up Reserve} = \text{Bypass} + \text{Bypass Change with Up Reserve}$
and

$\text{Bypass with Up Reserve} \leq \text{Bypass Capacity with Up Reserve}$

• Bypass Change with Up Reserve

Type: Series Slot

Units: Flow

Description: The amount Bypass would change to fully deploy all upward reserves

Information: Often this variable will be negative (or 0, and thus the slot Upper Bound will often be set to 0); however it could be positive if the reservoir has a minimum bypass requirement as a percentage of total outflow. It could be constrained to be 0, directly or indirectly, by RPL policy, or it could be allowed to be greater than or less than zero. This slot will get added regardless of which Spill method is selected. If the selected Spill method does not include Bypass, then the slot should not be referenced in the RPL optimization policy.

Defined by: $\text{Bypass with Up Reserve} = \text{Bypass} + \text{Bypass Change with Up Reserve}$

- **Bypass Capacity with Up Reserve**

Type: Series Slot
Units: Flow
Description: The maximum Bypass given full deployment of upward reserves
Information: This variable is defined by the Bypass Capacity approximation as a function of Storage with Up Reserve. This slot will get added regardless of which Spill method is selected. Users will not typically need to do anything with this slot.
Defined by: $\text{Bypass Capacity with Up Reserve} = f(\text{Storage with Up Reserve})$

- **Outflow with Up Reserve**

Type: Series Slot
Units: Flow
Description: The amount of total Outflow at full deployment of upward reserves
Information:
Defined by: $\text{Outflow with Up Reserve} = \text{Turbine Release with Up Reserve} + \text{Spill with Up Reserve}$

- **Storage with Up Reserve**

Type: Series Slot
Units: Volume
Description: The end of timestep Storage if full upward reserves were deployed
Information: This variable definition differs from the standard Storage variable only in the Outflow term. The previous Storage and the Net Inflow are the same. Net Inflow represents all gains and losses, including Inflow, Hydrologic Inflow, Precipitation, Evaporation, and so on.
Defined by: $\text{Storage with Up Reserve} = \text{Storage}[t - 1] + (\text{NetInflow} - \text{Outflow with Up Reserve}) \times \text{Timestep}$

- **Pool Elevation with Up Reserve**

Type: Series Slot
Units: Length
Description: The end of timestep Pool Elevation if full upward reserves were deployed
Information: This variable is defined by the Pool Elevation linear approximation as a function of Storage with Up Reserve.
Defined by: $\text{Pool Elevation with Up Reserve} = f(\text{Storage with Up Reserve})$

- **Tailwater Elevation with Up Reserve**

Type: Series Slot
Units: Length
Description: The average Tailwater Elevation if full upward reserves were deployed
Information: This variable is defined by the Tailwater Elevation linear approximation as a function of Outflow with Up Reserve and the Tailwater Base Value (if applicable based on the selected Optimization Tailwater method). Tailwater Base Value is unaffected by the Reserves modeling and will be the same for both Tailwater Elevation and Tailwater Elevation with Up Reserves. The Tailwater Elevation

linear approximation that is applied is dependent on the selected method in the Optimization Tailwater category.

Defined by: Tailwater Elevation with Up Reserve = $f(\text{Outflow with Up Reserve, Tailwater Base Value})$

- **Temp Tailwater Lookup with Up Reserve**

Type: Series Slot

Units: Length

Description: This slot only applies when the Base Value Plus Lookup Table method is selected in the Tailwater category. It represents the lookup value from the Tailwater Table as a function of Outflow with Up Reserve using the Tailwater linear approximation.

Information: This slot will get added regardless of which Tailwater method is selected but will not get used unless the Base Value Plus Lookup method is selected. Users should not need to do anything with this slot.

Defined by: Temp Tailwater Lookup with Up Reserve = $f(\text{Outflow with Up Reserve})$

- **PSA Head Factor with Up Reserve**

Type: Series Slot

Units: Flow

Description: This slot only applies when the Power Surface Approximation method is selected in the Optimization Power category. It represents the component of Operating Head that is dependent on Storage with Up Reserve and Spill with Up Reserve. It is calculated using the same coefficients used for the Standard PSA Head Factor variable.

Information: This slot will get added regardless of which Optimization Power method is selected but will not get used unless the Power Surface Approximation method is selected. Users should not need to do anything with this slot.

Defined by: PSA Head Adjustment with Up Reserve = $f(\text{Storage with Up Reserve, Spill with Up Reserve})$

- **Turbine Release with Down Reserve**

Type: Series Slot

Units: Flow

Description: The Turbine Release at full deployment of downward reserves

Information:

Defined by: Turbine Release with Down Reserve = Turbine Release + Turbine Change with Down Reserve
and

Turbine Release with Down Reserve $\leq$ Turbine Capacity with Down Reserve

- **Turbine Change with Down Reserve**

Type: Series Slot

Units: Flow

Description: The amount Turbine Release would change to fully deploy all downward reserves

Information: This variable is expected to always be negative or zero, and thus the slot configuration Upper Bound should be set to 0.

Defined by: Turbine Release with Down Reserve = Turbine Release + Turbine Change with Down Reserve

• Turbine Capacity with Down Reserve

Type: Series Slot
Units: Flow
Description: The maximum Turbine Release possible given the full deployment of downward reserves
Information: This variable is defined by the Turbine Capacity approximation as a function of Operating Head with Down Reserve.
Defined by: Turbine Capacity with Down Reserve = $f(\text{Operating Head with Down Reserve})$

• Operating Head with Down Reserve

Type: Series Slot
Units: Flow
Description: The Operating Head at full deployment of downward reserves.
Information:
Defined by: Operating Head with Down Reserve = $\frac{\text{Pool Elevation with Down Reserve} + \text{Pool Elevation}[t - 1]}{2}$
 – Tailwater Elevation with Down Reserve

• Spill with Down Reserve

Type: Series Slot
Units: Flow
Description: The amount of total Spill at full deployment of downward reserves
Information: This could be equal to Spill, or it could be greater than Spill if flow is allowed to be shifted from Turbine Release to Spill during deployment. This depends on how the RPL policy is formulated.
Defined by: Spill with Down Reserve = Unregulated Spill with Down Reserve + Regulated Spill with Down Reserve + Bypass with Down Reserve

• Unregulated Spill with Down Reserve

Type: Series Slot
Units: Flow
Description: The amount of Unregulated Spill at full deployment of downward reserves
Information: This quantity is defined by the Unregulated Spill linear approximation as a function of Storage with Down Reserve. This slot will get added regardless of which Spill method is selected. If the selected Spill method does not include Unregulated Spill, then the value of this variable will automatically be set to zero.
Defined by: Unregulated Spill with Down Reserve = $f(\text{Storage with Down Reserve})$

• Regulated Spill with Down Reserve

Type: Series Slot
Units: Flow
Description: The amount of Regulated Spill at full deployment of downward reserves

- Information:** This could be equal to Regulated Spill, or it could be greater than Regulated Spill if flow is allowed to be shifted from Turbine Release to Spill during deployment. This depends on how the RPL policy is formulated. This slot will get added regardless of which Spill method is selected. If the selected Spill method does not include Regulated Spill, then the value of this variable will automatically be set to zero, and the slot should not be referenced in the RPL optimization policy.
- Defined by:** $\text{Regulated Spill with Down Reserve} =$
 $\text{Regulated Spill} + \text{Regulated Spill Change with Down Reserve}$
 and
 $\text{Regulated Spill with Down Reserve} \leq \text{Regulated Spill Capacity with Down Reserve}$

• Regulated Spill Change with Down Reserve

- Type:** Series Slot
- Units:** Flow
- Description:** The amount Regulated Spill would change to fully deploy all downward reserves
- Information:** This variable will typically be non-negative. It could be constrained to be 0, directly or indirectly, by RPL policy, or it could be allowed to be greater than zero. This slot will get added regardless of which Spill method is selected. If the selected Spill method does not include Regulated Spill, then the slot should not be referenced in the RPL optimization policy.
- Defined by:** $\text{Regulated Spill with Down Reserve} =$
 $\text{Regulated Spill} + \text{Regulated Spill Change with Down Reserve}$

• Regulated Spill Capacity with Down Reserve

- Type:** Series Slot
- Units:** Flow
- Description:** The maximum Regulated Spill given full deployment of downward reserves
- Information:** This variable is defined by the Regulated Spill Capacity approximation as a function of Storage with Down Reserve. This slot will get added regardless of which Spill method is selected. Users will not typically need to do anything with this slot
- Defined by:** $\text{Regulated Spill Capacity with Down Reserve} = f(\text{Storage with Down Reserve})$

• Bypass with Down Reserve

- Type:** Series Slot
- Units:** Flow
- Description:** The amount of Bypass at full deployment of downward reserves
- Information:** This could be equal to Bypass, or it could be greater than Bypass if flow is allowed to be shifted from Turbine Release to Bypass during deployment. This depends on how the RPL policy is formulated. This slot will get added regardless of which Spill method is selected. If the selected Spill method does not include Bypass, then the value of this variable will automatically be set to zero, and the slot should not be referenced in the RPL optimization policy.
- Defined by:** $\text{Bypass with Down Reserve} = \text{Bypass} + \text{Bypass Change with Down Reserve}$
 and
 $\text{Bypass with Down Reserve} \leq \text{Bypass Capacity with Down Reserve}$

• Bypass Change with Down Reserve

Type:	Series Slot
Units:	Flow
Description:	The amount Bypass would change to fully deploy all downward reserves
Information:	This variable will typically be non-negative. It could be constrained to be 0, directly or indirectly, by RPL policy, or it could be allowed to be greater than zero. This slot will get added regardless of which Spill method is selected. If the selected Spill method does not include Bypass, then the slot should not be referenced in the RPL optimization policy.
Defined by:	$\text{Bypass with Down Reserve} = \text{Bypass} + \text{Bypass Change with Down Reserve}$

• Bypass Capacity with Down Reserve

Type:	Series Slot
Units:	Flow
Description:	The maximum Bypass given full deployment of downward reserves
Information:	This variable is defined by the Bypass Capacity approximation as a function of Storage with Down Reserve. This slot will get added regardless of which Spill method is selected. Users will not typically need to do anything with this slot
Defined by:	$\text{Bypass Capacity with Down Reserve} = f(\text{Storage with Down Reserve})$

• Outflow with Down Reserve

Type:	Series Slot
Units:	Flow
Description:	The amount of total Outflow at full deployment of downward reserves
Information:	
Defined by:	$\text{Outflow with Down Reserve} = \text{Turbine Release with Down Reserve} + \text{Spill with Down Reserve}$

• Storage with Down Reserve

Type:	Series Slot
Units:	Volume
Description:	The end of timestep Storage if full downward reserves were deployed
Information:	This variable definition differs from the standard Storage variable only in the Outflow term. The previous Storage and the Net Inflow are the same. Net Inflow represents all gains and losses, including Inflow, Hydrologic Inflow, Precipitation, Evaporation, and so on.
Defined by:	$\text{Storage with Down Reserve} = \text{Storage}[t - 1] + (\text{NetInflow} - \text{Outflow with Down Reserve}) \times \text{Timestep}$

• Pool Elevation with Down Reserve

Type:	Series Slot
Units:	Length
Description:	The end of timestep Pool Elevation if full downward reserves were deployed
Information:	This variable is defined by the Pool Elevation linear approximation as a function of Storage with Down Reserve.

Defined by: Pool Elevation with Down Reserve = $f(\text{Storage with Down Reserve})$

- **Tailwater Elevation with Down Reserve**

Type: Series Slot

Units: Length

Description: The average Tailwater Elevation if full downward reserves were deployed

Information: This variable is defined by the Tailwater Elevation linear approximation as a function of Outflow with Down Reserve and the Tailwater Base Value (if applicable based on the selected Optimization Tailwater method). Tailwater Base Value is unaffected by the Reserves modeling and will be the same for both Tailwater Elevation and Tailwater Elevation with Up Reserves. The Tailwater Elevation linear approximation that is applied is dependent on the selected method in the Optimization Tailwater category.

Defined by: Tailwater Elevation with Down Reserve = $f(\text{Outflow with Down Reserve, Tailwater Base Value})$

- **Temp Tailwater Lookup with Down Reserve**

Type: Series Slot

Units: Length

Description: This slot only applies when the Base Value Plus Lookup Table method is selected in the Tailwater category. It represents the lookup value from the Tailwater Table as a function of Outflow with Down Reserve using the Tailwater linear approximation.

Information: This slot will get added regardless of which Tailwater method is selected but will not get used unless the Base Value Plus Lookup method is selected. Users should not need to do anything with this slot.

Defined by: Temp Tailwater Lookup with Down Reserve = $f(\text{Outflow with Down Reserve})$

- **PSA Head Factor with Down Reserve**

Type: Series Slot

Units: Flow

Description: This slot only applies when the Power Surface Approximation method is selected in the Optimization Power category. It represents the component of Operating Head that is dependent on Storage with Down Reserve and Spill with Up Reserve. It is calculated using the same coefficients used for the Standard PSA Head Factor variable.

Information: This slot will get added regardless of which Optimization Power method is selected but will not get used unless the Power Surface Approximation method is selected. Users should not need to do anything with this slot.

Defined by: PSA Head Adjustment with Down Reserve = $f(\text{Storage with Down Reserve, Spill with Down Reserve})$

Method Details

Sample RPL Goal Set Implementation

Within a RPL Goal set, any constraint that should not be violated when deploying reserves should include a constraint on the standard variables as well as a duplicated constraint on the corresponding reserves variables. For example, assume that a reservoir has a maximum pool elevation that cannot be violated when reserves are

deployed, and that maximum pool elevation is stored in a series slot on a data object called Res_Data.PE_Max. The RPL goal might look like:

```
REPEATED MAXIMIN
  FOR(DATETIME t IN @"Start Timestep" TO @"Finish Timestep") DO
    ADD CONSTRAINT Res.Pool Elevation[t] <= Res_Data.PE_Max[t]
    ADD CONSTRAINT Res.Pool Elevation with Down Reserve[t] <= Res_Data.PE_Max[t]
  END FOR
END REPEATED MAXIMIN
```

The reservoir might also have a target elevation constraint that is a lower priority and can be violated by the deployment of reserves. In this case, the target goal would only include a constraint on the Pool Elevation slot but not a constraint on the Pool Elevation with Down Reserve slot.

Referencing any of the reserve slots in a RPL policy statement will typically draw a large number of additional variables and constraints into the optimization problem. For performance reasons, the user will probably not want these additional constraints to be added unless they are necessary. Assume, for example, that it is known prior to a run that a particular reservoir is not available to provide reserves during select hours. This may be indicated by setting an input value of 0 MW for those hours in a data object series slot called Res_Data.Down_Reserve_Max (similarly for Up Reserve). It would not be necessary to model the reserve versions of constraints for those time steps because it is already known that reserves will be zero. In this case the goal with the Pool Elevation constraints shown previously might be written as:

```
REPEATED MAXIMIN
  FOR(DATETIME t IN @"Start Timestep" TO @"Finish Timestep") DO
    ADD CONSTRAINT Res.Pool Elevation[t] <= Res_Data.PE_Max[t]
    IF(Res_Data.Down_Reserve_Max[t] != 0 MW)
      ADD CONSTRAINT Res.Pool Elevation with Down Reserve[t] <= Res_-
        Data.PE_Max[t]
    END IF
  END FOR
END REPEATED MAXIMIN
```

In this way, the constraint on Pool Elevation with Down Reserve would only be added when the reservoir is available to provide reserves (the max reserves are not constrained to zero).

Users could then write policy on the reserves to be carried at individual projects. Assume that the data object series slots Res_Data.Up_Reserve_Min and Res_Data.Up_Reserve_Max store the minimum upward reserves that the reservoir must carry and the maximum upward reserves the project can be credited respectively. A goal for the reservoir might be:

```
REPEATED MAXIMIN
  FOR(DATETIME t IN @"Start Timestep" TO @"Finish Timestep") DO
    IF(Res_Data.Up_Reserve_Max[t] != 0 MW)
      ADD CONSTRAINT Res.Up Reserve[t] >= Res_Data.Up_Reserve_Min[t]
      ADD CONSTRAINT Res.Up Reserve[t] <= Res_Data.Up_Reserve_Max[t]
    END IF
  END FOR
END REPEATED MAXIMIN
```

Chapter 5

Optimization Objects and Methods

These two constraints may not necessarily be included in the same goal.

Note: The first constraint based on the minimum required reserves will constrain the physical operations of the reservoir in order to meet the required reserve obligation. The second constraint, based on the maximum credited reserves will not directly constrain the physical operations, rather it will only limit the amount of reserves that can be counted. In other words, the solution may operate the reservoir such that, according to physical limits and other specified constraints, the reservoir may have a larger amount of reserves available, but this constraint allows the user to say that only a limited amount will be counted.

Users may then write goals which sum reserves across multiple projects to meet a system reserve requirement.

```
REPEATED MAXIMIN
  FOR(DATETIME t IN @"Start Timestep" TO @"Finish Timestep") DO
    ADD CONSTRAINT (FOR(OBJECT res IN ProjectsAvailableForUpReserve(t)) SUM
      res.Up Reserve[t]
    END FOR)
    >= System_Data.Up_Reserve_Min[t]
  END FOR
END REPEATED MAXIMIN
```

In this goal, ProjectsAvailableForUpReserve represents a user-defined function that returns a list of reservoirs that can carry reserves, again to prevent adding numerous constraints on the reserves version of variables unnecessarily. The details of this function would be model-specific.

Optimization Spill

The Optimization Spill methods determine how spill is calculated for the reservoir and generates physical constraints that correspond to the selected methods.

This category is dependent on selection of the Independent Linearizations method for the Optimization Power category. However, the method selected in the Optimization Spill category must match the corresponding non-optimization method in the Spill category.

Spill is an Optimization decision variable. The following constraint is always generated for a reservoir:

$$\text{Outflow} = \text{Turbine release (or Release)} + \text{Spill} \quad (5.2)$$

The spill methods generate values applicable to the following additional constraint, where some of these terms may be omitted if they do not apply to the selected spill method.

$$\text{Spill} = \text{Regulated Spill} + \text{Unregulated Spill} + \text{Bypass}, \quad (5.3)$$

As applicable, the following constraints are also added:

$$\text{Bypass} \leq \text{Bypass Capacity} \quad (5.4)$$

$$\text{Regulated Spill} \leq \text{Regulated Spill Capacity} \quad (5.5)$$

$$\text{Unregulated Spill} = \text{Unregulated Spill Capacity} \quad (5.6)$$

See “Spill” in *Objects and Methods* for details on non-optimization Spill methods.

Slots Specific to This Method

• Bypass Capacity

Type:	Series Slot
Units:	Flow
Description:	Bypass capacity
Information:	
Defined by:	Numerical 2-D Approximation in terms of storage, based upon the Bypass Capacity Table.

• Bypass Capacity Table

Type:	Table Slot
Units:	Volume vs Flow
Description:	Storage vs corresponding maximum bypass spill values
Information:	
Defined by:	Internally developed based on Bypass Table and Elevation Volume Table relationships. For each pool elevation in the Bypass Table, the Bypass Capacity Table has a row relating Storage to Bypass Capacity. The Pool Elevations are converted to Storage using the Elevation Volume Table.

• Regulated Spill Capacity

Type:	Series Slot
Units:	Flow
Description:	Regulated spill capacity
Information:	
Defined by:	Numerical 2-D Approximation in terms of storage, based upon the Regulated Spill Capacity Table.

• Regulated Spill Capacity Table

Type:	Table Slot
Units:	Volume vs Flow
Description:	Storage vs corresponding maximum regulated spill values
Information:	
Defined by:	Internally developed based on Regulated Spill Table and Elevation Volume Table relationships. For each pool elevation in the Regulated Spill Table, the Regulated Spill Capacity Table has a row relating Storage to Regulated Spill Capacity. The Pool Elevations are converted to Storage using the Elevation Volume Table.

• Regulated Spill or Bypass LP Param

Type:	Table Slot
--------------	------------

Units:	Volume
Description:	Specifies the Storage points use to take the tangent, line and piecewise approximations for Regulated Spill Capacity linearization and Bypass Spill Capacity linearization.
Information:	
Defined by:	User input

- **Unregulated Spill Linearization Table**

Type:	Table Slot
Units:	Volume vs Flow
Description:	Storage vs corresponding unregulated spill values
Information:	
Defined by:	Internally developed based on Unregulated Spill Table and Elevation Volume Table relationships. For each pool elevation in the Unregulated Spill Table, the Unregulated Spill Capacity Table has a row relating Storage to Unregulated Spill Capacity. The Pool Elevations are converted to Storage using the Elevation Volume Table.

- **Unregulated Spill LP Param**

Type:	Table Slot
Units:	Volume
Description:	Specifies the Storage points use to take the tangent, line and piecewise approximations for Unregulated Spill Linearization Table linearization
Information:	
Defined by:	User input

■ None

If this method is selected the slot bounds in the slot configuration dialog are set to zero. No additional constraints are generated.

■ Opt Monthly Spill

This method is not functional in RPL Optimization.

This method sets the lower and upper bounds on spill. The lower bound is set to zero and the default upper bound is set to a very big number (9,999,999 cms). The default upper bound can be revised in the Spill slot configuration, Upper Bound parameter. No additional constraints are generated beyond these bounds.

■ Opt Unregulated

If this method is selected, only unregulated spill is considered, and the following constraint is added to the LP:

$$\text{Spill} = \text{Unregulated Spill} \quad (5.7)$$

■ Opt Regulated

When this method is selected, only regulated spill is considered. The lower bound on spill is set to zero, and the following constraint is added to the LP:

$$\text{Spill} = \text{Regulated Spill} \quad (5.8)$$

■ Opt Regulated and Unregulated

When this method is selected, unregulated and regulated spill are considered, and the following constraint is added to the LP:

$$\text{Spill} = \text{Regulated Spill} + \text{Unregulated Spill} \quad (5.9)$$

■ Opt Regulated and Bypass

When this method is selected, only regulated and bypass spill are considered, the lower bound on spill is set to zero, and the following constraint is added to the LP:

$$\text{Spill} = \text{Regulated Spill} + \text{Bypass} \quad (5.10)$$

■ Opt Regulated, Bypass and Unregulated

When this method is selected, unregulated, regulated, and bypass spill are considered, and the following constraint is added to the LP:

$$\text{Spill} = \text{Regulated Spill} + \text{Unregulated Spill} + \text{Bypass} \quad (5.11)$$

■ Opt Bypass, Regulated and Unregulated

When this method is selected, unregulated, regulated, and bypass spill are considered, and the following constraint is added to the LP:

$$\text{Spill} = \text{Bypass} + \text{Regulated Spill} + \text{Unregulated Spill} \quad (5.12)$$

Optimization Tailwater

Only a subset of available methods are supported in Optimization. These methods are: Linked or Input, Base Value Only, Base Value Plus Lookup Table, Stage Flow Lookup Table, and Coefficients Table. Selection of a method other than these results in an error. The method selected in the Optimization Tailwater category should correspond to the one selected in the simulation Tailwater category.

Tailwater Elevation may be part of the Optimization problem, and it is handled differently for each method. Below is a description of the behavior of Tailwater Elevation.

See “[Tailwater](#)” in *Objects and Methods* for details on the Simulation methods.

■ Opt Linked or Input

If Tailwater Elevation is not input, it is replaced as follows:

$$\text{TailwaterBaseValue: Tailwater Elevation} = \text{Tailwater Base Value} \quad (5.13)$$

Slots Specific to This Method

- **Tailwater Base Value**

Type: Series
Units: Length
Description: See “[Linked or Input](#)” in *Objects and Methods* for details.

■ Opt Base Value Only

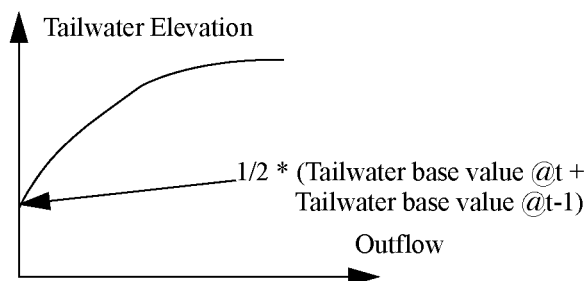
If Tailwater Elevation is not input, it is replaced as follows:

$$(\text{Tailwater Base Value}(t) + \text{Tailwater Base Value}(t-1)) / 2 \quad (5.14)$$

Slots Specific to This Method

- **Tailwater Base Value**

Type: Series
Units: Length
Description: See “[Base Value Only](#)” in *Objects and Methods* for details.
Tailwater outflow relationship. Not drawn to scale.



■ Opt Base Value Plus Lookup Table

If Tailwater Elevation is not input, this slot is replaced by a mathematical expression based on Tailwater Base Value and a numerical approximation of Tailwater Elevation as a function of Outflow (Numerical 2-D Approximation), as defined by the user in the Tailwater Table.

Tailwater Elevation is constrained as follows:

$$(\text{tailwaterBaseValue}(t) + \text{tailwaterBaseValue}(t-1)) / 2 + \text{tempTWLookupValue} \quad (5.15)$$

where tempTWLookupValue is a Numerical 2-D Approximation of increase in Tailwater Elevation due to flow, as described in the Tailwater Table. The Tailwater Table Lookup LP Param table values for outflow are used as approximation points, indexing the Tailwater Table to determine tempTWLookupValue from the Tailwater Elevation column.

Slots Specific to This Method

• Tailwater Base Value

Type: Series
Units: Length
Description: See “Base Value Plus Lookup Table” in *Objects and Methods* for details.

• Tailwater Table

Type: Series Slot
Units: Flow vs Length
Description: Reservoir outflow vs either the tailwater elevation or the tailwater elevation increment
Information: If the Tailwater Base Value is non-zero, the Tailwater Table gives values of incremental increase in Tailwater Elevation over the Base value. If the Tailwater Base Value is zero, the table simply gives the Tailwater Elevation values. The Tailwater Table should have increasing values of outflow. Tailwater Elevation is required to be a convex function of outflow. The preferred order of approximation is substitution, piece-wise, two-point line, tangent. See Object / Simulation documentation for details.
I/O: User input

• Tailwater Table Lookup LP Param

Type: Table Slot
Units: Flow, Flow, Flow
Description: LP Parameter approximation points for Outflow.
Information: The Tailwater Table Lookup LP Param table values for outflow are used as approximation points indexing the Tailwater Table to determine tempTWLookupValue from the Tailwater Elevation column.
I/O: User input only
Links: Not linkable

• temp Tailwater Lookup Value

Type: Series
Units: Length
Description: Numerical 2-D Approximation of increase in Tailwater Elevation due to flow as described in the Tailwater Table.

■ Opt Stage Flow Lookup Table

Tailwater Elevation is numerically approximated as a function of Stage and Flow, as defined by the user in the Stage Flow Tailwater Table. The tables are queried depending on method selection in the Tailwater Linearization Automation category, as follows:

- If the selected method in the Tailwater Linearization Automation category is None, the user-input Tailwater LP Param table values are used.
- For the Range Input Automation method, the automatically developed points are used.

Tailwater Elevation is approximated numerically (3-D approximation) as a function of Outflow and Tailwater Base Value based on the Stage Flow Tailwater Table. The Tailwater Elevation LP Param table Tailwater Base Value and flow values are used as approximation points indexing the Downstream Stage and Outflow columns of the Stage Flow Tailwater Table, respectively.

See “Stage Flow Lookup Table” in *Objects and Methods* for details.

Slots Specific to This Method

• Tailwater Base Value

Type:	Series
Units:	Length
Description:	See “Stage Flow Lookup Table” in <i>Objects and Methods</i> for details.

• Stage Flow Tailwater Table

Type:	Table Slot
Units:	Flow vs Length vs Length
Description:	Reservoir Outflow vs Downstream Elevation (Tailwater Base Value) vs Tailwater Elevation
Information:	Data must be entered into the table in increasing blocks of the same Outflow value for the three-dimensional table interpolation to work correctly. For every block of same Outflows in column 1, Downstream Stages should be listed in increasing order in column 2, and the corresponding Tailwater Elevations in column 3. Tailwater elevation is required to be a concave function of Outflow. The preferred order of approximation is substitution, piece-wise, two-point line, tangent. Internally, the Stage Flow Tailwater Table is rearranged to use the Stage as the primary index (column) with outflow as the secondary index (column). This rearranged table may be labeled as “Convolved Stage Flow Tailwater Table” in output messaging. Also, because of this rearrangement, it is desirable to repeat stage values for each outflow.
Defined by:	User input

• Tailwater Elevation LP Param

Type:	Table Slot
Units:	Length, Flow, Flow, Flow
Description:	Specifies the fixed tailwater base value point and the outflow points used to take the tangent, line and piecewise approximations for tailwater elevation linearization

Information:	This table is used for linearization. The best Tailwater Base Value point to choose for tangent approximation would be an outflow equal to the expected value of outflow during the run; for the line approximation, the minimum and maximum values expected during the run; for piecewise approximation, the minimum and maximum values expected during the run plus additional intermediate values to more closely fit the tailwater curve.
Defined by:	User input unless the selected method in the Tailwater Linearization Automation category is not None.

• Tail Water Reference Elevation

Type:	Table Slot
Units:	Length
Description:	Lowest Reservoir discharge Elevation when there are no backwater effects from a downstream pool (reservoir)
Information:	Although this is part of the Stage Flow Lookup Table method (and its corresponding Opt method), <i>this slot does not influence optimization</i> . See Object / Simulation documentation for details.
I/O:	User input

■ Opt Coefficients Table

This method sets up the physical Tailwater Elevation constraints using the same equation and coefficients used in the Coefficients Table tailwater method.

$$\begin{aligned} \text{Tailwater Elevation} = & \text{Constant Coeff}[t] + \text{Constant Coeff}[t - 1] + \\ & \text{Outflow Coeff}[t] \times \text{flow} + \text{Outflow Coeff}[t - 1] \times \text{Outflow}[t - 1] + \\ & \text{TW Base Val Coeff}[t] \times \text{TailwaterBaseValueTemp}[t] + \\ & \text{TW Base Val Coeff}[t - 1] \times \text{Tailwater BaseValue}[t - 1] + \\ & \text{TW Elev Coeff}[t - 1] \times \text{Tailwater Elevation}[t - 1] \end{aligned}$$

See “Coefficients Table” in *Objects and Methods* for details.

Slots Specific to This Method

• Tailwater Base Value

Type:	Series
Units:	Length
Description:	See “Coefficients Table” in <i>Objects and Methods</i> for details.

• Tailwater Coefficients

Type:	Table
Units:	multi
Description:	See “Coefficients Table” in <i>Objects and Methods</i> for details.
Information:	If the Power Surface Approximation method is selected, then there are additional restrictions on the coefficients that can be specified. With the Power Surface Approximation, having terms for Outflow[t-1], Tailwater BaseVal[t] and Tailwater[t-1] are not allowed.

Power

Only a subset of available Simulation methods are available in Optimization. Selection of a method other than those that follow will result in an error.

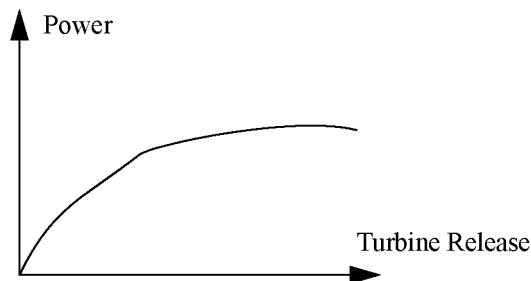
If the Optimization problem uses Power, either directly or indirectly, then the Power slot is added to the problem. Before being passed to the optimization solver, the Power slot is numerically approximated as a function of Operating Head and Turbine Release (Numerical 3-D Approximation).

See “Power” in *Objects and Methods* for details about the Simulation methods.

Slots Specific to This Method

- **Power LP Param**

Type:	Table Slot
Units:	Length and Flow
Description:	Specifies the Operating Head and the flow points used to take the tangent, line and piecewise approximations for Power linearization
Information:	This table must be user input if the Independent Linearizations method is selected in the Optimization Power category. Otherwise it is unused. The best Operating Head to choose should be close to the expected head during optimized period. Tangent approximation is generally not used, nor helpful as it often results in non-zero power for zero flow. The suggested points for a line approximation are 0 flow and best turbine flow for the entire plant. The suggested points for piecewise linearization are 0 flow, 1 unit best turbine flow, 2 units best turbine flow ... n units best turbine flow, and maximum turbine flow. Power turbine release relationship for a fixed operating head valve. Not drawn to scale.



■ Plant Power Coefficient

Slots Specific to This Method

In the RPL Optimization mode, the following slot requires special handling.

- **Best Turbine Flow**

Type:	Agg Series Slot
Units:	Flow

Description: Flow associated with the most efficient power generation for an associated Operating Head

Information:

Defined by: Numerical 2-D Approximation in terms of Operating Head, based upon an internal best turbine flow table. For the Plant Power Coefficient method (selected in the Power category) the internal table is the Best Turbine Q table. The Best Turbine Q table is required to have increasing values of Operating Head and Best Capacity and be a concave function of Operating Head. The preferred order of approximation is substitution, piece-wise, tangent, two-point line. The Best Turbine Flow LP Param table values are used as approximation points indexing the best turbine flow table.

If the Optimization problem uses Best Turbine Flow, either directly or indirectly, then this slot is added to the problem. Before being passed to the optimization solver, this slot is numerically approximated as a function of Operating Head (Numerical 2-D Approximation). The relationship between Operating Head and Best Turbine Flow will come from one of two places, depending on other method selection.

If the selected method in the Power category is Plant Efficiency Curve, then the relationship is automatically developed from the Plant Power Table.

For the Plant Power Coefficient method, the user-input Best Turbine Q table defines the relationship.

For other Plant Calculation category methods, Best Turbine Flow must be input.

• Best Turbine Flow LP Param

Type: Table Slot

Units: Length

Description: Specifies the Operating Head points in the best turbine flow relationship used to take the tangent, line and piecewise approximations for Best Turbine Flow linearization

Information: The best operating head points to choose for a piecewise approximation are generally: minimum Operating Head, Operating Head at max capacity, and maximum Operating Head. These three points typically define most of the shape of the Best Turbine Flow curve.

Defined by: User input

• Best Turbine Q

Type: Table Slot

Units: Length vs Flow

Description: Table defining the relationship between Operating Head and the most efficient power generation.

Information: This table is used if the Power category has the Plant Power Coefficient method selected. If the Power category has the Plant Efficiency Curve method selected, then this table is not used, but a similar relationship is automatically developed from the Plant Power Table.

Defined by: User input

• Plant Power Table

Type: Table Slot

Chapter 5

Optimization Objects and Methods

Units: Length vs Flow vs Power

Description: Table defining the relationship between Operating Head, Turbine Release, and Power at selected points of operation. In building the Plant Power table, the following approach might be taken. For a given Operating Head create $n + 2$ rows of data, where n is the number of units comprising the Plant. The first row will reflect no flow through the turbines. The next row will reflect the preferred level of flow through the first unit normally activated. Subsequent rows will reflect incremental flow levels as additional turbines come on line at their respective preferred levels of flow. The final row will reflect the flow of all available units running at maximum flow capacity.

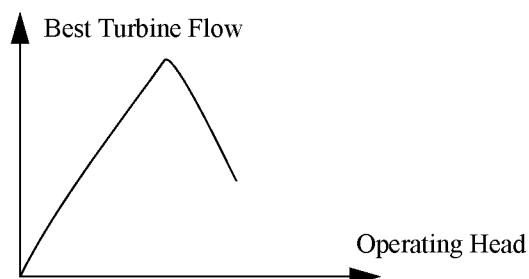
Information: Data must be entered into the table in increasing, concave blocks of the same Operating Head for the three-dimensional table interpolation to work correctly. For every block of the same Operating Head in column 1, Turbine Release and the corresponding Power should be listed in increasing, concave order in columns 2 and 3. There must be a point of zero Turbine Release and zero Power for each Operating Head.

The following table is an example of the proper way to formulate the Plant Power Table. This table can be derived from the Best Turbine Q, Best Power Coefficient, Max Turbine Q and Max Power Coefficient slots.

Operating Head	Turbine Release	Power
30	0	0
30	100	100
30	200	175
40	0	0
40	100	125
40	220	195
50	0	0
50	110	147
50	250	205

Defined by: User input

Best turbine flow - operating head relationship. Not drawn to scale.



■ Plant Efficiency Curve

The Plant Efficiency Curve method approximates Best Power Flow the same as the Plant Power Coefficient method.

■ Peak Power

In RPL Optimization, this method creates the following turbine release constraint. No actual power value is calculated.

$$\text{Turbine Release} \leq \text{Power Plant Capacity Fraction} * \text{Turbine Capacity} * \text{Number of Units.} \quad (5.16)$$

■ Peak and Base

In RPL Optimization, this method creates the following turbine release constraint. No actual power value is calculated.

$$\text{Turbine Release} \leq \text{Power Plant Capacity Fraction} * \text{Turbine Capacity} * \text{Number of Units.} \quad (5.17)$$

■ Unit Power Table

See “Unit Power” on page 301 for details about the Optimization formulation.

See “Unit Power Table” in *Objects and Methods* for the Simulation algorithm.

Slots Specific to This Method

See “Unit Power Table” in *Objects and Methods* for slot descriptions.

Method Details

When the Unit Power Table method is selected, the following categories are available:

- [Avoidance Zones](#), page 96
- [Cavitation](#), page 97
- [, page 99](#)
- [Head Loss](#), page 101
- [Startup](#), page 141

The status of a unit at a given timestep can be one of the following:

- Available
- Unavailable
- Must run

At the highest level, an entire plant may never be able to provide an ancillary service like Regulation. The user can indicate this by selecting None in the appropriate category. On the other hand, the following slot settings allow users to specify unit availability / must run for individual timesteps for generation and ancillary services. Without any inputs, the units default to being available.

Chapter 5

Optimization Objects and Methods

If the Unit Power Table method is selected, but not Frequency Regulation, the user can specify through optimization RPL policy.

Note: Setting certain slots may be possible but not preferred.

- Unit u must generate at time t :
 - Unit Is Generating $[t,u] = 1$, or
 - Unit Energy $[t,u] = \text{value}$
- Unit u is unavailable at time t :
 - Unit Is Generating $[t,u] = 0$, or
 - Unit Energy $[t,u] = 0$

If the Frequency Regulation method is selected, the user could specify through optimization policy (RPL):

- Unit u must regulate up at time t :
 - Regulation Up $[t,u] = \text{value}$
- Unit u must regulate down at time t :
 - Regulation Down $[t,u] = \text{value}$
- Unit u must regulate at time t :
 - Regulation $[t,u] = \text{value}$ (which implies Regulation Up $[t,u] = \text{value}$ and Regulation Down $[t,u] = \text{value}$)

If a unit is unavailable for some type of regulation, then a value can be 0.

■ Turbine Capacity

Turbine Capacity is numerically approximated before being passed to the optimization solver. The slot is approximated as a function of Operating Head (Numerical 2-D Approximation).

The relationship between Operating Head and Turbine Capacity depends on the method selected in the Power category, as follows:

- If Plant Efficiency Curve is selected, the relationship is automatically developed from the Plant Power Table.
- If Peak Power or Peak and Base is selected, the relationship comes from the Best Generator Flow table.
- If none of these methods is selected, the relationship is contained in the user-input Max Turbine Q slot.

Slots Specific to This Method

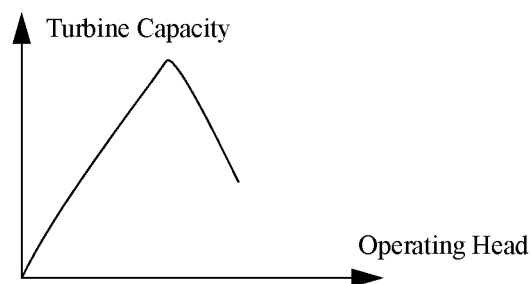
• Turbine Capacity

Type:	Agg Series Slot
Units:	Flow
Description:	Flow capacity of the turbines
Information:	
Defined by:	Numerical 2-D Approximation in terms of Operating Head, based upon a maximum turbine capacity table. This capacity table is determined in various ways according to the Power method:

The Turbine Capacity LP Param table values are used as approximation points indexing the selected maximum turbine capacity table. The maximum turbine capacity table is required to have increasing values of Operating Head. Turbine Capacity in that table is required to be a concave function of Operating Head. The preferred order of approximation is substitution, piece-wise, tangent, two-point line.

- **Turbine Capacity LP Param**

Type:	Table Slot
Units:	Length vs Length vs Length
Description:	Specifies the operational head points used to take the tangent, line and piecewise approximations for Turbine Capacity linearization
Information:	For the piecewise approximation the best operating head points to choose are generally: minimum Operating Head, Operating Head at max capacity, and maximum Operating Head. These three points typically define most of the shape of the Turbine Capacity curve.
Defined by:	User input, unless the selected method in the Power Linearization Automation category is not None. Turbine capacity operating head relationship. Not drawn to scale.



Startup

This category depends on selecting the Unit Power Table method (see “Unit Power Table” on page 139), and describes how the monetary cost associated with starting up or shutting down a unit (turbine) will be modeled. There are two methods in this category, one which does not model these costs (effectively assigning them a value of 0) and one which uses a table describing the combined costs for starting up and shutting down a unit.

■ None

This is the default, no action method.

■ Unit Lumped Cost

For each unit, this method lumps the cost of startup and shutdown into one value.

See “Unit-level Constraints” on page 309 for details about added constraints.

Slots Specific to This Method

See “[Unit Lumped Cost Method](#)” in *Objects and Methods* for slot descriptions.

Modeling a Level Power Reservoir

Steps to follow in setting up a level power reservoir for optimization

1. Set up a running simulation model, using methods that are compatible with optimization for Power, Tailwater, Spill, and so on.
2. Chose an Optimization Power method.
3. Choose an Optimization Head Computation method. In Optimization, None must be chosen.
4. Select the Optimization Spill method corresponding to the method selected in the Spill category.
5. Fill in the LP Param tables related to Power, Turbine Capacity and Best Turbine Flow (if plant power is used). Alternatively, the Power Linearization Automation method can also be chosen to automatically determine the values for the LP Param tables.
6. Select the Optimization Tailwater method corresponding to the method selected in Tailwater category. If Opt Base Value Plus Lookup Table or Opt Stage Flow Lookup Table is selected, fill in the Tailwater Elevation LP Param table. Alternatively, select a Tailwater Linearization Automation method (Range Input Automation is currently available).
7. If future value is needed, select a method in the Future Value category, followed by the Optimization Future Value category, and if desired, Cumul Stor Val Linearization Automation.
8. Selection of the remaining methods and linearizations can be done in any order. Pool elevation linearizations, Pool Elevation Linearization Automation, Evaporation and Precipitation, Optimize Evaporation Computation, Evaporation Linearization Automations, Bank Storage, Hydrologic Inflow, Energy In Storage, and Diversion from Reservoir. The appropriate data must be entered for each method.

Pumped Storage Reservoir

The Pumped Storage Reservoir object models a level reservoir where there are facilities that can release water to generate power but can also be pump water into the reservoir to increase storage.

General Slots

- **Canal Flow**

Type:	Agg Series
Units:	Flow
Description:	Flow into (out of) the reservoir from (to) a canal
Information:	
Defined by:	Explicit Optimization variable in the mass balance constraint

• **Diversion**

Type: Series Slot
Units: Flow
Description: Flow from the reservoir to a diverting object
Information:
Defined by: Explicit Optimization variable in the mass balance constraint

• **Elevation Volume Table**

Type: Table
Units: Length vs Volume
Description: Table relating elevation of the reservoir to volume stored in the reservoir
Information:
I/O: Input only
Defined by: Input only

• **Energy**

Type: Agg Series Slot
Units: Energy
Description: Product of the power generated by flow through the turbines and the length of the timestep.
Information:
Defined by: Replaced by Power * Timestep Length
Description:

• **Flow FROM Pumped Storage**

Type: Agg Series Slot
Units: Flow
Description: Flow into the reservoir from a pumped storage reservoir
Information:
Defined by: Explicit Optimization variable in the mass balance constraint. This slot should be linked to Outflow on a Pumped Storage object. The Pumped Storage object constrains its Outflow.

• **Flow TO Pumped Storage**

Type: Agg Series Slot
Units: Flow
Description: Flow out of the reservoir into a pumped storage reservoir
Information:
Defined by: Explicit Optimization variable in the mass balance constraint. This slot should be linked to Pumped Flow on a Pumped Storage object.

• **Inflow**

Type: MultiSlot

Units: Flow
Description: Inflow into the reservoir from upstream
Information:
Defined by: Explicit Optimization variable in the mass balance constraint
Information:

• Operating Head

Type: Agg Series Slot
Units: Length
Description: Elevation difference between the average Pool Elevation and the average Tailwater Elevation during a timestep
Information:
Defined by: Replacement by $(\text{Pool Elevation}(t) + \text{Pool Elevation}(t-1)) / 2 - \text{Tailwater Elevation}$

• Outflow

Type: Agg Series Slot
Units: Flow
Description: Outflow from the reservoir
Information:
Defined by: Explicit Optimization variable as $\text{Outflow} = \text{Turbine Release} + \text{Spill}$
Information:

• Pool Elevation

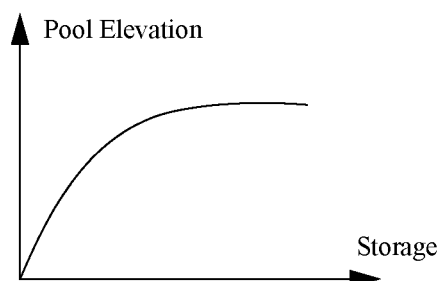
Type: Agg Series Slot
Units: Length
Description: Elevation of the water surface of the Reservoir
Information: When Pool Elevation is a part of the optimization problem, as it is in all conceivable RiverWare Optimization applications, this slot is numerically approximated as a function of Storage (Numerical 2-D Approximation). The relationship between Pool Elevation and Storage will come from the user-input Elevation Volume Table. The table will be queried either using user-input points defined in the Pool Elevation LP Param table.
Defined by: Numerical 2-D Approximation in terms of Storage, based upon the Elevation Volume Table. The Pool Elevation LP Param table values are used as approximation points indexing the Elevation Volume Table. The Elevation Volume Table should have increasing values of Pool Elevation and Storage. Storage is required to be a concave function of Pool Elevation. The preferred order of approximation is substitution, piece-wise, tangent, two-point line.

• Pool Elevation LP Param

Type: Table Slot
Units: Volume
Description: Specifies the Storage points used to take the tangent, line and piecewise approximations for Pool Elevation linearization

Information: This table is used for linearization unless Pool Elevation Linearization Automation category has selected Plant Automation. The best Storage point to choose for tangent approximation would be the expected storage expected during the run; for the line approximation, the expected maximum and minimum Storage; for piecewise approximation, use points that cover the full range of expected Storage during the run with intermediate points such that a piecewise linear curve reasonably approximates the actual curve.

Defined by: User input
Pool elevation storage relationship. Not drawn to scale.



• Power

Type: Agg Series Slot

Units: Power

Description: Power generated by flow through the turbines

Information:

Defined by: Numerical 3-D Approximation in terms of Operating Head and Turbine Release. Approximation is based on the Plant Power Table. The Power LP Param table contains a value for Operating Head used to index the Operating Head column of the Plant Power Table. This approximated value, therefore, reduces the Power to a function of Turbine Release at the given Operating Head.

The flow values in the Power LP Param table are then used as approximation points indexing the Turbine Release column of the Plant Power Table. The Plant Power Table should have increasing values of Operating Head and Turbine Release. Power should be a concave function of Operating Head, but concavity is not strictly enforced; mild non-concave regions are permissible to allow for round-off error, and so on. The preferred order of approximation is substitution, piece-wise, two-point line, tangent.

• Return Flow

Type: MultiSlot

Units: Flow

Description: Flow returning from a diversion object

Information:

Defined by: Explicit Optimization variable in the mass balance constraint (see Storage)

• Spill

Type: Agg Series Slot

Units: Flow

Description: Sum of the Bypass, Regulated Spill and Unregulated Spill
Information:
Defined by: Explicit Optimization variable as $\text{Spill} = \text{Bypass} + \text{Regulated Spill} + \text{Unregulated Spill}$

• Storage

Type: Series Slot
Units: Volume
Description: Volume of water stored in the reservoir
Information:
Defined by: Explicit Optimization variable as $\text{Storage} = \text{Storage}(t-1) + \text{Precipitation Volume} - \text{Evaporation} - \text{Change in Bank Storage} + \text{timestep} * (\text{Inflow} + \text{Canal Flow} + \text{Flow TO Pumped Storage} + \text{Hydrologic Inflow Net} + \text{Return Flow} - (\text{Outflow} + \text{Diversion} + \text{Flow FROM Pumped Storage}))$

• Tailwater Base Value

Type: Series Slot
Units: Length
Description: Elevation of tailwater or base elevation used to compute elevation of tailwater
Information:
Defined by: Explicit Optimization variable should be input or linked. See [“Pump Power Numerical Approximation” in Optimization](#) for descriptions of related constraints and other objects to which the slot is linked.

• Tailwater Elevation

Type: Agg Series Slot
Units: Length
Description: Water surface elevation on the downstream side of the dam
Information:
Defined by: Various approaches dependent on the method selected in the Optimization Tailwater category.

• Turbine Capacity LP Param

Type: Table
Units: Length, Length, Length
Description: LP Param table for turbine capacity
Information: See power methods for details

• Turbine Capacity

Type: Agg Series Slot
Units: Flow
Description: Flow capacity of the entire power plant’s turbines
Information:

Defined by: Numerical 2-D Approximation in terms of Operating Head, based upon a maximum turbine capacity table. This capacity table is determined in various ways according to the Power method.

• Turbine Release

Type: Agg Series Slot

Units: Flow

Description: Flow through the turbines of a power reservoir

Information:

Defined by: Explicit Optimization variable as Turbine Release $\leq$ Power Plant Cap Fraction * Turbine Capacity

User Methods

The following categories and methods are available for use in the Pumped Storage Reservoir object in Optimization.

Because of dependency relationships, you may not be able to see them in your model until you change other methods. In the discussion below, you will see the other methods that need to be selected to enable a given method to be used in your model. When building a model, you need to review this to ensure that required dependencies are satisfied so that the desired methods are available for use.

User method categories are as follows.

- [Avoidance Zones](#), page 148
- [Bank Storage](#), page 148
- [Cavitation](#), page 148
- [Diversion from Reservoir](#), page 149
- [Energy in Storage](#), page 149
- [Inflow](#), page 151
- [Future Value](#), page 151
- [Head Loss](#), page 152
- [Hydrologic Inflow](#), page 153
- [Optimization Evaporation](#), page 153
- [Optimization Future Value](#), page 156
- [Optimization Power](#), page 157
- [Optimization Reserves](#), page 170
- [Optimization Spill](#), page 182
- [Optimization Tailwater](#), page 185
- [Power](#), page 191

- [Startup](#), page 197

Avoidance Zones

This category depends on selecting the Unit Power Table method (see [“Unit Power Table”](#) on page 194) and contains methods for modeling the existence of undesirable regions of operation for turbines. There are two methods in this category, one which does not model avoidance zones at all, and one which

■ None

This the default, no action method; avoidance zones are not considered.

■ Unit Head Based Avoidance Zones

This method allows the user to specify a table that defines the conditions in which the turbines should not be operated.

See [“Unit-level Constraints”](#) on page 309 for descriptions of added constraints.

Slots Specific to This Method

See [“Unit Head Based Avoidance Zones”](#) in *Objects and Methods* for slot descriptions.

Bank Storage

Not all methods are functional in RPL optimization. Of the available methods, None and CRSS Bank Storage are supported.

■ None

See [“None”](#) in *Objects and Methods* for details.

■ CRSS Bank Storage

See [“CRSS Bank Storage”](#) in *Objects and Methods* for details.

Cavitation

This category depends on selecting the Unit Power Table method (see [“Unit Power Table”](#) on page 194) and contains methods for dealing with the problem of cavitation on turbines. Cavitation is the sudden formation and collapse of low-pressure bubbles in liquids by means of mechanical forces and this process can cause damage to turbines under certain operating conditions.

■ None

This is the default, no action method.

■ Unit Head and Tailwater Based Regions

This method allows the user to specify the regions of operation in which cavitation does *not* occur, so these regions can be avoided. These regions can be dependent on both operating head and tailwater.

See “[Unit-level Constraints](#)” on page 309 for descriptions of added constraints.

Slots Specific to This Method

See “[Unit Head and Tailwater Based Regions](#)” in *Objects and Methods* for slot descriptions.

Diversion from Reservoir

Not all methods in this category are supported in RPL optimization. Of the available methods, None and Available Flow Based Diversion are supported; selection of any other method results in an error at the start of run.

■ None

See “[None](#)” for details.

■ Available Flow Based Diversion

See “[Available Flow Based Diversion](#)” for details.

Energy in Storage

In Optimization, None and EIS Table Lookup are supported.

■ None

No Energy in storage is considered.

■ EIS Table Lookup

With this method selected, Energy in Storage is considered as a function of Pool Elevation.

If the optimization problem uses Energy In Storage, either directly or indirectly, then this slot is added to the problem. Before being passed to the optimization solver, this slot is numerically approximated as a function of Pool Elevation (Numerical 2-D Approximation).

The relationship between Pool Elevation and Energy In Storage will come from the user-input Energy In Storage Table. The table will be queried either using user-input points defined in the Energy In Storage LP Param table or using automatically calculated points, depending on method selection in the Pool Elevation Linearization

Chapter 5

Optimization Objects and Methods

Automation category. If the selected method in the Pool Elevation Linearization Automation is None, the user-input Energy In Storage LP Param table values are used.

For other Pool Elevation Linearization Automation methods (Initial Target Input, Min Difference or Range Input, Min Difference) the same points automatically developed for the Pool Elevation linearization will be used.

See “EIS Table Lookup” in *Objects and Methods* for details on the Simulation method.

Slots Specific to This Method

• Energy In Storage

Type:	Series Slot
Units:	Energy vs Power
Description:	Energy in Storage in the reservoir
Information:	
Defined by:	Numerical 2-D Approximation in terms of Pool Elevation, based upon the Energy In Storage Table. The Energy In Storage LP Param table values are used as approximation points indexing the Energy In Storage Table. The Energy In Storage Table should have increasing values of Pool Elevation and Energy In Storage. Energy In Storage is required to be a convex function of Pool Elevation. The preferred order of approximation is substitution, piece-wise, tangent, two-point line.

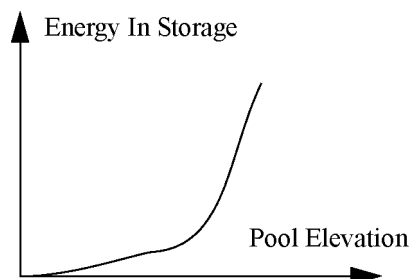
• Energy in Storage LP Param

Type:	Table Slot
Units:	Length
Description:	Specifies the Pool Elevation points used to take the tangent, line and piecewise approximations for Energy In Storage linearization.
Information:	This table is used for linearization unless the Pool Elevation Linearization Automation category has a method selected other than None.
Defined by:	User input

• Energy in Storage Table

Type:	Table Slot
Units:	Length vs Energy
Description:	Table defining the relationship between Energy In Storage and Pool Elevation
Information:	
Defined by:	User input

Energy in storage pool elevation relationship. Not drawn to scale.



Future Value

This category and its methods are not dependent on other method selections.

■ None

See “None” in *Objects and Methods* for details.

■ Cumulative Storage Value Table

In RPL-Optimization, the Cumulative Storage Value is Numerically Approximated as described below.

See “Cumulative Storage Value Table” in *Objects and Methods* for details.

Slots Specific to This Method

• Cumulative Storage Value

Type:	Agg Series Slot
Units:	\$
Description:	Represents the future energy value of the current storage
Information:	
Defined by:	2-D approximation in terms of Anticipated Storage, based upon the Cumul Stor Val Table. The Cumul Stor Val LP Param table values (Storage) are used as approximation points indexing the Cumul Stor Val Table. The Cumul Stor Val Table should have increasing values of Storage and Cumulative Value. Cumulative Storage Value is required to be a concave function of Anticipated Storage. The preferred order of approximation is substitution, piece-wise, tangent, two-point line. The Cumul Stor Val Linearization Automation category’s Marginal Value to Table and Lin method can automate creation of the Cumul Stor Val LP Param table and the Cumul Stor Val Table.

• Anticipated Storage

Type:	Agg Series Slot
Units:	Volume

Description: The combination of the actual storage plus water that would be expected to enter the reservoir after the Current Timestep but has not yet, due to lagging.

Information:

Defined by:

- **Cumul Stor Val Table**

Type: Table Slot

Units: Volume vs Value

Description: The estimated total economic value of water stored in the reservoir for discrete storage values.

Information:

Defined by: User input or by automated procedure if Cumul Stor Val Linearization Automation category has selected the Marginal Value to Table and Lin method.

- **Marginal Storage Value Table**

Type: Table Slot

Units: Storage vs \$ Value

Description: Anticipated Storage versus Cumulative Storage Value per unit energy

Information: This table should be increasing in storage, and logically decreasing in marginal value

Defined by: Required input

- **Spill Cost**

Type: Agg Series Slot

Units: \$

Description:

Information:

Defined by:

Head Loss

This category depends on the Unit Power Table method (see “[Unit Power Table](#)” on page 194) and contains methods for modeling additional head loss that occurs. This head loss may come from the configuration of the penstocks for bringing water to the turbines.

■ None

In this method, there is no additional head loss to be used in the power calculation. In terms of penstock head loss, this method should be selected if the penstocks for the units are independent and the penstock losses are typically incorporated in the power data. Thus the power data is specified in terms of operating head.

■ Shared Penstock Head Loss

In this method, there is additional head loss that results because units share a common penstock. The operating head losses in the penstock depend on the total turbine release and are shared for all units. The net head is calculated by subtracting penstock losses from the operating head. The unit data and power must be specified in terms of unit Net Heads instead of Operating Head.

See “[Unit-level Constraints](#)” on page 309 for descriptions of additional constraints.

Slots Specific to This Method

See “[Shared Penstock Head Loss method](#)” in *Objects and Methods* for slot descriptions.

Hydrologic Inflow

If any method in this category is selected, hydrologic inflow is included in the reservoir mass balance. Optimization assumes hydrologic inflow to be known (data) and it is not solved for by the reservoir regardless of the method selected.

See “[Hydrologic Inflow](#)” in *Objects and Methods* for details.

Optimization Evaporation

This category can be used to model evaporation and precipitation.

■ None

This is the default method for the Optimization Evaporation category. It performs no calculations and requires that None also be selected for the Evaporation and Precipitation category.

■ Opt Input Evaporation

This method is analogous to the Input Evaporation method in simulation and requires the Input Evaporation method to be selected for the Evaporation and Precipitation category. Evaporation Rate and Precipitation Rate are entered as a time series. Evaporation is calculated as a product of Evaporation Rate, Average Surface Area over the timestep and Timestep length. Similarly Precipitation Volume is calculated as the product of Precipitation Rate, Average Surface Area and Timestep length.

Caution: The linearization of the Surface Area variable can result in a small approximation error in optimization for Evaporation and Precipitation Volume. This means there can be a small difference between the mass balances in the optimization solution and the post-optimization rulebased simulation when using this method. It is important to use care when setting the approximation points for Surface Area in order to reduce this approximation error. Refer to the information on the Surface Area LP Param slot below. Also caution should be used if applying this method at a 1 Month timestep. All rates in the optimization mass balance are converted to monthly volumes based on a 30-day month, regardless of the month. This will also produce a difference between the mass balances in the optimization solution and the post-optimization rulebased simulation for a 1 Month timestep.

Slots Specific to This Method

• Elevation Area Table

Type:	Table Slot
Units:	Length vs Area
Description:	Represents the Elevation-Surface Area relationship
Information:	This table must be input. It is used to derive the Volume Area Table.
Defined by:	User input

• Evaporation

Type:	Series Slot
Units:	Volume
Description:	The volume of water lost to evaporation over the timestep
Information:	If this slot contains user input, it is added directly to the mass balance constraint, otherwise it is defined by the expression below.
Defined by:	Either user input or the following constraint: $\text{Evaporation} = \text{EvaporationRate} \times \frac{\text{SurfaceArea}(t-1) + \text{SurfaceArea}}{2} \times \text{Timestep}$

• Evaporation Rate

Type:	Series Slot
Units:	Velocity
Description:	The rate, in length per time, at which water is lost to evaporation at each timestep
Information:	This slot can be set as user input. If it is not set as an input, and if Evaporation is not an input, this slot defaults to zero. If Evaporation is an input, this slot is not used.
Defined by:	User input or defaults to zero

• Precipitation Rate

Type:	Series Slot
Units:	Velocity
Description:	The rate, in length per time, at which water is gained from precipitation at each timestep
Information:	This slot can be set as user input. If it is not set as an input, it defaults to zero.
Defined by:	User input or defaults to zero

• Precipitation Volume

Type:	Series Slot
Units:	Volume
Description:	The volume of water gained from precipitation over the timestep
Information:	The Input Evaporation method will not allow this slot to be set as an input.
Defined by:	Explicit constraint:

$$\text{Precipitation Volume} = \text{PrecipitationRate} \times \frac{\text{SurfaceArea}(t-1) + \text{SurfaceArea}}{2} \times \text{Timestep}$$

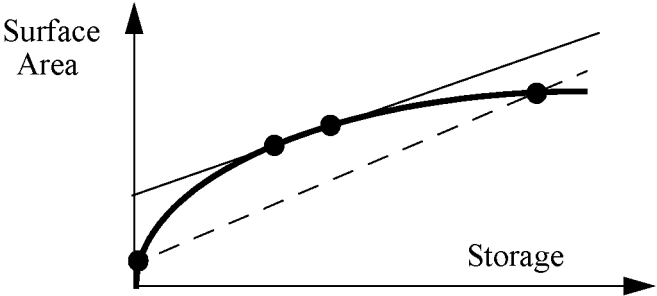
• Surface Area

Type:	Series Slot
Units:	Area
Description:	The area of the water surface at the end of the timestep
Information:	This slot is numerically approximated as a function of Storage (Numerical 2-D Approximation). The relationship between Pool Elevation and Storage comes from the automatically generated Volume Area Table. The table is queried using the user-input points defined in the Surface Area LP Param table.
Defined by:	Numerical 2-D Approximation in terms of Storage, based upon the Volume Area Table. The Surface Area LP Param table values are used as approximation points indexing the Volume Area Table. The preferred order of approximation is substitution, piecewise, two-point line, tangent. Most often the two-point line (secant) approximation will be used.

• Surface Area LP Param

Type:	Table Slot
Units:	Volume
Description:	Specifies the Storage points used to take the tangent, line and piecewise approximations for Surface Area linearization
Information:	The best Storage point to choose for tangent approximation would be the expected storage during the run; for line approximation, the expected maximum and minimum Storage; for piecewise approximation, use points that cover the full range of expected Storage during the run with intermediate points such that a piecewise linear curve reasonably approximates the actual curve. In most cases, the line (secant) approximation will be used. It is important to set these points carefully to minimize approximation error. The following figure represents two alternative selections of points for the line approximation in the Surface Area LP Param table. The linear approximation represented by the dashed line corresponds to the selection of points near the extremes of the Volume Area table. This approximation will tend to result in an under-estimation of Surface Area, and thus an under-estimation of Evaporation and Precipitation Volume. Evaporation losses in the post-optimization rulebased simulation would be greater than the losses approximated in the optimization solution. The linear approximation represented by the solid line corresponds to the selection of points closer together in the Volume Area Table, and is more similar to the Tangent approximation. This approximation would tend to result in an over estimation of Surface Area, and the losses due to Evaporation in the post-optimization rulebased simulation would be less than the approximation in the optimization solution.

Surface area vs storage relationship with two alternative line approximations. Not drawn to scale.



Defined by: User input

• **Volume Area Table**

Type: Table Slot
Units: Volume vs Area
Description: Represents the Storage Volume-Surface Area relationship
Information: This table is read-only and is automatically generated at the start of the run from the Elevation Area Table and the Elevation Volume table. The method starts by copying the Elevation Area Table and then replaces the Pool Elevation Values with the corresponding Storage values. The Storage values are linearly interpolated based on the Elevation Volume Table
Defined by: Automatically generated

Optimization Future Value

This category allows the optimization to provide slots relating to the future value of water. Its appearance is dependent on the Cumulative Storage Value Table method being selected for the Future Value category.

■ **None**

If this method is selected, the future value slots will not be visible, and no linearization will be attempted.

■ **Opt Cumulative Storage Value Table**

If this method is selected, the following slots are visible, and linearization is allowed.

Slots Specific to This Method

• **Cumul Stor Val LP Param**

Type: Table Slot
Units: Volume, Volume, Volume
Description: Specifies the storage points used to take the tangent, line and piecewise approximations for Cumul Stor Val Table linearization

Information:

Defined by: User input or by automated procedure if Cumul Stor Val Linearization Automation category has selected the Marginal Value to Table and Lin method.

Optimization Head Computation

Note: The methods in this category are no longer supported in RPL optimization.

Appearance of this category is dependent on selection of the Independent Linearizations method for the Optimization Power category.

Optimization Power

The Optimization Power category allows the user to select the types of linearizations to be used for linearizing various slots. This category has no dependencies.

■ None

This is the default method. It is an error to have this method selected for Optimization. No slots are added for this method.

■ Independent Linearizations

The Independent Linearizations method linearizes all the variables separately according to the user-selected methods. The variables that need to be linearized vary greatly depending on the other methods selected.

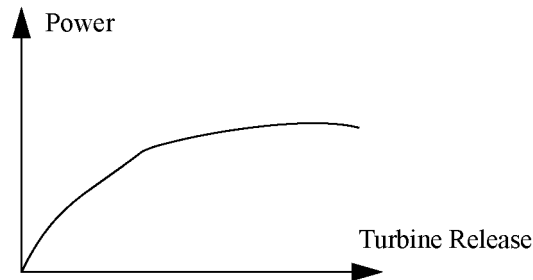
See the various Categories and Linearization Approaches for details.

Slots Specific to This Method

- **Power LP Param**

Type:	Table Slot
Units:	Length and Flow
Description:	Specifies the Operating Head and the flow points used to take the tangent, line and piecewise approximations for Power linearization
Information:	This table must be user input when the Independent Linearizations method is selected. The best Operating Head to choose should be close to the expected head during optimized period. Tangent approximation is generally not used, nor helpful as it often results in non-zero power for zero flow. The suggested points for a line approximation are 0 flow and best turbine flow for the entire plant. The suggested points for piecewise linearization are 0 flow, 1 unit best turbine flow, 2 units best turbine flow ... n units best turbine flow, and maximum turbine flow.
Defined by:	Input

Power turbine release relationship for a fixed operating head valve. Not to scale.



Method Details

If the Optimization problem uses Power, either directly or indirectly, the Power slot is added to the problem.

Before being passed to the Optimization solver, the Power slot is numerically approximated as a function of Operating Head and Turbine Release (Numerical 3-D Approximation).

The relationship between Operating Head and Turbine Release comes from the Plant Power Table. This table must be user input. The table is queried using the points defined in the Power LP Param table.

■ Power Coefficient

The Power Coefficient method models Power at each time step, as follows:

$$Power = \text{Turbine Release} \times \text{Power Coefficient Estimate}$$

Slots Specific to This Method

- **Power Coefficient Estimate**

Type:	Series Slot
Units:	PowerPerFlow
Description:	This represents the estimated Power Coefficient that gets used in the Optimization definition of Power.
Information:	The Power Coefficient Estimate is a required input for the run. It can either be input directly (manually or by DMI), or it could be set by an initialization rule. For example, an initialization rule could set the Power Coefficient Estimate based on a <i>Seed</i> (Slot Cache) value. If the Power Coefficient method is selected, and Power Coefficient Estimate is not an input for all time steps, then the run terminates with an error message.
Defined by:	Input

■ Power Surface Approximation

This method allows for the modeling of dynamic Operating Head to be incorporated into Optimization Power modeling.

It provides a significant improvement in the Power approximation error over the linearization using the Power LP Param table, which assumes a constant Operating Head over the entire run period. The improvement is especially

significant for projects that exhibit a wide range of Operating Head over the course of the run, particularly if the optimization policy is trying to maximize Power or set Power greater than or equal to a value.

The method introduces a new variable, PSA Head Factor, which represents the weighted contributions of Storage, Spill and Tailwater Base Value to Operating Head. The method generates a set of planes, the Power Surface, to constrain power as a function of Turbine Release and the PSA Head Factor.

The Power Surface can be thought of as pavement over a warped bridge that is level on one end and sloped and higher on the other end. The bridge ends represent zero Turbine Release and maximum Turbine Release respectively. In addition, the bridge is bowed concave across the center line everywhere except the level (zero Turbine Release) end. This bowing reflects Power as a function of the Head Factor for a fixed value of Turbine Release. The edges of the bridge represent Power as a function of Turbine Release for the low and high values of the Head Factor. Additional points defining a grid on the surface correspond to intermediate values of Turbine Release and the Head Factor. The user can define dimensions and points used for this grid. A piecewise linear surface is composed of cutting planes with each plane defining one triangle on the surface. A surface underneath the bridge composed of two planes defines a lower bound on Power given Turbine Release and Head Factor values.

Because only two planes can be defined for the lower bounds on power, additional approximation error can be introduced if the optimization policy has an incentive to minimize power. In these cases the optimization solution can have an incentive to under-approximate Power for a given Turbine Release, and thus the Post-optimization Rulebased Simulation will calculate a larger Power value than the optimization value for Power.

Slots Specific to This Method

• PSA Sample Input

Type:	Table Slot
Units:	Flow, Volume, Flow, Length
Description:	This table contains user-specified sample values for Turbine Release, Storage, Total Spill (if applicable) and Tailwater Base Value (if applicable) that are used by RiverWare to compute the PSA Head Factor Weights. The four parameters in this table represent all of the parameters that contribute to Operating Head. In other words, if a value is known for each of these parameters, it is possible to calculate the corresponding Operating Head.
Information:	The values in this table should span the full possible range for each parameter for the given run. For example, the Turbine Release column should contain a value of 0, the highest possible turbine release from the reservoir, and any other intermediate values specified by the user. Specifying tighter upper and lower bounds for each parameter will improve the approximation, but it is important that all possible values for each variable within the run are spanned by the range for that parameter in the table. The user can determine how many rows to include in the table. It is expected that typically, three to four values will be specified for each parameter. An example is shown below. The Spill column should only contain values if a method other than None has been selected in the Spill category. Otherwise the Spill column should be left empty (will display “NaN”). The Tailwater Base Value column should only contain values if the reservoir’s Tailwater Base Value slot is linked to the Pool Elevation slot of a downstream reservoir (that is, if the Downstream Reservoir Pool Elevation contributes to the

calculation of Operating Head). Otherwise the Tailwater Base Value column should be left empty.

Defined by: User input

Turbine Release	Storage	Spill (Total)	Tailwater Base Value
0	0	0	500
100	1000	50	510
200	2000	100	520

• PSA Head Factor Grid Lines

Type: Scalar Slot

Units: None

Description: The number of PSA Head Factor values for each Turbine Release value used when calculating the PSA Grid Points

Information: This must be an integer greater than or equal to 2. The value should typically range from 2 to 4. A larger number will improve the approximation but will increase run time.

Defined by: User input

• PSA Turbine Release Grid Lines

Type: Scalar Slot

Units: None

Description: The number of Turbine Release values for each PSA Head Factor value used when calculating the PSA Grid Points

Information: This must be an integer greater than or equal to 2. The value should typically range from 2 to 4. A larger number will improve the approximation but will increase run time.

Defined by: User input

• PSA Sample Output

Type: Table Slot

Units: Flow, Volume, Flow, Volume, Power

Description: A table containing all permutations of the values entered in the PSA Sample Input table slot along with the Power calculated for each combination of Sample Input values. This sample output is used to calculate the PSA Head Factor Weights.

Information: RiverWare will populate this table at the beginning of the run. For the PSA Sample Input table shown above, with three values in each of the four columns, this table would contain 81 rows, one for each permutation. For a given combination of Turbine Release, Storage, Spill and Tailwater Base Value, RiverWare can calculate the corresponding Power. The third column will contain the Downstream Storage values corresponding to the Tailwater Base Value entries in the PSA Sample Input slot.

Defined by: Populated by RiverWare at the beginning of the run.
For each row i in the table:

$$SamplePower_i = f(SampleTurbineRelease_i, SampleStorage_i, SampleSpill_i, SampleTailwaterBaseValue_i)$$

The calculation of SamplePower depends on the selected Power and Tailwater methods.

• PSA Head Factor Weights

Type:	Table Slot
Units:	PerTime, None, PerTime
Description:	The weights for Storage, Spill and Downstream Storage used when calculating PSA Head Factor
Information:	This is a 1 x 3 table with a single weight for Storage, Spill and Downstream Storage. The weights are calculated by RiverWare by performing a regression on the values in the PSA Sample Outputs table slot. If one of the parameters is not used (contains NaN for all rows in the PSA Sample Input table), the corresponding weight in this table slot will be zero.
Defined by:	Calculated by regression at the beginning of the run

• PSA Head Factor

Type:	Series Slot
Units:	Flow
Description:	This slot is a variable in the optimization solution and represents the weighted contribution of Storage, Spill and Downstream Storage to Operating Head and thus power. It can be thought of as a storage analog of Operating Head.
Information:	This variable is used in the automatically generated constraints that define Power. It will not, generally display any values, but the user can write a post-optimization rule that calculates the value of this variable at each time step using the equation shown below and the values in the PSA Head Factor Weights table slot.
Defined by:	$PSA\ Head\ Factor = \alpha \cdot Storage + \beta \cdot Spill + \gamma \cdot Downstream\ Storage$ The coefficients α, β and γ come from the PSA Head Factor Weights table slot.

• PSA Grid Points

Type:	Table Slot
Units:	Flow, Volume, Flow, Volume, Power, Flow
Description:	Points used to calculate the planes for the PSA Max Constraints and PSA Min Constraints, one row for each point with columns for Turbine Release, Storage, Spill, Downstream Storage, Power and PSA Head Factor. Turbine Release Column—Smallest and largest values from PSA Sample Input and evenly spaced intermediate values corresponding to the number of lines in the PSA Turbine Release Grid Lines scalar slot Storage, Spill and Downstream Storage Columns—Smallest and largest values from PSA Sample Input and evenly spaced intermediate values corresponding to the number of lines in the PSA Head Factor Grid Lines scalar slot Power Column—Determined by selected Power method, for each row i in the table $Power_i = f(TurbineRelease_i, Storage_i, Spill_i, DownstreamStorage_i)$ Head Factor Column—The coefficients α, β and γ come from the PSA Head Factor Weights table slot.

$$HeadFactor_i = \alpha \cdot Storage_i + \beta \cdot Spill_i + \gamma \cdot DownstreamStorage_i$$

- Information:** The number of rows (points) in this table equals the product of the number of lines in the PSA Head Factor Grid Lines and PSA Turbine Release Grid Lines scalar slots. For each grid point, RiverWare calculates the value of Power and the PSA Head Factor corresponding to the parameter values in that row.
- Defined by:** Populated by RiverWare at the beginning of the run.

• PSA Max Constraints

- Type:** Table Slot
- Units:** Flow, Flow, Flow, Flow, Flow, FLOW, PowerPerFlow, PowerPerFlow, Power
- Description:** This table contains the planes that define the upper bounds for power as a function of Turbine Release and PSA Head Factor, one row for each plane. Power is constrained to be less than or equal to the planes defined in this table. The first six columns contain three pairs of Turbine Release and PSA Head Factor values that define three points on the plane. The remaining three columns contain the corresponding Turbine Release Coefficient, Head Factor Coefficient and Constant Term that define the same plane and are used in the actual constraint expressions.
- Information:** The points used to define the planes come from the Turbine Release and Head Factor points in the PSA Grid Points table slot. RiverWare will not use all of the possible combinations of Turbine Release and Head Factor points but rather will only use the combinations necessary to define the Power Surface.
- Defined by:** Populated by RiverWare at the beginning of the run
For each row i in the PSA Max Constraints table slot a constraint is added to the optimization problem:
$$Power \leq u_i \cdot \text{Turbine Release} + v_i \cdot \text{PSA Head Factor} + w_i$$

The coefficients u_i , v_i and w_i are the Turbine Release Coefficient, Head Factor Coefficient and Constant Term from row i of the PSA Max Constraints table. Turbine Release and PSA Head Factor are the actual variables in the optimization problem.

• PSA Min Constraints

- Type:** Table Slot
- Units:** Flow, Flow, Flow, Flow, Flow, FLOW, PowerPerFlow, PowerPerFlow, Power
- Description:** This table contains the planes that define the lower bounds for power as a function of Turbine Release and PSA Head Factor, one row for each plane. Power is constrained to be greater than or equal to the planes defined in this table. The first six columns contain three pairs of Turbine Release and PSA Head Factor values that define three points on the plane. The remaining three columns contain the corresponding Turbine Release Coefficient, Head Factor Coefficient and Constant Term that define the same plane and are used in the actual constraint expressions
- Information:** This table will always contain two rows. Only two planes can be defined for the lower bounds on power.
- Defined by:** Populated by RiverWare at the beginning of the run.
For each row i in the PSA Min Constraints table slot, a constraint is added to the optimization problem:
$$Power \geq u_i \cdot \text{Turbine Release} + v_i \cdot \text{PSA Head Factor} + w_i$$

The coefficients u_i , v_i and w_i are the Turbine Release Coefficient, Head Factor Coefficient and Constant Term from row yc of the PSA Min Constraints table. Turbine Release and PSA Head Factor are the actual variables in the optimization problem.

Method Details

The following slots require input from the user:

- [PSA Sample Input](#), page 159
- [PSA Head Factor Grid Lines](#), page 160
- [PSA Turbine Release Grid Lines](#), page 160

RiverWare carries out the following steps to populate the remaining table slots associated with this method.

1. Populate the PSA Sample Output Table with all permutations of parameters in the PSA Sample Input table slot.
2. Calculate the PSA Head Factor Weights using a regression based on values in the PSA Sample Output table slot.
3. Add defining constraints for PSA Head Factor using the PSA Head Factor Weights.
4. Populate the PSA Grid Points table slot based on the number of grid lines specified by the user.
5. Generate upper bound planes using the PSA Grid Points and add the corresponding constraints on Power to the Optimization problem.
6. Generate lower bound planes using the PSA Grid Points and add the corresponding constraints on Power to the Optimization problem.

Power Surface Definition Details

This section provides details of the calculations carried out by RiverWare to generate the constraints that define the Power Surface. This material is included for reference only.

1. RiverWare populates the PSA Sample Output table slot. All permutations of values in the PSA Sample Input table slot are determined, and a row for each is added to the PSA Sample Output table slot. RiverWare may make some adjustments to prevent infeasible combinations. For example, the highest Turbine Release value may not be possible for all combinations of Storage, Spill and Tailwater Base Value. RiverWare will adjust the Turbine Release to use the maximum turbine release for the resulting Operating Head. The table is organized in blocks of constant Turbine Release, with the lowest Turbine Release block first and then increasing Turbine Release. The values in the remaining columns are ordered corresponding to increasing Operating Head. Storage is increasing. Spill and Tailwater Base Value (or Downstream Storage) are decreasing. For each combination of values (each row), RiverWare calculates the corresponding Operating Head and Power. The Power value is added to each row in the PSA Sample Output table. The details of these calculations are shown here for a given row i in the PSA Sample Output table.

Note: $Power_i$ is the only calculated quantity displayed in the table. Values from the intermediate calculations shown below are not displayed in the table.

$$PoolElevation_i = f(Storage_i)$$

Pool Elevation is calculated by interpolation on the reservoir Elevation Volume Table.

Chapter 5

Optimization Objects and Methods

If Tailwater Base Value is linked to Pool Elevation of a downstream reservoir:

$$TailwaterBaseValue_i = DownstreamPoolElevation_i = f(DownstreamStorage_i)$$

Tailwater Base Value is calculated by interpolation on the downstream reservoir Elevation Volume Table. Tailwater Base Value is only calculated from the Downstream Storage if either Base Value Plus Lookup Table, Stage Flow Lookup Table or Coefficients Table is the selected Tailwater method *and* Tailwater Base Value is linked to the Pool Elevation of the downstream reservoir. Otherwise Tailwater Base Value is unused, and thus Operating Head and Power are not dependent on Downstream Storage.

$$Outflow_i = TurbineRelease_i + Spill_i$$

Spill will only be non-zero if a method other than None is selected in the Spill category.

$$TailwaterElevation_i = f(Outflow_i, TailwaterBaseValue_i)$$

The tailwater calculation is dependent on the selected method in the Tailwater category.

$$OperatingHead_i = PoolElevation_i - TailwaterElevation_i$$

$$Power_i = f(TurbineRelease_i, OperatingHead_i)$$

The Power calculation is dependent on the selected method in the Power category.

2. RiverWare calculates the PSA Head Factor Weights using a combination of linear regression and averages.

FOR EACH Turbine Release value i

FOR EACH combination of Spill and Tailwater Base Value (or Downstream Storage), k , perform a linear regression to calculate a temporary Storage coefficient g_{ik} :

$$g_{ik} = \frac{n \sum (Power_j \times Storage_j) - \left(\left(\sum Power_j \right) \left(\sum Storage_j \right) \right)}{n \left(\sum Storage_j^2 \right) - \left(\sum Storage_j \right)^2}$$

where n is the number of Storage values.

Calculate the temporary Storage coefficient for the given Turbine Release, g_i :

$$g_i = \frac{\sum g_{ik}}{n}$$

where n is the number of Spill and Downstream Storage combinations.

FOR EACH combination of Storage and Downstream Storage, k , perform a linear regression to calculate a temporary Spill coefficient e_{ik} :

$$e_{ik} = \frac{n \sum (Power_j \times Spill_j) - \left(\left(\sum Power_j \right) \left(\sum Spill_j \right) \right)}{n \left(\sum Spill_j^2 \right) - \left(\sum Spill_j \right)^2}$$

where n is the number of Spill values.

Calculate the temporary Spill coefficient for the given Turbine Release, e_i :

$$e_i = \frac{\sum e_{ik}}{n}$$

where n is the number of Storage and Downstream Storage combinations.

FOR EACH combination of Storage and Spill, k , perform a linear regression to calculate a temporary Downstream Storage coefficient f_{ik} :

$$f_{ik} = \frac{n \sum (Power_j \times DownstreamStorage_j) - \left(\left(\sum Power_j \right) \left(\sum DownstreamStorage_j \right) \right)}{n \left(\sum DownstreamStorage_j^2 \right) - \left(\sum DownstreamStorage_j \right)^2}$$

where n is the number of Downstream Storage values.

Calculate the temporary Downstream Storage coefficient for the given Turbine Release, f_i :

$$f_i = \frac{\sum f_{ik}}{n}$$

where n is the number of Storage and Spill combinations.

Calculate average ratios over all Turbine Release values:

$$c = \frac{\sum (e_i / g_i)}{n}$$

$$d = \frac{\sum (f_i / g_i)}{n}$$

where n is the number of Turbine Release values.

Calculate the final Head Factor Weights:

$$\text{Storage Weight} = \frac{1}{c}$$

$$\text{Spill Weight} = -1$$

If Tailwater Base Value is linked to the downstream Pool Elevation:

$$\text{Downstream Storage Weight} = \frac{d}{c}$$

These three weight values are displayed in the three columns of the PSA Head Factor Weights table slot.

In the optimization solution, the PSA Head Factor is defined at a given time step by

- If Tailwater Base Value is linked to the downstream Pool Elevation:

$$\text{PSA Head Factor} = \text{StorageWeight} \times \text{Storage} + \text{SpillWeight} \times \text{Spill} + \text{DownstreamStorageWeight} \times \text{Downstream Storage}$$

- If Tailwater Base Value is not linked:

Chapter 5

Optimization Objects and Methods

$$\text{PSA Head Factor} = \text{StorageWeight} \times \text{Storage} + \text{SpillWeight} \times \text{Spill}$$

- RiverWare calculates the grid points for the PSA Grid Points table slot. The number of rows (points) in this table equals the product of the number of lines in the PSA Head Factor Grid Lines and PSA Turbine Release Grid Lines scalar slots. The smallest and largest values in the PSA Sample Input slot lead to the smallest and largest values for each of the parameters that affect power (Turbine Release, Storage, Spill and Downstream Storage). One row in this table will correspond to the smallest Turbine Release, smallest Storage, largest Spill and largest Downstream Storage, and thus smallest power. Another row will correspond to the largest Turbine Release, largest Storage, smallest Spill and smallest Downstream Storage, and thus the largest Power. RiverWare generates evenly spaced values between the extremes for each variable. Within the PSA Grid Points table, for a given Turbine Release value, Storage is increasing and Spill and Downstream Storage are decreasing. For each grid point, RiverWare calculates the value of Power and the PSA Head Factor corresponding to the parameter values in each row and adds these to the final two columns in the table.

For each row i in the table:

$$\text{Power}_i = f(\text{TurbineRelease}_i, \text{Storage}_i, \text{Spill}_i, \text{DownstreamStorage}_i)$$

The Power calculation depends on the method selected in the Power category.

$$\text{HeadFactor}_i = \alpha \cdot \text{Storage}_i + \beta \cdot \text{Spill}_i + \gamma \cdot \text{DownstreamStorage}_i$$

The coefficients α , β and γ come from the PSA Head Factor Weights table slot.

- RiverWare calculates the upper bound planes. These are entered in the PSA Max Constraints table slot, one row for each plane, where the planes represent Power as a function of Turbine Release and PSA Head Factor. For each combination of three points in the PSA Grid Points Table, RiverWare calculates the corresponding plane. For any three points, the plane is defined by three linear equations, which can be represented in matrix form.

$$\bar{P} = A\bar{u}$$

$$\bar{P} = \begin{bmatrix} P_1 \\ P_2 \\ P_3 \end{bmatrix} \quad A = \begin{bmatrix} QT_1 & HF_1 & 1 \\ QT_2 & HF_2 & 1 \\ QT_3 & HF_3 & 1 \end{bmatrix} \quad \bar{u} = \begin{bmatrix} u \\ v \\ w \end{bmatrix}$$

The coefficients that define the plane are solved for by

$$\bar{u} = A^{-1}\bar{P}$$

RiverWare only uses the planes that are necessary to define the Power Surface. Each plane is checked against all remaining points in the PSA Grid Points table. If the Power value for any of the remaining points lies above the given plane, then that plane is rejected (that is, it would over-constrain Power, resulting in an under-estimation of Power). For each plane i that is used, RiverWare adds the following constraint to the optimization problem for each time step:

$$\text{Power} \leq u_i \cdot \text{Turbine Release} + v_i \cdot \text{PSA Head Factor} + w_i$$

- RiverWare calculates the lower bound planes. These are entered in the PSA Min Constraints table slot, one row for each plane. Only two planes can be calculated for the lower bounds. The points used for the two planes are as follows:

Plane 1:

- Point A1 - Min Turbine Release, Max Head Factor
- Point B1 - Max Turbine Release, Max Head Factor
- Point C1 - Min Turbine Release, Min Head Factor

Plane 2:

- Point A2 - Min Turbine Release, Max Head Factor
- Point B2 - Max Turbine Release, Max Head Factor
- Point C2 - Max Turbine Release, Max Head Factor

The coefficients defining the planes are calculated using the same matrix calculation described for the upper bound planes. For each lower bound plane i , RiverWare adds the following constraint to the optimization problem for each time step:

$$Power \geq u_i \cdot \text{Turbine Release} + v_i \cdot \text{PSA Head Factor} + w_i$$

Note: Regarding the lower bound approximation, because only two planes can be defined for the lower bounds on power, additional approximation error can be introduced if the optimization policy has an incentive to minimize power. This can occur if Power is included in a Minimize objective. It can also occur if the optimization problem contains constraints that require Power to be less than or equal to value.

Note: These types of constraints can be introduced through multiple forms of RPL optimization goals. The first is basic less than or equal to constraint.

ADD CONSTRAINT Res.Power[t] ≤ Value

Policy that requires Power to be equal to a value also adds a less than or equal to constraint. The following RPL statement:

ADD CONSTRAINT Res.Power[t] = Value

actually adds the following two constraints to the optimization problem:

Res.Power[t] ≤ Value

Res.Power[t] ≥ Value

A constraint to require the sum of Power from multiple reservoirs to be less than or equal to a value will have a similar effect. A final manner in which this can occur is through policy that minimizes Power indirectly through user-defined variables. For example, assume a user-defined variable is defined by the following RPL constraint:

ADD CONSTRAINT Res.Power[t] + UserVariable[t] = Value1

Then later the variable is constrained by

ADD CONSTRAINT UserVariable[t] ≥ Value2

An equivalent constraint would be

ADD CONSTRAINT Res.Power[t] ≤ Value1 – Value2

It is possible to reduce the approximation error in these cases by adding constraints to the RPL Optimization Goal Set that essentially create more restrictive planes for the lower bound. These types of constraints are model-

specific. Contact CADSWES if assistance is required to add more restrictive lower bounds to the Power Surface Approximation.

Optimization Pump Power

The Optimization Pump Power category allows the user to specify the parameters for linearizing pumping power. This category has no dependencies.

■ None

This is the default method. No slots are added for this method.

■ Pump Independent Linearizations

The Independent Linearizations method linearizes all the variables separately.

Although the corresponding Simulation Pump Power method may model pumping at the unit level. This method can only model pumping at the plant level.

Slots Specific to This Method

- **Pump Power Lin Data**

Type:	Table Slot
Units:	Length, Flow, Power
Description:	3-D table used to determine plant pumping power using interpolation. This table is analogous to the Plant Power Table for the Plant Efficiency Curve Power Method. (See “ Plant Efficiency Curve ” in <i>Objects and Methods</i> .)
Information:	Data must be entered into the table in increasing, concave blocks of the same Operating Head for the three-dimensional table interpolation to work correctly. For every block of the same Operating Head in column 1, Pumped Flow and the corresponding Pump Power should be listed in increasing, concave order in columns 2 and 3. There must be a point of zero Pumped Flow and zero Pump Power for each Operating Head. The following table is an example of the proper way to formulate the Pump Power Lin Data table.

Operating Head	Pumped Flow	Pump Power
30	0	0
30	100	100
30	200	175
40	0	0
40	100	125
40	220	195

Operating Head	Pumped Flow	Pump Power
50	0	0
50	110	147
50	250	205

Defined by: Input

• Max Pump Q

Type: Table slot

Units: Length, Flow

Description: Operating Head vs. Pumped Flow for maximum pumped flow capacity for each Operating Head in the Pump Power Lin Data table

Information: The Operating Head values in this table should match the Operating Head values from the Pump Power Lin Data table, one row for each Operating Head. The Pump Q Capacity for each Operating Head should match the highest Pumped Flow value for that Operating Head from the Pump Power Lin Data table.

Defined by: Input

• Pump Power LP Param

Type: Table Slot

Units: Length and Flow

Description: Specifies the Operating Head and the flow points used to take the tangent, line and piecewise approximations for the Pump Power linearization

Information: This table must be user input. The best Operating Head to choose should be close to the expected head during optimized period. Tangent approximation is generally not used, nor helpful as it often results in non-zero pump power for zero flow. The suggested points for a line approximation are 0 flow and best pumped flow for the entire plant. The suggested points for piecewise linearization are 0 flow, 1 unit best pumped flow, 2 units best pumped flow ... n units best pumped flow, and maximum pumped flow.

Defined by: Input

• Pump Q Capacity

Type: Series Slot

Units: Flow

Description: The linearized maximum pumped flow for the entire plant at each timestep.

Information: This slot is computed during Optimization. It will not display values unless a Post-optimization Rule is written to explicitly set the slot value using the OptValue function.

Defined by: Numerical 2-D approximation in terms of Operating Head, based on the Max Pump Q table and the Pump Q Capacity LP Param Table

• Pump Q Capacity LP Param

Type: Table Slot

Units: Length, Length, Length

Description:	Specifies the Operating Head points used to take the tangent, line and piecewise approximation for the Pump Q Capacity linearization.
Information:	This table must be user input. In reality, only the Piecewise approximation is used, so only the Piecewise column is required to have data. The Operating Head values specified should cover the range of expected Operating Head for the run and should result in a concave curve of Pump Q Capacity vs. Operating Head. It is best to define this linearized curve of Pump Q Capacity such that it does not go above the full curve defined by the Pump Q Capacity table slot.

Method Details

If the Optimization problem uses Pump Power, either directly or indirectly, the Pump Power slot is added to the problem.

Before being passed to the Optimization solver, the Pump Power slot is numerically approximated as a function of Operating Head and Pumped Flow (Numerical 3-D Approximation).

The relationship between Operating Head and Pumped Flow comes from the Pump Power Lin Data table. This table must be user input. The table is queried using the points defined in the Pump Power LP Param table.

Optimization Reserves

This category depends on selecting either Plant Power Coefficient or Plant Efficiency Curve in the Power category. The methods in this category can be used to account for power reserve requirements in the optimization policy. There is no analogous simulation method associated with this category.

■ None

This is the default, no action method. No new slots will be added, and no new variables or constraints will be added to the optimization problem.

■ Constraint Based Single Timestep

This method introduces a set of duplicate variables that represent the value of standard variables assuming full deployment of either upward or downward reserves. In the RPL Optimization Goal Set, users can then write parallel constraints using the duplicate reserve variables for any constraint that cannot be violated while deploying reserves. The purpose of the new variables and their parallel constraints is so that the only reserves that are credited to a hydropower project are those that can actually be reasonably deployed. In other words, it will not *count* reserves that could not be deployed without violating specified constraints. The new method also introduces new variables for upward and downward reserves, which allow the user to write policy on the reserves themselves, for example, project minimum reserve requirements or total system reserve requirements. This method only models reserves from the generating portion of the Pumped Storage Reservoir. It does not model reserves that might be provided by pumping.

One significant assumption is made that the deployment of these reserves are temporary and only meant to cover a single timestep. It is assumed that in the event reserves are deployed in actual operations, adjustments would be made for later time periods, including reestablishing reserves. With this assumption, the consequences of deploying reserves at a reservoir are limited to that reservoir and that timestep; constraints on downstream

reservoirs and later timesteps are not considered when calculating the reserves that can be credited at a given timestep.

The deployment variables are constrained similarly to the original variables with two exceptions. First, the mass balance constraints with other reservoirs or intermediate reaches are omitted (within RiverWare). Second, if there are any constraints that can be violated during deployment then these constraints can also be omitted (within the Optimization Goal Set). The deployment variables will typically use the same linear approximations as the original variables. For example, Pool Elevation with Up Reserve will be approximated using the Pool Elevation LP Param table slot.

Note: This method has no analogous simulation method. The slots introduced by this method can be referenced in the RPL optimization policy to constrain the solution, but they will not, generally, display values after the post-optimization rulebased simulation like the standard variables, such as Outflow or Pool Elevation. The user does have the option of writing customized post-optimization rules to set values in these slots based on user-specified logic or using the OptValue or OptValuePiecewise predefined functions.

Slots Specific to This Method

• Up Reserve

Type:	Series Slot
Units:	Power
Description:	The upward reserves available; the amount the reservoir could increase generation on the given timestep
Information:	Introducing this variable in the optimization policy forces a two-point line approximation to be used for both Power and Power with Up Reserve if Independent Linearizations is selected in the Optimization Power category. This may introduce additional approximation error.
Defined by:	$Up\ Reserve = Power\ with\ Up\ Reserve - Power$

• Down Reserve

Type:	Series Slot
Units:	Power
Description:	The downward reserves available; the amount the reservoir could decrease generation on the given timestep
Information:	Introducing this variable forces a two-point line approximation to be used for both Power and Power with Down Reserve if Independent Linearizations is selected in the Optimization Power category. This may introduce additional approximation error.
Defined by:	$Down\ Reserve = Power - Power\ with\ Down\ Reserve$

• Power with Up Reserve

Type:	Series Slot
Units:	Power
Description:	The average Power generated if full upward reserves were deployed
Information:	This variable is defined by the Power approximation as a function of Turbine Release with Up Reserve and a combination of other variables. The set of other

variables depends on which Power approximation is being applied. The Power approximation that is applied is dependent on the selected method in the Optimization Power category.

Defined by: $\text{Power with Up Reserve} = f(\text{Turbine Release with Up Reserve}, \dots)$

• Power with Down Reserve

Type: Series Slot

Units: Power

Description: The average Power generated if full downward reserves were deployed

Information: This variable is defined by the Power linear approximation as a function of Turbine Release with Down Reserve and a combination of other variables. The set of other variables depends on which Power approximation is being applied. The Power approximation that is applied is dependent on the selected method in the Optimization Power category.

Defined by: $\text{Power with Down Reserve} = f(\text{Turbine Release with Down Reserve}, \dots)$

• Turbine Release with Up Reserve

Type: Series Slot

Units: Flow

Description: The Turbine Release at full deployment of upward reserves

Information: This variable is constrained to be less than or equal to Turbine Capacity with Up Reserve.

Defined by: $\text{Turbine Release with Up Reserve} = \text{Turbine Release} + \text{Turbine Change with Up Reserve}$
and

$\text{Turbine Release with Up Reserve} \leq \text{Turbine Capacity with Up Reserve}$

• Turbine Change with Up Reserve

Type: Series Slot

Units: Flow

Description: The amount Turbine Release would change to fully deploy all upward reserves

Information: This variable is expected to always be non-negative, and thus the slot configuration lower bound should be set to 0.

Defined by: $\text{Turbine Release with Up Reserve} = \text{Turbine Release} + \text{Turbine Change with Up Reserve}$

• Turbine Capacity with Up Reserve

Type: Series Slot

Units: Flow

Description: The maximum Turbine Release possible given the full deployment of upward reserves

Information: This variable is defined by the Turbine Capacity approximation as a function of Operating Head with Up Reserve.

Defined by: $\text{Turbine Capacity with Up Reserve} = f(\text{Operating Head with Up Reserve})$

• Operating Head with Up Reserve

Type: Series Slot

Units: Flow

Description: The Operating Head at full deployment of upward reserves.

Information:

Defined by: Operating Head with Up Reserve =

$$\frac{\text{Pool Elevation with Up Reserve} + \text{Pool Elevation}[t - 1]}{2} - \text{Tailwater Elevation with Up Reserve}$$

• Spill with Up Reserve

Type: Series Slot

Units: Flow

Description: The amount of total Spill at full deployment of upward reserves

Information: This could be equal to Spill, or it could be less than Spill if flow is allowed to be shifted from Spill to Turbine Release during deployment. This depends on how the RPL policy is formulated.

Defined by: Spill with Up Reserve =

$$\text{Unregulated Spill with Up Reserve} + \text{Regulated Spill with Up Reserve} + \text{Bypass with Up Reserve}$$

• Unregulated Spill with Up Reserve

Type: Series Slot

Units: Flow

Description: The amount of Unregulated Spill at full deployment of upward reserves

Information: This quantity is defined by the Unregulated Spill linear approximation as a function of Storage with Up Reserve. This slot will get added regardless of which Spill method is selected. If the selected Spill method does not include Unregulated Spill, then the value of this variable will automatically be set to zero.

Defined by: Unregulated Spill with Up Reserve = $f(\text{Storage with Up Reserve})$

• Regulated Spill with Up Reserve

Type: Series Slot

Units: Flow

Description: The amount of Regulated Spill at full deployment of upward reserves

Information: This could be equal to Regulated Spill, or it could be less than Regulated Spill if flow is allowed to be shifted from Spill to Turbine Release during deployment, or it could be greater than Regulated Spill if the reservoir has a minimum spill requirement as a percentage of total outflow. This depends on how the RPL policy is formulated. This slot will get added regardless of which Spill method is selected. If the selected Spill method does not include Regulated Spill, then the value of this variable will automatically be set to zero, and the slot should not be referenced in the RPL optimization policy.

Defined by: Regulated Spill with Up Reserve = Regulated Spill + Regulated Spill Change with Up Reserve
and

$$\text{Regulated Spill with Up Reserve} \leq \text{Regulated Spill Capacity with Up Reserve}$$

- **Regulated Spill Change with Up Reserve**

Type:	Series Slot
Units:	Flow
Description:	The amount Regulated Spill would change to fully deploy all upward reserves
Information:	Often this variable will be negative (or 0, and thus the slot Upper Bound will often be set to 0); however it could be positive if the reservoir has a minimum spill requirement as a percentage of total outflow. It could be constrained to be 0, directly or indirectly, by RPL policy, or it could be allowed to be greater than or less than zero. This slot will get added regardless of which Spill method is selected. If the selected Spill method does not include Regulated Spill, then the slot should not be referenced in the RPL optimization policy.
Defined by:	$\text{Regulated Spill with Up Reserve} = \text{Regulated Spill} + \text{Regulated Spill Change with Up Reserve}$

- **Regulated Spill Capacity with Up Reserve**

Type:	Series Slot
Units:	Flow
Description:	The maximum Regulated Spill given full deployment of upward reserves
Information:	This variable is defined by the Regulated Spill Capacity approximation as a function of Storage with Up Reserve. This slot will get added regardless of which Spill method is selected. Users will not typically need to do anything with this slot.
Defined by:	$\text{Regulated Spill Capacity with Up Reserve} = f(\text{Storage with Up Reserve})$

- **Bypass with Up Reserve**

Type:	Series Slot
Units:	Flow
Description:	The amount of Bypass at full deployment of upward reserves
Information:	This could be equal to Bypass, or it could be less than Bypass if flow is allowed to be shifted from Bypass to Turbine Release during deployment, or it could be greater than Bypass if the reservoir has a minimum bypass requirement as a percentage of total outflow. This depends on how the RPL policy is formulated. This slot will get added regardless of which Spill method is selected. If the selected Spill method does not include Bypass, then the value of this variable will automatically be set to zero, and the slot should not be referenced in the RPL optimization policy.
Defined by:	$\text{Bypass with Up Reserve} = \text{Bypass} + \text{Bypass Change with Up Reserve}$ and $\text{Bypass with Up Reserve} \leq \text{Bypass Capacity with Up Reserve}$

- **Bypass Change with Up Reserve**

Type:	Series Slot
Units:	Flow
Description:	The amount Bypass would change to fully deploy all upward reserves
Information:	Often this variable will be negative (or 0, and thus the slot Upper Bound will often be set to 0); however it could be positive if the reservoir has a minimum bypass requirement as a percentage of total outflow. It could be constrained to be 0, directly or indirectly, by RPL policy, or it could be allowed to be greater than or less than

zero. This slot will get added regardless of which Spill method is selected. If the selected Spill method does not include Bypass, then the slot should not be referenced in the RPL optimization policy.

Defined by: $\text{Bypass with Up Reserve} = \text{Bypass} + \text{Bypass Change with Up Reserve}$

• Bypass Capacity with Up Reserve

Type: Series Slot

Units: Flow

Description: The maximum Bypass given full deployment of upward reserves

Information: This variable is defined by the Bypass Capacity approximation as a function of Storage with Up Reserve. This slot will get added regardless of which Spill method is selected. Users will not typically need to do anything with this slot.

Defined by: $\text{Bypass Capacity with Up Reserve} = f(\text{Storage with Up Reserve})$

• Outflow with Up Reserve

Type: Series Slot

Units: Flow

Description: The amount of total Outflow at full deployment of upward reserves

Information:

Defined by: $\text{Outflow with Up Reserve} = \text{Turbine Release with Up Reserve} + \text{Spill with Up Reserve}$

• Storage with Up Reserve

Type: Series Slot

Units: Volume

Description: The end of timestep Storage if full upward reserves were deployed

Information: This variable definition differs from the standard Storage variable only in the Outflow term. The previous Storage and the Net Inflow are the same. Net Inflow represents all gains and losses, including Inflow, Hydrologic Inflow, Precipitation, Evaporation, and so on.

Defined by: $\text{Storage with Up Reserve} = \text{Storage}[t - 1] + (\text{NetInflow} - \text{Outflow with Up Reserve}) \times \text{Timestep}$

• Pool Elevation with Up Reserve

Type: Series Slot

Units: Length

Description: The end of timestep Pool Elevation if full upward reserves were deployed

Information: This variable is defined by the Pool Elevation linear approximation as a function of Storage with Up Reserve.

Defined by: $\text{Pool Elevation with Up Reserve} = f(\text{Storage with Up Reserve})$

• Tailwater Elevation with Up Reserve

Type: Series Slot

Units: Length

Description: The average Tailwater Elevation if full upward reserves were deployed

- Information:** This variable is defined by the Tailwater Elevation linear approximation as a function of Outflow with Up Reserve and the Tailwater Base Value (if applicable based on the selected Optimization Tailwater method). Tailwater Base Value is unaffected by the Reserves modeling and will be the same for both Tailwater Elevation and Tailwater Elevation with Up Reserves. The Tailwater Elevation linear approximation that is applied is dependent on the selected method in the Optimization Tailwater category.
- Defined by:** $\text{Tailwater Elevation with Up Reserve} = f(\text{Outflow with Up Reserve}, \text{Tailwater Base Value})$

• Temp Tailwater Lookup with Up Reserve

- Type:** Series Slot
- Units:** Length
- Description:** This slot only applies when the Base Value Plus Lookup Table method is selected in the Tailwater category. It represents the lookup value from the Tailwater Table as a function of Outflow with Up Reserve using the Tailwater linear approximation.
- Information:** This slot will get added regardless of which Tailwater method is selected but will not get used unless the Base Value Plus Lookup method is selected. Users should not need to do anything with this slot.
- Defined by:** $\text{Temp Tailwater Lookup with Up Reserve} = f(\text{Outflow with Up Reserve})$

• PSA Head Factor with Up Reserve

- Type:** Series Slot
- Units:** Flow
- Description:** This slot only applies when the Power Surface Approximation method is selected in the Optimization Power category. It represents the component of Operating Head that is dependent on Storage with Up Reserve and Spill with Up Reserve. It is calculated using the same coefficients used for the Standard PSA Head Factor variable.
- Information:** This slot will get added regardless of which Optimization Power method is selected but will not get used unless the Power Surface Approximation method is selected. Users should not need to do anything with this slot.
- Defined by:** $\text{PSA Head Adjustment with Up Reserve} = f(\text{Storage with Up Reserve}, \text{Spill with Up Reserve})$

• Turbine Release with Down Reserve

- Type:** Series Slot
- Units:** Flow
- Description:** The Turbine Release at full deployment of downward reserves
- Information:**
- Defined by:** $\text{Turbine Release with Down Reserve} = \text{Turbine Release} + \text{Turbine Change with Down Reserve}$
and
 $\text{Turbine Release with Down Reserve} \leq \text{Turbine Capacity with Down Reserve}$

• Turbine Change with Down Reserve

- Type:** Series Slot
- Units:** Flow

Description: The amount Turbine Release would change to fully deploy all downward reserves
Information: This variable is expected to always be negative or zero, and thus the slot configuration Upper Bound should be set to 0.
Defined by: Turbine Release with Down Reserve = Turbine Release + Turbine Change with Down Reserve

• Turbine Capacity with Down Reserve

Type: Series Slot
Units: Flow
Description: The maximum Turbine Release possible given the full deployment of downward reserves
Information: This variable is defined by the Turbine Capacity approximation as a function of Operating Head with Down Reserve.
Defined by: Turbine Capacity with Down Reserve = $f(\text{Operating Head with Down Reserve})$

• Operating Head with Down Reserve

Type: Series Slot
Units: Flow
Description: The Operating Head at full deployment of downward reserves.
Information:
Defined by: Operating Head with Down Reserve = $\frac{\text{Pool Elevation with Down Reserve} + \text{Pool Elevation}[t - 1]}{2}$
 – Tailwater Elevation with Down Reserve

• Spill with Down Reserve

Type: Series Slot
Units: Flow
Description: The amount of total Spill at full deployment of downward reserves
Information: This could be equal to Spill, or it could be greater than Spill if flow is allowed to be shifted from Turbine Release to Spill during deployment. This depends on how the RPL policy is formulated.
Defined by: Spill with Down Reserve = Unregulated Spill with Down Reserve + Regulated Spill with Down Reserve + Bypass with Down Reserve

• Unregulated Spill with Down Reserve

Type: Series Slot
Units: Flow
Description: The amount of Unregulated Spill at full deployment of downward reserves
Information: This quantity is defined by the Unregulated Spill linear approximation as a function of Storage with Down Reserve. This slot will get added regardless of which Spill method is selected. If the selected Spill method does not include Unregulated Spill, then the value of this variable will automatically be set to zero.
Defined by: Unregulated Spill with Down Reserve = $f(\text{Storage with Down Reserve})$

- **Regulated Spill with Down Reserve**

Type:	Series Slot
Units:	Flow
Description:	The amount of Regulated Spill at full deployment of downward reserves
Information:	This could be equal to Regulated Spill, or it could be greater than Regulated Spill if flow is allowed to be shifted from Turbine Release to Spill during deployment. This depends on how the RPL policy is formulated. This slot will get added regardless of which Spill method is selected. If the selected Spill method does not include Regulated Spill, then the value of this variable will automatically be set to zero, and the slot should not be referenced in the RPL optimization policy.
Defined by:	$\text{Regulated Spill with Down Reserve} = \text{Regulated Spill} + \text{Regulated Spill Change with Down Reserve}$ and $\text{Regulated Spill with Down Reserve} \leq \text{Regulated Spill Capacity with Down Reserve}$

- **Regulated Spill Change with Down Reserve**

Type:	Series Slot
Units:	Flow
Description:	The amount Regulated Spill would change to fully deploy all downward reserves
Information:	This variable will typically be non-negative. It could be constrained to be 0, directly or indirectly, by RPL policy, or it could be allowed to be greater than zero. This slot will get added regardless of which Spill method is selected. If the selected Spill method does not include Regulated Spill, then the slot should not be referenced in the RPL optimization policy.
Defined by:	$\text{Regulated Spill with Down Reserve} = \text{Regulated Spill} + \text{Regulated Spill Change with Down Reserve}$

- **Regulated Spill Capacity with Down Reserve**

Type:	Series Slot
Units:	Flow
Description:	The maximum Regulated Spill given full deployment of downward reserves
Information:	This variable is defined by the Regulated Spill Capacity approximation as a function of Storage with Down Reserve. This slot will get added regardless of which Spill method is selected. Users will not typically need to do anything with this slot
Defined by:	$\text{Regulated Spill Capacity with Down Reserve} = f(\text{Storage with Down Reserve})$

- **Bypass with Down Reserve**

Type:	Series Slot
Units:	Flow
Description:	The amount of Bypass at full deployment of downward reserves
Information:	This could be equal to Bypass, or it could be greater than Bypass if flow is allowed to be shifted from Turbine Release to Bypass during deployment. This depends on how the RPL policy is formulated. This slot will get added regardless of which Spill method is selected. If the selected Spill method does not include Bypass, then the

value of this variable will automatically be set to zero, and the slot should not be referenced in the RPL optimization policy.

Defined by: Bypass with Down Reserve = Bypass + Bypass Change with Down Reserve
and
 $Bypass \text{ with Down Reserve} \leq Bypass \text{ Capacity with Down Reserve}$

• Bypass Change with Down Reserve

Type: Series Slot
Units: Flow
Description: The amount Bypass would change to fully deploy all downward reserves
Information: This variable will typically be non-negative. It could be constrained to be 0, directly or indirectly, by RPL policy, or it could be allowed to be greater than zero. This slot will get added regardless of which Spill method is selected. If the selected Spill method does not include Bypass, then the slot should not be referenced in the RPL optimization policy.
Defined by: Bypass with Down Reserve = Bypass + Bypass Change with Down Reserve

• Bypass Capacity with Down Reserve

Type: Series Slot
Units: Flow
Description: The maximum Bypass given full deployment of downward reserves
Information: This variable is defined by the Bypass Capacity approximation as a function of Storage with Down Reserve. This slot will get added regardless of which Spill method is selected. Users will not typically need to do anything with this slot
Defined by: $Bypass \text{ Capacity with Down Reserve} = f(\text{Storage with Down Reserve})$

• Outflow with Down Reserve

Type: Series Slot
Units: Flow
Description: The amount of total Outflow at full deployment of downward reserves
Information:
Defined by: Outflow with Down Reserve = Turbine Release with Down Reserve + Spill with Down Reserve

• Storage with Down Reserve

Type: Series Slot
Units: Volume
Description: The end of timestep Storage if full downward reserves were deployed
Information: This variable definition differs from the standard Storage variable only in the Outflow term. The previous Storage and the Net Inflow are the same. Net Inflow represents all gains and losses, including Inflow, Hydrologic Inflow, Precipitation, Evaporation, and so on.
Defined by: Storage with Down Reserve = $\text{Storage}[t - 1] + (\text{NetInflow} - \text{Outflow with Down Reserve}) \times \text{Timestep}$

- **Pool Elevation with Down Reserve**

Type: Series Slot
Units: Length
Description: The end of timestep Pool Elevation if full downward reserves were deployed
Information: This variable is defined by the Pool Elevation linear approximation as a function of Storage with Down Reserve.
Defined by: Pool Elevation with Down Reserve = $f(\text{Storage with Down Reserve})$

- **Tailwater Elevation with Down Reserve**

Type: Series Slot
Units: Length
Description: The average Tailwater Elevation if full downward reserves were deployed
Information: This variable is defined by the Tailwater Elevation linear approximation as a function of Outflow with Down Reserve and the Tailwater Base Value (if applicable based on the selected Optimization Tailwater method). Tailwater Base Value is unaffected by the Reserves modeling and will be the same for both Tailwater Elevation and Tailwater Elevation with Up Reserves. The Tailwater Elevation linear approximation that is applied is dependent on the selected method in the Optimization Tailwater category.
Defined by: Tailwater Elevation with Down Reserve = $f(\text{Outflow with Down Reserve}, \text{Tailwater Base Value})$

- **Temp Tailwater Lookup with Down Reserve**

Type: Series Slot
Units: Length
Description: This slot only applies when the Base Value Plus Lookup Table method is selected in the Tailwater category. It represents the lookup value from the Tailwater Table as a function of Outflow with Down Reserve using the Tailwater linear approximation.
Information: This slot will get added regardless of which Tailwater method is selected but will not get used unless the Base Value Plus Lookup method is selected. Users should not need to do anything with this slot.
Defined by: Temp Tailwater Lookup with Down Reserve = $f(\text{Outflow with Down Reserve})$

- **PSA Head Factor with Down Reserve**

Type: Series Slot
Units: Flow
Description: This slot only applies when the Power Surface Approximation method is selected in the Optimization Power category. It represents the component of Operating Head that is dependent on Storage with Down Reserve and Spill with Up Reserve. It is calculated using the same coefficients used for the Standard PSA Head Factor variable.
Information: This slot will get added regardless of which Optimization Power method is selected but will not get used unless the Power Surface Approximation method is selected. Users should not need to do anything with this slot.
Defined by: PSA Head Adjustment with Down Reserve = $f(\text{Storage with Down Reserve}, \text{Spill with Down Reserve})$

Method Details

Sample RPL Goal Set Implementation

Within a RPL Goal set, any constraint that should not be violated when deploying reserves should include a constraint on the standard variables as well as a duplicated constraint on the corresponding reserves variables. For example, assume that a reservoir has a maximum pool elevation that cannot be violated when reserves are deployed, and that maximum pool elevation is stored in a series slot on a data object called Res_Data.PE_Max. The RPL goal might look like:

```
REPEATED MAXIMIN
  FOR(DATETIME t IN @"Start Timestep" TO @"Finish Timestep") DO
    ADD CONSTRAINT Res.Pool Elevation[t] <= Res_Data.PE_Max[t]
    ADD CONSTRAINT Res.Pool Elevation with Down Reserve[t] <= Res_Data.PE_Max[t]
  END FOR
END REPEATED MAXIMIN
```

The reservoir might also have a target elevation constraint that is a lower priority and can be violated by the deployment of reserves. In this case, the target goal would only include a constraint on the Pool Elevation slot but not a constraint on the Pool Elevation with Down Reserve slot.

Referencing any of the reserve slots in a RPL policy statement will typically draw a large number of additional variables and constraints into the optimization problem. For performance reasons, the user will probably not want these additional constraints to be added unless they are necessary. Assume, for example, that it is known prior to a run that a particular reservoir is not available to provide reserves during select hours. This may be indicated by setting an input value of 0 MW for those hours in a data object series slot called Res_Data.Down_Reserve_Max (similarly for Up Reserve). It would not be necessary to model the reserve versions of constraints for those time steps because it is already known that reserves will be zero. In this case the goal with the Pool Elevation constraints shown previously might be written as:

```
REPEATED MAXIMIN
  FOR(DATETIME t IN @"Start Timestep" TO @"Finish Timestep") DO
    ADD CONSTRAINT Res.Pool Elevation[t] <= Res_Data.PE_Max[t]
    IF(Res_Data.Down_Reserve_Max[t] != 0 MW)
      ADD CONSTRAINT Res.Pool Elevation with Down Reserve[t] <=
        Res_Data.PE_Max[t]
    END IF
  END FOR
END REPEATED MAXIMIN
```

In this way, the constraint on Pool Elevation with Down Reserve would only be added when the reservoir is available to provide reserves (the max reserves are not constrained to zero).

Users could then write policy on the reserves to be carried at individual projects. Assume that the data object series slots Res_Data.Up_Reserve_Min and Res_Data.Up_Reserve_Max store the minimum upward reserves that the reservoir must carry and the maximum upward reserves the project can be credited respectively. A goal for the reservoir might be:

```
REPEATED MAXIMIN
  FOR(DATETIME t IN @"Start Timestep" TO @"Finish Timestep") DO
```

Chapter 5

Optimization Objects and Methods

```
IF(Res_Data.Up_Reserve_Max[t] != 0 MW)
  ADD CONSTRAINT Res.Up Reserve[t] >= Res_Data.Up_Reserve_Min[t]
  ADD CONSTRAINT Res.Up Reserve[t] <= Res_Data.Up_Reserve_Max[t]
END IF
END FOR
END REPEATED MAXIMIN
```

These two constraints may not necessarily be included in the same goal.

Note: The first constraint based on the minimum required reserves will constrain the physical operations of the reservoir in order to meet the required reserve obligation. The second constraint, based on the maximum credited reserves will not directly constrain the physical operations, rather it will only limit the amount of reserves that can be counted. In other words, the solution may operate the reservoir such that, according to physical limits and other specified constraints, the reservoir may have a larger amount of reserves available, but this constraint allows the user to say that only a limited amount will be counted.

Users may then write goals which sum reserves across multiple projects to meet a system reserve requirement.

```
REPEATED MAXIMIN
  FOR(DATETIME t IN @"Start Timestep" TO @"Finish Timestep") DO
    ADD CONSTRAINT (FOR(OBJECT res IN ProjectsAvailableForUpReserve(t)) SUM res.Up
    Reserve[t]
    END FOR)
    >= System_Data.Up_Reserve_Min[t]
  END FOR
END REPEATED MAXIMIN
```

In this goal, ProjectsAvailableForUpReserve represents a user-defined function that returns a list of reservoirs that can carry reserves, again to prevent adding numerous constraints on the reserves version of variables unnecessarily. The details of this function would be model-specific.

Optimization Spill

The Optimization Spill methods determine how spill is calculated for the reservoir and generates physical constraints that correspond to the selected methods.

This category is dependent on selection of the Independent Linearizations method for the Optimization Power category. However, the method selected in the Optimization Spill category must match with the corresponding non-optimization method in the Spill category.

Spill is an Optimization decision variable. The following constraint is always generated for a reservoir:

$$\text{Outflow} = \text{Turbine release (or Release)} + \text{Spill} \quad (5.18)$$

The spill methods generate values applicable the following additional constraint, where some of these terms may be omitted if they do not apply to the selected spill method.

$$\text{Spill} = \text{Regulated Spill} + \text{Unregulated Spill} + \text{Bypass}, \quad (5.19)$$

As applicable, the following constraints are also added:

$$\text{Bypass} \leq \text{Bypass Capacity} \quad (5.20)$$

$$\text{Regulated Spill} \leq \text{Regulated Spill Capacity} \quad (5.21)$$

$$\text{Unregulated Spill} = \text{Unregulated Spill Capacity} \quad (5.22)$$

See “[Spill](#)” in *Objects and Methods* for details about non-optimization Spill methods.

Slots Specific to This Method

- **Bypass Capacity**

Type: Series Slot
Units: Flow
Description: Bypass capacity
Information:
Defined by: Numerical 2-D Approximation in terms of storage, based upon the Bypass Capacity Table.

- **Bypass Capacity Table**

Type: Table Slot
Units: Volume vs Flow
Description: Storage vs corresponding maximum bypass spill values
Information:
Defined by: Internally developed based on Bypass Table and Elevation Volume Table relationships. For each pool elevation in the Bypass Table, the Bypass Capacity Table has a row relating Storage to Bypass Capacity. The Pool Elevations are converted to Storage using the Elevation Volume Table.

- **Regulated Spill Capacity**

Type: Series Slot
Units: Flow
Description: Regulated spill capacity
Information:
Defined by: Numerical 2-D Approximation in terms of storage, based upon the Regulated Spill Capacity Table.

- **Regulated Spill Capacity Table**

Type: Table Slot
Units: Volume vs Flow
Description: Storage vs corresponding maximum regulated spill values

Information:

Defined by: Internally developed based on Regulated Spill Table and Elevation Volume Table relationships. For each pool elevation in the Regulated Spill Table, the Regulated Spill Capacity Table has a row relating Storage to Regulated Spill Capacity. The Pool Elevations are converted to Storage using the Elevation Volume Table.

• **Regulated Spill or Bypass LP Param**

Type: Table Slot

Units: Volume

Description: Specifies the Storage points use to take the tangent, line and piecewise approximations for Regulated Spill Capacity linearization and Bypass Spill Capacity linearization.

Information:

Defined by: User input

• **Unregulated Spill Linearization Table**

Type: Table Slot

Units: Volume vs Flow

Description: Storage vs corresponding unregulated spill values

Information:

Defined by: Internally developed based on Unregulated Spill Table and Elevation Volume Table relationships. For each pool elevation in the Unregulated Spill Table, the Unregulated Spill Capacity Table has a row relating Storage to Unregulated Spill Capacity. The Pool Elevations are converted to Storage using the Elevation Volume Table.

• **Unregulated Spill LP Param**

Type: Table Slot

Units: Volume

Description: Specifies the Storage points use to take the tangent, line and piecewise approximations for Unregulated Spill Linearization Table linearization

Information:

Defined by: User input

■ **None**

If this method is selected the slot bounds in the slot configuration dialog are set to zero. No additional constraints are generated.

■ **Opt Monthly Spill**

This method is not functional in RPL Optimization.

This method sets the lower and upper bounds on spill. The lower bound is set to zero and the default upper bound is set to a very big number (9,999,999 cms). The default upper bound can be revised in the Spill slot configuration, Upper Bound parameter. No additional constraints are generated beyond these bounds.

■ **Opt Unregulated**

If this method is selected, only unregulated spill is considered and the following constraint is added to the LP:

$$\text{Spill} = \text{Unregulated Spill} \quad (5.23)$$

■ **Opt Regulated**

When this method is selected, only regulated spill is considered. The lower bound on spill is set to zero and the following constraint is added to the LP:

$$\text{Spill} = \text{Regulated Spill} \quad (5.24)$$

■ **Opt Regulated and Unregulated**

When this method is selected, unregulated and regulated spill are considered and the following constraint is added to the LP:

$$\text{Spill} = \text{Regulated Spill} + \text{Unregulated Spill} \quad (5.25)$$

■ **Opt Regulated and Bypass**

When this method is selected, only regulated and bypass spill are considered and the lower bound on spill is set to zero and the following constraint is added to the LP:

$$\text{Spill} = \text{Regulated Spill} + \text{Bypass} \quad (5.26)$$

■ **Opt Regulated, Bypass and Unregulated**

When this method is selected, unregulated, regulated and bypass spill are considered and the following constraint is added to the LP:

$$\text{Spill} = \text{Regulated Spill} + \text{Unregulated Spill} + \text{Bypass} \quad (5.27)$$

■ **Opt Bypass, Regulated and Unregulated**

When this method is selected, unregulated, regulated and bypass spill are considered and the following constraint is added to the LP:

$$\text{Spill} = \text{Bypass} + \text{Regulated Spill} + \text{Unregulated Spill} \quad (5.28)$$

Optimization Tailwater

Only a subset of available Simulation methods are supported in Optimization. These methods are: Linked or Input, Base Value Only, Base Value Plus Lookup Table, Stage Flow Lookup Table, and Coefficients Table. Selection of a

Chapter 5

Optimization Objects and Methods

method other than these will result in an error. The method selected in the Optimization Tailwater category should correspond to the one selected in the simulation Tailwater category.

Tailwater Elevation may be part of the Optimization problem and it is handled differently for each method. Below is a description of the behavior of Tailwater Elevation.

See “Tailwater” in *Objects and Methods* for details on the Simulation methods.

■ Opt Linked or Input

If Tailwater Elevation is not input, it is replaced as follows:

$$\text{TailwaterBaseValue: Tailwater Elevation} = \text{Tailwater Base Value} \quad (5.29)$$

Slots Specific to This Method

- **Tailwater Base Value**

Type: Series
Units: Length
Description: See “Linked or Input” in *Objects and Methods* for details.

■ Opt Base Value Only

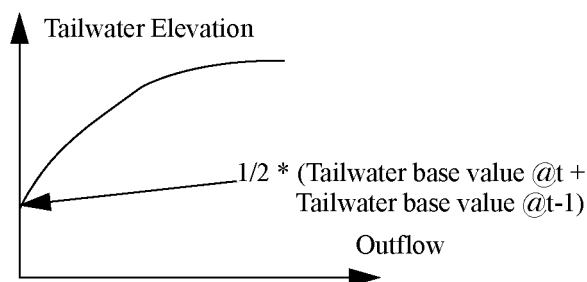
If Tailwater Elevation is not input, it is replaced as follows:

$$(\text{Tailwater Base Value}(t) + \text{Tailwater Base Value}(t-1)) / 2 \quad (5.30)$$

Slots Specific to This Method

- **Tailwater Base Value**

Type: Series
Units: Length
Description: See “Base Value Only” in *Objects and Methods* for details.
Tailwater outflow relationship. Not drawn to scale.



■ Opt Base Value Plus Lookup Table

If Tailwater Elevation is not input, this slot is replaced by a mathematical expression based on Tailwater Base Value and a numerical approximation of Tailwater Elevation as a function of Outflow (Numerical 2-D Approximation), as defined by the user in the Tailwater Table.

Tailwater Elevation is constrained as follows:

$$(\text{tailwaterBaseValue}(t) + \text{tailwaterBaseValue}(t-1))/2 + \text{tempTWLookupValue} \quad (5.31)$$

where tempTWLookupValue is a Numerical 2-D Approximation of increase in Tailwater Elevation, due to flow as described in the Tailwater Table. The Tailwater Table Lookup LP Param table values for outflow are used as approximation points, indexing the Tailwater Table to determine tempTWLookupValue from the Tailwater Elevation column.

Slots Specific to This Method

• Tailwater Base Value

Type: Series
Units: Length
Description: See “Base Value Plus Lookup Table” in *Objects and Methods* for details.

• Tailwater Table

Type: Series Slot
Units: Flow vs Length
Description: Reservoir outflow vs either the tailwater elevation or the tailwater elevation increment
Information: If the Tailwater Base Value is non-zero, the Tailwater Table gives values of incremental increase in Tailwater Elevation over the Base value. If the Tailwater Base Value is zero, the table simply gives the Tailwater Elevation values. The Tailwater Table should have increasing values of outflow. Tailwater Elevation is required to be a convex function of outflow. The preferred order of approximation is substitution, piece-wise, two-point line, tangent. See Object / Simulation documentation for details.
I/O: User input

• Tailwater Table Lookup LP Param

Type: Table Slot
Units: Flow, Flow, Flow
Description: LP Parameter approximation points for Outflow.
Information: The Tailwater Table Lookup LP Param table values for outflow are used as approximation points indexing the Tailwater Table to determine tempTWLookupValue from the Tailwater Elevation column.
I/O: User input only
Links: Not linkable

- **temp Tailwater Lookup Value**

Type:	Series
Units:	Length
Description:	Numerical 2-D Approximation of increase in Tailwater Elevation due to flow as described in the Tailwater Table.

■ Opt Stage Flow Lookup Table

Tailwater Elevation is numerically approximated as a function of Stage and Flow, as defined by the user in the Stage Flow Tailwater Table.

The tables are queried depending on method selection in the Tailwater Linearization Automation category, as follows.

- If the selected method in the Tailwater Linearization Automation category is None, the user-input Tailwater LP Param table values are used.
- For the Range Input Automation method, the automatically calculated points are used.

Tailwater Elevation is approximated numerically (3-D approximation), as a function of Outflow and Tailwater Base Value based on the Stage Flow Tailwater Table. The Tailwater Elevation LP Param table Tailwater Base Value and flow values are used as approximation points, indexing the Downstream Stage and Outflow columns of the Stage Flow Tailwater Table, respectively.

See “[Stage Flow Lookup Table](#)” in *Objects and Methods* for details on the Simulation method.

Slots Specific to This Method

- **Tailwater Base Value**

Type:	Series
Units:	Length
Description:	See “ Stage Flow Lookup Table ” in <i>Objects and Methods</i> for details.

- **Stage Flow Tailwater Table**

Type:	Table Slot
Units:	Flow vs Length vs Length
Description:	Reservoir Outflow vs Downstream Elevation (Tailwater Base Value) vs Tailwater Elevation
Information:	Data must be entered into the table in increasing blocks of the same Outflow value for the three-dimensional table interpolation to work correctly. For every block of same Outflows in column 1, Downstream Stages should be listed in increasing order in column 2, and the corresponding Tailwater Elevations in column 3. Tailwater elevation is required to be a concave function of Outflow. The preferred order of approximation is substitution, piece-wise, two-point line, tangent. Internally, the Stage Flow Tailwater Table is rearranged to use the Stage as the primary index (column) with outflow as the secondary index (column). This rearranged table may be labeled as “Convolved Stage Flow Tailwater Table” in output messaging. Also, because of this rearrangement, it is desirable to repeat stage values for each outflow.
Defined by:	User input

• Tailwater Elevation LP Param

Type:	Table Slot
Units:	Length, Flow, Flow, Flow
Description:	Specifies the fixed tailwater base value point and the outflow points used to take the tangent, line and piecewise approximations for tailwater elevation linearization
Information:	This table is used for linearization. The best Tailwater Base Value point to choose for tangent approximation would be an outflow equal to the expected value of outflow during the run; for the line approximation, the minimum and maximum values expected during the run; for piecewise approximation, the minimum and maximum values expected during the run plus additional intermediate values to more closely fit the tailwater curve.
Defined by:	User input unless the selected method in the Tailwater Linearization Automation category is not None.

• Tail Water Reference Elevation

Type:	Table Slot
Units:	Length
Description:	Lowest Reservoir discharge Elevation when there are no backwater effects from a downstream pool (reservoir)
Information:	Although this is part of the Stage Flow Lookup Table method (and its corresponding Opt method), this slot does not influence optimization . See Object / Simulation documentation for details.
I/O:	User input

■ Opt Coefficients Table

This method sets up the physical Tailwater Elevation constraints using the same equation and coefficients used in the Coefficients Table tailwater method.

$$\begin{aligned} \text{Tailwater Elevation} = & \text{Constant Coeff}[t] + \text{Constant Coeff}[t - 1] + \\ & \text{Outflow Coeff}[t] \times \text{flow} + \text{Outflow Coeff}[t - 1] \times \text{Outflow}[t - 1] + \\ & \text{TW Base Val Coeff}[t] \times \text{TailwaterBaseValueTemp}[t] + \\ & \text{TW Base Val Coeff}[t - 1] \times \text{Tailwater BaseValue}[t - 1] + \\ & \text{TW Elev Coeff}[t - 1] \times \text{Tailwater Elevation}[t - 1] \end{aligned}$$

See “Coefficients Table” in *Objects and Methods* for details.

Slots Specific to This Method

• Tailwater Base Value

Type:	Series
Units:	Length
Description:	See “Coefficients Table” in <i>Objects and Methods</i> for details.

• Tailwater Coefficients

Type:	Table
Units:	multi
Description:	See “Coefficients Table” in <i>Objects and Methods</i> for details.

Defined by: If the Power Surface Approximation method is selected, then there are additional restrictions on the coefficients that can be specified. With the Power Surface Approximation, having terms for Outflow[t-1], Tailwater BaseVal[t] and Tailwater[t-1] are not allowed.

■ Pump Power Numerical Approximation

If the Optimization problem uses Pump Power, either directly or indirectly, this slot is added to the problem.

Before being passed to the optimization solver, this slot is numerically approximated as a function of Operating Head and Pumped Flow (Numerical 3-D Approximation). The relationship between Operating Head and Pumped Flow comes from the user-input Pump Power Lin Data table. The table is queried using the points defined in the Pump Power LP Param table. The values in the Pump Power LP Param table may be entirely user input, or may include an automatically calculated value for Operating Head. If the Operating Head is not available at the beginning of the run, this value will be assigned automatically by RiverWare.

Slots Specific to This Method

• Pump Power

Type: Agg Series Slot
Units: Flow
Description: Power required to operate the pumps
Information:
Defined by: Numerical 3-D Approximation in terms of Operating Head and Pumped Flow based on the Pump Power Lin Data table. The Pump Power LP Param table contains a value for Operating head used to index the Operating Head column of the Pump Power Lin Data table. This approximated value, therefore, reduces the Pump Power to a function of Pumped Flow at the given Operating Head. The flow values in the Pump Power LP Param table are then used as approximation points indexing the Pumped Flow column of the Pump Power Lin Data table. The Pump Power Lin Data table should have increasing values of Operating Head and Pumped Flow. Pump Power should be a convex function of Operating Head, but concavity is not strictly enforced; mild non-convex regions are permissible to allow for round-off error, and so on. The preferred order of approximation is substitution, piece-wise, tangent, two-point line.

• Pump Power Lin Data

Type: Table Slot
Units: Length vs Flow vs Power
Description: Table defining the relationship between Operating Head, Pumped Flow and power expended (Pump Power).
Information:
Defined by: User input

• Pump Power LP Param

Type: Agg Series Slot
Units: Length, Flow, Flow, Flow

Description:	Specifies the Operating Head and the flow points used to take the tangent, line and piecewise approximations for Pump Power Lin Data linearizations.
Information:	This slot is the pumping equivalent of the Power LP Param table. The best Operating Head to choose should be close to the expected head during optimized period. Tangent approximation is generally not used, nor helpful as it often results in non-zero power for zero flow. The suggested points for a line approximation are 0 flow and the preferred pumping flow. The suggested points for piecewise linearization are 0 flow, 1 unit preferred pumping flow, 2 units preferred pumping flow ... n units preferred pumping flow, and maximum pumping flow.

Pool Elevation Linearization Automation

Note: This category is no longer supported in RPL optimization.

Power

Only a subset of available Simulation methods are available in Optimization. Selection of a method other than the ones that follow results in an error.

If the Optimization problem uses Power, either directly or indirectly, the Power slot is added to the problem. Before being passed to the optimization solver, the Power slot is numerically approximated as a function of Operating Head and Turbine Release (Numerical 3-D Approximation).

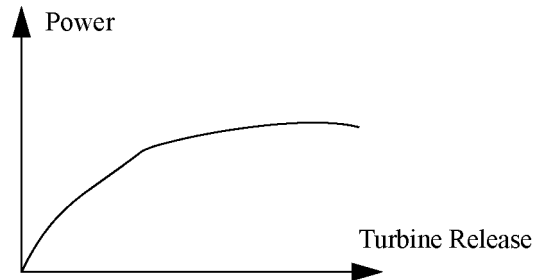
See “[Power](#)” in *Objects and Methods* for details on the Simulation methods.

Slots Specific to This Method

- **Power LP Param**

Type:	Table Slot
Units:	Length and Flow
Description:	Specifies the Operating Head and the flow points used to take the tangent, line and piecewise approximations for Power linearization
Information:	This table must be user input if the Independent Linearizations method is selected in the Optimization Power category. Otherwise it is unused. The best Operating Head to choose should be close to the expected head during optimized period. Tangent approximation is generally not used, nor helpful as it often results in non-zero power for zero flow. The suggested points for a line approximation are 0 flow and best turbine flow for the entire plant. The suggested points for piecewise linearization are 0 flow, 1 unit best turbine flow, 2 units best turbine flow ... n units best turbine flow, and maximum turbine flow.

Power turbine release relationship for a fixed operating head valve. Not drawn to scale.



■ Plant Power Coefficient

Slots Specific to This Method

In the RPL Optimization mode, the following slot requires special handling.

- **Best Turbine Flow**

Type: Agg Series Slot

Units: Flow

Description: Flow associated with the most efficient power generation for an associated Operating Head

Information:

Defined by: Numerical 2-D Approximation in terms of Operating Head, based upon an internal best turbine flow table. For the Plant Power Coefficient method (selected in the Power category) the internal table is the Best Turbine Q table. The Best Turbine Q table is required to have increasing values of Operating Head and Best Capacity and be a concave function of Operating Head. The preferred order of approximation is substitution, piece-wise, tangent, two-point line. The Best Turbine Flow LP Param table values are used as approximation points indexing the best turbine flow table.

If the Optimization problem uses Best Turbine Flow, either directly or indirectly, then this slot is added to the problem. Before being passed to the optimization solver, this slot is numerically approximated as a function of Operating Head (Numerical 2-D Approximation).

The relationship between Operating Head and Best Turbine Flow depends on what other methods are selected, as follows:

If the selected method in the Power category is Plant Efficiency Curve, the relationship is automatically developed from the Plant Power Table.

For the Plant Power Coefficient method, the user-input Best Turbine Q table defines the relationship.

For other Plant Calculation category methods, Best Turbine Flow must be input.

- **Best Turbine Flow LP Param**

Type: Table Slot

Units: Length

- Description:** Specifies the Operating Head points in the best turbine flow relationship used to take the tangent, line and piecewise approximations for Best Turbine Flow linearization
- Information:** The best operating head points to choose for a piecewise approximation are generally: minimum Operating Head, Operating Head at max capacity, and maximum Operating Head. These three points typically define most of the shape of the Best Turbine Flow curve.
- Defined by:** User input

• Best Turbine Q

- Type:** Table Slot
- Units:** Length vs Flow
- Description:** Table defining the relationship between Operating Head and the most efficient power generation.
- Information:** This table is used if the Power category has the Plant Power Coefficient method selected. If the Power category has the Plant Efficiency Curve method selected, then this table is not used, but a similar relationship is automatically developed from the Plant Power Table.
- Defined by:** User input

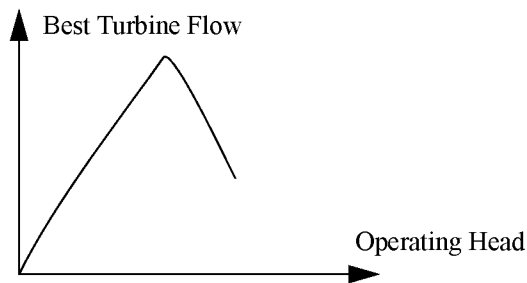
• Plant Power Table

- Type:** Table Slot
- Units:** Length vs Flow vs Power
- Description:** Table defining the relationship between Operating Head, Turbine Release, and Power at selected points of operation. In building the Plant Power table, the following approach might be taken. For a given Operating Head create $n + 2$ rows of data, where n is the number of units comprising the Plant. The first row will reflect no flow through the turbines. The next row will reflect the preferred level of flow through the first unit normally activated. Subsequent rows will reflect incremental flow levels as additional turbines come on line at their respective preferred levels of flow. The final row will reflect the flow of all available units running at maximum flow capacity.
- Information:** Data must be entered into the table in increasing, concave blocks of the same Operating Head for the three-dimensional table interpolation to work correctly. For every block of the same Operating Head in column 1, Turbine Release and the corresponding Power should be listed in increasing, concave order in columns 2 and 3. There must be a point of zero Turbine Release and zero Power for each Operating Head. The following table is an example of the proper way to formulate the Plant Power Table. This table can be derived from the Best Turbine Q, Best Power Coefficient, Max Turbine Q and Max Power Coefficient slots.

Operating Head	Turbine Release	Power
30	0	0
30	100	100
30	200	175
40	0	0
40	100	125

Operating Head	Turbine Release	Power
40	220	195
50	0	0
50	110	147
50	250	205

Defined by: User input
Best turbine flow – operating head relationship. Not drawn to scale.



■ Plant Efficiency Curve

The Plant Efficiency Curve method approximates Best Power Flow the same as the Plant Power Coefficient method.

■ Peak Power

In RPL Optimization, this method creates the following turbine release constraint. No actual power value is calculated.

$$\text{Turbine Release} \leq \text{Power Plant Capacity Fraction} * \text{Turbine Capacity} * \text{Number of Units.} \quad (5.32)$$

■ Peak and Base

In RPL Optimization, this method creates the following turbine release constraint. No actual power value is calculated.

$$\text{Turbine Release} \leq \text{Power Plant Capacity Fraction} * \text{Turbine Capacity} * \text{Number of Units.} \quad (5.33)$$

■ Unit Power Table

See “Unit Power” on page 301 for the Optimization formulation.

See “Unit Power Table” in *Objects and Methods* the Simulation algorithm.

Slots Specific to This Method

See “Unit Power Table” in *Objects and Methods* for slot descriptions.

Method Details

When the Unit Power Table method is selected, the following method categories are available:

- [Avoidance Zones](#), page 207
- [Cavitation](#), page 208
- [,](#) page 210
- [Head Loss](#), page 212
- [Startup](#), page 241

The status of a unit at a given timestep can be one of the following:

- Available
- Unavailable
- Must run

At the highest level, an entire plant may never be able to provide an ancillary service like Regulation. The user can indicate this by selecting None in the appropriate category. On the other hand, the following slot settings allow users to specify unit availability / must run for individual time steps for generation and ancillary services. Without any inputs, the units default to being available.

If the Unit Power Table method is selected, but not Frequency Regulation, the user can specify through optimization RPL policy.

Note: Setting certain slots may be possible but not preferred.

- Unit u must generate at time t :
 - Unit Is Generating $[t,u] = 1$, or
 - Unit Energy $[t,u] = \text{value}$
- Unit u is unavailable at time t :
 - Unit Is Generating $[t,u] = 0$, or
 - Unit Energy $[t,u] = 0$

If the Frequency Regulation method is selected, the user could specify through optimization policy (RPL):

- Unit u must regulate up at time t :
 - Regulation Up $[t,u] = \text{value}$
- Unit u must regulate down at time t :
 - Regulation Down $[t,u] = \text{value}$
- Unit u must regulate at time t :
 - Regulation $[t,u] = \text{value}$ (which implies Regulation Up $[t,u] = \text{value}$ and Regulation Down $[t,u] = \text{value}$)

If a unit is unavailable for some type of regulation, a value can be 0.

■ Turbine Capacity

Turbine Capacity is numerically approximated before being passed to the optimization solver. The slot is approximated as a function of Operating Head (Numerical 2-D Approximation).

The relationship between Operating Head and Turbine Capacity depends on the method selected in the Power category, as follows:

- If Plant Efficiency Curve is selected, the relationship is automatically developed from the Plant Power Table.
- If Peak Power or Peak and Base is selected, the relationship comes from the Best Generator Flow table.
- If none of these methods are selected, the relationship is contained in the user-input Max Turbine Q slot.

Slots Specific to This Method

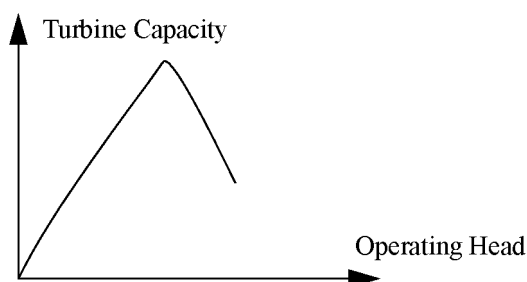
• Turbine Capacity

Type:	Agg Series Slot
Units:	Flow
Description:	Flow capacity of the turbines
Information:	
Defined by:	Numerical 2-D Approximation in terms of Operating Head, based upon a maximum turbine capacity table. This capacity table is determined in various ways according to the Power method: The Turbine Capacity LP Param table values are used as approximation points indexing the selected maximum turbine capacity table. The maximum turbine capacity table is required to have increasing values of Operating Head. Turbine Capacity in that table is required to be a concave function of Operating Head. The preferred order of approximation is substitution, piece-wise, tangent, two-point line.

• Turbine Capacity LP Param

Type:	Table Slot
Units:	Length vs Length vs Length
Description:	Specifies the operational head points used to take the tangent, line, and piecewise approximations for Turbine Capacity linearization.
Information:	For the piecewise approximation the best operating head points to choose are generally: minimum Operating Head, Operating Head at max capacity, and maximum Operating Head. These three points typically define most of the shape of the Turbine Capacity curve. The following figure illustrates the relationship between Turbine Capacity and Operating Head (not drawn to scale).
Defined by:	User input, unless the selected method in the Power Linearization Automation category is not None.

Turbine capacity operating head relationship. Not drawn to scale.



Startup

This category depends on selecting the Unit Power Table method (see [“Unit Power Table”](#) on page 194) and describes how the monetary cost associated with starting up or shutting down a unit (turbine) will be modeled.

The methods in this category are as follows:

- One that does *not* model these costs, effectively assigning them a value of 0.
- One that uses a table describing the combined costs for starting up and shutting down a unit.

■ None

This is the default, no action method.

■ Unit Lumped Cost

For each unit, this method lumps the cost of startup and shutdown into one value.

See [“Unit-level Constraints”](#) on page 309 for descriptions of additional constraints.

Slots Specific to This Method

See [“Unit Lumped Cost Method”](#) in *Objects and Methods* for slot descriptions.

Modeling a Pumped Storage Reservoir

Use the following steps to set up a pumped power reservoir for optimization.

1. Set up a running simulation model, using methods that are compatible with optimization for Power, Tailwater, Spill, and so on.
2. Chose an Optimization Power method.
3. Choose an Optimization Head Computation method. In Optimization, None must be chosen.
4. Select the Optimization Spill method corresponding to the method selected in the Spill category.

Chapter 5

Optimization Objects and Methods

5. Fill in the LP Param tables related to Power, Turbine Capacity and Best Turbine Flow (if plant power is used). Alternatively, the Power Linearization Automation method can also be chosen to automatically determine the values for the LP Param tables.
6. Select the Optimization Tailwater method corresponding to the method selected in Tailwater category. If Opt Base Value Plus Lookup Table or Opt Stage Flow Lookup Table is selected, fill in the Tailwater Elevation LP Param table. Alternatively, select a Tailwater Linearization Automation method (Range Input Automation is currently available).
7. If future value is needed, select a method in the Future Value category, followed by the Optimization Future Value category, and if desired, Cumul Stor Val Linearization Automation.
8. Selection of the remaining methods and linearizations can be done in any order. Pool elevation linearizations, Pool Elevation Linearization Automation, Evaporation and Precipitation, Optimize Evaporation Computation, Evaporation Linearization Automations, Bank Storage, Hydrologic Inflow, Energy In Storage, and Diversion from Reservoir. The appropriate data must be entered for each method.

Reach

In RPL Optimization, a constraint is always created to ensure mass balance. This constraint takes the following form with actual values or expressions comprising the variables shown, being dependent on the configuration of the reach object:

$$0 = \sum \text{Inflows} - \text{Outflow} + \text{Return Flow} - \text{Diversion} \quad (5.34)$$

Inflows may include Inflow, Local Inflow, time lagging effects, and GainLoss consideration.

Return Flow and Diversion may be excluded, if model configuration does not require usage of the terms.

General Slots

• Inflow

Type:	Agg Series
Units:	Flow
Description:	flow at entrance to the object
Information:	
Defined by:	Explicit Optimization variable in the mass balance constraint

• Outflow

Type:	Agg Series
Units:	Flow
Description:	flow at exit from the object
Information:	
Defined by:	Explicit Optimization variable in the mass balance constraint

User Methods

User method categories in Optimization are as follows.

- [Diversion from Reach](#), page 199
- [Gain Loss](#), page 200
- [Gain Loss](#), page 200
- [Local Inflow and Solution Direction](#), page 200
- [Numerical Approximation](#), page 202
- [Routing](#), page 201

Diversion from Reach

This category is dependent on selection of one of the following methods in the Routing category: No Routing, Variable Time Lag, Impulse Response, Muskingum, Muskingum Cunge, Muskingum Cunge Improved, Kinematic, Kinematic Improved, or MacCormack.

Note: No Routing is the only one of these methods supported by RPL Optimization.

Of the available methods, None and Available Flow Based Diversion are supported. Selection of any other method results in an error during run initialization.

See “[Diversion from Reach](#)” in *Objects and Methods* for details on the Simulation method.

■ None

This method is the default for this category and should be selected if no diversion is desired.

■ Available Flow Based Diversion

In RPL Optimization, this method adds a constraint to the optimization problem. The constraint simply specifies that the value for the Available For Diversion slot is the Upper Bound on Available For Diversion (on the slot configuration) or the Inflow minus any Minimum Diversion Bypass.

Slots Specific to This Method

• Available For Diversion

Type:	Agg Series Slot
Units:	Flow
Description:	the divertable flow in the reach
Information:	
Defined by:	a user-input constant or a function of inflow, depending on model configuration. If a maximum value has been assigned to the Available For Diversion slot, then the Available For Diversion slot is set to that maximum value. Otherwise, if the Min Diversion Bypass category specifies a method other than None, the slot is set equal

to Inflow - Minimum Diversion Bypass. Otherwise, if a minimum value has been assigned to the Outflow slot, the Available For Diversion slot is set to Inflow - that minimum value. Otherwise, the Available For Diversion slot is set to Inflow.

Gain Loss

Of the available methods, None and Constant Gain Loss are supported; selection of any other method may result in an error during the start of run.

See “Gain Loss” in *Objects and Methods* for details on the Simulation methods.

■ None

This method is the default for this category. It should be used if gains and losses are not required in the model.

■ Constant Gain Loss

This method allows the user to specify constant-value parameters for GainLoss. As such, the parameters are used directly in the RPL Optimization problem. No special consideration of slot values is necessary in RPL Optimization.

In defining the Inflows expression described at the beginning of this section, the following formula applies:

$$\text{Inflows} = (\text{Preliminary Inflows} * \text{GainLoss Coeff}) + \text{GainLoss} \quad (5.35)$$

Preliminary Inflows may equal Inflow, Inflow + Local Inflow, or appropriate time lagged inflow. See Routing “Routing” on page 201 for additional detail on the lagged inflow.

See “Constant Gain Loss” in *Objects and Methods* for details on the Simulation method.

Local Inflow and Solution Direction

The reach object considers Local Inflow in its mass balance calculation if specified. In RPL Optimization Local Inflow is included as one of the Inflows if specified.

See “Local Inflow and Solution Direction” in *Objects and Methods* for details on the Simulation methods.

■ Min Diversion Bypass

Under certain configurations, the Available Flow Based Diversion method in the Diversion From Reach category may look for the Minimum Diversion Bypass value to calculate Available For Diversion flow. (See Diversion From Reach, above.) If it is desirable that a Minimum Diversion Bypass value be specified, the Input Min Bypass method should be selected.

If a Min Diversion Bypass method is selected and a value is specified, the Available For Diversion constraint will include the Minimum Diversion Bypass. See “Available Flow Based Diversion” on page 199 for details.

See “Min Diversion Bypass” in *Objects and Methods* for details on the Simulation method.

Routing

Of the available methods, only No Routing, Time Lag and Impulse Response are supported in RPL optimization; selection of any other method will result in an error during begin run.

See “Routing” in *Objects and Methods* for details on the Simulation methods.

■ No Routing

This method provides no routing functionality but adds consideration of Return Flow to the mass balance.

Slots Specific to This Method

• Return Flow

Type:	Multi Slot
Units:	flow
Description:	Return flow into the reach
Information:	Enters at the bottom of the reach and so is not lagged by the reach nor is it subject to gains or losses. Return Flow must be defined, either by input, a link or a constraint. It will not default to 0 if undefined.
Defined by:	Explicit Optimization variable in the mass balance constraint: $0 = InflowExpression - Outflow + ReturnFlow - Diversion$ where Inflow Expression depends on model configuration and may include Inflow, Local Inflow, and GainLoss consideration. Inflow may be linked to a slot on another object.

■ Time Lag

In RPL Optimization, Time Lag routing results in a modified mass balance constraint that includes the lagging effect. Selection of this method excludes usage of Return Flow and Diversion values in the optimization problem. Routing occurs prior to any gain and loss consideration.

$$\sum Inflows = (Inflow(lagINT - 1) + LocalInflow(lagINT - 1)) \times flowFraction1 + (Inflow(lagINT) + LocalInflow(LagINT)) \times flowFraction2$$

where:

- lagINT refers to the timestep on or after the point in time equal to the current timestep minus LagTime.
- lagINT – 1 refers to the timestep prior to LagINT.
- flowFraction1 equals the fraction of a timestep used to weigh the inflow associated with lagINT – 1; flowFraction1 equals the fractional remainder of the LagTime / timestep (for example, 0.7 if LagTime = 1.7 timesteps and 0 if LagTime is a multiple of the timestep).
- flowFraction2 equals the fraction of a timestep used to weigh the inflow associated with lagINT; flowFraction2 equals 1 – flowFraction1. (for example, 0.3, if LagTime = 1.7 timesteps and 1 if LagTime is a multiple of the timestep).

Slots Specific to This Method

• LagTime

Type:	Table Slot
Units:	Time
Description:	lag time or travel time of flow through the reach
Information:	a single value
Defined by:	User input

■ Impulse Response

Impulse Response routing results in a modified mass balance constraint where the reach Outflow is the weighted sum of the current and previous timesteps' Inflows. Selection of this method excludes usage of Return Flow and Diversion values in the optimization problem. Any gains or losses are added to or subtracted from the routed flow.

$$Outflow = C_0 Inflow_t + C_1 Inflow_{t-1} + \dots + C_{ncoeff-2} Inflow_{t-ncoeff-2} + C_{ncoeff-1} Inflow_{t-(ncoeff-1)} + TotalGainLoss$$

Slots Specific to This Method

• Lag Coeff

Type:	Table Slot
Units:	No Units
Description:	impulse response coefficients
Information:	There must be the same number of values in the Lag Coeff table as the value given in Num of Coeff
Defined by:	User input

• Num. of Coeff

Type:	Scalar Slot
Units:	No Units
Description:	number of impulse response coefficients
Information:	There must be the same number of values in the Lag Coeff table as the value given in Num of Coeff
Defined by:	User input

Numerical Approximation

No Reach slots require numerical approximation.

Modeling a Reach

To use a Reach object in optimization, set up a running simulation model using methods that are compatible with Optimization for the following categories: Routing, Local Inflow and Solution Direction, Gain Loss, Min Diversion Bypass and Diversion from Reach. Input the required data for each method.

Slope Power Reservoir

The Slope Power Reservoir object is similar to the Level Power Reservoir but with added functionality to model the *wedge* storage or backwater storage effects of a sloped water surface.

General Slots

• Backwater Elevation

Type: Agg Series
Units: Length
Description: Elevation of the reservoir at the top of the dam

• Canal Flow

Type: Agg Series
Units: Flow
Description: Flow into (out of) the reservoir from (to) a canal
Information:
Defined by: Explicit Optimization variable in the mass balance constraint

• Diversion

Type: Series Slot
Units: Flow
Description: Flow from the reservoir to a diverting object
Information:
Defined by: Explicit Optimization variable in the mass balance constraint

• Energy

Type: Agg Series Slot
Units: Energy
Description: Product of the power generated by flow through the turbines and the length of the timestep.
Information:
Defined by: Replaced by Power * Timestep Length

- **Flow FROM Pumped Storage**

Type: Agg Series Slot
Units: Flow
Description: Flow into the reservoir from a pumped storage reservoir
Information:
Defined by: Explicit Optimization variable in the mass balance constraint. This slot should be linked to Outflow on a Pumped Storage object. The Pumped Storage object constrains its Outflow.

- **Flow TO Pumped Storage**

Type: Agg Series Slot
Units: Flow
Description: Flow out of the reservoir into a pumped storage reservoir
Information:
Defined by: Explicit Optimization variable in the mass balance constraint. This slot should be linked to Pumped Flow on a Pumped Storage object.

- **Inflow**

Type: MultiSlot
Units: Flow
Description: Inflow into the reservoir from upstream
Information:
Defined by: Explicit Optimization variable in the mass balance constraint

- **Inflow 2**

Type: Agg Series
Units: Flow
Description: An additional inflow into the reservoir

- **Operating Head**

Type: Agg Series Slot
Units: Length
Description: Elevation difference between the average Pool Elevation and the average Tailwater Elevation during a timestep
Information:
Defined by: Replacement by $(\text{Pool Elevation}(t) + \text{Pool Elevation}(t-1)) / 2 - \text{Tailwater Elevation}$

- **Outflow**

Type: Agg Series Slot
Units: Flow
Description: Outflow from the reservoir
Information:
Defined by: Explicit Optimization variable as $\text{Outflow} = \text{Turbine Release} + \text{Spill}$

• Pool Elevation

Type: Agg Series Slot
Units: Length
Description: Elevation of the water surface of the Reservoir at the dam
Information: When Pool Elevation is a part of the optimization problem, as it is in all conceivable

• Power

Type: Agg Series Slot
Units: Power
Description: Power generated by flow through the turbines
Information:
Defined by: Numerical 3-D Approximation in terms of Operating Head and Turbine Release. Approximation is based on the Plant Power Table. The Power LP Param table contains a value for Operating Head used to index the Operating Head column of the Plant Power Table. This approximated value, therefore, reduces the Power to a function of Turbine Release at the given Operating Head. The flow values in the Power LP Param table are then used as approximation points indexing the Turbine Release column of the Plant Power Table. The Plant Power Table should have increasing values of Operating Head and Turbine Release. Power should be a concave function of Operating Head, but concavity is not strictly enforced; mild non-concave regions are permissible to allow for round-off error, and so on. The preferred order of approximation is substitution, piece-wise, two-point line, tangent.

• Return Flow

Type: MultiSlot
Units: Flow
Description: Flow returning from a diversion object
Information:
Defined by: Explicit Optimization variable in the mass balance constraint (see Storage)

• Spill

Type: Agg Series Slot
Units: Flow
Description: Sum of the Bypass, Regulated Spill and Unregulated Spill
Information:
Defined by: Explicit Optimization variable as $\text{Spill} = \text{Bypass} + \text{Regulated Spill} + \text{Unregulated Spill}$

• Storage

Type: Agg Series Slot
Units: Volume
Description: Volume of water stored in the reservoir
Information:

Defined by: Explicit Optimization variable as $\text{Storage} = \text{Storage}(t-1) + \text{Precipitation Volume} - \text{Evaporation} - \text{Change in Bank Storage} + \text{timestep} * (\text{Inflow} + \text{Inflow2 Canal Flow} + \text{Flow TO Pumped Storage} + \text{Hydrologic Inflow Net} + \text{Return Flow} - (\text{Outflow} + \text{Diversion} + \text{Flow FROM Pumped Storage}))$

• Tailwater Base Value

Type: Agg Series Slot
Units: Length
Description: Elevation of tailwater or base elevation used to compute elevation of tailwater
Information:
Defined by: Explicit Optimization variable should be input or linked. See “[Optimization Tailwater](#)” in *Optimization* for details about related constraints and other objects to which the slot is linked.

• Tailwater Elevation

Type: Agg Series Slot
Units: Length
Description: Water surface elevation on the downstream side of the dam
Information:
Defined by: Various approaches dependent on the method selected in the Optimization Tailwater category.

• Turbine Capacity LP Param

Type: Table
Units: Length, Length, Length
Description: LP Param table for turbine capacity
Information: See power methods for details

• Turbine Release

Type: Agg Series Slot
Units: Flow
Description: Flow through the turbines of a power reservoir
Information:
Defined by: Explicit Optimization variable as $\text{Turbine Release} \leq \text{Power Plant Cap Fraction} * \text{Turbine Capacity}$

User Methods

The following categories and methods are available for use in the Slope Power Reservoir object in Optimization. Because of dependency relationships, you may not be able to see them in your model until you change other methods. In the discussion below, you will see the other methods that need to be selected to enable a given method to be used in your model. When building a model, you need to review this to ensure that required dependencies are satisfied so that the desired methods are available for use.

User method categories in Optimization are as follows.

- [Avoidance Zones](#), page 207
- [Bank Storage](#), page 208
- [Cavitation](#), page 208
- [Diversion from Reservoir](#), page 208
- [Energy in Storage](#), page 209
- [,](#) page 210
- [Future Value](#), page 210
- [Head Loss](#), page 212
- [Hydrologic Inflow](#), page 212
- [Optimization Backwater](#), page 212
- [Optimization Evaporation](#), page 213
- [Optimization Future Value](#), page 216
- [Optimization Power](#), page 216
- [Optimization Reserves](#), page 227
- [Optimization Spill](#), page 228
- [Optimization Tailwater](#), page 231
- [Power](#), page 235
- [Slope Storage](#), page 241
- [Slope Storage Coefficients](#), page 241
- [Startup](#), page 241

Avoidance Zones

This category depends on selecting the Unit Power Table method (see “[Unit Power Table](#)” on page 239) and contains methods for modeling the existence of undesirable regions of operation for turbines. There are two methods in this category, one which does not model avoidance zones at all, and one which

■ None

This the default, no action method. Avoidance zones are not considered.

■ Unit Head Based Avoidance Zones

This method allows the user to specify a table that defines the conditions in which the turbines should not be operated.

See [“Unit-level Constraints”](#) on page 309 for descriptions of added constraints.

Slots Specific to This Method

See [“Unit Head Based Avoidance Zones”](#) in *Objects and Methods* for slot descriptions.

Bank Storage

Not all methods are functional in RPL optimization. Of the available methods, None and CRSS Bank Storage are supported.

■ None

See [“None”](#) in *Objects and Methods* for details.

■ CRSS Bank Storage

See [“CRSS Bank Storage”](#) in *Objects and Methods* for details.

Cavitation

Cavitation is the sudden formation and collapse of low-pressure bubbles in liquids by means of mechanical forces. This process can cause damage to turbines under certain operating conditions.

This category depends on selecting the Unit Power Table method (see [“Unit Power Table”](#) on page 239) and includes methods for dealing with the problem of cavitation on turbines.

■ None

This is the default, no action method.

■ Unit Head and Tailwater Based Regions

This method allows the user to specify the regions of operation in which cavitation does *not* occur, so these regions can be avoided. These regions can depend on both operating head and tailwater.

See [“Unit-level Constraints”](#) on page 309 for descriptions of added constraints.

Slots Specific to This Method

See [“Unit Head and Tailwater Based Regions”](#) in *Objects and Methods* for slot descriptions.

Diversion from Reservoir

Not all methods in this category are supported in RPL optimization. Of the available methods, None and Available Flow Based Diversion are supported; selecting any other method results in an error during begin run.

■ None

See “None” for details.

■ Available Flow Based Diversion

See “Diversion Power Efficiency Curve” for details.

Energy in Storage

In Optimization, currently None and EIS Table Lookup are supported.

■ None

No Energy in storage is considered.

■ EIS Table Lookup

With this method selected, Energy in Storage is considered as a function of Pool Elevation.

If the optimization problem uses Energy In Storage, either directly or indirectly, then this slot is added to the problem. Before being passed to the optimization solver, this slot is numerically approximated as a function of Pool Elevation (Numerical 2-D Approximation).

The relationship between Pool Elevation and Energy In Storage comes from the user-input Energy In Storage Table.

The table is queried either using user-input points defined in the Energy In Storage LP Param table, or using automatically calculated points, depending on method selection in the Pool Elevation Linearization Automation category, as follows.

- If the selected method in the Pool Elevation Linearization Automation is None, the user-input Energy In Storage LP Param table values are used.
- For other Pool Elevation Linearization Automation methods (Initial Target Input, Min Difference or Range Input, Min Difference), the same points automatically developed for the Pool Elevation linearization are used.

See “EIS Table Lookup” in *Objects and Methods* for details on the Simulation method.

Slots Specific to This Method

• Energy In Storage

Type:	Agg Series Slot
Units:	Energy vs Power
Description:	Energy in Storage in the reservoir
Information:	
Defined by:	Numerical 2-D Approximation in terms of Pool Elevation, based upon the Energy In Storage Table. The Energy In Storage LP Param table values are used as approximation points indexing the Energy In Storage Table. The Energy In Storage

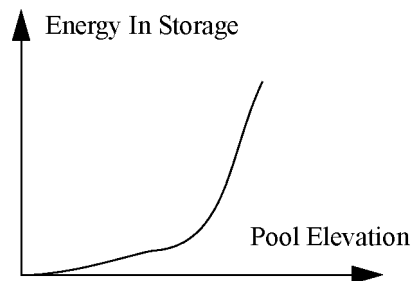
Table should have increasing values of Pool Elevation and Energy In Storage. Energy In Storage is required to be a convex function of Pool Elevation. The preferred order of approximation is substitution, piece-wise, tangent, two-point line.

• Energy in Storage LP Param

Type:	Table Slot
Units:	Length
Description:	Specifies the Pool Elevation points used to take the tangent, line and piecewise approximations for Energy In Storage linearization.
Information:	This table is used for linearization unless the Pool Elevation Linearization Automation category has a method selected other than None.
Defined by:	User input

• Energy in Storage Table

Type:	Table Slot
Units:	Length vs Energy
Description:	Table defining the relationship between Energy In Storage and Pool Elevation
Information:	
Defined by:	User input Energy in storage pool elevation relationship. Not drawn to scale.



Future Value

This category and its methods are not dependent on other method selections.

■ None

See “None” in *Objects and Methods* for details.

■ Cumulative Storage Value Table

In RPL Optimization, the Cumulative Storage Value is Numerically Approximated as described below.

See “Cumulative Storage Value Table” in *Objects and Methods* for details.

Slots Specific to This Method

• Cumulative Storage Value

Type:	Agg Series Slot
Units:	\$
Description:	Represents the future energy value of the current storage
Information:	
Defined by:	2-D approximation in terms of Anticipated Storage, based upon the Cumul Stor Val Table. The Cumul Stor Val LP Param table values (Storage) are used as approximation points indexing the Cumul Stor Val Table. The Cumul Stor Val Table should have increasing values of Storage and Cumulative Value. Cumulative Storage Value is required to be a concave function of Anticipated Storage. The preferred order of approximation is substitution, piece-wise, tangent, two-point line. The Cumul Stor Val Linearization Automation category's Marginal Value to Table and Lin method can automate creation of the Cumul Stor Val LP Param table and the Cumul Stor Val Table.

• Anticipated Storage

Type:	Agg Series Slot
Units:	Volume
Description:	The combination of the actual storage plus water that would be expected to enter the reservoir after the Current Timestep but has not yet, due to lagging.
Information:	
Defined by:	

• Cumul Stor Val Table

Type:	Table Slot
Units:	Volume vs Value
Description:	The estimated total economic value of water stored in the reservoir for discrete storage values.
Information:	
Defined by:	User input or by automated procedure if Cumul Stor Val Linearization Automation category has selected the Marginal Value to Table and Lin method.

• Marginal Storage Value Table

Type:	Table Slot
Units:	Storage vs \$ Value
Description:	Anticipated Storage versus Cumulative Storage Value per unit energy
Information:	This table should be increasing in storage, and logically decreasing in marginal value
Defined by:	Required input

• Spill Cost

Type:	Agg Series Slot
Units:	\$

Description:

Information:

Defined by:

Head Loss

This category depends on the Unit Power Table method (see [“Unit Power Table”](#) on page 239) and contains methods for modeling additional head loss that occurs. This head loss may come from the configuration of the penstocks for bringing water to the turbines.

■ None

In this method, there is no additional head loss to be used in the power calculation. In terms of penstock head loss, this method should be selected if the penstocks for the units are independent and the penstock losses are typically incorporated in the power data. Thus the power data is specified in terms of operating head.

■ Shared Penstock Head Loss method

In this method, there is additional head loss that results because units share a common penstock. The operating head losses in the penstock depend on the total turbine release and are shared for all units. The net head is calculated by subtracting penstock losses from the operating head. The unit data and power must be specified in terms of unit Net Heads instead of Operating Head.

See [“Unit-level Constraints”](#) on page 309 for descriptions of the added constraints.

Slots Specific to This Method

See [“Shared Penstock Head Loss method”](#) in *Objects and Methods* for slot descriptions.

Hydrologic Inflow

If any method in this category is selected, hydrologic inflow is included in the reservoir mass balance. Optimization assumes hydrologic inflow to be known (data) and it is not solved for by the reservoir regardless of the method selected.

See [“Hydrologic Inflow”](#) in *Objects and Methods* for details on the Simulation methods.

Optimization Backwater

■ None

This is the default method for this category. It does nothing. No defining constraints are added automatically. When this method is selected, Pool Elevation and Backwater Elevation must be configured as user-defined variables, and they must be defined by constraints in the optimization goal set. (See [“User-defined Variables”](#) on page 76.)

Slots Specific to This Method

None

■ Lambda

Note: The documentation is under development.

■ Independent Linearizations

Note: This method is not supported in RPL optimization.

Optimization Evaporation

This category can be used to model evaporation and precipitation.

■ None

This is the default method for this category. It does no calculations, and it requires that None be selected for the Evaporation and Precipitation category.

■ Opt Input Evaporation

This method is analogous to the Input Evaporation method in simulation, and it requires the Input Evaporation method to be selected for the Evaporation and Precipitation category. Evaporation Rate and Precipitation Rate are entered as a time series. Evaporation is calculated as a product of Evaporation Rate, Average Surface Area over the timestep and Timestep length. Similarly Precipitation Volume is calculated as the product of Precipitation Rate, Average Surface Area and Timestep length.

Evaporation and Precipitation are calculated based on a static Volume-Surface Area relationship. In reality the Volume-Surface Area relationship may not be static for a slope storage reservoir, so results from the Opt Input Evaporation method should be treated with caution when used on a slope power reservoir object.

Caution: The linearization of the Surface Area variable can result in a small approximation error in optimization for Evaporation and Precipitation Volume. This means there can be a small difference between the mass balances in the optimization solution and the post-optimization rulebased simulation when using this method. It is important to use care when setting the approximation points for Surface Area in order to reduce this approximation error. Refer to the information on the Surface Area LP Param slot below. Also caution should be used if applying this method at a 1 Month timestep. All rates in the optimization mass balance are converted to monthly volumes based on a 30-day month, regardless of the month. This will also produce a difference between the mass balances in the optimization solution and the post-optimization rulebased simulation for a 1 Month timestep.

Slots Specific to This Method

- **Elevation Area Table**

Type: Table Slot
Units: Length vs Area

Description:	Represents the Elevation-Surface Area relationship
Information:	This table must be input. It is used to derive the Volume Area Table.
Defined by:	User input

• Evaporation

Type:	Series Slot
Units:	Volume
Description:	The volume of water lost to evaporation over the timestep
Information:	If this slot contains user input, it is added directly to the mass balance constraint, otherwise it is defined by the expression below.
Defined by:	Either user input or the following constraint: $\text{Evaporation} = \text{EvaporationRate} \times \frac{\text{SurfaceArea}(t-1) + \text{SurfaceArea}}{2} \times \text{Timestep}$

• Evaporation Rate

Type:	Series Slot
Units:	Velocity
Description:	The rate, in length per time, at which water is lost to evaporation at each timestep
Information:	This slot can be set as user input. If it is not set as an input, and if Evaporation is not an input, this slot defaults to zero. If Evaporation is an input, this slot is not used.
Defined by:	User input or defaults to zero

• Precipitation Rate

Type:	Series Slot
Units:	Velocity
Description:	The rate, in length per time, at which water is gained from precipitation at each timestep
Information:	This slot can be set as user input. If it is not set as an input, it defaults to zero.
Defined by:	User input or defaults to zero

• Precipitation Volume

Type:	Series Slot
Units:	Volume
Description:	The volume of water gained from precipitation over the timestep
Information:	The Input Evaporation method will not allow this slot to be set as an input.
Defined by:	Explicit constraint: $\text{Precipitation Volume} = \text{PrecipitationRate} \times \frac{\text{SurfaceArea}(t-1) + \text{SurfaceArea}}{2} \times \text{Timestep}$

• Surface Area

Type:	Series Slot
Units:	Area

Description: The area of the water surface at the end of the timestep

Information: This slot is numerically approximated as a function of Storage (Numerical 2-D Approximation). The relationship between Pool Elevation and Storage comes from the automatically generated Volume Area Table. The table is queried using the user-input points defined in the Surface Area LP Param table.

Defined by: Numerical 2-D Approximation in terms of Storage, based upon the Volume Area Table. The Surface Area LP Param table values are used as approximation points indexing the Volume Area Table. The preferred order of approximation is substitution, piecewise, two-point line, tangent. Most often the two-point line (secant) approximation will be used.

• Surface Area LP Param

Type: Table Slot

Units: Volume

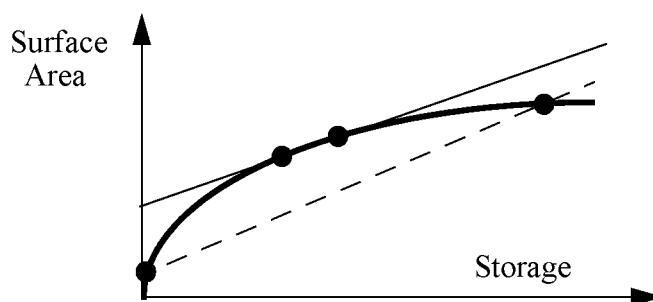
Description: Specifies the Storage points used to take the tangent, line and piecewise approximations for Surface Area linearization.

Information: The best Storage point to choose for tangent approximation would be the expected storage during the run; for line approximation, the expected maximum and minimum Storage; for piecewise approximation, use points that cover the full range of expected Storage during the run with intermediate points such that a piecewise linear curve reasonably approximates the actual curve. In most cases, the line (secant) approximation will be used. It is important to set these points carefully to minimize approximation error.

Defined by: User input

The following figure represents two alternative selections of points for the line approximation in the Surface Area LP Param table. The linear approximation represented by the dashed line corresponds to the selection of points near the extremes of the Volume Area table. This approximation will tend to result in an under-estimation of Surface Area, and thus an under-estimation of Evaporation and Precipitation Volume. Evaporation losses in the post-optimization rulebased simulation would be greater than the losses approximated in the optimization solution. The linear approximation represented by the solid line corresponds to the selection of points closer together in the Volume Area Table, and is more similar to the Tangent approximation. This approximation would tend to result in an over estimation of Surface Area, and the losses due to Evaporation in the post-optimization rulebased simulation would be less than the approximation in the optimization solution.

Surface area vs storage relationship with two alternative line approximations. Not drawn to scale.



- **Volume Area Table**

Type:	Table Slot
Units:	Volume vs Area
Description:	Represents the Storage Volume-Surface Area relationship
Information:	This table is read-only and is automatically generated at the start of the run from the Elevation Area Table and the Elevation Volume table. The method starts by copying the Elevation Area Table and then replaces the Pool Elevation Values with the corresponding Storage values. The Storage values are linearly interpolated based on the Elevation Volume Table
Defined by:	Automatically generated

Optimization Future Value

This category allows the optimization to provide slots relating to the future value of water. Its appearance is dependent on selecting the Cumulative Storage Value Table method for the Future Value category.

■ None

If this method is selected, the future value slots will not be visible, and no linearization will be attempted.

■ Opt Cumulative Storage Value Table

If this method is selected, the following slots will be visible, and linearization will be allowed.

Slots Specific to This Method

- **Cumul Stor Val LP Param**

Type:	Table Slot
Units:	Volume, Volume, Volume
Description:	Specifies the storage points used to take the tangent, line and piecewise approximations for Cumul Stor Val Table linearization
Information:	
Defined by:	User input or by automated procedure if Cumul Stor Val Linearization Automation category has selected the Marginal Value to Table and Lin method.

Optimization Power

The Optimization Power category allows the user to select the types of linearizations to be used for linearizing various slots. For Independent Linearizations, all slots are linearized separately according to the user-specified methods. This category has no dependencies.

■ None

This is the default method. It is an error to select this method for Optimization. No slots are added for this method.

■ Independent Linearizations

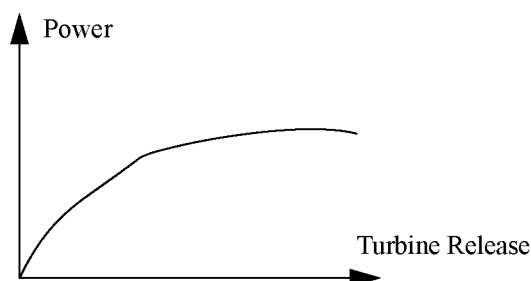
The Independent Linearizations method linearizes all the variables separately, according to the user-selected methods. The variables that need to be linearized vary greatly depending on the other methods selected.

See the various Categories and Linearization Approaches for details.

Slots Specific to This Method

- **Power LP Param**

Type:	Table Slot
Units:	Length and Flow
Description:	Specifies the Operating Head and the flow points used to take the tangent, line and piecewise approximations for Power linearization
Information:	This table must be user input when the Independent Linearizations method is selected. The best Operating Head to choose should be close to the expected head during optimized period. Tangent approximation is generally not used, nor helpful as it often results in non-zero power for zero flow. The suggested points for a line approximation are 0 flow and best turbine flow for the entire plant. The suggested points for piecewise linearization are 0 flow, 1 unit best turbine flow, 2 units best turbine flow ... n units best turbine flow, and maximum turbine flow.
Defined by:	Input Power turbine release relationship for a fixed operating head valve. Not drawn to scale.



Method Details

If the Optimization problem uses Power, either directly or indirectly, the Power slot is added to the problem.

Before being passed to the Optimization solver, the Power slot is numerically approximated as a function of Operating Head and Turbine Release (Numerical 3-D Approximation).

The relationship between Operating Head and Turbine Release comes from the Plant Power Table. This table must be user input. The table is queried using the points defined in the Power LP Param table.

■ Power Coefficient

The Power Coefficient method models Power at each time step as Turbine Release multiplied by a Power Coefficient Estimate.

Slots Specific to This Method

• Power Coefficient Estimate

Type:	Series Slot
Units:	PowerPerFlow
Description:	This represents the estimated Power Coefficient that gets used in the Optimization definition of Power.
Information:	The Power Coefficient Estimate is a required input for the run. It can either be input directly (manually or by DMI), or it could be set by an initialization rule. For example, an initialization rule could set the Power Coefficient Estimate based on a <i>Seed</i> (Slot Cache) value. If the Power Coefficient method is selected, and Power Coefficient Estimate is not an input for all time steps, then the run terminates with an error message.
Defined by:	Input Power at each time step is then defined as: $Power = \text{Turbine Release} \times \text{Power Coefficient Estimate}$

■ Power Surface Approximation

This method allows for the modeling of dynamic Operating Head to be incorporated into Optimization Power modeling. It provides a significant improvement in the Power approximation error over the linearization using the Power LP Param table, which assumes a constant Operating Head over the entire run period. The improvement is especially significant for projects that exhibit a wide range of Operating Head over the course of the run, particularly if the optimization policy is trying to maximize Power or set Power greater than or equal to a value.

The method introduces a new variable, PSA Head Factor, which represents the weighted contributions of Storage, Spill and Tailwater Base Value to Operating Head. The method generates a set of planes, the Power Surface, to constrain power as a function of Turbine Release and the PSA Head Factor.

The Power Surface can be thought of as pavement over a warped bridge that is level on one end and sloped and higher on the other end. The bridge ends represent zero Turbine Release and maximum Turbine Release respectively. In addition, the bridge is bowed concave across the center line everywhere except the level (zero Turbine Release) end. This bowing reflects Power as a function of the Head Factor for a fixed value of Turbine Release. The edges of the bridge represent Power as a function of Turbine Release for the low and high values of the Head Factor. Additional points defining a grid on the surface correspond to intermediate values of Turbine Release and the Head Factor. The user can define dimensions and points used for this grid. A piecewise linear surface is composed of cutting planes with each plane defining one triangle on the surface. A surface underneath the bridge composed of two planes defines a lower bound on Power given Turbine Release and Head Factor values.

Because only two planes can be defined for the lower bounds on power, additional approximation error can be introduced if the optimization policy has an incentive to minimize power. In these cases the optimization solution can have an incentive to under-approximate Power for a given Turbine Release, and thus the Post-optimization Rulebased Simulation will calculate a larger Power value than the optimization value for Power.

Slots Specific to This Method

• PSA Sample Input

Type:	Table Slot
--------------	------------

- Units:** Flow, Volume, Flow, Length
- Description:** This table contains user-specified sample values for Turbine Release, Storage, Total Spill (if applicable) and Tailwater Base Value (if applicable) that are used by RiverWare to compute the PSA Head Factor Weights. The four parameters in this table represent all of the parameters that contribute to Operating Head. In other words, if a value is known for each of these parameters, it is possible to calculate the corresponding Operating Head.
- Information:** The values in this table should span the full possible range for each parameter for the given run. For example, the Turbine Release column should contain a value of 0, the highest possible turbine release from the reservoir, and any other intermediate values specified by the user. Specifying tighter upper and lower bounds for each parameter will improve the approximation, but it is important that all possible values for each variable within the run are spanned by the range for that parameter in the table. The user can determine how many rows to include in the table. It is expected that typically, three to four values are specified for each parameter. An example is shown below. The Spill column should only contain values if a method other than None has been selected in the Spill category. Otherwise the Spill column should be left empty (will display “NaN”). The Tailwater Base Value column should only contain values if the reservoir’s Tailwater Base Value slot is linked to the Pool Elevation slot of a downstream reservoir (that is, if the Downstream Reservoir Pool Elevation contributes to the calculation of Operating Head). Otherwise the Tailwater Base Value column should be left empty.
- Defined by:** User input

Turbine Release	Storage	Spill (Total)	Tailwater Base Value
0	0	0	500
100	1000	50	510
200	2000	100	520

• PSA Head Factor Grid Lines

- Type:** Scalar Slot
- Units:** None
- Description:** The number of PSA Head Factor values for each Turbine Release value used when calculating the PSA Grid Points
- Information:** This must be an integer greater than or equal to 2. The value should typically range from 2 to 4. A larger number will improve the approximation but will increase run time.
- Defined by:** User input

• PSA Turbine Release Grid Lines

- Type:** Scalar Slot
- Units:** None
- Description:** The number of Turbine Release values for each PSA Head Factor value used when calculating the PSA Grid Points
- Information:** This must be an integer greater than or equal to 2. The value should typically range from 2 to 4. A larger number will improve the approximation but will increase run time.

Defined by: User input

• PSA Sample Output

Type: Table Slot

Units: Flow, Volume, Flow, Volume, Power

Description: A table containing all permutations of the values entered in the PSA Sample Input table slot along with the Power calculated for each combination of Sample Input values. This sample output is used to calculate the PSA Head Factor Weights.

Information: RiverWare will populate this table at the beginning of the run. For the PSA Sample Input table shown above, with three values in each of the four columns, this table would contain 81 rows, one for each permutation. For a given combination of Turbine Release, Storage, Spill and Tailwater Base Value, RiverWare can calculate the corresponding Power. The third column will contain the Downstream Storage values corresponding to the Tailwater Base Value entries in the PSA Sample Input slot.

Defined by: Populated by RiverWare at the beginning of the run.
For each row i in the table:

$$SamplePower_i = f(SampleTurbineRelease_i, SampleStorage_i, SampleSpill_i, SampleTailwaterBaseValue_i)$$

The calculation of SamplePower depends on the selected Power and Tailwater methods.

• PSA Head Factor Weights

Type: Table Slot

Units: PerTime, None, PerTime

Description: The weights for Storage, Spill and Downstream Storage used when calculating PSA Head Factor

Information: This is a 1 x 3 table with a single weight for Storage, Spill and Downstream Storage. The weights are calculated by RiverWare by performing a regression on the values in the PSA Sample Outputs table slot. If one of the parameters is not used (contains NaN for all rows in the PSA Sample Input table), the corresponding weight in this table slot will be zero.

Defined by: Calculated by regression at the beginning of the run

• PSA Head Factor

Type: Series Slot

Units: Flow

Description: This slot is a variable in the optimization solution and represents the weighted contribution of Storage, Spill and Downstream Storage to Operating Head and thus power. It can be thought of as a storage analog of Operating Head.

Information: This variable is used in the automatically generated constraints that define Power. It will not, generally display any values, but the user can write a post-optimization rule that calculates the value of this variable at each time step using the equation shown below and the values in the PSA Head Factor Weights table slot.

Defined by: $PSA\ Head\ Factor = \alpha \cdot Storage + \beta \cdot Spill + \gamma \cdot Downstream\ Storage$

The coefficients α , β and γ come from the PSA Head Factor Weights table slot.

• PSA Grid Points

Type:	Table Slot
Units:	Flow, Volume, Flow, Volume, Power, Flow
Description:	Points used to calculate the planes for the PSA Max Constraints and PSA Min Constraints, one row for each point with columns for Turbine Release, Storage, Spill, Downstream Storage, Power and PSA Head Factor. Turbine Release Column—Smallest and largest values from PSA Sample Input and evenly spaced intermediate values corresponding to the number of lines in the PSA Turbine Release Grid Lines scalar slot Storage, Spill and Downstream Storage Columns—Smallest and largest values from PSA Sample Input and evenly spaced intermediate values corresponding to the number of lines in the PSA Head Factor Grid Lines scalar slot Power Column—Determined by selected Power method, for each row i in the table $Power_i = f(TurbineRelease_i, Storage_i, Spill_i, DownstreamStorage_i)$ Head Factor Column—The coefficients α, β and γ come from the PSA Head Factor Weights table slot. $HeadFactor_i = \alpha \cdot Storage_i + \beta \cdot Spill_i + \gamma \cdot DownstreamStorage_i$
Information:	The number of rows (points) in this table equals the product of the number of lines in the PSA Head Factor Grid Lines and PSA Turbine Release Grid Lines scalar slots. For each grid point, RiverWare calculates the value of Power and the PSA Head Factor corresponding to the parameter values in that row.
Defined by:	Populated by RiverWare at the beginning of the run.

• PSA Max Constraints

Type:	Table Slot
Units:	Flow, Flow, Flow, Flow, Flow, FLOW, PowerPerFlow, PowerPerFlow, Power
Description:	This table contains the planes that define the upper bounds for power as a function of Turbine Release and PSA Head Factor, one row for each plane. Power is constrained to be less than or equal to the planes defined in this table. The first six columns contain three pairs of Turbine Release and PSA Head Factor values that define three points on the plane. The remaining three columns contain the corresponding Turbine Release Coefficient, Head Factor Coefficient and Constant Term that define the same plane and are used in the actual constraint expressions.
Information:	The points used to define the planes come from the Turbine Release and Head Factor points in the PSA Grid Points table slot. RiverWare will not use all of the possible combinations of Turbine Release and Head Factor points but rather will only use the combinations necessary to define the Power Surface.
Defined by:	Populated by RiverWare at the beginning of the run For each row i in the PSA Max Constraints table slot a constraint is added to the optimization problem: $Power \leq u_i \cdot \text{Turbine Release} + v_i \cdot \text{PSA Head Factor} + w_i$ The coefficients u_i, v_i and w_i are the Turbine Release Coefficient, Head Factor Coefficient and Constant Term from row i of the PSA Max Constraints table. Turbine Release and PSA Head Factor are the actual variables in the optimization problem.

• PSA Min Constraints

Type:	Table Slot
Units:	Flow, Flow, Flow, Flow, Flow, FLOW, PowerPerFlow, PowerPerFlow, Power
Description:	This table contains the planes that define the lower bounds for power as a function of Turbine Release and PSA Head Factor, one row for each plane. Power is constrained to be greater than or equal to the planes defined in this table. The first six columns contain three pairs of Turbine Release and PSA Head Factor values that define three points on the plane. The remaining three columns contain the corresponding Turbine Release Coefficient, Head Factor Coefficient and Constant Term that define the same plane and are used in the actual constraint expressions
Information:	This table will always contain two rows. Only two planes can be defined for the lower bounds on power.
Defined by:	Populated by RiverWare at the beginning of the run For each row i in the PSA Min Constraints table slot, a constraint is added to the optimization problem: $Power \geq u_i \cdot \text{Turbine Release} + v_i \cdot \text{PSA Head Factor} + w_i$ The coefficients u_i, v_i and w_i are the Turbine Release Coefficient, Head Factor Coefficient and Constant Term from row i of the PSA Min Constraints table. Turbine Release and PSA Head Factor are the actual variables in the optimization problem.

Method Details

The following slots require input from the user:

- [PSA Sample Input](#), page 218
- [PSA Head Factor Grid Lines](#), page 219
- [PSA Turbine Release Grid Lines](#), page 219

RiverWare carries out the following steps to populate the remaining table slots associated with this method.

1. Populate the PSA Sample Output Table with all permutations of parameters in the PSA Sample Input table slot.
2. Calculate the PSA Head Factor Weights using a regression based on values in the PSA Sample Output table slot.
3. Add defining constraints for PSA Head Factor using the PSA Head Factor Weights.
4. Populate the PSA Grid Points table slot based on the number of grid lines specified by the user.
5. Generate upper bound planes using the PSA Grid Points and add the corresponding constraints on Power to the Optimization problem.
6. Generate lower bound planes using the PSA Grid Points and add the corresponding constraints on Power to the Optimization problem.

Power Surface Definition Details

This section provides details of the calculations carried out by RiverWare to generate the constraints that define the Power Surface. This material is included for reference only.

1. RiverWare populates the PSA Sample Output table slot. All permutations of values in the PSA Sample Input table slot are determined, and a row for each is added to the PSA Sample Output table slot. RiverWare may make some adjustments to prevent infeasible combinations. For example, the highest Turbine Release value may not be possible for all combinations of Storage, Spill and Tailwater Base Value. RiverWare will adjust the Turbine Release to use the maximum turbine release for the resulting Operating Head. The table is organized in blocks of constant Turbine Release, with the lowest Turbine Release block first and then increasing Turbine Release. The values in the remaining columns are ordered corresponding to increasing Operating Head. Storage is increasing. Spill and Tailwater Base Value (or Downstream Storage) are decreasing. For each combination of values (each row), RiverWare calculates the corresponding Operating Head and Power. The Power value is added to each row in the PSA Sample Output table. The details of these calculations are shown here for a given row i in the PSA Sample Output table.

Note: $Power_i$ is the only calculated quantity displayed in the table. Values from the intermediate calculations shown below are not displayed in the table.

$$PoolElevation_i = f(Storage_i)$$

Pool Elevation is calculated by interpolation on the reservoir Elevation Volume Table.

If Tailwater Base Value is linked to Pool Elevation of a downstream reservoir:

$$TailwaterBaseValue_i = DownstreamPoolElevation_i = f(DownstreamStorage_i)$$

Tailwater Base Value is calculated by interpolation on the downstream reservoir Elevation Volume Table. Tailwater Base Value is only calculated from the Downstream Storage if either Base Value Plus Lookup Table, Stage Flow Lookup Table or Coefficients Table is the selected Tailwater method *and* Tailwater Base Value is linked to the Pool Elevation of the downstream reservoir. Otherwise Tailwater Base Value is unused, and thus Operating Head and Power are not dependent on Downstream Storage.

$$Outflow_i = TurbineRelease_i + Spill_i$$

Spill will only be non-zero if a method other than None is selected in the Spill category.

$$TailwaterElevation_i = f(Outflow_i, TailwaterBaseValue_i)$$

The tailwater calculation is dependent on the selected method in the Tailwater category.

$$OperatingHead_i = PoolElevation_i - TailwaterElevation_i$$

$$Power_i = f(TurbineRelease_i, OperatingHead_i)$$

The Power calculation is dependent on the selected method in the Power category.

2. RiverWare calculates the PSA Head Factor Weights using a combination of linear regression and averages.

FOR EACH Turbine Release value i

FOR EACH combination of Spill and Tailwater Base Value (or Downstream Storage), k , perform a linear regression to calculate a temporary Storage coefficient g_{ik} :

$$g_{ik} = \frac{n \sum (Power_j \times Storage_j) - \left(\left(\sum Power_j \right) \left(\sum Storage_j \right) \right)}{n \left(\sum Storage_j^2 \right) - \left(\sum Storage_j \right)^2}$$

Chapter 5

Optimization Objects and Methods

where n is the number of Storage values.

Calculate the temporary Storage coefficient for the given Turbine Release, g_i :

$$g_i = \frac{\sum g_{ik}}{n}$$

where n is the number of Spill and Downstream Storage) combinations.

FOR EACH combination of Storage and Downstream Storage, k , perform a linear regression to calculate a temporary Spill coefficient e_{ik} :

$$e_{ik} = \frac{n \sum (Power_j \times Spill_j) - \left(\left(\sum Power_j \right) \left(\sum Spill_j \right) \right)}{n \left(\sum Spill_j^2 \right) - \left(\sum Spill_j \right)^2}$$

where n is the number of Spill values.

Calculate the temporary Spill coefficient for the given Turbine Release, e_i :

$$e_i = \frac{\sum e_{ik}}{n}$$

where n is the number of Storage and Downstream Storage combinations.

FOR EACH combination of Storage and Spill, k , perform a linear regression to calculate a temporary Downstream Storage coefficient f_{ik} :

$$f_{ik} = \frac{n \sum (Power_j \times DownstreamStorage_j) - \left(\left(\sum Power_j \right) \left(\sum DownstreamStorage_j \right) \right)}{n \left(\sum DownstreamStorage_j^2 \right) - \left(\sum DownstreamStorage_j \right)^2}$$

where n is the number of Downstream Storage values.

Calculate the temporary Downstream Storage coefficient for the given Turbine Release, f_i :

$$f_i = \frac{\sum f_{ik}}{n}$$

where n is the number of Storage and Spill combinations.

Calculate average ratios over all Turbine Release values:

$$c = \frac{\sum (e_i / g_i)}{n}$$

$$d = \frac{\sum (f_i / g_i)}{n}$$

where n is the number of Turbine Release values.

Calculate the final Head Factor Weights:

$$\text{Storage Weight} = \frac{1}{c}$$

$$\text{Spill Weight} = -1$$

If Tailwater Base Value is linked to the downstream Pool Elevation:

$$\text{Downstream Storage Weight} = \frac{d}{c}$$

These three weight values are displayed in the three columns of the PSA Head Factor Weights table slot.

In the optimization solution the PSA Head Factor is defined at a given time step by

- If Tailwater Base Value is linked to the downstream Pool Elevation:

$$\text{PSA Head Factor} = \text{StorageWeight} \times \text{Storage} + \text{SpillWeight} \times \text{Spill} + \text{DownstreamStorageWeight} \times \text{Downstream Storage}$$

- If Tailwater Base Value is not linked:

$$\text{PSA Head Factor} = \text{StorageWeight} \times \text{Storage} + \text{SpillWeight} \times \text{Spill}$$

3. RiverWare calculates the grid points for the PSA Grid Points table slot. The number of rows (points) in this table equals the product of the number of lines in the PSA Head Factor Grid Lines and PSA Turbine Release Grid Lines scalar slots. The smallest and largest values in the PSA Sample Input slot lead to the smallest and largest values for each of the parameters that affect power (Turbine Release, Storage, Spill and Downstream Storage). One row in this table will correspond to the smallest Turbine Release, smallest Storage, largest Spill and largest Downstream Storage, and thus smallest power. Another row will correspond to the largest Turbine Release, largest Storage, smallest Spill and smallest Downstream Storage, and thus the largest Power. RiverWare generates evenly spaced values between the extremes for each variable. Within the PSA Grid Points table, for a given Turbine Release value, Storage is increasing and Spill and Downstream Storage are decreasing. For each grid point, RiverWare calculates the value of Power and the PSA Head Factor corresponding to the parameter values in each row and adds these to the final two columns in the table.

For each row i in the table:

$$\text{Power}_i = f(\text{TurbineRelease}_i, \text{Storage}_i, \text{Spill}_i, \text{DownstreamStorage}_i)$$

The Power calculation depends on the method selected in the Power category.

$$\text{HeadFactor}_i = \alpha \cdot \text{Storage}_i + \beta \cdot \text{Spill}_i + \gamma \cdot \text{DownstreamStorage}_i$$

The coefficients α , β and γ come from the PSA Head Factor Weights table slot.

4. RiverWare calculates the upper bound planes. These are entered in the PSA Max Constraints table slot, one row for each plane, where the planes represent Power as a function of Turbine Release and PSA Head Factor. For each combination of three points in the PSA Grid Points Table, RiverWare calculates the corresponding plane. For any three points, the plane is defined by three linear equations, which can be represented in matrix form.

$$\bar{P} = A\bar{u}$$

Chapter 5

Optimization Objects and Methods

$$\bar{P} = \begin{bmatrix} P_1 \\ P_2 \\ P_3 \end{bmatrix}, A = \begin{bmatrix} QT_1 & HF_1 & 1 \\ QT_2 & HF_2 & 1 \\ QT_3 & HF_3 & 1 \end{bmatrix} \quad \bar{u} = \begin{bmatrix} u \\ v \\ w \end{bmatrix}$$

The coefficients that define the plane are solved for by

$$\bar{u} = A^{-1}\bar{P}$$

RiverWare only uses the planes that are necessary to define the Power Surface. Each plane is checked against all remaining points in the PSA Grid Points table. If the Power value for any of the remaining points lies above the given plane, then that plane is rejected (that is, it would over-constrain Power, resulting in an under-estimation of Power). For each plane i that is used, RiverWare adds the following constraint to the optimization problem for each time step:

$$Power \leq u_i \cdot \text{Turbine Release} + v_i \cdot \text{PSA Head Factor} + w_i$$

5. RiverWare calculates the lower bound planes. These are entered in the PSA Min Constraints table slot, one row for each plane. Only two planes can be calculated for the lower bounds. The points used for the two planes are as follows:

Plane 1:

- Point A1 - Min Turbine Release, Max Head Factor
- Point B1 - Max Turbine Release, Max Head Factor
- Point C1 - Min Turbine Release, Min Head Factor

Plane 2:

- Point A2 - Min Turbine Release, Max Head Factor
- Point B2 - Max Turbine Release, Max Head Factor
- Point C2 - Max Turbine Release, Max Head Factor

The coefficients defining the planes are calculated using the same matrix calculation described for the upper bound planes. For each lower bound plane i , RiverWare adds the following constraint to the optimization problem for each time step:

$$Power \geq u_i \cdot \text{Turbine Release} + v_i \cdot \text{PSA Head Factor} + w_i$$

Note: Regarding the lower bound approximation, because only two planes can be defined for the lower bounds on power, additional approximation error can be introduced if the optimization policy has an incentive to minimize power. This can occur if Power is included in a Minimize objective. It can also occur if the optimization problem contains constraints that require Power to be less than or equal to value.

Note: These types of constraints can be introduced through multiple forms of RPL optimization goals. The first is basic less than or equal to constraint.

ADD CONSTRAINT Res.Power[t] $\leq$ Value

Policy that requires Power to be equal to a value also adds a less than or equal to constraint. The following RPL statement:

```
ADD CONSTRAINT Res.Power[t] = Value
```

actually adds the following two constraints to the optimization problem:

```
Res.Power[t] ≤ Value
```

```
Res.Power[t] ≥ Value
```

A constraint to require the sum of Power from multiple reservoirs to be less than or equal to a value will have a similar effect. A final manner in which this can occur is through policy that minimizes Power indirectly through user-defined variables. For example, assume a user-defined variable is defined by the following RPL constraint:

```
ADD CONSTRAINT Res.Power[t] + UserVariable[t] = Value1
```

Then later the variable is constrained by

```
ADD CONSTRAINT UserVariable[t] ≥ Value2
```

An equivalent constraint would be

```
ADD CONSTRAINT Res.Power[t] ≤ Value1 - Value2
```

It is possible to reduce the approximation error in these cases by adding constraints to the RPL Optimization Goal Set that essentially create more restrictive planes for the lower bound. These types of constraints are model-specific. Contact CADSWES if assistance is required to add more restrictive lower bounds to the Power Surface Approximation.

Optimization Reserves

This category depends on selecting either Plant Power Coefficient or Plant Efficiency Curve in the Power category. The methods in this category can be used to account for power reserve requirements in the optimization policy. There is no analogous simulation method associated with this category.

Note: The methods in this category have not been fully implemented for Slope Power Reservoirs. The selected method should be left as the default, None. Otherwise the run terminates with an error message stating that the method selection is not allowed for Slope Power Reservoirs.

■ None

This is the default, no action method. No new slots are added, and no new variables or constraints are added to the Optimization problem.

This method should always be selected for Slope Power Reservoirs.

■ Constraint Based Single Timestep

Caution: Although this method is visible on Slope Power Reservoirs, it is not fully implemented. Selecting this method on Slope Power Reservoir causes the run to terminate with an error.

Optimization Spill

The Optimization Spill methods determine how spill is calculated for the reservoir and generates physical constraints that correspond to the selected methods.

This category is dependent on selection of the Independent Linearizations method for the Optimization Power category. However, the method selected in the Optimization Spill category must match with the corresponding non-optimization method in the Spill category.

Spill is an Optimization decision variable. The following constraint is always generated for a reservoir:

$$\text{Outflow} = \text{Turbine release (or Release)} + \text{Spill} \quad (5.36)$$

The spill methods generate values applicable to the following additional constraint, where some of these terms may be omitted if they do not apply to the selected spill method.

$$\text{Spill} = \text{Regulated Spill} + \text{Unregulated Spill} + \text{Bypass}, \quad (5.37)$$

As applicable, the following constraints are also added:

$$\text{Bypass} \leq \text{Bypass Capacity} \quad (5.38)$$

$$\text{Regulated Spill} \leq \text{Regulated Spill Capacity} \quad (5.39)$$

$$\text{Unregulated Spill} = \text{Unregulated Spill Capacity} \quad (5.40)$$

See “Spill” in *Objects and Methods* for details on non-optimization Spill methods.

Slots Specific to This Method

• Bypass Capacity

Type:	Series Slot
Units:	Flow
Description:	Bypass capacity
Information:	
Defined by:	Numerical 2-D Approximation in terms of storage, based upon the Bypass Capacity Table.

• Bypass Capacity Table

Type:	Table Slot
Units:	Volume vs Flow
Description:	Storage vs corresponding maximum bypass spill values
Information:	
Defined by:	Internally developed based on Bypass Table and Elevation Volume Table relationships. For each pool elevation in the Bypass Table, the Bypass Capacity

Table has a row relating Storage to Bypass Capacity. The Pool Elevations are converted to Storage using the Elevation Volume Table.

- **Regulated Spill Capacity**

Type: Series Slot
Units: Flow
Description: Regulated spill capacity
Information:
Defined by: Numerical 2-D Approximation in terms of storage, based upon the Regulated Spill Capacity Table.

- **Regulated Spill Capacity Table**

Type: Table Slot
Units: Volume vs Flow
Description: Storage vs corresponding maximum regulated spill values
Information:
Defined by: Internally developed based on Regulated Spill Table and Elevation Volume Table relationships. For each pool elevation in the Regulated Spill Table, the Regulated Spill Capacity Table has a row relating Storage to Regulated Spill Capacity. The Pool Elevations are converted to Storage using the Elevation Volume Table.

- **Regulated Spill or Bypass LP Param**

Type: Table Slot
Units: Volume
Description: Specifies the Storage points use to take the tangent, line and piecewise approximations for Regulated Spill Capacity linearization and Bypass Spill Capacity linearization.
Information:
Defined by: User input

- **Unregulated Spill Linearization Table**

Type: Table Slot
Units: Volume vs Flow
Description: Storage vs corresponding unregulated spill values
Information:
Defined by: Internally developed based on Unregulated Spill Table and Elevation Volume Table relationships. For each pool elevation in the Unregulated Spill Table, the Unregulated Spill Capacity Table has a row relating Storage to Unregulated Spill Capacity. The Pool Elevations are converted to Storage using the Elevation Volume Table.

- **Unregulated Spill LP Param**

Type: Table Slot
Units: Volume

Description: Specifies the Storage points use to take the tangent, line and piecewise approximations for Unregulated Spill Linearization Table linearization

Information:

Defined by: User input

■ None

If this method is selected the slot bounds in the slot configuration dialog are set to zero. No additional constraints are generated.

■ Opt Monthly Spill

Note: This method is not functional in RPL Optimization.

This method sets the lower and upper bounds on spill. The lower bound is set to zero and the default upper bound is set to a very big number (9,999,999 cms). The default upper bound can be revised in the Spill slot configuration, Upper Bound parameter. No additional constraints are generated beyond these bounds.

■ Opt Unregulated

If this method is selected only unregulated spill is considered and the following constraint is added to the LP:

$$\text{Spill} = \text{Unregulated Spill} \quad (5.41)$$

■ Opt Regulated

When this method is selected only regulated spill is considered, the lower bound on spill is set to zero, and the following constraint is added to the LP:

$$\text{Spill} = \text{Regulated Spill} \quad (5.42)$$

■ Opt Regulated and Unregulated

When this method is selected unregulated and regulated spill are considered, and the following constraint is added to the LP:

$$\text{Spill} = \text{Regulated Spill} + \text{Unregulated Spill} \quad (5.43)$$

■ Opt Regulated and Bypass

When this method is selected only regulated and bypass spill are considered, the lower bound on spill is set to zero, and the following constraint is added to the LP:

$$\text{Spill} = \text{Regulated Spill} + \text{Bypass} \quad (5.44)$$

■ Opt Regulated, Bypass and Unregulated

When this method is selected unregulated, regulated, and bypass spill are considered, and the following constraint is added to the LP:

$$\text{Spill} = \text{Regulated Spill} + \text{Unregulated Spill} + \text{Bypass} \quad (5.45)$$

■ Opt Bypass, Regulated and Unregulated

When this method is selected unregulated, regulated, and bypass spill are considered, and the following constraint is added to the LP:

$$\text{Spill} = \text{Bypass} + \text{Regulated Spill} + \text{Unregulated Spill} \quad (5.46)$$

Optimization Tailwater

Only a subset of available methods are supported in Optimization. These methods are: Linked or Input, Base Value Only, Base Value Plus Lookup Table, Stage Flow Lookup Table, and Coefficients Table. Selection of a method other than those will result in an error. The method selected in the Optimization Tailwater category should correspond to the one selected in the simulation Tailwater category.

Tailwater Elevation may be part of the Optimization problem and it is handled differently for each method. Below is a description of the behavior of Tailwater Elevation.

See “Tailwater” in *Objects and Methods* for details on the Simulation methods.

■ Opt Linked or Input

If Tailwater Elevation is not input, it is replaced as follows:

$$\text{Tailwater Elevation} = \text{Tailwater Base Value}$$

Slots Specific to This Method

- **Tailwater Base Value**

Type: Series
Units: Length
Description: See “Linked or Input” in *Objects and Methods* for details.

■ Opt Base Value Only

If Tailwater Elevation is not input, it is replaced as follows:

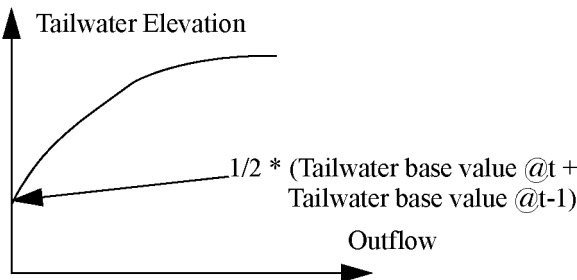
$$(\text{Tailwater Base Value}(t) + \text{Tailwater Base Value}(t-1)) / 2$$

Slots Specific to This Method

- **Tailwater Base Value**

Type: Series
Units: Length
Description: See “Base Value Only” in *Objects and Methods* for details.

Tailwater Outflow relationship. Not drawn to scale.



■ **Opt Base Value Plus Lookup Table**

If Tailwater Elevation is not input, this slot is replaced by a mathematical expression based on Tailwater Base Value and a numerical approximation of Tailwater Elevation as a function of Outflow (Numerical 2-D Approximation), as defined by the user in the Tailwater Table.

Tailwater Elevation is constrained as follows:

$$((\text{tailwaterBaseValue}(t) + \text{tailwaterBaseValue}(t-1)) / 2) + \text{tempTWLookupValue}$$

where tempTWLookupValue is a Numerical 2-D Approximation of increase in Tailwater Elevation due to flow, as described in the Tailwater Table. The Tailwater Table Lookup LP Param table values for outflow are used as approximation points indexing the Tailwater Table to determine tempTWLookupValue from the Tailwater Elevation column.

Slots Specific to This Method

• Tailwater Base Value	
Type:	Series
Units:	Length
Description:	See “Base Value Plus Lookup Table” in <i>Objects and Methods</i> for details.
• Tailwater Table	
Type:	Series Slot
Units:	Flow vs Length
Description:	Reservoir outflow vs either the tailwater elevation or the tailwater elevation increment
Information:	If the Tailwater Base Value is non-zero, the Tailwater Table gives values of incremental increase in Tailwater Elevation over the Base value. If the Tailwater Base Value is zero, the table simply gives the Tailwater Elevation values. The Tailwater Table should have increasing values of outflow. Tailwater Elevation is required to be a convex function of outflow. The preferred order of approximation is substitution, piece-wise, two-point line, tangent. See Object / Simulation documentation for details.
I/O:	User input

• Tailwater Table Lookup LP Param

Type:	Table Slot
Units:	Flow, Flow, Flow
Description:	LP Parameter approximation points for Outflow.
Information:	The Tailwater Table Lookup LP Param table values for outflow are used as approximation points indexing the Tailwater Table to determine tempTWLookupValue from the Tailwater Elevation column.
I/O:	User input only
Links:	Not linkable

• temp Tailwater Lookup Value

Type:	Series
Units:	Length
Description:	Numerical 2-D Approximation of increase in Tailwater Elevation due to flow, as described in the Tailwater Table.

■ Opt Stage Flow Lookup Table

Tailwater Elevation is numerically approximated as a function of Stage and Flow, as defined by the user in the Stage Flow Tailwater Table. The tables are queried depending on the method selected in the Tailwater Linearization Automation category, as follows:

- If None is selected, the user-input Tailwater LP Param table values are used.
- I Range Input Automation is selected, the automatically developed points are used.

Tailwater Elevation is approximated numerically (3-D approximation) as a function of Outflow and Tailwater Base Value, based on the Stage Flow Tailwater Table. The Tailwater Elevation LP Param table Tailwater Base Value and flow values are used as approximation points, indexing the Downstream Stage and Outflow columns of the Stage Flow Tailwater Table, respectively.

See “Stage Flow Lookup Table” in *Objects and Methods* for details.

Slots Specific to This Method

• Tailwater Base Value

Type:	Series
Units:	Length
Description:	See “Stage Flow Lookup Table” in <i>Objects and Methods</i> for details.

• Stage Flow Tailwater Table

Type:	Table Slot
Units:	Flow vs Length vs Length
Description:	Reservoir Outflow vs Downstream Elevation (Tailwater Base Value) vs Tailwater Elevation
Information:	Data must be entered into the table in increasing blocks of the same Outflow value for the three-dimensional table interpolation to work correctly. For every block of

same Outflows in column 1, Downstream Stages should be listed in increasing order in column 2, and the corresponding Tailwater Elevations in column 3. Tailwater elevation is required to be a concave function of Outflow.

The preferred order of approximation is substitution, piece-wise, two-point line, tangent. Internally, the Stage Flow Tailwater Table is rearranged to use the Stage as the primary index (column), with outflow as the secondary index (column). This rearranged table may be labeled as “Convolved Stage Flow Tailwater Table” in output messaging. Also, because of this rearrangement, it is desirable to repeat stage values for each outflow.

Defined by: User input

• Tailwater Elevation LP Param

Type: Table Slot

Units: Length, Flow, Flow, Flow

Description: Specifies the fixed tailwater base value point and the outflow points used to take the tangent, line and piecewise approximations for tailwater elevation linearization

Information: This table is used for linearization. The best Tailwater Base Value point to choose for tangent approximation would be an outflow equal to the expected value of outflow during the run; for the line approximation, the minimum and maximum values expected during the run; for piecewise approximation, the minimum and maximum values expected during the run plus additional intermediate values to more closely fit the tailwater curve.

Defined by: User input, unless the selected method in the Tailwater Linearization Automation category is not None

• Tail Water Reference Elevation

Type: Table Slot

Units: Length

Description: Lowest Reservoir discharge Elevation when there are no backwater effects from a downstream pool (reservoir)

Information: Although this is part of the Stage Flow Lookup Table method (and its corresponding Opt method), *this slot does not influence optimization*. See Object / Simulation documentation for details.

I/O: User input

■ Opt Coefficients Table

This method sets up the physical Tailwater Elevation constraints using the same equation and coefficients used in the Coefficients Table tailwater method.

$$\begin{aligned} \text{Tailwater Elevation} = & \text{Constant Coeff}[t] + \text{Constant Coeff}[t - 1] + \\ & \text{Outflow Coeff}[t] \times \text{flow} + \text{Outflow Coeff}[t - 1] \times \text{Outflow}[t - 1] + \\ & \text{TW Base Val Coeff}[t] \times \text{TailwaterBaseValueTemp}[t] + \\ & \text{TW Base Val Coeff}[t - 1] \times \text{Tailwater BaseValue}[t - 1] + \\ & \text{TW Elev Coeff}[t - 1] \times \text{Tailwater Elevation}[t - 1] \end{aligned}$$

See “Coefficients Table” in *Objects and Methods* for details.

Slots Specific to This Method

• Tailwater Base Value

Type: Series
Units: Length
Description: See “Coefficients Table” in *Objects and Methods* for details.

• Tailwater Coefficients

Type: Table
Units: multi
Description: See “Coefficients Table” in *Objects and Methods* for details.
Defined by: If the Power Surface Approximation method is selected, then there are additional restrictions on the coefficients that can be specified. With the Power Surface Approximation, having terms for Outflow[t-1], Tailwater BaseVal[t] and Tailwater[t-1] are not allowed.

Power

Only a subset of available Simulation methods are available in Optimization. Selection of a method other than those that follow will result in an error.

If the Optimization problem uses Power, either directly or indirectly, then the Power slot is added to the problem. Before being passed to the optimization solver, this slot is numerically approximated as a function of Operating Head and Turbine Release (Numerical 3-D Approximation).

See “Power” in *Objects and Methods* for details on the Simulation methods.

Slots Specific to This Method

• Power

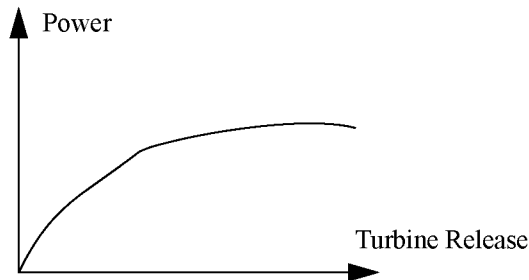
Type: Agg Series Slot
Units: Power
Description: Power generated by flow through the turbines
Information:
Defined by: Numerical 3-D Approximation in terms of Operating Head and Turbine Release. Approximation is based on the Plant Power Table.
 The Power LP Param table contains a value for Operating Head used to index the Operating Head column of the Plant Power Table. This approximated value, therefore, reduces the Power to a function of Turbine Release.
 The flow values in the Power LP Param table are then used as approximation points, indexing the Turbine Release column of the Plant Power Table.
 The Plant Power Table should have increasing values of Operating Head and Turbine Release. Power should be a concave function of Operating Head, but concavity is not strictly enforced. The preferred order of approximation is substitution, piece-wise, two-point line, tangent.

- **Power LP Param**

Type:	Table Slot
Units:	Length and Flow
Description:	Specifies the Operating Head and the flow points used to take the tangent, line, and piecewise approximations for Power linearization.
Information:	This table must be user input if the Independent Linearizations method is selected in the Optimization Power category; otherwise it is unused. The best Operating Head to choose should be close to the expected head during optimized period. Tangent approximation is generally not used, nor helpful, as it often results in non-zero power for zero flow. The suggested points for a line approximation are 0 flow and best turbine flow for the entire plant. The suggested points for piecewise linearization are 0 flow, 1 unit best turbine flow, 2 units best turbine flow ... n units best turbine flow, and maximum turbine flow.

Defined by:

Power Turbine Release relationship, for a fixed Operating Head value. Not drawn to scale.



■ Plant Power Coefficient

Slots Specific to This Method

In the RPL Optimization mode, the following slot requires special handling.

- **Best Turbine Flow**

Type:	Agg Series Slot
Units:	Flow
Description:	Flow associated with the most efficient power generation for an associated Operating Head
Information:	If the Optimization problem uses Best Turbine Flow, either directly or indirectly, this slot is added to the problem. Before being passed to the Optimization solver, this slot is numerically approximated as a function of Operating Head (Numerical 2-D Approximation). The relationship between Operating Head and Best Turbine Flow depends on other methods selected, as follows. If the selected method in the Power category is Plant Efficiency Curve, the relationship is automatically developed from the Plant Power Table.

For the Plant Power Coefficient method, the user-input Best Turbine Q table defines the relationship.

For other Plant Calculation category methods, Best Turbine Flow must be input.

Defined by: Numerical 2-D Approximation in terms of Operating Head, based upon an internal best turbine flow table. For the Plant Power Coefficient method (selected in the Power category) the internal table is the Best Turbine Q table. The Best Turbine Q table is required to have increasing values of Operating Head and Best Capacity, and be a concave function of Operating Head.

The preferred order of approximation is substitution, piece-wise, tangent, two-point line. The Best Turbine Flow LP Param table values are used as approximation points, indexing the best turbine flow table.

• Best Turbine Flow LP Param

Type: Table Slot

Units: Length

Description: Specifies the Operating Head points in the best turbine flow relationship used to take the tangent, line and piecewise approximations for Best Turbine Flow linearization

Information: The best operating head points to choose for a piecewise approximation are generally: minimum Operating Head, Operating Head at max capacity, and maximum Operating Head. These three points typically define most of the shape of the Best Turbine Flow curve.

Defined by: User input

• Best Turbine Q

Type: Table Slot

Units: Length vs Flow

Description: Table defining the relationship between Operating Head and the most efficient power generation.

Information: This table is used if the Power category has the Plant Power Coefficient method selected. If the Power category has the Plant Efficiency Curve method selected, then this table is not used, but a similar relationship is automatically developed from the Plant Power Table.

Defined by: User input

• Plant Power Table

Type: Table Slot

Units: Length vs Flow vs Power

Description: Table defining the relationship between Operating Head, Turbine Release, and Power at selected points of operation. In building the Plant Power table, the following approach might be taken.

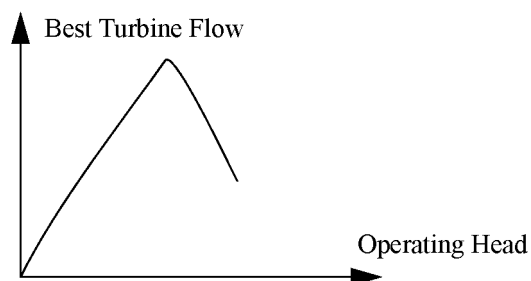
For a given Operating Head, create $n + 2$ rows of data, where n is the number of units comprising the Plant. The first row reflects no flow through the turbines. The next row reflects the preferred level of flow through the first unit normally activated. Subsequent rows reflect incremental flow levels as additional turbines come on line at their respective preferred levels of flow. The final row reflects the flow of all available units running at maximum flow capacity.

Information: Data must be entered into the table in increasing, concave blocks of the same Operating Head for the 3-D table interpolation to work correctly. For every block of the same Operating Head in column 1, Turbine Release and the corresponding Power should be listed in increasing, concave order in columns 2 and 3.

There must be a point of zero Turbine Release and zero Power for each Operating Head. The following table is an example of the proper way to formulate the Plant Power Table. This table can be derived from the Best Turbine Q, Best Power Coefficient, Max Turbine Q and Max Power Coefficient slots.

Operating Head	Turbine Release	Power
30	0	0
30	100	100
30	200	175
40	0	0
40	100	125
40	220	195
50	0	0
50	110	147
50	250	205

Best Turbine Flow – Operating Head relationship. Not drawn to scale.



Defined by: User input

■ Plant Efficiency Curve

The Plant Efficiency Curve method approximates Best Power Flow the same as the Plant Power Coefficient method.

■ Peak Power

In RPL Optimization, this method creates the following turbine release constraint. No actual power value is calculated.

$$\text{Turbine Release} \leq \text{Power Plant Capacity Fraction} * \text{Turbine Capacity} * \text{Number of Units.} \quad (5.47)$$

■ Peak and Base

In RPL Optimization, this method creates the following turbine release constraint. No actual power value is calculated.

$$\text{Turbine Release} \leq \text{Power Plant Capacity Fraction} * \text{Turbine Capacity} * \text{Number of Units.} \quad (5.48)$$

■ Unit Power Table

See “Unit Power” on page 301 for the Optimization formulation.

See “Unit Power Table” in *Objects and Methods* for the Simulation algorithm.

Slots Specific to This Method

See “Unit Power Table” in *Objects and Methods* for slot descriptions.

Method Details

When the Unit Power Table method is selected, the following method categories are available:

- [Avoidance Zones](#), page 207
- [Cavitation](#), page 208
- [,](#) page 210
- [Head Loss](#), page 212
- [Startup](#), page 241

The status of a unit at a given timestep can be one of the following:

- Available
- Unavailable
- Must run

At the highest level, an entire plant may never be able to provide an ancillary service like Regulation. The user can indicate this by selecting None in the appropriate category. On the other hand, the following slot settings allow users to specify unit availability / must run for individual time steps for generation and ancillary services. Without any inputs, the units default to being available.

If the Unit Power Table method is selected, but not Frequency Regulation, the user can specify through optimization RPL policy.

Note: Setting certain slots may be possible but not preferred.

- Unit u must generate at time t:
 - Unit Is Generating [t,u] = 1, or
 - Unit Energy [t,u] = value
- Unit u is unavailable at time t:
 - Unit Is Generating [t,u] = 0, or

Chapter 5

Optimization Objects and Methods

- Unit Energy $[t,u] = 0$

If the Frequency Regulation method is selected, the user could specify through optimization policy (RPL):

- Unit u must regulate up at time t :
 - Regulation Up $[t,u] = \text{value}$
- Unit u must regulate down at time t :
 - Regulation Down $[t,u] = \text{value}$
- Unit u must regulate at time t :
 - Regulation $[t,u] = \text{value}$ (which implies Regulation Up $[t,u] = \text{value}$ and Regulation Down $[t,u] = \text{value}$)

If a unit is unavailable for some type of regulation, then a value can be 0.

■ Turbine Capacity

Turbine Capacity is numerically approximated before being passed to the optimization solver. The slot is approximated as a function of Operating Head (Numerical 2-D Approximation).

The relationship between Operating Head and Turbine Capacity depends on the method selected in the Power category, as follows.

- If Plant Efficiency Curve is selected, the relationship is automatically developed from the Plant Power Table.
- If Peak Power or Peak and Base is selected, the relationship comes from the Best Generator Flow table.
- If none of these methods is chosen, the relationship is contained in the user-input Max Turbine Q slot.

Slots Specific to This Method

• Turbine Capacity

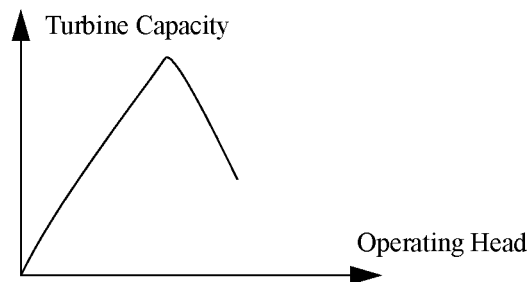
Type:	Agg Series Slot
Units:	Flow
Description:	Flow capacity of the turbines
Information:	
Defined by:	Numerical 2-D Approximation in terms of Operating Head, based upon a maximum turbine capacity table. This capacity table is determined in various ways according to the Power method: The Turbine Capacity LP Param table values are used as approximation points, indexing the selected maximum turbine capacity table. The maximum turbine capacity table is required to have increasing values of Operating Head. Turbine Capacity in that table is required to be a concave function of Operating Head. The preferred order of approximation is substitution, piecewise, tangent, two-point line.

• Turbine Capacity LP Param

Type:	Table Slot
Units:	Length vs Length vs Length
Description:	Specifies the operational head points used to take the tangent, line and piecewise approximations for Turbine Capacity linearization

Information: For the piecewise approximation the best operating head points to chose are generally: minimum Operating Head, Operating Head at max capacity, and maximum Operating Head. These three points typically define most of the shape of the Turbine Capacity curve.

Defined by: The following figure illustrates the relationship of User input, unless the selected method in the Power Linearization Automation category is not None.
Turbine Capacity Operating Head relationship. Not drawn to scale.



Slope Storage

This category is used to specify how the wedge is to be modeled.

See “[Slope Storage](#)” in *Objects and Methods* for details.

Slope Storage Coefficients

This category is used to specify whether to use Weighting Coefficients or Impulse Response coefficients to model the wedge. Both methods are supported in RPL optimization.

See “[Slope Storage Coefficients](#)” in *Objects and Methods* for details.

Startup

This category depends on selecting the Unit Power Table method (see “[Unit Power Table](#)” on page 239) and describes how the monetary cost associated with starting up or shutting down a unit (turbine) will be modeled. There are two methods in this category, one which does not model these costs (effectively assigning them a value of 0) and one which uses a table describing the combined costs for starting up and shutting down a unit.

■ None

This is the default, no action method.

■ Unit Lumped Cost

For each unit, this method lumps the cost of startup and shutdown into one value. See [“Unit-level Constraints”](#) on page 309 for descriptions of added constraints.

Slots Specific to This Method

See [“Unit Lumped Cost Method”](#) in *Objects and Methods* for slot descriptions.

Modeling a Slope Power Reservoir

Steps to follow in setting up a level power reservoir for optimization

1. Set up a running simulation model, using methods that are compatible with optimization for Power, Tailwater, Spill, and so on.
2. Choose a method in the Slope Storage Coefficients category.
3. Fill in the Backwater Lambda Coefficients, LP Parameters, and Restrictions table.
4. Chose an Optimization Power method.
5. Choose an Optimization Head Computation method. In Optimization, None must be chosen.
6. Select the Optimization Spill method corresponding to the method selected in the Spill category.
7. Fill in the LP Param tables related to Power, Turbine Capacity and Best Turbine Flow (if plant power is used). Alternatively, the Power Linearization Automation method can also be chosen to automatically determine the values for the LP Param tables.
8. Select the Optimization Tailwater method corresponding to the method selected in Tailwater category. If Opt Base Value Plus Lookup Table or Opt Stage Flow Lookup Table is selected, fill in the Tailwater Elevation LP Param table. Alternatively, select a Tailwater Linearization Automation method (Range Input Automation is currently available).
9. If future value is needed, select a method in the Future Value category, followed by the Optimization Future Value category, and if desired, Cumul Stor Val Linearization Automation.
10. Selection of the remaining methods and linearizations can be done in any order. Pool elevation linearizations, Pool Elevation Linearization Automation, Evaporation and Precipitation, Optimize Evaporation Computation, Evaporation Linearization Automations, Bank Storage, Hydrologic Inflow, Energy In Storage, and Diversion from Reservoir. The appropriate data must be entered for each method.

Storage Reservoir

The Storage Reservoir object is the least complicated of all the reservoirs. The only process performed by the Storage Reservoir is the storage of water. No power production facilities exist on the Storage Reservoir.

General Slots

• Canal Flow

Type: Agg Series
Units: Flow
Description: Flow into (out of) the reservoir from (to) a canal
Information:
Defined by: Explicit Optimization variable in the mass balance constraint

• Diversion

Type: Series Slot
Units: Flow
Description: Flow from the reservoir to a diverting object
Information:
Defined by: Explicit Optimization variable in the mass balance constraint

• Elevation Volume Table

Type: Table
Units: Length vs Volume
Description: Table relating elevation of the reservoir to volume stored in the reservoir
Information:
I/O: Input only
Defined by: Input only

• Flow FROM Pumped Storage

Type: Agg Series Slot
Units: Flow
Description: Flow into the reservoir from a pumped storage reservoir
Information:
Defined by: Explicit Optimization variable in the mass balance constraint. This slot should be linked to Outflow on a Pumped Storage object. The Pumped Storage object constrains its Outflow.

• Flow TO Pumped Storage

Type: Agg Series Slot
Units: Flow
Description: Flow out of the reservoir into a pumped storage reservoir
Information:
Defined by: Explicit Optimization variable in the mass balance constraint. This slot should be linked to Pumped Flow on a Pumped Storage object.

- **Inflow**

Type: MultiSlot
Units: Flow
Description: Inflow into the reservoir from upstream
Information:
Defined by: Explicit Optimization variable in the mass balance constraint

- **Max Release**

Type: Table Slot
Units: Length vs Flow
Description: Table relating Pool Elevation to maximum release
Information: See power methods for details

- **Outflow**

Type: Agg Series Slot
Units: Flow
Description: Outflow from the reservoir
Information:
Defined by: Explicit Optimization variable as $\text{Outflow} = \text{Turbine Release} + \text{Spill}$

- **Pool Elevation**

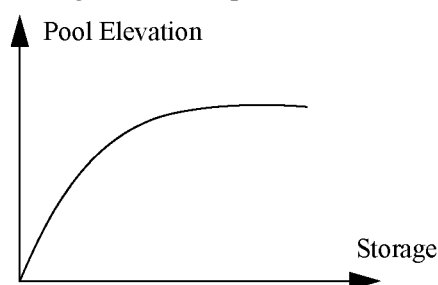
Type: Agg Series Slot
Units: Length
Description: Elevation of the water surface of the Reservoir
Information: When Pool Elevation is a part of the optimization problem, as it is in all conceivable RiverWare Optimization applications, this slot is numerically approximated as a function of Storage (Numerical 2-D Approximation). The relationship between Pool Elevation and Storage will come from the user-input Elevation Volume Table. The table will be queried either using user-input points defined in the Pool Elevation LP Param table.
Defined by: Numerical 2-D Approximation in terms of Storage, based upon the Elevation Volume Table. The Pool Elevation LP Param table values are used as approximation points indexing the Elevation Volume Table. The Elevation Volume Table should have increasing values of Pool Elevation and Storage. Storage is required to be a concave function of Pool Elevation. The preferred order of approximation is substitution, piece-wise, tangent, two-point line.

- **Pool Elevation LP Param**

Type: Table Slot
Units: Volume
Description: Specifies the Storage points used to take the tangent, line and piecewise approximations for Pool Elevation linearization
Information: This table is used for linearization unless Pool Elevation Linearization Automation category has selected Plant Automation. The best Storage point to choose for tangent approximation would be the expected storage expected during the run; for

the line approximation, the expected maximum and minimum Storage; for piecewise approximation, use points that cover the full range of expected Storage during the run with intermediate points such that a piecewise linear curve reasonably approximates the actual curve.

Defined by: User input
Pool Elevation Storage relationship. Not drawn to scale.



• Return Flow

Type: MultiSlot
Units: Flow
Description: Flow returning from a diversion object
Information:
Defined by: Explicit Optimization variable in the mass balance constraint (see Storage)

• Spill

Type: Agg Series Slot
Units: Flow
Description: Sum of the Bypass, Regulated Spill and Unregulated Spill
Information:
Defined by: Explicit Optimization variable as $\text{Spill} = \text{Bypass} + \text{Regulated Spill} + \text{Unregulated Spill}$

• Storage

Type: Agg Series Slot
Units: Volume
Description: Volume of water stored in the reservoir
Information:
Defined by: Explicit Optimization variable as:
$$\text{Storage} = \text{Storage}(t-1) + \text{Precipitation Volume} - \text{Evaporation} - \text{Change in Bank Storage} + \text{timestep} * (\text{Inflow} + \text{Canal Flow} + \text{Flow TO Pumped Storage} + \text{Hydrologic Inflow Net} + \text{Return Flow} - (\text{Outflow} + \text{Diversion} + \text{Flow FROM Pumped Storage}))$$

User Methods

The following categories and methods are available for use in the Storage Reservoir object in Optimization.

Because of dependency relationships, you may not be able to see them in your model until you change other methods. In the discussion below, you will see the other methods that need to be selected to enable a given method to be used in your model. When building a model, you need to review this to ensure that required dependencies are satisfied so that the desired methods are available for use.

User method categories in Optimization are as follows.

- [Bank Storage](#), page 246
- [Diversion From Reservoir](#), page 247
- [Energy in Storage](#), page 247
- [Evaporation and Precipitation](#), page 248
- [Future Value](#), page 249
- [Hydrologic Inflow](#), page 250
- [Optimization Evaporation](#), page 250
- [Optimization Future Value](#), page 253
- [Optimization Spill](#), page 254
- [Load](#), page 258
- [Modified Load](#), page 258
- [Preferred Customer](#), page 259
- [Regulation](#), page 259
- [Block Economic Value](#), page 261
- [Linear Economic Value](#), page 262
- [Thermal Unit Replacement Value](#), page 263

Bank Storage

Not all methods are functional in RPL optimization. Of the available methods, None and CRSS Bank Storage are supported.

None

See “None” in *Objects and Methods* for details.

CRSS Bank Storage

See “CRSS Bank Storage” in *Objects and Methods* for details.

Diversion From Reservoir

Not all methods in this category are supported in RPL optimization. Of the available methods, None and Available Flow Based Diversion are supported; selecting any other method results in an error at the start of run.

■ None

See [“None”](#) for details.

■ Available Flow Based Diversion

See [“Available Flow Based Diversion”](#) for details.

Energy in Storage

In Optimization, currently None and EIS Table Lookup are supported.

■ None

No Energy in storage is considered.

■ EIS Table Lookup

With this method selected, Energy in Storage is considered as a function of Pool Elevation.

If the optimization problem uses Energy In Storage, either directly or indirectly, then this slot is added to the problem. Before being passed to the optimization solver, this slot is numerically approximated as a function of Pool Elevation (Numerical 2-D Approximation).

The relationship between Pool Elevation and Energy In Storage will come from the user-input Energy In Storage Table. The table will be queried, either using user-input points defined in the Energy In Storage LP Param table, or using automatically calculated points, depending on method selection in the Pool Elevation Linearization Automation category.

- If the selected method in the Pool Elevation Linearization Automation is None, the user input Energy In Storage LP Param table values are used.
- For other Pool Elevation Linearization Automation methods (Initial Target Input, Min Difference or Range Input, Min Difference), the same points automatically developed for the Pool Elevation linearization are used.

See [“EIS Table Lookup”](#) in *Objects and Methods* for details on the Simulation method.

Slots Specific to This Method

• Energy In Storage

Type: Series Slot
Units: Energy vs Power

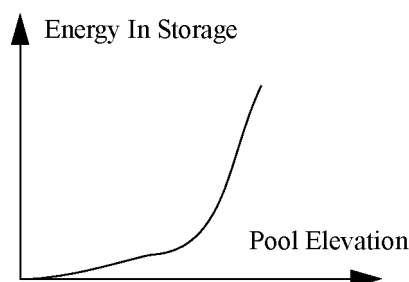
Description: Energy in Storage in the reservoir
Information:
Defined by: Numerical 2-D Approximation in terms of Pool Elevation, based upon the Energy In Storage Table. The Energy In Storage LP Param table values are used as approximation points indexing the Energy In Storage Table. The Energy In Storage Table should have increasing values of Pool Elevation and Energy In Storage. Energy In Storage is required to be a convex function of Pool Elevation. The preferred order of approximation is substitution, piece-wise, tangent, two-point line.

• Energy in Storage LP Param

Type: Table Slot
Units: Length
Description: Specifies the Pool Elevation points used to take the tangent, line and piecewise approximations for Energy In Storage linearization.
Information: This table is used for linearization unless the Pool Elevation Linearization Automation category has a method selected other than None.
Defined by: User input

• Energy in Storage Table

Type: Table Slot
Units: Length vs Energy
Description: Table defining the relationship between Energy In Storage and Pool Elevation
Information:
Defined by: User input
 Energy in storage pool elevation relationship. Not drawn to scale.



Evaporation and Precipitation

The Input Evaporation method is the only Evaporation and Precipitation method that can be modeled in conjunction with Optimization.

If the Input Evaporation method is selected, the Opt Input Evaporation method must be selected for the Optimization Evaporation category. Otherwise, None should be selected for the Evaporation and Precipitation category, and None should be selected for the Optimization Evaporation category.

When Evaporation and Precipitation Volume exist on the reservoir, they are added as part of the mass balance constraint.

Future Value

This category and its methods are not dependent on other method selections.

■ None

See “None” in *Objects and Methods* for details.

■ Cumulative Storage Value Table

In RPL Optimization, the Cumulative Storage Value is Numerically Approximated as described below.

See “Cumulative Storage Value Table” in *Objects and Methods* for details.

Slots Specific to This Method

• Cumulative Storage Value

Type:	Agg Series Slot
Units:	\$
Description:	Represents the future energy value of the current storage
Information:	
Defined by:	2-D approximation in terms of Anticipated Storage, based upon the Cumul Stor Val Table. The Cumul Stor Val LP Param table values (Storage) are used as approximation points indexing the Cumul Stor Val Table. The Cumul Stor Val Table should have increasing values of Storage and Cumulative Value. Cumulative Storage Value is required to be a concave function of Anticipated Storage. The preferred order of approximation is substitution, piece-wise, tangent, two-point line. The Cumul Stor Val Linearization Automation category’s Marginal Value to Table and Lin method can automate creation of the Cumul Stor Val LP Param table and the Cumul Stor Val Table.

• Anticipated Storage

Type:	Agg Series Slot
Units:	Volume
Description:	The combination of the actual storage plus water that would be expected to enter the reservoir after the Current Timestep but has not yet, due to lagging.
Information:	
Defined by:	

• Cumul Stor Val Table

Type:	Table Slot
Units:	Volume vs Value

Description: The estimated total economic value of water stored in the reservoir for discrete storage values.

Information:

Defined by: User input or by automated procedure if Cumul Stor Val Linearization Automation category has selected the Marginal Value to Table and Lin method.

- **Marginal Storage Value Table**

Type: Table Slot

Units: Storage vs \$ Value

Description: Anticipated Storage versus Cumulative Storage Value per unit energy

Information: This table should be increasing in storage, and logically decreasing in marginal value

Defined by: Required input

- **Spill Cost**

Type: Agg Series Slot

Units: \$

Description:

Information:

Defined by:

Hydrologic Inflow

If any method in this category is selected, hydrologic inflow is included in the reservoir mass balance. Optimization assumes hydrologic inflow to be known (data) and it is not solved for by the reservoir regardless of the method selected.

See “[Hydrologic Inflow](#)” in *Objects and Methods* for details.

Optimization Evaporation

This category can be used to model evaporation and precipitation.

■ None

This is the default method for the Optimization Evaporation category. It performs no calculations and requires that None also be selected for the Evaporation and Precipitation category.

■ Opt Input Evaporation

This method is analogous to the Input Evaporation method in simulation and requires the Input Evaporation method to be selected for the Evaporation and Precipitation category. Evaporation Rate and Precipitation Rate are entered as a time series. Evaporation is calculated as a product of Evaporation Rate, Average Surface Area over the

timestep and Timestep length. Similarly Precipitation Volume is calculated as the product of Precipitation Rate, Average Surface Area and Timestep length.

Caution: The linearization of the Surface Area variable can result in a small approximation error in optimization for Evaporation and Precipitation Volume. This means there can be a small difference between the mass balances in the optimization solution and the post-optimization rulebased simulation when using this method. It is important to use care when setting the approximation points for Surface Area in order to reduce this approximation error. Refer to the information on the Surface Area LP Param slot below. Also caution should be used if applying this method at a 1 Month timestep. All rates in the optimization mass balance are converted to monthly volumes based on a 30-day month, regardless of the month. This will also produce a difference between the mass balances in the optimization solution and the post-optimization rulebased simulation for a 1 Month timestep.

Slots Specific to This Method

• Elevation Area Table

Type: Table Slot
Units: Length vs Area
Description: Represents the Elevation-Surface Area relationship
Information: This table must be input. It is used to derive the Volume Area Table.
Defined by: User input

• Evaporation

Type: Series Slot
Units: Volume
Description: The volume of water lost to evaporation over the timestep
Information: If this slot contains user input, it is added directly to the mass balance constraint, otherwise it is defined by the expression below.
Defined by: Either user input or the following constraint:

$$\text{Evaporation} = \text{EvaporationRate} \times \frac{\text{SurfaceArea}(t-1) + \text{SurfaceArea}}{2} \times \text{Timestep}$$

• Evaporation Rate

Type: Series Slot
Units: Velocity
Description: The rate, in length per time, at which water is lost to evaporation at each timestep
Information: This slot can be set as user input. If it is not set as an input, and if Evaporation is not an input, this slot defaults to zero. If Evaporation is an input, this slot is not used.
Defined by: User input or defaults to zero

• Precipitation Rate

Type: Series Slot
Units: Velocity
Description: The rate, in length per time, at which water is gained from precipitation at each timestep

Information: This slot can be set as user input. If it is not set as an input, it defaults to zero.
Defined by: User input or defaults to zero

• Precipitation Volume

Type: Series Slot
Units: Volume
Description: The volume of water gained from precipitation over the timestep
Information: The Input Evaporation method will not allow this slot to be set as an input.
Defined by: Explicit constraint:

$$\text{Precipitation Volume} = \text{PrecipitationRate} \times \frac{\text{SurfaceArea}(t-1) + \text{SurfaceArea}}{2} \times \text{Timestep}$$

• Surface Area

Type: Series Slot
Units: Area
Description: The area of the water surface at the end of the timestep
Information: This slot is numerically approximated as a function of Storage (Numerical 2-D Approximation). The relationship between Pool Elevation and Storage comes from the automatically generated Volume Area Table. The table is queried using the user-input points defined in the Surface Area LP Param table.
Defined by: Numerical 2-D Approximation in terms of Storage, based upon the Volume Area Table. The Surface Area LP Param table values are used as approximation points indexing the Volume Area Table. The preferred order of approximation is substitution, piecewise, two-point line, tangent. Most often the two-point line (secant) approximation will be used.

• Surface Area LP Param

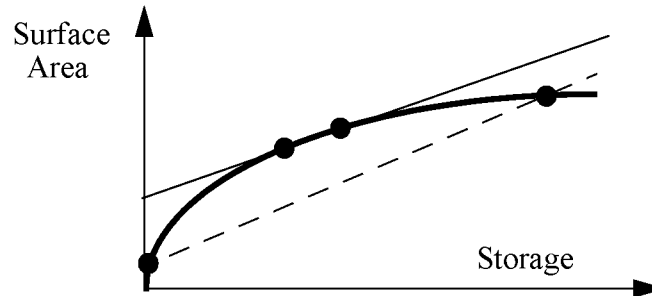
Type: Table Slot
Units: Volume
Description: Specifies the Storage points used to take the tangent, line and piecewise approximations for Surface Area linearization
Information: The best Storage point to choose for tangent approximation would be the expected storage during the run; for line approximation, the expected maximum and minimum Storage; for piecewise approximation, use points that cover the full range of expected Storage during the run with intermediate points such that a piecewise linear curve reasonably approximates the actual curve. In most cases, the line (secant) approximation will be used. It is important to set these points carefully to minimize approximation error.
Defined by: User input

• Volume Area Table

Type: Table Slot
Units: Volume vs Area
Description: Represents the Storage Volume-Surface Area relationship

Information: This table is read-only and is automatically generated at the start of the run from the Elevation Area Table and the Elevation Volume table. The method starts by copying the Elevation Area Table and then replaces the Pool Elevation Values with the corresponding Storage values. The Storage values are linearly interpolated based on the Elevation Volume Table.

Defined by: Automatically generated
Surface area vs storage relationship, with two alternative line approximations. Not drawn to scale.



The linear approximation represented by the dashed line corresponds to the selection of points near the extremes of the Volume Area table. This approximation tends to result in an under-estimation of Surface Area, and thus an under-estimation of Evaporation and Precipitation Volume. Evaporation losses in the post-optimization rulebased simulation would be greater than the losses approximated in the optimization solution.

The linear approximation represented by the solid line corresponds to the selection of points closer together in the Volume Area Table, and is more similar to the Tangent approximation. This approximation would tend to result in an over-estimation of Surface Area, and the losses due to Evaporation in the post-optimization rulebased simulation would be less than the approximation in the optimization solution.

Optimization Future Value

This category allows the optimization to provide slots relating to the future value of water. Its appearance is dependent on selecting the Cumulative Storage Value Table method for the Future Value category.

None

If this method is selected, the future value slots will not be visible, and no linearization will be attempted.

Opt Cumulative Storage Value Table

If this method is selected, the Cumul Stor Val LP Param slot is visible and linearization is allowed.

Slots Specific to This Method

- **Cumul Stor Val LP Param**

Type:	Table Slot
Units:	Volume, Volume, Volume
Description:	Specifies the storage points used to take the tangent, line and piecewise approximations for Cumul Stor Val Table linearization
Information:	
Defined by:	User input or by automated procedure if Cumul Stor Val Linearization Automation category has selected the Marginal Value to Table and Lin method.

Optimization Spill

The Optimization Spill methods determine how spill is calculated for the reservoir and generates physical constraints that correspond to the selected methods.

This category is dependent on selection of the Independent Linearizations method for the Optimization Power category. However, the method selected in the Optimization Spill category must match with the corresponding non-optimization method in the Spill category.

Spill is an Optimization decision variable. The following constraint is always generated for a reservoir:

$$\text{Outflow} = \text{Turbine release (or Release)} + \text{Spill} \quad (5.49)$$

The spill methods generate values applicable to the following additional constraint, where some of these terms may be omitted if they do not apply to the selected spill method.

$$\text{Spill} = \text{Regulated Spill} + \text{Unregulated Spill} + \text{Bypass}, \quad (5.50)$$

Depending on the method selected, some of the following slots will be added. Other slots will be used from the non-optimization method selected.

As applicable, the following constraints are also added.

$$\text{Bypass} \leq \text{Bypass Capacity} \quad (5.51)$$

$$\text{Regulated Spill} \leq \text{Regulated Spill Capacity} \quad (5.52)$$

$$\text{Unregulated Spill} = \text{Unregulated Spill Capacity} \quad (5.53)$$

See “[Spill](#)” in *Objects and Methods* for details on non-optimization Spill methods.

Slots Specific to This Method

- **Bypass Capacity**

Type:	Series Slot
--------------	-------------

Units: Flow
Description: Bypass capacity
Information:
Defined by: Numerical 2-D Approximation in terms of storage, based upon the Bypass Capacity Table.

- **Bypass Capacity Table**

Type: Table Slot
Units: Volume vs Flow
Description: Storage vs corresponding maximum bypass spill values
Information:
Defined by: Internally developed based on Bypass Table and Elevation Volume Table relationships. For each pool elevation in the Bypass Table, the Bypass Capacity Table has a row relating Storage to Bypass Capacity. The Pool Elevations are converted to Storage using the Elevation Volume Table.

- **Regulated Spill Capacity**

Type: Series Slot
Units: Flow
Description: Regulated spill capacity
Information:
Defined by: Numerical 2-D Approximation in terms of storage, based upon the Regulated Spill Capacity Table.

- **Regulated Spill Capacity Table**

Type: Table Slot
Units: Volume vs Flow
Description: Storage vs corresponding maximum regulated spill values
Information:
Defined by: Internally developed based on Regulated Spill Table and Elevation Volume Table relationships. For each pool elevation in the Regulated Spill Table, the Regulated Spill Capacity Table has a row relating Storage to Regulated Spill Capacity. The Pool Elevations are converted to Storage using the Elevation Volume Table.

- **Regulated Spill or Bypass LP Param**

Type: Table Slot
Units: Volume
Description: Specifies the Storage points use to take the tangent, line and piecewise approximations for Regulated Spill Capacity linearization and Bypass Spill Capacity linearization.
Information:
Defined by: User input

- **Unregulated Spill Linearization Table**

Type:	Table Slot
Units:	Volume vs Flow
Description:	Storage vs corresponding unregulated spill values
Information:	
Defined by:	Internally developed based on Unregulated Spill Table and Elevation Volume Table relationships. For each pool elevation in the Unregulated Spill Table, the Unregulated Spill Capacity Table has a row relating Storage to Unregulated Spill Capacity. The Pool Elevations are converted to Storage using the Elevation Volume Table.

- **Unregulated Spill LP Param**

Type:	Table Slot
Units:	Volume
Description:	Specifies the Storage points use to take the tangent, line and piecewise approximations for Unregulated Spill Linearization Table linearization
Information:	
Defined by:	User input

■ None

If this method is selected the slot bounds in the slot configuration dialog are set to zero. No additional constraints are generated.

■ Opt Monthly Spill

Note: This method is not functional in RPL Optimization.

This method sets the lower and upper bounds on spill. The lower bound is set to zero and the default upper bound is set to a very big number (9,999,999 cms). The default upper bound can be revised in the Spill slot configuration, Max Value parameter. No additional constraints are generated beyond these bounds.

■ Opt Unregulated

If this method is selected only unregulated spill is considered and the following constraint is added to the LP:

$$\text{Spill} = \text{Unregulated Spill} \quad (5.54)$$

■ Opt Regulated

When this method is selected only regulated spill is considered. The lower bound on spill is set to zero and the following constraint is added to the LP:

$$\text{Spill} = \text{Regulated Spill} \quad (5.55)$$

■ Opt Regulated and Unregulated

When this method is selected unregulated and regulated spill are considered and the following constraint is added to the LP:

$$\text{Spill} = \text{Regulated Spill} + \text{Unregulated Spill} \quad (5.56)$$

■ Opt Regulated and Bypass

When this method is selected only regulated and bypass spill are considered and the lower bound on spill is set to zero and the following constraint is added to the LP:

$$\text{Spill} = \text{Regulated Spill} + \text{Bypass} \quad (5.57)$$

■ Opt Regulated, Bypass and Unregulated

When this method is selected unregulated, regulated and bypass spill are considered and the following constraint is added to the LP:

$$\text{Spill} = \text{Regulated Spill} + \text{Unregulated Spill} + \text{Bypass} \quad (5.58)$$

■ Opt Bypass, Regulated and Unregulated

When this method is selected unregulated, regulated and bypass spill are considered and the following constraint is added to the LP:

$$\text{Spill} = \text{Bypass} + \text{Regulated Spill} + \text{Unregulated Spill} \quad (5.59)$$

■ Release Capacity Numerical Approximation

When Release Capacity is part of the Optimization problem, this slot is numerically approximated as a function of Pool Elevation (Numerical 2-D Approximation). The relationship between Pool Elevation and Release Capacity comes from the user-input Max Release Table. The table is queried using user-input points defined in the Release LP Param table.

Modeling a Storage Reservoir

Use these steps to set up a Storage reservoir for Optimization.

1. Set up a running simulation model, using methods that are compatible with optimization for Spill.
2. Select the Optimization Spill method corresponding to the method selected in the Spill category.
3. Fill in the LP Param table related to Release.
4. If future value is needed, select a method in the Future Value category, followed by the Optimization Future Value category, and if desired, Cumul Stor Val Linearization Automation.
5. Selection of the remaining methods and linearizations can be done in any order. Pool elevation linearizations, Pool Elevation Linearization Automation, Evaporation and Precipitation, Optimize Evaporation Computation,

Evaporation Linearization Automations, Bank Storage, Hydrologic Inflow, Energy In Storage, and Diversion from Reservoir. The appropriate data must be entered for each method.

Thermal

The Thermal object represents the economics of power generation. In optimization, the thermal object can influence the solution. In addition to allowing a modeler to maximize the economic value of hydropower, the thermal object allows a modeler to write constraints and objectives on the power-related multislots. For example, total system hydropower generation or capacity can be maximized or constrained to meet a minimum level. See “General Slots” in *Objects and Methods* for thermal object documentation.

User Methods

The following categories and methods are available for use in the Thermal object in Optimization.

- [Load](#), page 258
- [Modified Load](#), page 258
- [Preferred Customer](#), page 259
- [Regulation](#), page 259
- [Block Economic Value](#), page 261
- [Linear Economic Value](#), page 262
- [Thermal Unit Replacement Value](#), page 263

Load

These methods determine if hourly power load data should be part of the economic valuation of hydropower. See “Load” in *Objects and Methods* for details.

Modified Load

This category is dependent on selecting the Hourly Load method from the Load category.

■ None

This is the default, no action method.

■ Calculate Modified Load

The Modified Load is defined by the following constraint, when all of these power sources are selected.

$$\begin{aligned} \text{Load Energy} \leq & \text{Modified Load} & (5.60) \\ & + \text{Hydro Generation} \\ & - \text{Preferred Customer Energy} \\ & + \text{Pumped Storage Generation} \\ & - \text{Pumped Storage Pumping} \\ & + \text{Allocated Detail Energy} \end{aligned}$$

If any of the power sources are NaN in the Modified Load Power Sources Used table, the terms for these power sources are omitted from the constraint.

In the policy, the following types of sample statements can be added.

```
REPEATED MAXIMIN
  FOREACH (DATETIME date IN @“Start Timestep” TO @ “Finish Timestep”
    ADD CONSTRAINT Thermal.Modified Load[date] <= 0.0
  END FOREACH
END MAXIMIN
```

This constraint minimizes the largest values of modified load.

See [“Calculate Modified Load” in *Objects and Methods*](#) for details on the Simulation method.

Slots Specific to This Method

See [“Calculate Modified Load” in *Objects and Methods*](#) for slot descriptions.

Preferred Customer

Note: Although still shown in the interface, this method no longer does anything in the RPL-based optimization.

A group of reservoirs may be obligated to meet the power demands of preferred customers before this energy is used for other purposes. For example, if a group of reservoirs is owned by another entity, the energy demand of their customers must be met before the energy is coordinated with the other reservoirs in an economic objective, and in this sense, their customers are *preferred*. In such a situation, some— perhaps all—of these reservoirs are also treated as *allocated energy* that can be flexibly used in coordination with the remaining reservoirs.

The methods in this category model the energy needs of the preferred customers. The preferred customers are identified by membership in a “Preferred Customer” subbasin. This is a predefined type of subbasin.

See [“Preferred Customer” in *Objects and Methods*](#) for the methods in this category.

Regulation

Regulation is one of the ancillary services that hydropower plants can supply in addition to power to increase the reliability and flexibility of the power system to adjust to fluctuations in power demand and supply. When a plant is regulating it will follow the load within some prescribed band of power rather than generating a fixed amount of power. Typically, regulation is a valuable service that can be provided efficiently by hydropower compared to alternative power sources. In locations where regulation is marketed it usually commands a solid premium above

the price of the power generated. This value is partially reduced by the increased maintenance costs associated with regulation.

See “[Regulation](#)” in *Objects and Methods* for the methods in this category.

Economic Valuation User Methods

These methods model the economic valuation of hydropower during an optimization run using the Optimization controller. The category doesn’t exist in the Simulation and Rulebased Simulation controllers. The values are based either explicitly or implicitly on the replacement of thermal power sources and the associated cost savings. (The descriptions of possible optimization policy in this section are not intended to exactly show the syntax of the policy statements. This is in part because the policy syntax differs under the old Optimization controller and the new Optimization controllers.)

The methods in this category exactly parallel the methods in the Thermal Replacement Value category for simulation, with the exception that no attempt is made to attribute value to particular energy sources. Thus, simulation methods that are identical except for the order of evaluation map to a single optimization method. This optimization method is dependent on one of the appropriate simulation methods being selected.

All of the methods use the slot Avoided Operating Cost to represent the explicit or implicit savings to the thermal system from hydropower generation. The methods differ in how they compute Avoided Operating Cost. It is assumed that the user will write an objective function that either directly or indirectly maximizes Avoided Operating Cost.

In this description, anywhere it says Net Avoided Cost or Avoided Operating Cost, it should have either Thermal, Block, or Linear added to the beginning; that is, Linear Net Avoided Cost, depending on method selections.

Following are possible objectives:

- [Objective 1: Maximize Net Avoided Cost](#)
- [Objective 2: Maximize Avoided Operating Cost + Total Cumulative Storage Value \(Final timestep\)](#)
- [Objective 3: Maximize Avoided Operating Cost + Total Cumulative Storage Value \(Final timestep\) – Total Cumulative Storage Value \(Initial timestep\)](#)

Objective 1: Maximize Net Avoided Cost

Net Avoided Cost is defined as the Avoided Operating Cost minus the value of energy used for generation and energy lost to spill:

$$NetAvoidedCost = AvoidedOperatingCost - FutureValueOfUsedEnergy - SpillCost$$

Future Value of Used Energy and Spill Cost are multislots that are assumed to be linked to the individual values on power reservoirs. The individual reservoir calculations use a power coefficient and a dollar value to translate releases into these costs.

An alternative to penalizing the Future Value of Used Energy and Spill Cost is to maximize the Total Cumulative Storage Value, a multislot with the expected value of all water remaining in storage. This multislot is expected to be linked to the Cumulative Storage Value on the individual reservoirs, where it is a piecewise linear function of storage.

Objective 2:

Maximize Avoided Operating Cost + Total Cumulative Storage Value (Final timestep)

Optionally, the objective can subtract the following term, Total Cumulative Storage Value (Initial timestep).

Objective 3:

Maximize Avoided Operating Cost + Total Cumulative Storage Value (Final timestep) – Total Cumulative Storage Value (Initial timestep)

Subtracting this constant term does not effect the optimal solution, but the reported objective function value will now reflect the *change* in total Cumulative Storage Value, typically a more meaningful number.

In some cases a utility may be allocating additional energy from outside the basin. If data is specified for the slots described below, the optimization will automatically create variables for each column of the Allocated Detail Energy Slot. These variables are constrained to minimum and maximum values:

$$AllocatedMinimum(t) \leq AllocatedDetailEnergy(t) \leq AllocatedMaximum(t)$$

The Allocated Detail Energy is further constrained to meet periodic totals where the period is a multiple of the timesteps of the run. For example, a model with a 6 Hour timestep, might have a total daily energy target. More generally, the period is specified by the timesteps with non-zero values for Allocated Total Energy. The Allocated Detail Energy is required to meet the totals. For a daily period the constraint is:

$$\sum_{t \in day} AllocatedDetailEnergy(t) = AllocatedTotalEnergy(day) \quad \forall days$$

The user can further constrain allocated energy by selecting the Preferred Customer method from the Preferred Customer category.

Block Economic Value

None

No evaluation is made.

Calculate Block Economic Value

The economic value of hydropower is set equal to (or replaced by) a piecewise linear function of hydropower generated. The piecewise function can vary by time period. While there is no direct linkage to thermal sources of power, one interpretation of the piecewise linear function is that it represents the coordinated reduction in other power sources as a result of hydropower generation. This method does not attempt to meet an explicit load, rather the load is assumed to be included in the process that generates the piecewise linear functions.

The piecewise linear functions are represented by two slots: maximum of Hydro Block Use slot (same for all t) and Hydro Block Costs. The Hydro Block Costs are the slope of the piecewise linear segments, or alternatively the value of a unit of power. The piecewise linear function is assumed to be concave: the costs are assumed to decrease as the block number increases.

Chapter 5

Optimization Objects and Methods

The method name indicates which hydropower resources should be evaluated by the piecewise linear function. Any power sources that precede the “Block” keyword are assumed to already be included in the piecewise linear function and are not evaluated. The order of power sources after the “Block” keyword doesn’t matter for optimization. In this case, the power sources included are as follows:

- Allocated energy
- Conventional hydropower
- Pumped storage units (both pumping and generation)
- Unevaluated: None.

A constraint is added to limit the Hydro Block Use credits to the net generation from all sources minus the Preferred Customer Energy demands that must first be met by the generation.

$$\sum_{blocks} HydroBlockUse(block,t) \leq HydroGeneration(t) + AllocatedDetailEnergy(t) + PumpStorageGeneration(t) - PumpStoragePumping(t) - PreferredCustomerEnergy(t) \quad \forall t \quad (5.61)$$

The constraint is written as an inequality rather than an equality to allow for piecewise linearization of power. This constraint is triggered by writing an objective function that includes maximizing Avoided Operating Cost. In addition to this constraint, Avoided Operating Cost is set equal to (or replaced by) the piecewise linear value of power:

$$BlockAvoidedOperatingCost = \sum_{blocks} HydroBlockCosts(block,t) \times HydroBlockUse(block,t) \quad (5.62)$$

Slots Specific to This Method

See “Calculate Block Economic Value” in *Objects and Methods* for slot descriptions.

Linear Economic Value

None

No evaluation is made.

Calculate Linear Economic Value

The economic value of hydropower is set equal to (or replaced by) a linear function of hydropower generated. The function can vary by time period. This method, in essence, models a block valuation but with only one block.

$$LinearAvoidedOperatingCost \leq LinearHydroCosts(t) \times (HydroGeneration(t) + AllocatedDetailEnergy(t) + PumpStorageGeneration(t) - PumpStoragePumping(t) - PreferredCustomerEnergy(t)) \quad \forall t \quad (5.63)$$

The constraint is written as an inequality rather than an equality to allow for piecewise linearization of power. This constraint is triggered by writing an objective function that includes maximizing Linear Avoided Operating Cost.

Slots Specific to This Method

See “[Calculate Linear Economic Value](#)” in *Objects and Methods* for slot descriptions.

Thermal Unit Replacement Value

None

No evaluation is made.

Calculate Thermal Unit Replacement Value

The economic value of hydropower is set equal to (or replaced by) the savings from replacing generation from thermal power sources.

A constraint is added to meet the load. This load must be met (or exceeded) by a combination of the following power sources: thermal energy, power reservoir energy, allocated energy (from outside the basin), and net energy from pumped storage facilities (generation - pumping).

$$\sum_{units} ThermalEnergy(unit,t) + HydroGeneration(t) + AllocatedDetailEnergy(t) + PumpStorageGeneration(t) - PumpStoragePumping(t) \geq$$

$$LoadEnergy(t) + PreferredCustomerEnergy(t) \quad \forall t$$

The constraint is written as an inequality, rather than an equality, to allow for piecewise linearization of power. The assumption is that an objective function will be written that will tend to maximize the energy generated for a given amount of turbine release. For example, an objective function minimizing the cost of thermal generation would be sufficient.

Additional terms are available for writing more complex objective functions. In this method, Avoided Operating Cost is defined as the reduction in the cost of thermal generation, the thermal costs of meeting the load with only thermal sources minus the cost of a combined hydropower and thermal solution.

$$ThermalAvoidedOperatingCost(t) = ThermalOnlyCost(t) -$$

$$\sum ThermalUnitCost(unit,t) \times ThermalEnergy(unit,t) \quad \forall t$$

Thermal Energy is further constrained by unit capacity and availability:

$$0 \leq ThermalEnergy(unit,t) \leq ThermalUnitAvailabilities(unit,t) \times ThermalUnitCapacities(unit,t) \quad \forall units, t$$

Slots Specific to This Method

See “[Calculate Thermal Unit Replacement Value](#)” in *Objects and Methods* for slot descriptions.

Modeling a Thermal Object

Use these steps to convert a model created prior to RiverWare 5.1 to RiverWare 5.1.

1. Extract model information using an old executable.
 - a. Load the model.
 - b. Deactivate any policy referencing the Thermal Object.
 - c. Save the Opt Goal set: we will load it later.
 - d. Record the selected thermal method: we will later use the power source ordering.
 - e. Record Avoided Operating Cost upper and lower bounds.
 - f. Record the Block End Tolerance: slot is replicating and changing to scalar slot.
 - g. Record the reporting method: we will reinstate it.
2. Put information into model using 5.1 executable.
 - a. Change the thermal methods as desired for each Linear, Block, or Thermal category.
 1. Set the power source ordering for each selected method.

For example: Nan for sources that used to come before Block or Therm in the old method name; 1,2, and so on, for sources after Block or Therm.
 2. Avoided Operating Cost for each selected method; for example, Linear Avoided Operating Cost.

View Configuration

Lower Bound = 0 (or available from earlier executable unless this is a new method)

Upper Bound = 1e7 (for example) (or available from earlier executable unless this is a new method)
 3. Import costs if they weren't transferred from the previous model for some reason.
 4. Set Hydro Block Use bounds for block method if missing.
 5. Set (Block or Thermal) End Tolerance.
 - b. Set the reporting methods for simulation as desired (optional).

One reporting category for each selected power economic evaluation method.

Typically, set the method in this category to report the same thing used in an optimization objective function.
 - c. Load the goal set and replace any thermal slot references to old slots with the new method-specific versions of the slots.
3. The Total Hydro Capacity slot on the Thermal object should have values by the end of simulation either by linking to Hydro Capacity on power reservoirs (plant power methods) or by setting the slot to input (Unit Power Table method). The economic value of power generation can be limited by Total Hydro Capacity. In particular, negative or invalid values for Total Hydro Capacity result are treated as zero capacity.

Chapter 5

Optimization Objects and Methods

Chapter 6

Setting Up and Running an Optimization Model

This section provides a broad outline of the process of creating a RiverWare model designed for use with the Optimization controller.

Topics

- [Select Timestep and Run Range, page 267](#)
- [Set Up and Test a Simulation Model, page 268](#)
- [Select Optimization Methods, page 268](#)
- [Add a Thermal Object, page 270](#)
- [Enter Lower and Upper Bounds, page 271](#)
- [Provide Approximation Points, page 275](#)
- [Create an Optimization Goal Set, page 279](#)
- [Create a Post-optimization Ruleset, page 280](#)
- [Run Parameters, page 282](#)
- [Run the Model, page 282](#)

Select Timestep and Run Range

The timestep and run range for Optimization are set in the Run Control dialog, just as for Simulation models. The same timesteps are available for Optimization as for Simulation; however due to the nature of Optimization, some additional issues should be considered when setting the timestep and run range for Optimization.

During an optimization run a linear program (LP) is created which represents the physical and policy constraints for the system. The time required to solve the LP is related to the size of the model, including the number of objects, the number of timesteps in the run, and the number of constraints. Thus to permit a simulation that runs in an acceptable time, one might need to consider increasing the timestep length and/or shortening the run duration. It is important to realize that the run time for Optimization does not increase linearly with the number of timesteps. The run time tends to increase with something between the square and the cube of the number of timesteps.

Another reason for limiting the duration of an optimization run is that long optimization runs can exhibit *perfect foresight*. That is, since each LP solution optimizes over the full run duration, it can incorporate behavior that would not be plausible for operators who at any point in time do not have perfect knowledge of future events. Thus

Chapter 6

Setting Up and Running an Optimization Model

as a rule of thumb, optimization models should have a run duration no longer than the time frame for which accurate forecasts are likely to exist for the river basin operations.

Caution: Although all timestep lengths are available for Optimization, it is not recommended that a 1 Month timestep be used with Optimization. Optimization does not have a concept of *current timestep*. Thus a 30 day month is used for all calculations and constraints that incorporate the timestep length, for example, conversions between volume and flow and between power and energy. As such, the mass balance in the Optimization solution will not match the mass balance of the Post-Optimization Rulebased Simulation for a 1 Month timestep.

Set Up and Test a Simulation Model

Before running a model with the Optimization controller, the model should be set up and run in Simulation mode. This allows for checking that all required input data have been provided and also allows for calibration of the physical model. A model should be validated using Simulation (or Rulebased Simulation) before trying to use it for Optimization. Also, recall that a complete Optimization run actually consists of a sequence of three runs using the Simulation, Optimization and Rulebased Simulation controllers (see [“Standard Controller Sequence”](#) on page 31), so a working Simulation model is a requirement for running with Optimization.

One common approach is to run the model in Simulation using historic Inflow and Outflow data as inputs and allowing the model to calculate Storage, Pool Elevation, Power, etc, which would be checked against the historic values. Then the Outflow inputs would be removed when running in Optimization to let the Optimization solution set the Outflows.

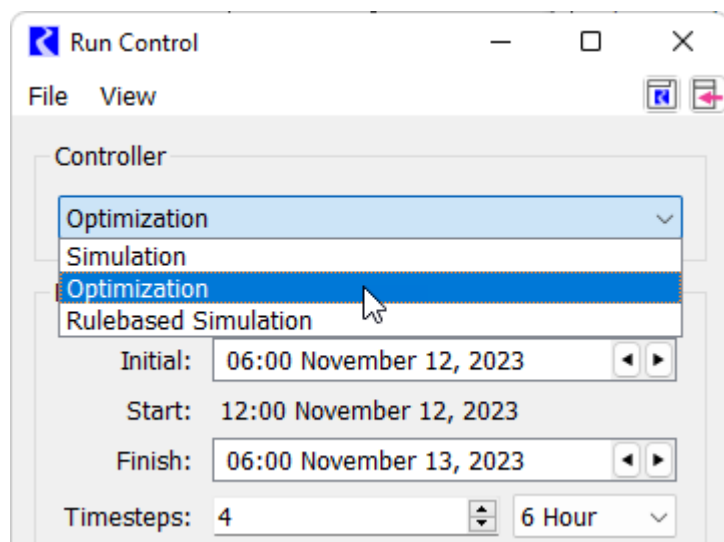
Note: When setting up a Simulation or Rulebased Simulation model that will eventually be used for Optimization, it is important to keep in mind that not all objects and Simulation methods are supported by Optimization. Thus the object and method selections in the Simulation model must be made such that they are compatible with Optimization. See [“Optimization Objects and Methods”](#) on page 87 for descriptions of the objects and methods that are supported by Optimization.

Select Optimization Methods

Many method categories apply only to the optimization controller, and some method selections which are appropriate for Simulation or Rulebased Simulation might not be so for an optimization model. Therefore, one must visit several categories and make appropriate selections. See [“Optimization Objects and Methods”](#) on page 87 for descriptions of the optimization-specific categories and methods.

Note: Not all Simulation objects are supported in Optimization. See [“Optimization Objects and Methods”](#) on page 87 for a list of the only objects that can be included.

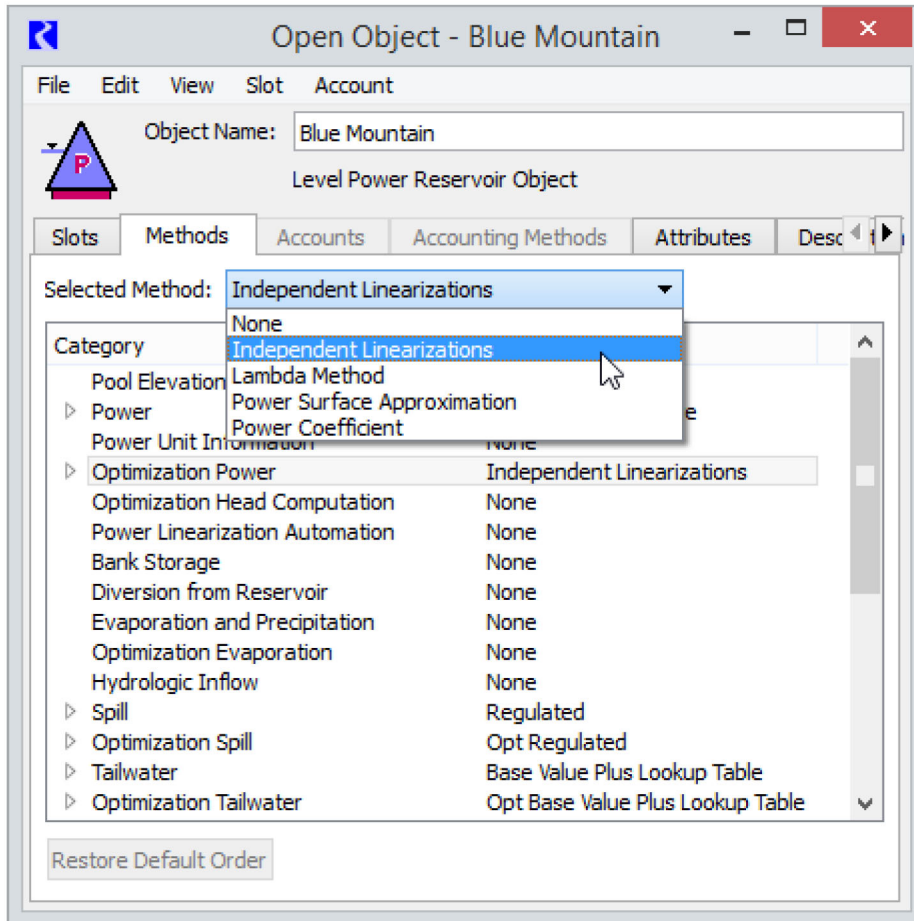
The Optimization method categories are only visible on simulation objects when the Optimization controller is the active controller selected on the **Run Control** dialog.



Chapter 6

Setting Up and Running an Optimization Model

Once the Optimization has been selected, Optimization method selections can be made in the same manner as Simulation methods on the Methods tab of each simulation object (or through the multi-object method selector).



In some cases, a single method is selected for both Simulation and Optimization. For example, if the Time Lag routing method is selected for a reach object, then the appropriate routing constraints will be added in Optimization based on the Time Lag provided for the Simulation method. There is no need in this case to select an additional method for Optimization.

In other cases, an Optimization method must be selected that corresponds to a Simulation method. For example if the Base Value Plus Lookup Table method is selected for the Tailwater category, then Opt Base Value Plus Lookup Table should be selected in the Optimization Tailwater category. The optimization method will add slots that will be used to linearize the Tailwater variables. The linearization will use a combination of slots specific to the Optimization method as well as the Tailwater Tables from the Simulation method. See [“Optimization Objects and Methods”](#) on page 87 for additional information on the selection of appropriate methods for Optimization.

Add a Thermal Object

A single thermal object models basin-wide quantities such as the total value of storage across all reservoirs in the model. The thermal object also includes economic modeling of basin-wide power generation. A thermal object is

not required for an Optimization model, but it can facilitate the modeling of common hydropower objectives. See “Thermal” on page 258 for additional information on the thermal object.

The thermal object acts as a summarizing object for many variables that may be included in an optimization policy. It summarizes information by linking to each of the reservoir or other objects of interest. For example, to compute the total energy produced in the model, the Thermal.Hydro Generation slot should be linked to each PowerReservoir.Energy slot. Other possible links include:

- Thermal.Hydro Generation <-----> PowerReservoir.Energy
- Thermal.Total Cumulative Storage Value <-----> Reservoir.Cumulative Storage Value
- Thermal.Spill Cost <-----> Reservoir.Spill Cost
- Thermal.Future Value of Used Energy <-----> Reservoir.Future Value of Used Energy
- Thermal.Energy in Storage <-----> Reservoir.Energy in Storage
- Thermal.Hydro Capacity <-----> Reservoir.Hydro Capacity

If a thermal object is used in the model, these links will need to be created.

Enter Lower and Upper Bounds

Bounds on Variables

Upper and lower bounds must be entered for all series slots that are decision variables. The values should be reasonable. They represent the absolute minimum and maximum for the slot. Often these minimum and maximum values are based on the actual physical minimum and maximum. For example, the Pool Elevation Lower Bound should be the bottom of the reservoir and the Upper Bound should be the top of the reservoir as found in the Elevation Volume table. Turbine Release should be limited to zero as the lower bound and turbine capacity for the upper bound. In other cases, the absolute physical minimum or maximum may not be well-defined, such as maximum Inflow (or if economic variables are included in the model).

These variable bounds remain constant and are automatically converted by RiverWare into hard constraints on the variable for every time step in the optimization solution. In these cases variables without an obviously defined minimum or maximum, the slot bounds should not unnecessarily constrain the problem. These types of variables will typically be limited, either directly or indirectly, through the constraints in the Optimization Goal Set. It is also important, however, that the slot bounds not be set arbitrarily large or small (i.e. orders of magnitude different than the realistic limits) as this can result in numerical instability due to scaling issues in the optimization problem. There is a tendency to enter data just to get the model to run. This can lead to later errors and difficulty debugging when these values inadvertently limit the solution of the problem.

It is important that slot bounds not be confused with policy constraints, nor be used to apply policy constraints. For example, a hydro plant might have a minimum generation requirement that it must meet, but the Power slot Lower Bound should still be zero. The minimum generation requirement would be set through a constraint in the optimization goal set. The slot bounds become hard constraints in the optimization problem and essentially remain constant in the model, whereas policy constraint values (usually stored in custom slots) are typically incorporated as soft constraints and may change for different runs.

Chapter 6

Setting Up and Running an Optimization Model

The slot bounds do not affect the Simulation solution. They are only applied in the Optimization solution. The Simulation solution is limited only by the physical characteristics described in table slots such as the Elevation Volume Table and Plant Power Table. Often it is helpful to set the slot bound slightly inside min/max values for the corresponding data table. This prevents problems of exceeding values in the data table associated with approximation error when going from Optimization to the Post-optimization RBS. For example, if the highest Pool Elevation in the Elevation Volume Table slot is 1000 ft, the Pool Elevation slot Upper Bound should be set to something like 999.95 ft. See [“Variable Bounds”](#) on page 75 for additional information on how variable bounds are used in the solution.

Setting Bounds

Note: On Agg Series Slots, a value must be entered for each column.

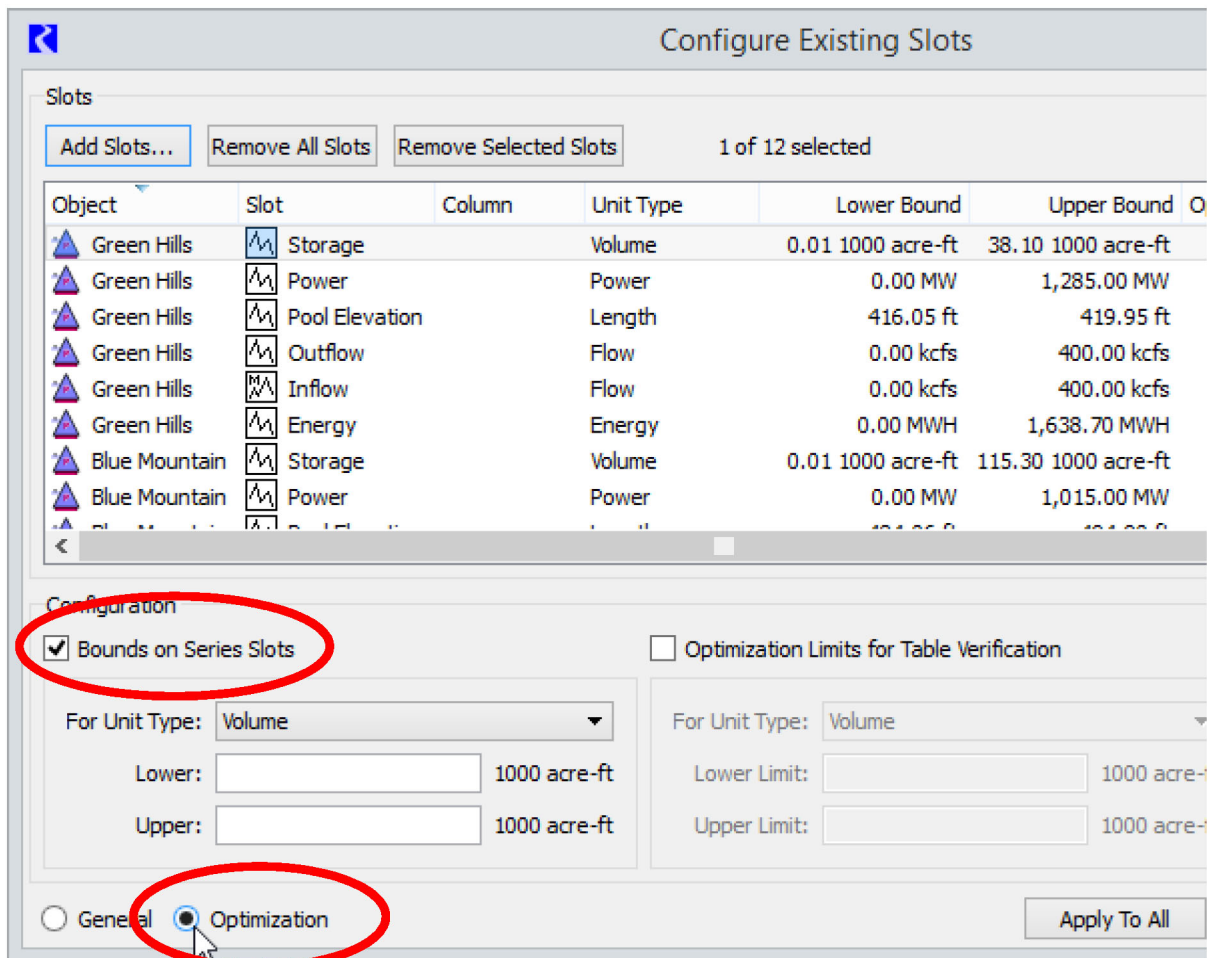
There are two ways to set the Bounds.

1. From each slot, open the Configuration dialog (**View**, then **Configure**) and enter the Upper and Lower Bound.

The screenshot shows the 'Configure Slot' dialog box for the 'Blue Mountain' object and 'Storage' slot. The 'Bounds' section is highlighted with a red circle. The 'Lower' bound is set to 0.01 and the 'Upper' bound is set to 115.30, both with a unit of 1000 acre-ft. The 'Convergence' section shows 'Type: Percent' and 'Value: 0.01'.

Field	Value	Unit
Lower	0.01	1000 acre-ft
Upper	115.30	1000 acre-ft

- Values on multiple slots can be set at once using the global slot configuration dialog accessed from the **Workspace**, then **Slots**, then **Configure Slots** menu (see “[Global Slot Configuration](#)” in *User Interface*). Select the **Optimization** option, and verify the **Bounds on Series Slots** check box is selected.



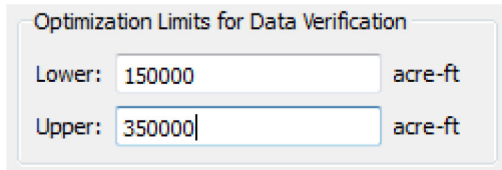
Optimization Limits for Data Verification

Certain Table Slots are verified to ensure they are increasing or satisfy requirements for concavity. Some tables meet the more stringent requirements of Optimization when a reservoir is within a *normal* operating region but violate these conditions during extreme events such as a flood when optimization would not be used. Users can optionally limit data checking in Optimization to the normal region by specifying Lower and Upper Limits. This allows extreme values to be in the table so that the model can be used for extreme events, but the extreme values are not used for Optimization purposes and they are ignored in Optimization verification in order to prevent the verification from failing. The specified range is configured in the Optimization Limits for Data Verification in the Table Slot configuration (shown in the screenshot) or using the Global Slot Configuration dialog described above (check the box for Optimization Limits for Table Verification). Some tables are automatically derived from other tables by RiverWare. For example spill tables based on elevations are automatically converted to spill tables based

Chapter 6

Setting Up and Running an Optimization Model

on storage. When the original tables have upper/lower limits, RiverWare automatically converts the original limits to their equivalent values for the derived tables, e.g. from elevation limits to storage limits.



Optimization Limits for Data Verification

Lower: 150000 acre-ft

Upper: 350000 acre-ft

For three-dimensional tables, this logic may be applied both to the range of z values and to x or y within a block of constant z values.

Only select table slots have limited data checking enabled. This is the current list of those slots for Reservoirs if the slots exist based on controller and method selections:

- Bypass Table
- Elevation Volume Table
- Energy In Storage Table
- Marginal Storage Value Table
- Regulated Spill Table
- Unregulated Spill Table
- Volume Area Table

The following tables are automatically created for Reservoirs by RiverWare by converting elevation based spill tables to storage based tables.

- Bypass Capacity Table
- Regulated Spill Capacity Table
- Unregulated Spill Linearization Table

The limits on these tables are set automatically based on the original tables' limits.

In addition to these slots, Power Reservoirs have the following slots verified if the slots exist based on controller and method selections:

- Maximum Turbine Q
- Plant Power Table
- Stage Flow Tailwater Table
- Tailwater Table

The following tables are automatically created for Power Reservoirs by RiverWare by converting elevation based spill tables to storage based tables.

- Auto Max Turbine Q
- Convolved Stage Flow Tailwater Table

The limits on these tables are set automatically based on the limits in the Plant Power Table and the Stage Flow Tailwater Table respectively.

If the limits values are omitted on a table slot, then there will be no lower limit or upper limit respectively on checking that the table conforms to optimization requirements. The full range is checked.

Provide Approximation Points

When one modeled quantity (slot) is related to one or two other modeled quantities by a non-linear function, then that function must be approximated as one or more linear functions. RiverWare supports three basic ways of approximating a two dimensional function:

- Tangent - the tangent to the function at a given point
- Secant - the line passing through two given points
- Piecewise - several lines connecting points on the function

See [“Variable Substitution”](#) on page 80 for details about each approximation technique.

Select Optimization methods use their own linearization method that does not depend on the tangent, secant and piecewise approximations. The information in this section does not apply for those methods. In the Optimization Power category, the Power Coefficient method and the Power Surface Approximation method are two such methods. See [“Power Coefficient”](#) on page 106 and [“Power Surface Approximation”](#) on page 107 for details. For those methods, a separate set of approximation parameters are entered as described in the section on each method. In the Optimization Tailwater category, the Opt Coefficients Table method uses the same table of coefficients as the corresponding Simulation method to define Tailwater Elevation (i.e. the Simulation method is already a linearized expression of Tailwater). See [“Opt Coefficients Table”](#) on page 135 for details.

Data Table Slots

Each slot being approximated is related to another slot by a data table. Each row of the table provides a point in the curve describing the relation between the two quantities. In the case of 3-dimensional functions, multiple curves are described in the table.

- 2-D example: Elevation Volume Table, relates Elevation to Storage for a Reservoir.
- 3-D example: Plant Power Table, describes Power as a function of the Operating Head and Turbine Release. The table is organized as several curves, corresponding to Power as a function of Turbine Release for various Operating Heads.

Optimization uses the same data tables that are used in Simulation, or generates the required table automatically based on tables used in Simulation. Thus, a model that runs in Simulation should contain all of required data in data tables to run in Optimization.

LP Param Table Slots

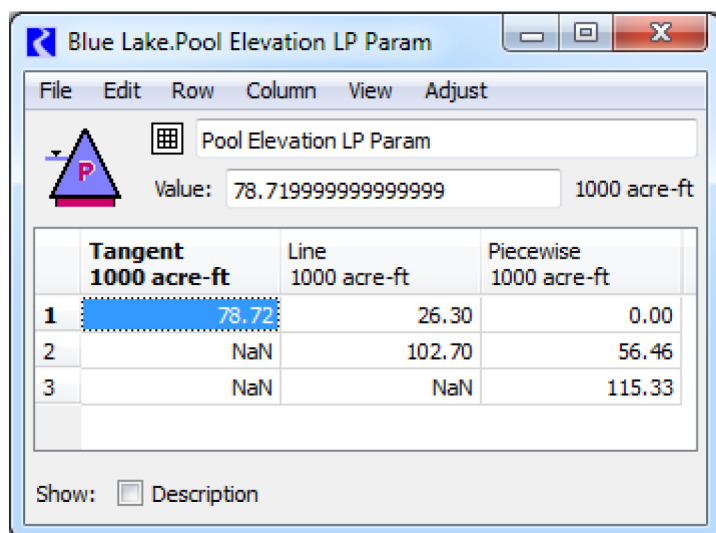
Linear programming parameter tables (LP Param Tables) are specific to Optimization and are required input for an Optimization model. These slots become visible when the Controller is switched to Optimization. The LP Param

Chapter 6

Setting Up and Running an Optimization Model

table slots contain the points to be used when each approximation technique is applied. Setting the values for the LP Param tables can be one of the more challenging aspects of setting up an Optimization model. Setting poor approximation points can result in model infeasibilities that are difficult to debug. [Figure 6.1](#) is an example of a Pool Elevation LP Param table. Basic guidelines for setting LP Param table values follow.

Figure 6.1

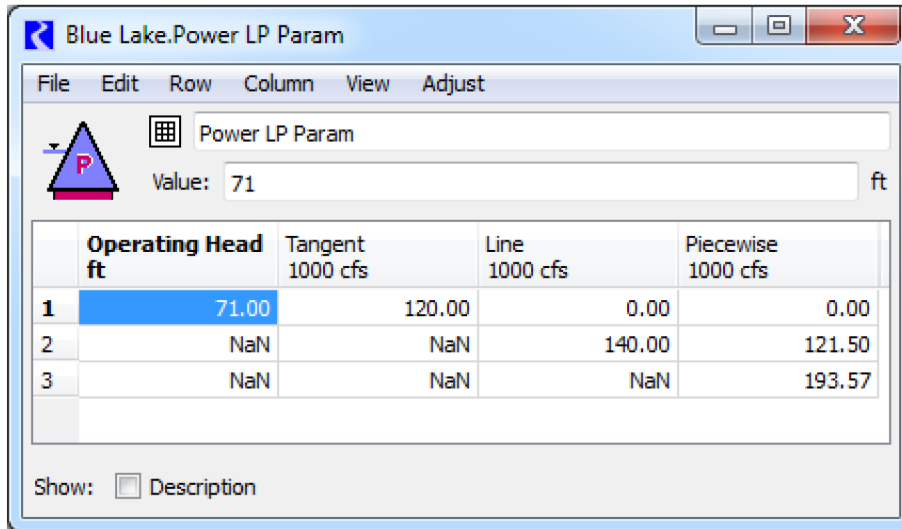


- The Tangent approximation requires a single point. The single approximation point for the tangent approximation should typically be set in the middle of the range of values expected to occur during the current run. The Tangent approximation will overestimate the approximated value for a concave function and underestimate the approximated value for a convex function. See [“Tangent Approximation”](#) on page 83 for details.
- The Line (Secant) approximation requires two points. They should typically be set near either end of the range of expected values. The Line approximation will overestimate a function at some points and underestimate the function at other points. See [“Two-point Line Approximation”](#) on page 83 for details.
- The Piecewise approximation requires two or more points. See [“Piecewise Approximation”](#) on page 84 for details. For piecewise approximation, often it is sufficient to choose the three points used for the Tangent and Secant approximations. In other cases, finer granularity might be required.

Note: Points are entered into the LP Param tables in terms of the independent variables not the dependent variable. For example, the values in the Pool Elevation LP Param table slot are in terms of Storage, not Pool Elevation.

For three-dimensional approximations, the approximation point for the second independent variable (first column in the table) should be somewhere near the middle of the expected operating range. An example of a three-dimensional LP Param table is shown below. For the Power approximation, Turbine Release is the first independent variable and has points set for the Tangent, Line and Piecewise approximations. Operating Head is the

second independent variable. A single value is set for Operating Head in the Power LP Param table, and Power is calculated based on this single Operating Head in the Optimization run.



Additional Guidelines

Turbine Capacity Approximation

The Turbine Capacity linearization will always use the Piecewise approximation. Poor Turbine Capacity approximation points can lead to runs that fail in the Post-optimization Rulebased Simulation due to flows that exceed the actual Turbine Capacity in RBS, once the approximation error is removed. When setting the Piecewise points in the Turbine Capacity LP Param table slot, four criteria should be met.

- First, the piecewise curve must be concave. The optimization solution will not behave correctly if the piecewise curve is not concave, and thus the optimization run terminates with an error if the approximation points define a piecewise curve that is not concave.
- Second, the approximation points should cover the full range of operating heads in the Plant Power Table. If the points do not cover the full range, the optimization solution will use a linear extrapolation beyond the highest and lowest approximation points. This can result in excessive approximation error for upper and lower operating heads.
- Third, the piecewise curve defined by the approximation points should never be greater than the curve defined by the Plant Power Table (Auto Max Turbine Q slot). This can result in the optimization solution allowing turbine releases that exceed the actual Turbine Capacity in the post-optimization simulation. In other words, the piecewise curve from the LP Param approximation should always be under the curve defined by the Auto Max Turbine Q table.
- Fourth, the approximation points should be set such that the difference between the piecewise curve and the Auto Max Turbine Q curve is minimized. This reduces the approximation error in the optimization solution. If the difference between the two curves is excessively large, the optimization solution may overly restrict the turbine release to a capacity significantly lower than the actual turbine capacity for the given operating head.

Chapter 6

Setting Up and Running an Optimization Model

One strategy that can help with picking points for the Turbine Capacity approximation is to plot the Auto Max Turbine Q slot and visually identify reasonable approximation points. The Auto Max Turbine Q slot is automatically populated by RiverWare at the start of the run (even if the run fails). It might be necessary to run the model first in order to populate the table.

Because the Turbine Capacity linearization always uses the piecewise approximation, it is not necessary to set points for the tangent and line approximations. If values are set for these other two approximations, it will not cause a problem. They will simply never be used. If in doubt, set points for all of the approximations and let RiverWare select the appropriate approximation points to use. If values are missing for an approximation that RiverWare tries to use, the run terminates with an error message notifying the user of the missing values.

Power Approximation

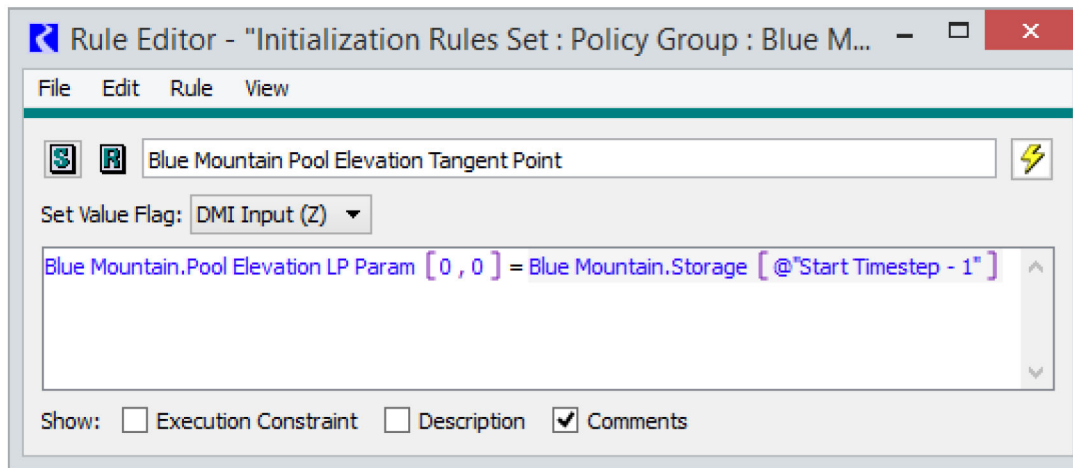
The power approximation tends to be the linear approximation that introduces the most approximation error into the optimization solution. If the Independent Linearizations method is selected in the Optimization Power category, the Power LP Param table is used to define the Power approximation. The table contains an additional initial column for Operating Head in addition to the Tangent, Line and Piecewise columns with Turbine Release values. In the Optimization solution, a single piecewise linear curve for Power as a function of Turbine Release is used. This corresponds to one block of constant head in the Plant Power Table. Operating head is held constant at the value specified in the Power LP Param table. The remaining columns in the Power LP Param table set the Turbine Release for the approximation points.

Typically the Operating Head value in the Power LP Param table should be set to the average Operating Head expected in the run. However, in some cases it might be less problematic to overestimate Power than to underestimate Power. In these cases, it might make sense to set the Operating Head value in the Power LP Param table near the lower end of the expected operating range. This would cause the Optimization solution to underestimate Power, and thus the Post-optimization Rulebased Simulation will calculate a large Power value than the Optimization.

Initialization Rules to Set Approximation Points

In some cases, it might make sense to set the approximation points based on the specific run conditions. For example, for a reservoir that has a large seasonal variation in Pool Elevation, it might not be possible to set Tangent and Line approximation points in the Pool Elevation LP Param table or an Operating Head point in the Power LP Param table that will result in sufficiently small approximation error across all conditions. In these cases, initialization rules can be used to set the approximation points such that the approximation error will be small for the expected operating range of the given run but would be large if operating in a different range in another season. For example, an initialization rule could set the Pool Elevation LP Param Tangent point equal to the initial Storage for the run as shown in [Figure 6.2](#).

Figure 6.2



A second rule could set the two Line points to the Tangent point plus or minus some expected delta. In this manner, the approximations will be updated automatically when the model runs, rather than needing to adjust the values manually at the start of each run.

Create an Optimization Goal Set

The Optimization policy must be cast into a set of prioritized goals. Like a RBS Ruleset, an Optimization Goal Set is a prioritized list of goals (written in RPL). However, whereas the rules in a RBS Ruleset are all executed at each timestep and set values, Optimization Goals are each executed once during a run from highest priority to lowest priority and have the effect of modifying or solving the current linear programming problem.

RBS rules are normally ordered to execute in an upstream to downstream order, object-by-object. Generally there is no need to order Optimization goals in an upstream to downstream order. The priorities of the goals should correspond to their *true* priorities in operations. The highest priority goals should correspond to the most important operating constraints, those that should essentially never be violated.

A single goal can contain constraints for multiple objects, but it is generally recommended that a single goal correspond to a single operating policy. For example, the highest priority goal might contain the license maximum Pool Elevation constraints for all reservoirs at all timesteps. The second goal might contain the license minimum Pool Elevation constraints for all reservoirs at all timesteps. This is generally better practice than placing the minimum and maximum elevation constraints in the same goal or placing the minimum elevation constraint in the same goal (same priority) as a minimum flow constraint. When multiple policies are combined in a single goal, it is not always obvious how the policies will trade off with one another if it is not possible to satisfy all constraints at that priority.

Medium priority goals generally contain constraints that you expect to get satisfied most of the time but that may not be possible to satisfy under some operating conditions, for example under extreme high or low flows. Lower priority goals typically contain target operating constraints, targets that you would like to meet under ideal conditions but are expected to be frequently violated due to higher priority constraints and input data (e.g. inflows). For example, you might have a low priority goal to keep flows equal across all reservoirs on each timestep, but most of the time you cannot keep flows exactly equal due to other requirements on the system. Typically the lowest priority contains an objective function to optimize with the remaining degrees of freedom given all higher priority

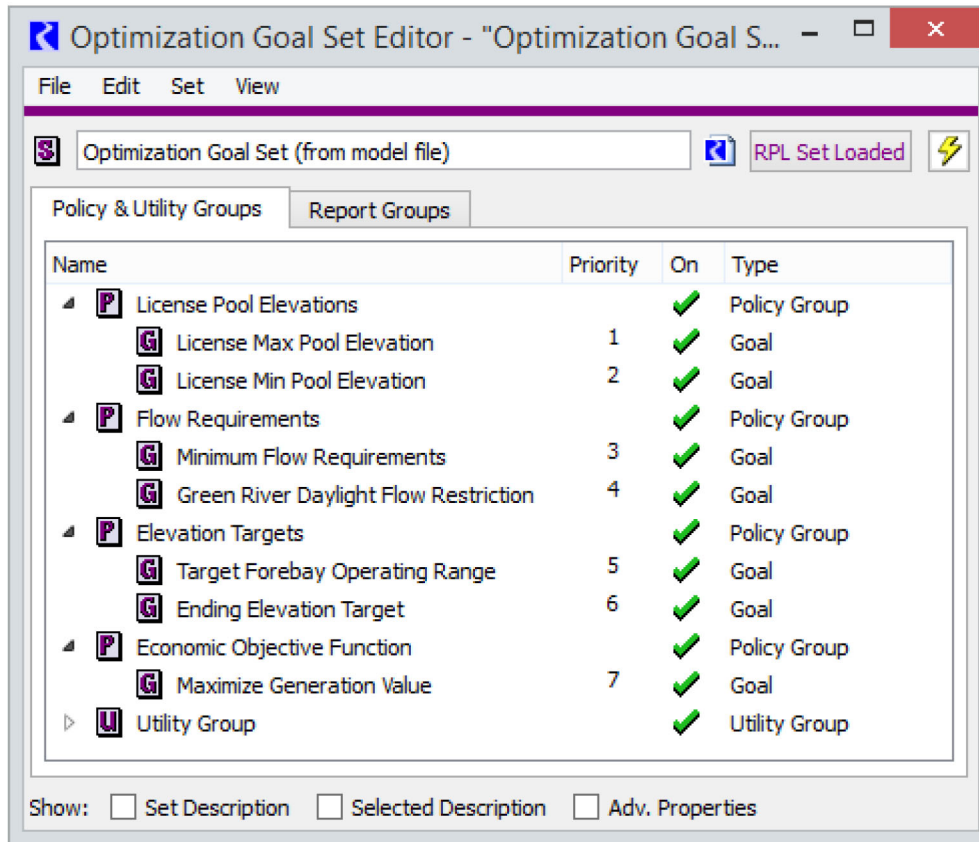
Chapter 6

Setting Up and Running an Optimization Model

constraints. For example, the objective might maximize the total value of hydropower generation during the run within the operational limits specified by higher priority constraints.

Figure 6.3 is a simple example of an Optimization Goal Set.

Figure 6.3



See “[RPL Optimization Goal Set](#)” on page 55 for additional information about the Optimization Goal Set. See “[Statements in Goal Sets](#)” on page 60 for details about the individual statements that make up Optimization Goals.

Create a Post-optimization Ruleset

RiverWare will automatically generate a Post-optimization Ruleset if there is no ruleset loaded when switching from the Optimization controller to the Rulebased Simulation controller upon completing a run with the Optimization controller. The automatically generated ruleset contains rules to set each reservoir Outflow slot to the Outflow value from the Optimization solution.

```
Reservoir.Outflow[] = OptValue(Reservoir.Outflow, @"t")
```

See “[Automatically Generated Ruleset](#)” on page 34 for details about the automatically generated ruleset.

In many cases it is necessary to customize the Post-optimization Ruleset to do more than simply set reservoir Outflows to the Optimization values. Some common uses for a custom ruleset might be:

- A need to set Turbine Release and Spill independently based on their Optimization values rather than just total Outflow.
- If other water uses, such as diversions, need to be specified in the RBS based on the Optimization solution.
- To refine the Optimization solution further in the Post-optimization RBS, possibly to make small adjustments to the flows once outputs have been calculated with the approximation errors removed.
- To add post-processing calculations to the Post-optimization rules.

A useful approach for creating a custom Post-optimization Ruleset is to begin with the automatically generated set. Then revise the rules as needed using the basic structure of the automatically generated set.

While the Post-optimization Ruleset is a special case of a ruleset used specifically in the context of an Optimization run, from a technically perspective, it is the same as any RBS Ruleset. As such it can contain any components that might be in a standard RBS Ruleset. For example, there is nothing to technically prevent the Post-optimization Ruleset from completely overwriting the Optimization solution. It is the use of the OptValue function when setting the slot in the ruleset that ties to the Post-optimization Rulebased Simulation to the Optimization solution. The Post-optimization Ruleset can make use of as much or as little of the Optimization solution as desired as long as it adheres to all standard requirements for a RBS Ruleset.

That being said, the standard approach is to have rules set flows based on the Optimization solution and let RiverWare calculate all remaining values in Simulation. When using this standard approach, it is strongly recommend to write the rules such that they maintain the overall mass balance from the Optimization solution. This is because the feasibility of the Optimization solution is dependent on the mass balance used in the Optimization solution. In addition the satisfaction of policies such as Pool Elevation constraints in the Optimization solution is dependent on the mass balance.

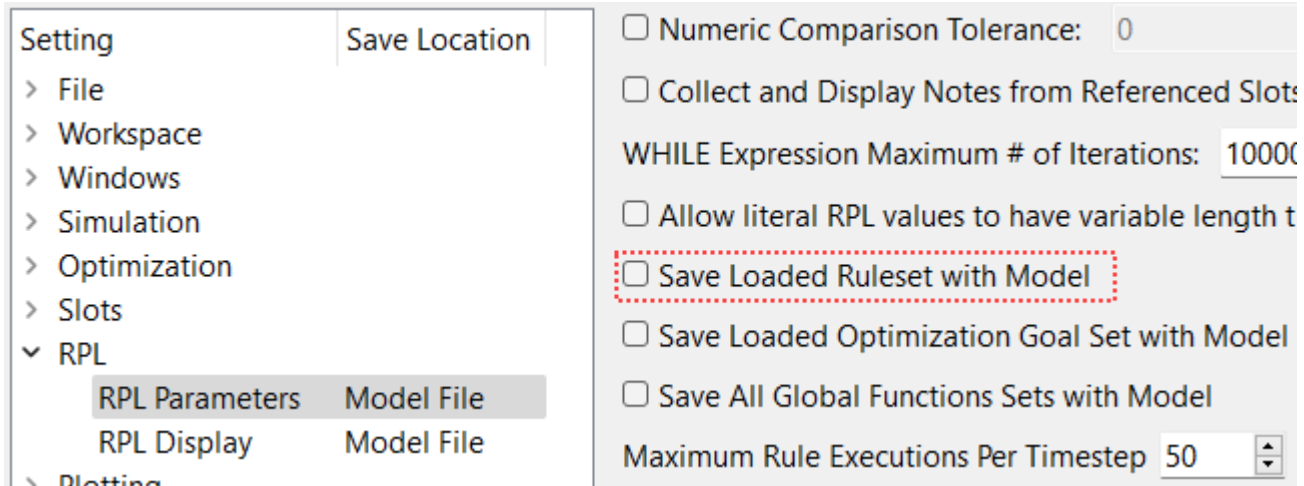
As an example, Post-optimization Rules that adjust the solution by making shifts between Spill and Turbine Release but use the same total Outflow as the Optimization solution would maintain the mass balance from the Optimization solution. Whereas rules that modified the total Outflow from the Optimization solution would alter the mass balance. In the later case, it would be possible to have a run where the Storage exceeded the limits in the Elevation Volume Table in the Post-optimization Rulebased Simulation, causing the run to abort, even though it was within the required limits in the Optimization solution (or less severely violate elevation constraint limits in the Post-optimization Rulebased Simulation when they were satisfied in the Optimization solution). Of course it is possible to include logic in the rules to check for these issues, but the more this type of logic is included in the ruleset, the more the solution will tend to deviate from the Optimization solution and become more of a Rulebased Simulation solution. Depending on the intended use of the model, this may or may not be acceptable.

Once a custom Post-optimization Ruleset has been created, it can be saved as a separate file just like a standard RBS Ruleset by selecting **File**, then **Save RBS Ruleset As** from the Ruleset Editor dialog and/or it can be saved in the model file. Once a Ruleset has been saved with the model file, then the next time the model is opened, the Ruleset will automatically be opened and loaded. To save the Ruleset with the model file it must first be loaded.

Chapter 6

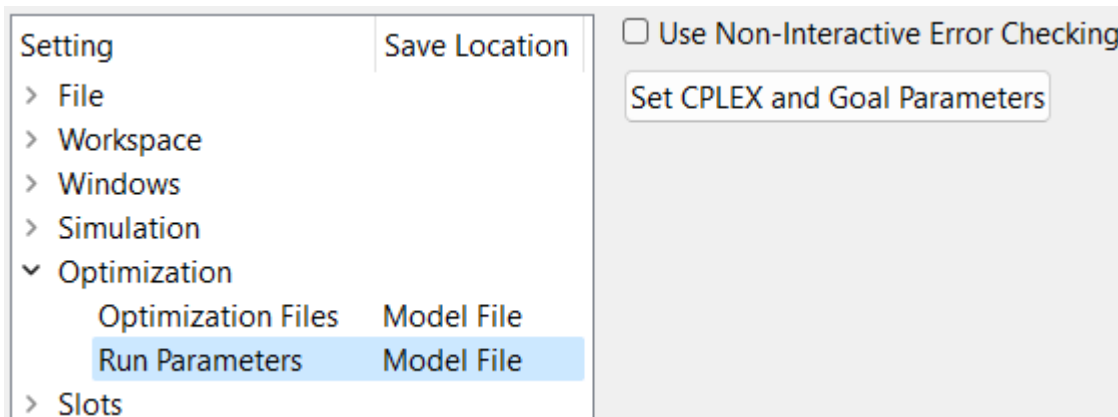
Setting Up and Running an Optimization Model

Then on the **Run Control** dialog select **Policy**, then **RPL Parameters**. In the Settings manager, check the box for **Save Loaded Ruleset with Model**. Close the dialog, and save the model with the Ruleset now included.



Run Parameters

Optimization run parameters, such as the LP method used and CPLEX parameters can be set on the Optimization Run Parameters page of the Settings Manager. A screenshot is shown below:



See the “[Settings Manager](#)” in *User Interface* for more information. Also, for more information on Optimization Run Parameters, contact riverware-support@colorado.edu.

Run the Model

Every run should be made as follows:

1. Switch the run controller to **Simulation**, and select **Start**. See “[Simulation](#)” on page 31 for details.

After the Simulation run, most slots of interest will still contain NaN.

2. Switch the run controller to **Optimization**, and click **Start**. See “[Optimization](#)” on page 32 for details.

The Optimization run will take considerably longer than the Simulation run. During the Optimization run, the Run Status dialog will report which priority is currently executing from the Optimization Goal Set rather than a timestep as for Simulation. After the Optimization run, all slots that contained NaN after the Simulation run will continue to display NaN. The Optimization solution is only stored internally and has not been returned to the workspace.

3. Switch the run controller to **Rulebased Simulation**. If no Ruleset is loaded either accept the option to (automatically) generate and load a Post-optimization Ruleset, or alternatively load a custom ruleset, and then click **Start**. See “[Post-optimization Rulebased Simulation](#)” on page 32 for details.

After running the Post-optimization Rulebased Simulation, all slots of interest should be populated with data.

Note: It is important to always run Simulation before running Optimization. Running Optimization without first running Simulation will not produce the desired results as data might be leftover from a previous run, or necessary preprocessing of data by initialization rules or expression slots will not be carried out.

An alternative to automate the three run sequence is to use a RiverWare script with actions to Set Controller and Execute Run for each of the three individual runs. See “[Managing Scripts](#)” in *Automation Tools* for details on how to configure and use a script.

Chapter 6

Setting Up and Running an Optimization Model

Chapter 7

Debugging and Solution Analysis

Topics

- [Optimization Debugging Tools](#), page 285
- [Priority-oriented Optimization Solution Analysis Tool](#), page 285
- [Tooltips on Variables](#), page 292
- [Saving Optimization Files](#), page 293

Optimization Debugging Tools

The main tools to use when debugging an optimization run are:

- The RPL debugger to investigate the policy. See “[RPL Debugger](#)” in *Debugging and Analysis* for details.
- The Priority-Oriented Optimization Solution Analysis Tool which provides information on the optimization solution, described below.

This section describes the solution analysis tool and other debugging features.

Priority-oriented Optimization Solution Analysis Tool

The Priority-Oriented Optimization Solution Analysis Tool provides detailed information from an optimization run. This includes the following:

- Objective values
- Satisfaction of derived objectives (e.g. Repeated Maximin) from goals with soft constraints
- Constraints frozen with each solution
- Variables frozen with each solution
- Dual prices of frozen constraints
- Reduced costs of frozen variables

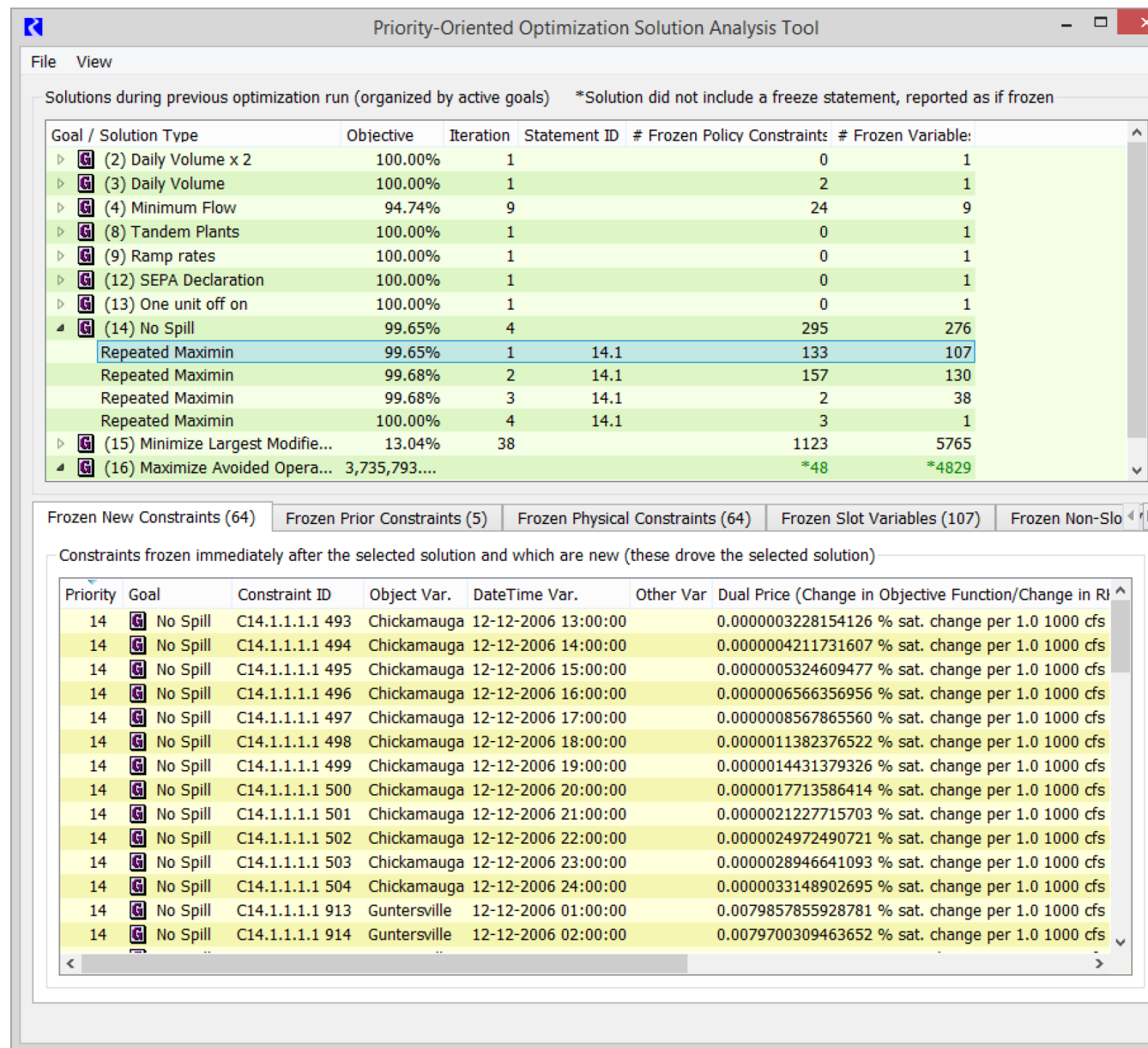
This information is valuable for understanding the optimization solution. For example, observing which constraints were frozen immediately after a given solution indicates which policies were driving the solution. Information

Chapter 7

Debugging and Solution Analysis

from the tool can also be helpful for debugging problems with an optimization run. For example, constraints that were frozen when not expected can at times indicate policy in the goal set that is having unintended consequences.

Figure 7.1



The tool has two panels. The top panel, the Solution Information panel, provides an overview of the solutions that occurred during a single optimization controller run. The tabs of the lower panel provide additional details; the focus of these details is controlled by user selection within the Solution Information panel.

The information displayed by the tool is:

- Collected during every optimization run.

- Invalidated and completely cleared by deletion of objects or slots from the workspace.
- Cleared whenever a new optimization run is initiated.
- Corresponds to the current run when paused within a run; otherwise it corresponds to the previous optimization run.
- Only guaranteed to display correctly when the currently loaded goal set was the previous run's goal set.
- Partially saved in the model file. The entire set of information can be saved to an external file and reloaded for later use.

During an optimization run, the display updates when the run is paused, stopped, terminated, or ends.

Accessing the Analysis Tool

When information about the last optimization run is available, the Priority-Oriented Optimization Solution Analysis Tool can be accessed in the following ways:

- From the **Run Control** dialog: **View**, then **Optimization Solution Analysis**.
- From the **Model Run Analysis** dialog: **File**, then **Optimization Solution Analysis**.
- From the Workspace: **Utilities**, then **Optimization Solution Analysis**.

Save/Load Problem

The information associated with the Solution Information panel is automatically saved within the model file. The information associated with the remaining panels is likely to be large and so is not saved with the model, but it can optionally be saved to a file in the optimization output directory.

To automatically save the problem each time:

1. Open the **Run Control** dialog.
2. Select the **Optimization** controller.
3. Select **View**, then **Optimization Run Parameters**.
4. Check the box for **Save Final Optimization Problem**.

When this option is selected, the constraint and variable information will be saved to a file in the optimization output directory when the model is saved.

To save the problem interactively for later use, use the **File**, then **Save Problem** operation to write the problem to a file.

To load the saved information, after opening the relevant model and loading the goal set, open the Solution Analysis Tool and select **File**, then **Load Problem** and choose the desired file.

Note: For the **File**, then **Load Problem** utility to function correctly, the active goals and their priorities in the problem being loaded must match those of the most recent optimization problem currently displayed in the Solution Information panel (i.e. must be from a run with the same goal set or at least a set that is the same at the top level of goal names and priorities). If the loaded problem is from a different goal set, one with different goals, different goal activation and/or different

prioritization than the most recent solution displayed in the Solution Analysis Tool, then the information displayed will not be consistent with the loaded problem.

In most cases, the **Priority-Oriented Optimization Solution Analysis Tool** will be used for a run just completed and this step of saving and loading the optimization solution is not necessary.

Solution Information Panel

The Solution Information Panel presents an overview of the optimization solution. The top level displays the active goals in the loaded goal set, in priority order. Clicking on the arrow at the left of each these *goal rows* will expand the row to display information about each solution corresponding to that goal (a *solution row*). See [Table 7.1](#) for details.

Table 7.1 Solution Information Panel Columns

Column Name	Goal Row	Solution Row
Goal/Solution Type	Priority and name of the goal	Type of solution from the executed statement in the associated goal, one of: Maximize Objective, Minimize Objective, Summation, Repeated Maximin, Single Maximin, Mixed Integer
Objective	The value of the first solution row that has a value; for Maximize or Minimize objectives this is the objective value; for Repeated Maximin it is the percent satisfaction of the derived objective on the first iteration	The objective value (with units); if the associated statement was a Maximin this is the percent satisfaction of the derived objective.
Iteration	For goals with Repeated Maximin, the total number of iterations; otherwise no value is displayed	For Repeated Maximin, the solution index (iteration number); otherwise no value is displayed
Statement ID	No value displayed	The first number indicates the priority of the goal. Subsequent numbers correspond to levels within the goal. (e.g. 22.1 indicates the first statement in goal 22).
Number of Frozen Policy Constraints	The number of policy constraints introduced by goals that were frozen while executing the goal (all iterations); physical constraints are not included in the count	The number of policy constraints introduced by goals that were frozen immediately after the corresponding solution; physical constraints are not included in the count. See also the following section.
Number of Frozen Variables	The number of variables frozen while executing the goal (all iterations)	The number of variables frozen immediately after the corresponding solution.

Right-clicking on a row in the Solution Information Panel will open a context menu that will allow to you open the corresponding goal.

Goal / Solution Type	Objective
▶  (20) Meet Spill Requirement	100.00 %
▶  (21) Spill Cap Instantaneous	100.00 %
▲  (22) Spill Cap 12 Hour Average	96.51 %
Repeated Maximin	96.51 %
Repeated Maximin	96.51 %
Repeated Maximin	96.51 %
Repeated Maximin	96.51 %
Repeated Maximin	96.51 %
Repeated Maximin	96.51 %

Solutions Without a Freeze Statement

In a typical policy, most problem solutions are immediately followed by the freezing of solution limiting variables and constraints. This freezing occurs automatically only for solutions that occur during execution of a Repeated Maximin statement; for freezing to occur after solutions caused by statements of other types, the policy must contain an explicit Freeze statement.

When a problem solution is not immediately followed by a freeze before the next solution, it is sometimes useful to know which constraints and variables would have been frozen had a Freeze Statement been executed. Thus, for these solutions the “# Frozen Constraints” and “# Frozen Variables” columns contain the number of constraints and variables that would have been frozen had the solution been immediately followed by a Freeze statement. These totals are displayed with a preceding asterisk (*) and a differently colored font.

Note: When more than two solutions occur before a freeze, a single variable or constraint might have been frozen at multiple solutions, but is reported for only the earliest solution.

Frozen Constraints Tabs

The lower panel presents five tabs. The first three present lists of constraints that were frozen:

Frozen New Constraints

Constraints frozen immediately after the selected solution, which were introduced by the goal at this priority (these drove the selected solution)

Frozen Prior Constraints

Constraints frozen immediately after the selected solution and which were introduced earlier

Physical Constraints

Non-policy constraints frozen immediately after the selected solution; typically physical constraints or variable definition constraints automatically added to the optimization problem by RiverWare

Information in the Frozen Constraints tabs corresponds to the currently selected solution in the Solution Information panel. If no solution is selected, the Frozen Constraints tabs will be blank. Also, if no constraints were frozen as a result of the selected solution, the Frozen Constraints Tabs will be blank. See [Table 7.2](#) for details.

Note: Not all columns are included in all three tabs.

Table 7.2 Frozen Constraints Tabs Columns

Column Name	Description
Priority	Priority of the goal that added the constraint to the optimization problem
Goal	Name of the goal that added the constraint to the optimization problem
Constraint ID	The ID of the constraint, for example C4.1.2.2.1 45. The first number indicates the goal priority number and each subsequent number corresponding to a statement at each different level within the goal. The final number is the individual permutation of variables from a set of nested FOR loops.
Object Var.	If the constraint is within a FOR statement, and if one of those statements contains an index variable of type Object, then this is the value of the outermost Object index.
DateTime Var.	Similar to the Object Var. column but for DateTime variables
Other Var.	The values of any FOR statement index variables not covered by the previous two columns
Dual Price(Change in Objective Function/Change in RHS)	The change in the objective function value per unit change in the right-hand side of the constraint. The units given are the units of the objective per units of the right-hand-side (user units).
Dual Price (raw)	The dual price value as reported by CPLEX
Shrink to ID	The ID of the constraint to which this constraint is shrinking, if there is one (i.e. a constraint added earlier that differs only in a less restrictive right-hand-side value)
Satisfaction	The percent satisfaction of the constraint

- Clicking on any column header will sort the table by that column.
- Columns can be reordered by clicking and dragging on the column header.
- Right-clicking on a constraint row presents the **Open Constraint's Goal** option. For constraints that were shrunk to another constraint (a value is present in the Shrink to ID column), there will also be the **Open Shrink To Constraint's Goal** option.

Frozen Variables Tabs

The remaining two tabs present information about Frozen Variables:

- **Frozen Slot Variables**—Variables associated directly with slots frozen immediately after the selected solution (these limited its objective function, are at bounds)
- **Frozen Non-Slot Variables**—Variables not associated directly with a slot frozen immediately after the selected solution (these limited its objective function, are at bounds)

Information in the Frozen Variables tabs correspond to the currently selected solution in the Solution Information panel. If no solution is selected, the Frozen Variables tabs will be blank. Also, if no variables were frozen as a result of the selected solution, the Frozen Variables tabs will be blank. See [Table 7.3](#) for details.

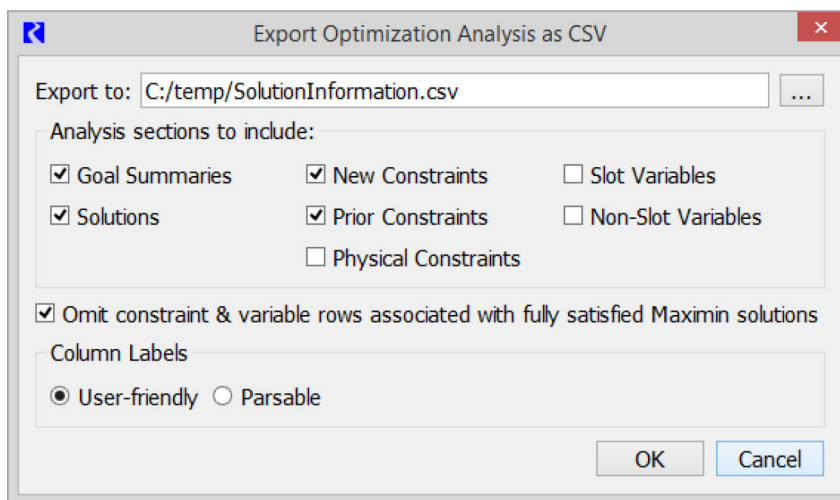
Table 7.3 Frozen Slot Variables Tab Columns

Column Name	Description
Object	The object associated with the variable
slot	The slot associated with the variable (just the slot name)
Date/Time	The timestep associated with the variable
Value	The value in the solution (in internal optimization units)
Reduced Cost (Change in Objective Function/Change in Variable)	The change in the objective function if the variable changes by 1 unit. The units given are the units of the objective per unit of the variable (user units).
Reduced Cost (raw)	The reduced cost value as reported by CPLEX

- The **Frozen Non-Slot Variable** tab has a single Name column in place of the Object, Slot and Date/Time columns.
- Selecting any column header sorts the table by that column.
- Columns can be reordered by clicking and dragging on the column header.

Export as CSV

To export the information in the analysis tool to a comma separated values (CSV) file, select **File**, then **Export Analysis (CSV)**. The following dialog opens, allowing you to configure the file output.



1. First, specify the file to use for export in the upper **Export to:** field.
2. Choose the **Analysis sections to include**. Each checkbox results in one or more rows for each optimization goal priority in the file.

The columns of the CSV file correspond to the columns shown in the analysis tool, presented in the tables above.

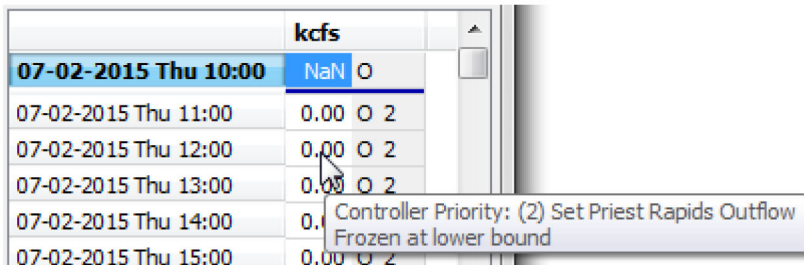
Chapter 7

Debugging and Solution Analysis

3. Specify whether to **Omit constraint & variable rows associated with fully satisfied Maximin solutions**. When checked, rows representing a fully satisfied solution are omitted.
4. For the column labels, choose one of the following options:
 - **User-friendly** with spaces between words
 - **Parsable** where all spaces are removed.
5. Select **OK** to generate the CSV file or **Cancel** to close without creating the file.

Tooltips on Variables

Tool tips provide information about frozen variables. The information is shown on the slot and SCT dialogs when you hover over the slot value.



	kcfs
07-02-2015 Thu 10:00	NaN
07-02-2015 Thu 11:00	0.00
07-02-2015 Thu 12:00	0.00
07-02-2015 Thu 13:00	0.00
07-02-2015 Thu 14:00	0.00
07-02-2015 Thu 15:00	0.00

This occurs when:

- The associated variable was frozen at the slot upper or lower bound.
- The associated variable was the only variable in a constraint that was frozen. When a constraint is frozen, RiverWare internally converts the inequality constraint to an equality constraint for the remainder of the solution. When there is only a single variable in the constraint, this is conceptually the equivalent of freezing the variable.

Examples include the following:

- **Frozen at lower bound**

This type of message occurs if the variable is frozen at one of the slot bounds.

- **Frozen by (3) Minimum Load 47.6% between limits set by 3.1.1.1 and 2.1.1.1.**

This type of message occurs if the variable is in a soft constraint that was not fully satisfied. In this example, the best the solution could do was get 47.6% of the way from the old limit set on the variable in goal 2 (statement 2.1.1.1) and the new limit set by goal 3 (statement 3.1.1.1).

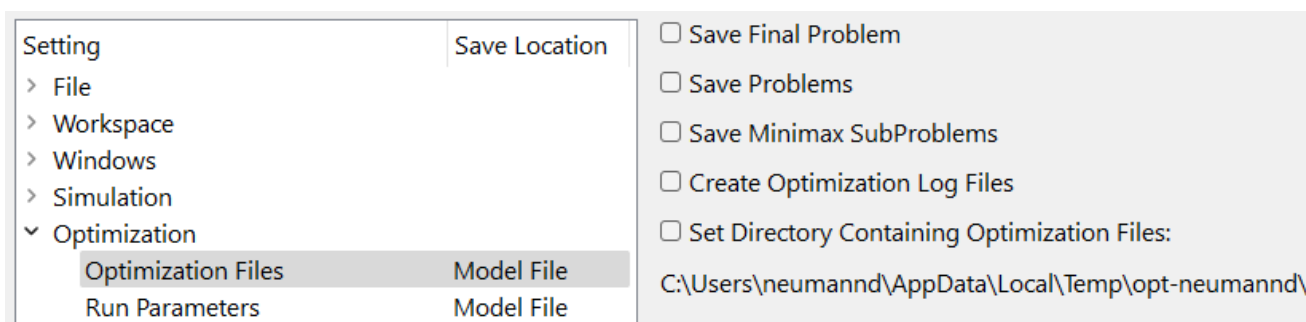
- **Frozen by (22) Maximize Generation Value at a limit set by 4.1.1.1.**

The occurs when the variable is in a constraint that is fully satisfied but forced to be tight (frozen at the constraint limit) by a lower priority constraint or objective. In this example, assume that the variable is Pool Elevation, and goal 4 has a constraint statement (4.1.1.1) that constrains Pool Elevation to be greater than or equal to some value (a minimum Pool Elevation). The tool tip is stating that the Maximize Generation Value objective at priority 22 forced the Pool Elevation to the minimum set by goal 4.

These text strings that appear in the tool tips can be exported with the values using output DMIs described. See “Optional keyword=value Pairs” in *Data Management Interface (DMI)* for details.

Saving Optimization Files

For in-depth debugging, it can sometimes be helpful to save the optimization problem (LP file) that RiverWare passes to CPLEX or the RiverWare Optimization Log. Saving of these files can be activated on the Optimization Files page of the Settings Manager shown in the screenshot.



See the “Settings Manager” in *User Interface* for more information.

For more information on saving and using optimization files, contact riverware-support@colorado.edu.

Chapter 7

Debugging and Solution Analysis

Appendix A

Technical Appendix

Topics

- [Optimization Units and Scaling, page 295](#)
- [Mixed-integer Programming, page 296](#)

Optimization Units and Scaling

Optimization computations are carried out in the same RiverWare standard units used for Simulation computations with one difference. As with Simulation, values are converted automatically from display units to internal units for computations, but for Optimization, that internal conversion might also include scaling. For example, storage volumes in standard units are often on the order of 10^9m^3 , whereas elevation changes might be on the order of 10^{-1}m . Introducing variables with such a wide range of magnitudes can lead to instability in an Optimization solution. Storage and elevation values are therefore scaled when added to the Optimization problem so that they will tend to be of similar orders of magnitude.

RiverWare determines how to scale values based on their unit types, and all conversions are carried out internally. The unit scaling is specified in the Optimization Run Parameters. To modify the unit scaling, in the Run Control, set the controller to **Optimization**. Then select **View** then **Optimization Run Parameters**. In the Optimization Run Parameters dialog box, select **Set CPLEX and Goal Parameters**. In the Set Optimization Solver Parameters dialog box, select the tree-view arrow for **RiverWare Units**, and select the desired unit type. [Table A.1](#) shows the default unit scaling for each unit type. Unit types not listed will always use the standard RiverWare internal units.

Table A.1 Optimization Units - Default Scaling Values

Unit Type	Parameter	Default Scale (with standard unit)
None (dimensionless)	UNITS SCALE NOUNITS	1
Length	UNITS SCALE LENGTH	10 m
Area	UNITS SCALE AREA	$1\text{e}7 \text{ m}^2$
Volume	UNITS SCALE VOLUME	$1\text{e}8 \text{ m}^3$
Flow	UNITS SCALE FLOW	1000 cms
Power	UNITS SCALE POWER	1000 MW
Energy	UNITS SCALE ENERGY	1000 MWH
Time	UNITS SCALE TIME	$1\text{e}5 \text{ s}$

Table A.1 Optimization Units - Default Scaling Values

Unit Type	Parameter	Default Scale (with standard unit)
Money	UNITS SCALE VALUE	1e5 \$
PowerPerFlow	UNITS SCALE POWER PER FLOW	1 MW/cms
FlowPerTime	UNITS SCALE FLOW PER TIME	1e-2 cmss
Velocity	UNITS SCALE VELOCITY	1e-4 m/s
PowerPerTime	UNITS SCALE POWER PER TIME	1e-2 MW/s
FlowSquared	UNITS SCALE FLOW SQUARED	1e6 m ⁶ /s ²
VolumeSquared	UNITS SCALE VOLUME SQUARED	1e16 m ⁶

Ideally the scaling is such that coefficients on variables in the internal representation of the optimization problem are close to 1.0, within one or two orders of magnitude. Very small coefficients in particular can lead to numeric instability. The default unit scaling values generally result in reasonable scaling for most RiverWare optimization models with a timestep of 1 Hour or 6 Hour. The user, therefore, does not typically need to be concerned with scaling unless the timestep is larger or smaller. For example, for a model with a 5 Minute timestep, it may be necessary to decrease the Time unit scaling by a factor of 10 or 100. When the Time scaling is reduced by a factor of 10, other scaling adjustments are required for consistency.

- Area, Volume: Reduce by a factor of 10
- Volume Squared: Reduce by a factor of 100
- FlowPerTime, PowerPerTime, Velocity: Increase by a factor of 10

If Time scaling is increased, the direction of the other adjustments needs to be reversed.

Knowledge of the unit scaling may also be necessary when viewing the full LP problem generated for debugging purposes. Values in the LP text file will be the scaled values. For assistance with questions about scaling, contact RiverWare Support: riverware-support@colorado.edu

Mixed-integer Programming

Some user-selectable engineering methods require a mixed integer programming solution. The Unit Power Table method, for example, requires integer programming to model the start-up and shut-down of individual turbines. Mixed integer programming adds substantial computational time to an Optimization model.

Note: It is recommended that users contact RiverWare Support before attempting to use methods that require mixed-integer programming. Inquiries can be sent to: riverware-support@colorado.edu

Mixed Integer Programs are optimization problems that contain both continuous and integer variables. In the river and reservoir modeling context, many of the variables are continuous. For example, outflow, pool elevation, and storage are all continuous variables. In contrast, hydropower turbines are operated such that a unit is either off or fully on. Thus, the Unit Power Table method has introduced some integer and binary (zero or one) variables. More specifically, the number of units generating at a given plant during a given time step is an integer variable and the decision to operate a single unit is a binary variable; the unit is either off or on. The modeling of integer and binary variables is sufficiently similar that the term *integer* is used to refer to both integer and binary variables.

Integer variables are also used to model discontinuities. For example, avoidance and cavitation zones are regions where power cannot be generated without damaging the equipment. These zones create a discontinuity in the power curve, and integer variables are used to model this discontinuity.

RiverWare solves this using Mixed Integer Linear Programs, but we will generally omit the word *linear*. The *linear* restriction reflects the fact that all non-linear functions have been replaced by linear or piecewise linear functions. The use of linear approximation allows the optimization to solve faster. We will also omit the word *mixed* and refer to Mixed-integer Linear Programming as simply Integer Programming.

Integer Programs are more difficult to solve than linear programs. For this reason, RiverWare has limited use of integer variables and the heuristic solution methods used are discussed in more detail in the rest of this section.

Variables That Bring Integer Programming Into the Problem

Two conditions are necessary for Integer Programming in RiverWare; integer programming is only allowed in the lowest priority policy and the policy must directly or indirectly use integer variables.

Integer Programming is limited to the lowest priority for performance reasons. To include integer variables at additional priorities would lead to what it called Integer Goal Programming (IGP). While there is no theoretical barrier, IGP is too computational challenging at this time. Consequently, RiverWare only allows integer variables in the final priority. If integer variables are referenced within a previous objective, RiverWare will relax the integrality constraints and solve the relaxed linear programming problem.

If a RiverWare policy doesn't reference integer variables then integer programming is not used. Currently the only source of integer variables is the Unit Power Table and related methods (see "[Unit Power Table](#)" on page 139). Therefore, if only plant power methods are selected, integer variables will not be introduced and RiverWare will be solved as a Linear Goal Program.

The integer variables can be introduced either by directly referencing them in policy or by indirectly referencing them in policy. Integer variables are typically introduced indirectly by a user policy that optimizes the economic value of power using slots on the model's Thermal object. During optimization, these slot variables will be replaced with equivalent expressions involving Power and related quantities on the individual power reservoirs. In this way, if the Unit Power Table method is selected, constraints which model unit level power concerns will be introduced into the optimization problem.

In addition, the user might write policy constraints which refer to binary quantities. For example, the user might want to forbid scheduling *holes*; that is, turning a unit off for a single time period. One normally thinks of writing this constraint as

$$\text{Unit Is Generating } [u,t] \geq \text{Unit Is Generating } [u,t-1] + \text{Unit Is Generating } [u,t+1] - 1$$

However, to avoid referencing future time periods they will need to write this as:

$$\text{Unit Is Generating } [u,t-1] \geq \text{Unit Is Generating } [u,t-2] + \text{Unit Is Generating } [u,t] - 1$$

How Integer Programs Are Solved

Understanding the method used to solve integer programs is useful for understanding the solution information and adjusting parameters that control the solution process.

Integer programs are solved by first starting with a *linear programming relaxation*, for which integrality restrictions (i.e. that variables are either 0 or 1) are temporarily removed. From this relaxed solution two subproblems are created by branching on an integer variable. For example, the solver might branch on Unit 1 being used in the first period, with one branch requiring it to be used and the other branch requiring it not be used. This branching process can be continued with additional variables. The resulting process creates a tree structure that contains an optimal solution at one of the leaf nodes. Explicitly exploring the whole tree is impossible for all but the smallest problems. Fortunately, large portions of the tree can frequently be pruned off without exploration. The pruning is possible because the linear programming relaxation of a node in the tree may either be an integer solution, infeasible, or sufficiently suboptimal.

Timestep Approach

The integer programs formulated in RiverWare are sufficiently hard that an optional, but highly recommended heuristic has been added to speed up the solution process. The heuristic is an iterative timestep approach that focuses on solving early timesteps with integer variables while temporarily relaxing the integrality conditions for later timesteps. By relaxing the integrality constraints for most of the timesteps, the problem can be solved much more quickly. After obtaining a solution, the integer variables for the earliest timesteps are frozen at their integer values, and some previously relaxed variables are treated as integer variables. The problem is resolved, and the process continues until all timesteps have been modeled as integer variables and frozen. For each iteration the integer variables are constrained both by the frozen integer variables for previous time steps and the linear programming relaxation for future timesteps.

There are two time windows that are advanced at each iteration in this process, the range of (non-frozen) integer variables in the problem, and the possibly smaller range of integer variables that should be frozen after the problem is solved. The size of each of these windows is an important parameter of this process. By default, one time step is used for each of these time windows, but the user can increase these values by setting appropriate parameters, **Mip Integer Window Size** and **Mip Freeze Window Size**. If both variables are set to the number of timesteps in the run, then the MIP will be solved as a single problem containing all of the integer variables. At this time, we do not recommend increasing the window sizes beyond 1 because of the dramatic increase in computation time.

This process may find the optimal solution, but it is not guaranteed. During testing, the timestep approach has found high quality, feasible solutions in a fraction of the time to find an optimal solution and prove it is optimal.

Defining Constraints

See [“Mathematical Formulation of MIP for Unit Power”](#) on page 301 for details on the mathematical formulation of the mixed integer programming to model unit power.

Additional Parameters for Controlling This Solution

CPLEX provides a variety of parameters to tune the solution of optimization problems. RiverWare makes these parameters accessible to the user. In addition, RiverWare has added some additional parameters to the list that are related to RiverWare's use of goal programming and time windows. All of the parameters have default values determined by CPLEX or by CADSWES. In some cases CADSWES has replaced the default values of CPLEX parameters with values that have performed better for RiverWare models. These changes in the CPLEX parameters are in the `cplex.par` file packaged with RiverWare. Similarly, any non-default values for CADSWES parameters are contained in the `goal.par` file.

The vast majority of the parameters need never be examined, but some of them are potentially useful for integer programming. While branch and bound will theoretically find an optimal solution given enough time, the exponential growth of the search tree means that the time to do so is prohibitive for many practical problems. Instead, the tree can be used in a heuristic manner to find near optimal solutions. By limiting the number of solutions, the time allowed, and/or the optimality tolerance, the search can be terminated early with a good solution.

Setting Parameters

The parameters (both for CPLEX and RiverWare specific parameters) are set through the RiverWare interface. From the Run Control dialog, select **Optimization** as the controller and then select the menu option **View > Optimization Run Parameters**. From the Optimization Run Parameters dialog, select **Set CPLEX** and **Goal Parameters**. The parameters are organized by categories. Select the desired parameter and change the current value to the desired value. Select either **Apply** (to save and continue with other parameter changes) or **Ok** (to save and exit the parameter dialog). If you check the **Save non-default settings to parameter file** box these settings will be saved to a file called `cplex.init.par`, and this file will be used to initialize parameter settings for future RiverWare sessions.

Lower Cutoff Parameter

This parameter usually will not be effective when the timestep approach is used (see [“Timestep Approach”](#) on page 298). However, when optimizing the entire problem, this parameter can be very influential because of the structure of the branch and bound solution method. For maximization problems, the lower cutoff parameter will help prune the search tree until an integer feasible solution is found (an upper cutoff parameter can be used for minimization problems). Ideally, this parameter will be set high enough to eliminate large portions of the tree without eliminating all of the feasible integer solutions. Since the optimal integer solution isn't known in advance, use of this parameter requires a little experimentation.

A starting point for the experimentation is the linear programming relaxation. By definition, integer solutions cannot improve on this value. For a maximization problem this means the optimal integer solution will be less than or equal to the linear programming relaxation and the lower cutoff parameter should certainly be set less than the objective function value for the linear programming relaxation. Our limited experimentation with test models suggests cutoff values ranging from about 90% of the linear programming relaxation to just below the linear programming relaxation.

Some experimentation is unavoidable with this parameter. If the lower cutoff value is set too high the search will end without a feasible solution. If the value is set too low (or there are too many time steps) the run will continue until some parameter stops the run or the user terminates the run. In both cases there won't be an integer solution.

Appendix A

Technical Appendix

If the problem is small enough to be solved, a reasonable cutoff value can typically be determined with a couple of tries.

The objective function of the linear programming relaxation can be obtained approximately by terminating the run with a little delay after the last objective has started. The delay is necessary to give CPLEX time to solve the relaxation. After terminating, the objective function of the best node in the tree will be printed in the RiverWare diagnostics window. This may be slightly less than the linear programming relaxation. In general this value will decrease as branch and bound proceeds. The number of nodes processed in the tree are also printed in the diagnostic. Unfortunately, for technical reasons the value can't be printed while CPLEX is still running. The objective function is printed in *optimization units*, which are typically scaled from user units for numerical stability. For this reason, direct interpretation of the value is not advised. However, these are the same units that should be used for the lower cutoff value.

The lower cutoff parameter is in the **MIP Tolerance Parameters** category.

Limiting the Number of Solutions

Sometimes, branch and bound will find a good or even optimal solution early in an optimization and spend the bulk of its time proving optimality or making relatively minor improvements in the solution. Limiting the number of solutions can improve the solution time. However, testing has shown that setting this parameter overly low while using the timestep approach can lead to integer infeasibility in the middle of the timestep iterations.

The solutions parameter is in the **MIP Limit Parameters** category.

Time Limit

Optimization problems can take a long time to solve. A time limit parameter allows a solver to exit gracefully when the solver takes too long. For integer programs, exiting when the time limit expires will not generate an error if a feasible solution has been found. Thus, a time limit parameter can be used to truncate exploration of the branch and bound tree.

The time limit parameter in the **General Parameters** category. The default value is set to 3000 seconds (50 minutes).

Optimality Tolerance

Optimality tolerance parameters define how close the solution must be to optimal to be considered close enough, either as a fraction of optimality (0.0 - 1.0) or as an absolute number (in optimization units). Even on relatively easy problems, proving 100% optimality (an optimality tolerance of zero) can require very large computation times. A small optimality tolerance (e.g., e-4 to e-6 absolute) generally yields a great improvement in runtime, and for certain problems larger tolerances (e.g., 1-10% fractional) can also be beneficial. Experimenting with these parameters may prove worthwhile.

The fractional and absolute optimality tolerance parameters are in the **MIP Tolerance Parameters** category and are called respectively, **mipgap** and **absmipgap**.

Unit Power

At the end of optimization, the optimized values are known but haven't been returned to the workspace yet. The challenge is to know what variables will be returned to drive the post-optimization RBS run. When using the Unit Power Table method, the Rulebased Simulation can be driven in several ways:

Optimization could return the individual Unit Turbine Releases and some other piece of information to cause the object to dispatch and solve. For example, the additional information could be the Outflow, plant Turbine Release or the "U" flag (implementation soon) on Turbine Release. In fact, it is recommended, to return the individual Unit Turbine Release and total Outflow. This approach calculates power very similarly to the optimization calculation and is perhaps the most natural way to do a post-optimization simulation. Also, given unit turbine releases will always produce an answer.

Note: If a regulation method is selected, optimization will also need to return Unit Flow Reduction for Regulation and Unit Flow Addition for Regulation. This approach is more reliable than the next approach and will be the only one supported at this time.

Optimization could return the individual unit energy, the total Spill, and some other piece of information to cause the object to dispatch and solve. For example, the additional information could be the Plant Energy or the "U" flag (implementation soon) on Energy. This approach would be similar to the first option, but would return unit energies instead of unit flows. In both of these options, given the individual unit values (and even spill) would not be enough to cause the reservoir to dispatch. A plant level value must be specified. Given unit energies, there is the potential to have an infeasible solution in simulation. An operating head and flow could be calculated that because of linearization errors, cannot meet the given energies. This approach is not supported or recommended.

For the time being, the first option is the only supported. The rule should return the plant level Outflow and the Unit Turbine Release, marking both with rule flags. The plant level value will cause dispatching in simulation. With these two approaches, the user has options on what should be returned from optimization and how the object should solve. Experimentation will allow the user to determine the variables that provides the best solution for the application.

Mathematical Formulation of MIP for Unit Power

This section presents an improved mathematical formulation of the relationship between power and other quantities. This formulation models power generation at the unit level to capture the fact that power generation occurs at one power reservoir with a heterogeneous collection of turbines. Integer variables allow the user to model the fact that these units have discrete states (off, generating power, spinning, etc.) and transitions between states are potentially costly.

[Table A.2](#) summarizes information concerning the slots used by the Unit Power Method, methods in the related dependent categories, and methods on the thermal object. The "Symbol" column indicates the symbolic notation that will be used in subsequent sections to refer to the slot.

The slot dimensionality column is included to give a rough idea of the resources required by these methods. This column uses the following abbreviations:

- T = The number of time steps.
- U = The number of units.
- P = Average number of points in the Unit Power Table for a single unit.

Appendix A

Technical Appendix

- H = The number of distinct head values in the Unit Power Table.
- TW = The number of distinct Tailwater values in the Unit Power Cavitation Table
- A = The number of distinct avoidance zones in the Avoidance Zone Table
- Q = The number of Turbine Release values in the Shared Penstock Loss Table
- B = The number of blocks in the Block Regulation slots

Table A.2 Unit Power Method slots and related methods

Symbol	Slot Name	Type	# of Rows	# of Col.'s	Status
Unit Power Method Slots					
	Number of Units	TableSlot	1	1	Input
	Unit Power Table	TableSlot	P	3*U	Input
	Unit Priority Table	TableSlot	U	1	Input
UIG	Unit Is Generating	AggSeriesSlot	T	U	Input / Output
UP	Unit Power	AggSeriesSlot	T	U	Input / Output
UEQT	Unit (Expected) Turbine Release	AggSeriesSlot	T	U	Input / Output
NUG	Number Units Generating	SeriesSlot	T	1	Input / Output
PP	(Plant) Power	SeriesSlot	T	3	Exists
PEQT	(Plant Expected) Turbine Release	SeriesSlot	T	3	Exists
PQS	(Plant) Spill	AggSeriesSlot	T	3	Exists
MinPPE	Minimum Power Elevation	TableSlot	U	1	Input
PE	Pool Elevation	AggSeriesSlot	T	2	Exists
MinPE	Min value of Pool Elevation Slot	Configuration			Exists
TW	Tailwater Elevation	AggSeriesSlot	T	2	Exists
Q	(Plant) Outflow	AggSeriesSlot	T	3	Exists
Startup Cost - Unit Lumped Cost method Slots					
USU	Unit Startup	AggSeriesSlot	T	U	Input / Output
USD	Unit Shutdown	AggSeriesSlot	T	U	Input / Output
NUSU	Number Units Startup	SeriesSlot	T	1	Input / Output
NUSD	Number Units Shutdown	SeriesSlot	T	1	Input / Output
	Unit Startup Cost Table	TableSlot	U	1	Input
USC	Unit Startup Cost	AggSeriesSlot	T	U	Output
PSC	Plant Startup Cost	SeriesSlot	T	1	Output
Shared Penstock Head Loss method Slots					

Table A.2 Unit Power Method slots and related methods

Symbol	Slot Name	Type	# of Rows	# of Col.'s	Status
	Shared Penstock Head Loss Table	TableSlot	Q	2	Input
	Shared Penstock Head Loss LP Param Table	TableSlot	3+	3	Input
SPH	Shared Penstock Head Loss	SeriesSlot	T	1	Output
Unit Head and Tailwater Based Regions method Slots					
	Unit Power Cavitation Table	TableSlot	~H*TW	4*U	Input
	Unit Cavitation Optimization Tolerances	TableSlot	U	2	Input
Unit Head Based Avoidance Zones method Slots					
	Unit Avoidance Zone Table	TableSlot	~H	3*U*A	Input
Unit Regulation method Slots					
	Unit Regulation Table	TableSlot	P	U*6	Computed
USMP	Unit Scheduled Mechanical Power	AggSeriesSlot	T	U	Output
USQT	Unit Scheduled Turbine Release	AggSeriesSlot	T	U	Output
URU	Unit Regulation Up	AggSeriesSlot	T	U	Input / Output
URD	Unit Regulation Down	AggSeriesSlot	T	U	Input / Output
UPRU	Unit Possible Regulation Up	AggSeriesSlot	T	U*P	Dynamic
UPRD	Unit Possible Regulation Down	AggSeriesSlot	T	U*P	Dynamic
UTSR	Unit Two Sided Regulation	AggSeriesSlot	T	U	Input / Output
UQAR	Unit Flow Addition For Regulation	AggSeriesSlot	T	U	Output
UQRR	Unit Flow Reduction For Regulation	AggSeriesSlot	T	U	Output
UOCPR	Unit Operating Cost Per Regulation	TableSlot	U	1	Input
UOC	Unit Operating Cost	AggSeriesSlot	T	U	Output
POC	Plant Operating Cost	AggSeriesSlot	T	U	Exists
PSMP	Plant Scheduled Mechanical Power	SeriesSlot	T	1	Output
PSQT	Plant Scheduled Turbine Release	SeriesSlot	T	1	Output
PRU	Plant Regulation Up	SeriesSlot	T	1	Input / Output
PRD	Plant Regulation Down	SeriesSlot	T	1	Input / Output
PR	(Plant Two-Sided) Regulation	SeriesSlot	T	3	Exists
PQAR	Plant Flow Addition For Regulation	SeriesSlot	T	1	Output
PQRR	Plant Flow Reduction For Regulation	SeriesSlot	T	1	Output

Appendix A

Technical Appendix

Table A.2 Unit Power Method slots and related methods

Symbol	Slot Name	Type	# of Rows	# of Col.'s	Status
Tailwater method specific slots affected by Unit Power Method					
TWBV	Tailwater Base Value	AggSeriesSlot	T	2	Exists
TWL	Temp Tailwater Lookup	SeriesSlot	T	1	Exists
SF	Stage Flow Tailwater Table	TableSlot		3	Exists
TWT	Tailwater Table	TableSlot		2	Exists
Thermal Object Slot					
SSC	System Startup Cost	MultiSlot	T	1	Input?/Output
Thermal Object Slots for one-sided regulation					
SRU	System Regulation Up	MultiSlot	T	1	Input?/Output
BRU	Block Regulation Up	AggSeriesSlot	T	B	Output
RUBC	Regulation Up Block Costs	AggSeriesSlot	T	B	Input
RUV	Regulation Up Value	SeriesSlot	T	1	Output
	Regulation Up Marginal Value	SeriesSlot	T	1	Output
	Regulation Up Previous Marginal Value	SeriesSlot	T	1	Output
SRD	System Regulation Down	MultiSlot	T	1	Input?/Output
BRD	Block Regulation Down	AggSeriesSlot	T	B	Output
RDBC	Regulation Down Block Costs	AggSeriesSlot	T	B	Input
RDV	Regulation Down Value	SeriesSlot	T	1	Output
	Regulation Down Marginal Value	SeriesSlot	T	1	Output
	Regulation Down Previous Marginal Value	SeriesSlot	T	1	Output

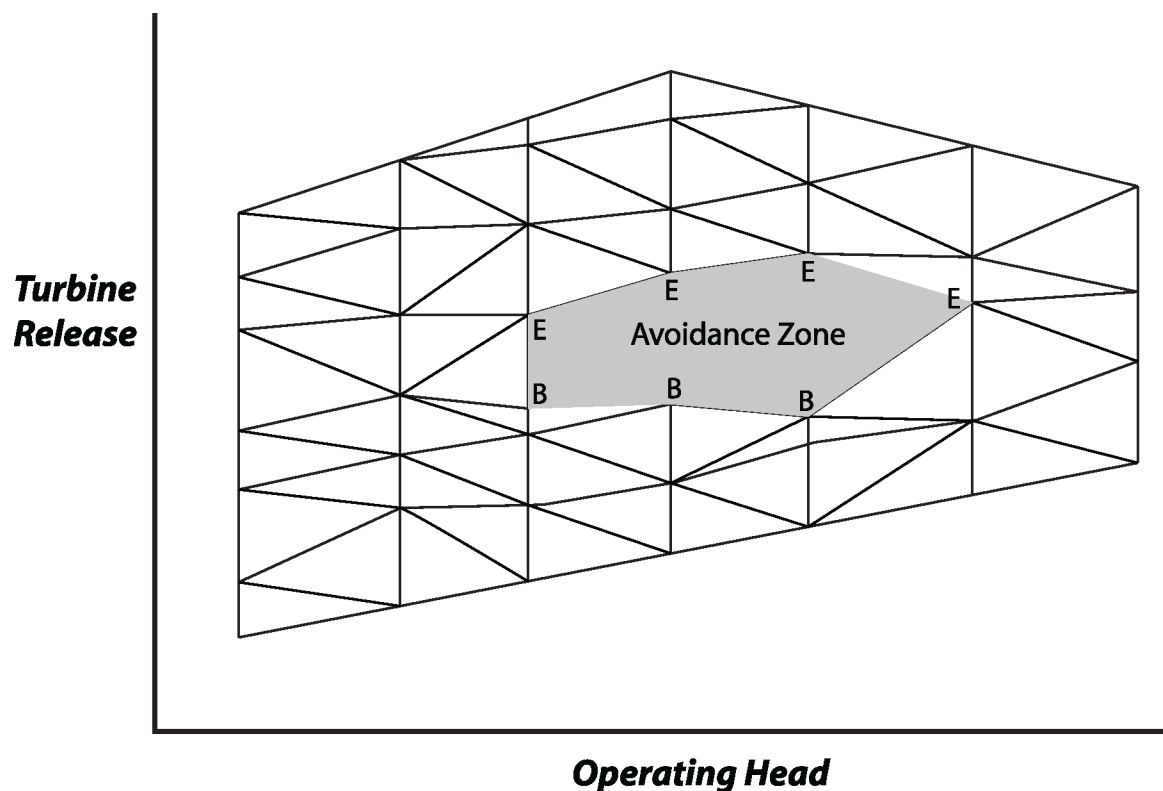
Triangulation

At the heart of the optimization controller's model of power is the triangulation algorithm for partitioning the power curve domain (specifically flow and head), which are described here. The method successfully excludes zones that are infeasible because of vibration, cavitation, etc.

The input for this method is a three-dimensional power table and a table indicating zones that must be avoided. It is assumed that the power table contains the end points of head that define the zone and for each head the flow values that define the zone. The points themselves are assumed to be feasible operating points unless they represent minimum or maximum values of head or flow. There may be multiple separate avoidance zones.

Figure A.1 is an example of a triangulated power domain with an avoidance zone. In this figure, each vertex corresponds the data in a Unit Power Table row which apply to a single unit, head (operating head or net head), and Turbine Release. Power, an additional dimension represented in that table, is not shown.

Figure A.1 Triangulation algorithm applied to data from the Unit Power Table for a single unit



Let us identify the avoidance zone values from the power table in the following way for the algorithm:

- B - the beginning, low flow, of an avoidance zone for a given head
- E - the end, high flow, of an avoidance zone for a given head. If either the maximal or minimal head has a single flow value, then that point is marked E.

Pseudocode for Triangulation

The basic idea is to iterate through the Head values in the (reduced) Unit Power Table, partitioning the vertical space between that head and the next into triangles. In general when deciding which of two potential triangles to create, RiverWare creates the one that has the lesser slope for the upper side.

```
// Triangulate between each pair of adjacent heads
For H1 = first head to next to last
  H2 = next H
  // Start with the minimum flow for each head
  Q1 = First Q for H1
  Q2 = First Q for H2
```

Appendix A Technical Appendix

```
// At each step increment one of the flows to form a triangle
while (next Q1 != NULL || next Q2 != Null)
    // If only Q1 can be incremented or incrementing Q1 leads to a flatter triangle
    // top, increment Q1 and form
    // the triangle
    If (next Q2 = NULL || next Q1 - Q2 <= next Q2 - Q1)
        Create a Triangle with coordinates(H1.Q1, H1.next Q1, H2.Q2)
        Q1 = next Q1
    Else
        Create a Triangle with coordinates(H1.Q1, H2.next Q2, H2.Q2)
        Q2 = next Q2
    // If the two heads are at an avoidance zone, increment Q past it
    If (Q1 is marked B && Q2 is marked B or E)
        Q1 = next Q1
    If (Q2 is marked B && Q1 is marked B or E)
        Q2 = next Q2
```

If head is specified, then the triangles become line segments, and line segments become points.

Lambda Variables for Convex Combinations of Discrete Operating Points (Triangles)

In the context of optimization, a lambda set is (roughly) a discretization of a continuous space, so RiverWare uses this technique to model the discretization of the power curve domain into triangles. While solvers don't require it, usually it is assumed that lambda variables are constrained as follows:

```
0 <= Lambda_subscripts <= 1
Sigma (some subscripts, Lambda_subscripts) = 1
```

Lambda sets may additionally be constrained to be a specially ordered set of type 1 or 2, aka SOS1 or SOS2. The individual variables in an SOS are ordered and a reference row of weights is associated with each variable. An SOS1 may have at most one value that is nonzero while an SOS 2 may have at most two nonzero values and they must be adjacent. Solvers internally treat these sets as being similar to integer variables with branching to enforce the set conditions. The branching is based on weights in the reference row. The general idea is to calculate a weighted average of the variables using the reference row. The two branches are roughly that either variables on one side of the weighted average are zero or the variables on the other side are zero.

In addition, frequently the individual variables belonging to an SOS1 are also binary variables. The variables in a SOS2 are almost always continuous.

Power is a nonlinear function of both flow and head. Hence, when it is discretized, the domain of the function is two-dimensional. The SOS technique can be applied here as well. In this case, a combination of 3 points is used to define flow and head. Alexander Martin has described how to model this case as an SOS type k , where k is 3 in this case (A. Martin, "Approximation of Non-linear functions in Mixed Integer Programming" Workshop on Integer Programming and Continuous Optimization, Chemnitz University of Technology, November 7–9, 2004, www-user.tu-chemnitz.de/~helmberg/workshop04/martin.ppt). Martin then modifies a solver to directly branch on the SOS k , creating separate branches for each region of the domain.

Note: Some solvers have an SOS 3, but this typically does not refer to the same thing as it is used here.

A variant of his method that is compatible with CPLEX and other solvers is used here. The flow and head domains are partitioned into triangles and define a variable for each triangle, Lambda_1 . The triangles in an SOS 3 are analogous to the line segments in an SOS 2. The triangulation is described in section 2.7.

The following lambda variables are part of the formulation:

- Lambda_1rut : selection of triangular region r on unit u

This variable binary, technically an SOS k , Lambda_2 is used to implement this functionality. Therefore, there is no need for a reference row and its weights. This variable may represent 2 points instead of 3 if a feasible region is one-dimensional. For example most data sets will contain points with zero flow for several heads, but disallow small flows. Thus, the segment of zero flow between adjacent heads will be a Lambda_1 variable even though it is not a triangle. In theory, there could be a single isolated operating point that would need its own Lambda_1 variable, but this is unlikely in practice.

- Lambda_2h'ut : selection of a triangle with approximate head h' for unit u .

This is a binary variable, SOS1 for each t : reference row weights are the approximate head (half way between the two head values for the points in the triangle). The set of Lambda_1rut triangles with r in h' form an SOS1 for each h', u, t combination. The reference row weights are the average q value of the points in the triangle. In addition the sum is constrained to equal Lambda_2h'ut :

- Lambda_4hut : selection of head h

This is a continuous variable, SOS2 for each u, t : the reference row weights are head.

- Lambda_6hqut : combination of o and q

This is a continuous variable, not an SOS. The data should include $q=0$ for all possible heads, even those that cannot generate power.

The set of Lambda_6hqut points with the same o form an SOS2 for each h, u, t combination. The reference row weights are the q values.

Defining Constraints

This section presents the constraints which will be introduced into the optimization problem as power-related quantities are encountered in the optimization policy (when the Unit Power method is selected).

Notes on Variables and Notation

All variables are constrained to be positive unless otherwise noted.

Binary Variables

These variables are required to be zero or one, and will be enforced on user inputs in the corresponding slots at the beginning of the run.

- Unit Is Generating (UIGut)
- If the Startup method is selected

Appendix A Technical Appendix

- Unit Startup (USUut)
- Unit Shutdown (USDut)
- Lambda 6 variables (Lambda_6put)

Continuous Variables

- If the Shared Penstock Head Loss method is selected,
 - PNHt = Net head after losses in the shared penstock
 - SPHt = Shared penstock losses
- If the Unit Regulation method is selected,
 - RUFhqt = Fraction of possible regulation up at point indexed by hqu being used
 - RDFhqt = Fraction of possible regulation down at point indexed by hqu being used

Symbols

The following symbols will be used in the formulation below (see also slot summary table):

- X_p = The X component of point p in a data table (i.e., the value in the column labeled X)
- WQ_{ru} = turbine flow weighting of triangle r on unit u. Equal to the average of flow on the points r contains.
- WH_{ru} = head weighting of unit triangle r on unit u. Equal to the average of head on the points r contains.
- TWM_{pu} = minimum tailwater that allows point Lambda_6put to be feasible
- MinTW = minimum possible tailwater at the plant
- TW = plant tailwater elevation
- Q_{Tr} = maximum turbine release for triangle r

The following indexes are used:

- h' = approximate head index for power triangles with their center of mass within a tolerance of the operating
- r = triangle region of the power curve where convex combinations are allowed.
- u = unit index
- t = time
- p = point in a table. E.g., for the Unit Power table each point is a triplet (QT, H, P), so QT_p denotes the Flow component of p, H_p denotes the head component, and P_p denotes the Power component. Similarly, the Unit Regulation Table has sextets of (QT,H, RUL,QTUL,RDL, QDL). RUL_p denotes the Regulation Up Limit, the maximum power generation that can be attained by increasing flow at the specified operating head without passing through an avoidance or cavitation zone. Similarly, RDL_p is the Regulation Down Limit, $QTUL$ and $QTDL$ are the flow values associated with these power values.

Sigma indicates capital sigma used for a summation. The arguments following sigma in parenthesis are respectively the indexes to sum over and the expression summed.

Bold typeface for a variable indicates the constraint is a defining constraint for that variable.

The constraints are presented with an unspecified timestep (t) and unit (u). Instantiations will be created during the run only for those units and timesteps which are referenced by the user's policy.

Unit-level Constraints

1. Unit Is Generating is equivalent to selecting a triangular region above zero flow/generation
 "Unit Generation Constraint"

$$\text{Sigma} (r \text{ with } Q_{\text{Tru}} > 0, \text{Lambda_1rut}) = \text{UIGut}$$
2. Unit Start up and Shut down
 If the Unit Lumped Cost startup method is selected,
 "Unit Startup Constraint"

$$\text{UIGut} - \text{UIGut} - 1 = \text{USTu,t} - \text{USHu,t}$$
3. Operating Head or Net Head is discretized by Lambda_4
 If there is no additional head loss (No Method in the Head Loss category) is selected
 "Operating Head Unit Lambda Constraint"

$$\text{POHt} = \text{Sigma} (\text{head } h \text{ in Unit Power Table with unit } u, \text{Hhu} * \text{Lambda_4hut})$$
 Else If the Shared Penstock Head Loss method is selected
 "Net Head Unit Lambda Constraint"

$$\text{PNHt} = \text{Sigma} (\text{head } h \text{ in Unit Power Table with unit } u, \text{Hhu} * \text{Lambda_4hut})$$
4. Net Head Definition
 If the Shared Penstock Head Loss method is selected:
 "Net Head Constraint"

$$\text{PNHt} = \text{Oht} - \text{SPht} \text{ (Net Head = Operating Head - Shared Penstock Loss)}$$
5. Specified Power – Flow curve from Lambda_6 points
 If the Unit Frequency Regulation method is selected:
 "Unit Scheduled Mechanical Power Constraint"

$$\text{USMPut} = \text{Sigma} (p \text{ in Unit Power Table with unit } u, \text{Ppu} * \text{Lambda_6put})$$
 Else
 "Unit Power Constraint"

$$\text{UPut} = \text{Sigma} (p \text{ in Unit Power Table with unit } u, \text{Ppu} * \text{Lambda_6put})$$
 If the Unit Frequency Regulation method is selected:
 "Unit Scheduled Turbine Release Constraint"

$$\text{USQTut} = \text{Sigma} (p \text{ in Unit Power Table with unit } u, \text{QTpu} * \text{Lambda_6put})$$
 Else
 "Unit Expected Turbine Release Constraint"

$$\text{UEQTut} = \text{Sigma} (p \text{ in Unit Power Table with unit } u, \text{QTpu} * \text{Lambda_6put})$$
6. Tailwater Limit to Prevent Cavitation
 "Tailwater Cavitation Constraint"

Appendix A Technical Appendix

$T_{wt} - MinTW \geq \text{Sigma} (p \text{ in Unit Power Table with unit } u, (T_{Wmpu} - MinTW) * \text{Lambda_da_6put})$

Note: $T_{Wmpu} - MinTW$ should be 0 for most power points

7. Minimum Power Elevation

if $MinPPEu \neq NaN$

“Minimum Power Elevation Constraint”

$P_{et} - MinPE \geq (MinPPEu - MinPE) U_{IGut}$

8. Power – Flow regulation up equals a fraction of the limit

If the Unit Frequency Regulation method is selected,

“Unit Regulation Up Constraint”

$U_{RUut} = \text{Sigma} (p \text{ in Unit Regulation Table with unit } u, R_{ULpu} * U_{PRUut})$

“Unit Flow Addition for Regulation Constraint”

$U_{QARut} = \text{Sigma} (p \text{ in Unit Regulation Table with unit } u, Q_{TULpu} * U_{PRUut})$

“Unit Regulation Up Lambda_6 Constraint”

For all unit u , for all p in Unit Power Table

$U_{PRUut} \leq \text{Lambda_6pt}$

9. Power - Flow regulation down equals a fraction of the limit

If the Unit Frequency Regulation method is selected,

“Unit Regulation Down Constraint”

$U_{RDut} = \text{Sigma} (p \text{ in Unit Regulation Table with unit } u, R_{DLpu} * U_{PRDut})$

“Unit Flow Reduction for Regulation Constraint”

$U_{QRRut} = \text{Sigma} (p \text{ in Unit Regulation Table with unit } u, Q_{DLpu} * U_{PRDut})$

“Unit Regulation Down Lambda_6 Constraint”

For all unit u , for all p in Unit Regulation Table

$U_{PRDut} \leq \text{Lambda_6put}$

10. Two-sided regulation tied to regulation up and down

If the Unit Frequency Regulation method is selected,

“Two-sided Regulation Limited by Regulation Up”

$U_{TSRut} \leq U_{RUut}$

“Two-sided Regulation Limited by Regulation Down”

$U_{TSRut} \leq U_{RDut}$

11. Expected power and flow after regulating

If the Unit Frequency Regulation method is selected,

“Expected Unit Power Constraint”

$U_{Put} = U_{SMPut} + U_{RUut}/2 - U_{RDut}/2$

12. Unit Operating Costs

If the Unit Frequency Regulation method is selected,

“Unit Operating Cost Constraint”

$$UOCut = timestepHRS * UOCPR * (RUut + RDut)$$

13. Unit Startup Costs

Unit Startup Cost is replaced using the following definition:

$$USCut = \text{Unit Startup Cost Table } u * USTut$$

14. Unit Priority Constraints

Ordering the units will considerably reduce the search space, particularly if the some of the units are identical. In addition, this is often an operating policy.

For unit i with a higher priority than unit j :

“Precedence Constraint”

$$UIGit > UIGjt$$

Defining Constraints for Lambda Variables

1. Lambda1 Unity Constraint

$$\text{Sigma}(r \text{ with unit } u, \text{Lambda_1rut}) = 1$$

2. Lambda6 Unity Constraint

$$\text{Sigma}(p \text{ in Unit Power Table with unit } u, \text{Lambda_6put}) = 1$$

Note: The Lambda_2 unity constraint is omitted because it is implied by Lambda_1, and the Lambda_4 unity constraint because it is implied by Lambda_6.

3. Lambda1 Definition Constraint

For all h' :

$$\text{Sigma}(r \text{ in } h', \text{Lambda_1rut}) = \text{Lambda_2h't}$$

4. Lambda 2 Definition Constraint

For each Lambda_4 variable from the triangles, the sum of the adjacent Lambda_4 variables must be greater than the Lambda_2 variable:

For all h' :

$$\text{Sigma}(h \text{ adjacent to } h', \text{Lambda_4hut}) \geq \text{Lambda_2h'ut}$$

5. Triangle Definition Constraint

The sum of the use of the points of a triangle must be greater than the triangle variable:

For all r :

$$\text{Sigma}(p \text{ in } r, \text{Lambda_6put}) \geq \text{Lambda_1rut}$$

6. Lambda 6 Definition Constraint - the sum is constrained to equal Lambda_4hut

For all h :

$$\text{Sigma}(p \text{ with Head value } = h, \text{Lambda_6put}) = \text{Lambda_4hut}$$

Unit/Plant Relationship Constraints

Any plant variable, PV_x , where x represents the appropriate subscripts is defined by the sum of the unit variables, V_{ux} , indexed by unit u and x . Plant variables are only created if they are used directly or indirectly by policy. Thus, the defining constraint is as follows:

“Plant Unit V Constraint”

$PVx = \text{Sigma}(u, Vux)$

Redefining Existing Plant-level Linearizations

The modeling of power will be more accurate when lambda variables and SOS sets are used to replace the plant level linearization of pool elevation and tailwater. Both of these affect operating head. This has been temporarily postponed until a potential problem is addressed: the associated lambda variables might be frozen by an objective solved using linear programming before it reaches the power objective. If this were to occur, the intended model for integer programming would be disrupted. This is more likely to be a problem for pool elevation than tailwater because pool elevation is important for non-power policy, whereas tailwater is usually only important for power. Moreover, the error in existing methods is larger for tailwater than pool elevation. For these reasons, it is more likely beneficial to change the modeling of tailwater.

The pool elevation constraints are:

“Pool Elevation Lambda Constraint”

$PEt = \text{Sigma}(p \text{ in Elevation Volume Table}, PEp * \text{Lambda_7st})$

“Pool Elevation Lambda Storage Constraint”

$St = \text{Sigma}(p \text{ in Elevation Volume Table}, SEp * \text{Lambda_7st})$

“Pool Elevation Lambda Unity Constraint”

$\text{Sigma}(p \text{ in Elevation Volume Table}, \text{Lambda_7pt}) = 1$

Lambda_7 is an SOS2

If the tailwater method is optTWStageFlowLookupTable, the tailwater constraints define tailwater, outflow, and tailwater base value in terms of Lambda_8 points directly or indirectly in terms of Lambda_9 and Lambda_10:

“Tailwater Stage Flow Lambda Constraint”

$Twt = \text{Sigma}(p \text{ in Stage Flow Tailwater Table}, Twp * \text{Lambda_8pt})$

“Tailwater Stage Flow Lambda Flow Constraint”

$Qt = \text{Sigma}(\text{flow } q \text{ in Stage Flow Tailwater Table}, Qq * \text{Lambda_10qt})$

“Tailwater Stage Flow Lambda Base Value Constraint”

$TWBvt = \text{Sigma}(\text{tailwater base value } v \text{ in Stage Flow Tailwater Table}, TWBVv * \text{Lambda_9vt})$

“Tailwater Stage Flow Lambda Unity Constraint”

$\text{Sigma}(p \text{ in Stage Flow Tailwater Table}, \text{Lambda_8pt}) = 1$

“Tailwater Stage Flow Lambda Base Value Lambda Constraint”

For all v:

$\text{Sigma}(p \text{ in Stage Flow Tailwater Table with tailwater base value } = v, \text{Lambda_8pt}) = \text{Lambda_9vt}$

“Tailwater Stage Flow Lambda Flow Lambda Constraint”

For all q:

$\text{Sigma}(p \text{ in Stage Flow Tailwater Table with outflow } = q, \text{Lambda_8pt}) = \text{Lambda_10qt}$

Lambda8 is an SOSk implemented by making Lambda_9 and lambda_10 SOS2s. An alternative implementation would be triangles as used for power.

If the tailwater method is optTWBaseValuePlusLookupTable, the new tailwater constraints define temp tailwater lookup and outflow in terms of Lambda_11:


```

“Tailwater Lookup Lambda Constraint”
TWLt = Sigma(f, TWTF * Lambda_11ft)
“Tailwater Lookup Lambda Flow Constraint”
Qt = Sigma(f, TWQf * Lambda_11ft)
“Tailwater Lookup Unity Constraint”
Sigma(f, Lambda_11ft) = 1
Lambda_11 is an SOS2

```

For this method, the existing tailwater constraint is retained:

$$TWt = (TWBVt + TWBVt-1) / 2 + TWLt$$

Thermal Object

The thermal object has a number of multi-slots that are potential variables in an optimization problem. These multi-slots typically represent the system wide total of individual reservoir values that are linked to these slots. Equations for these sums are added automatically if a policy either directly or indirectly references one of the thermal object multi-slots.

Many of the existing variables and definitions are already on the thermal object. In addition, to the existing equations, the following new equations will be added.

System Regulation Up is greater than the sum of Block Regulation Up variables:

$$SRUt * timestepHRS \geq \text{Sigma}(b, BRUbt)$$

The Regulation Up Value equals the sum of the value of the Block Regulation Up variables:

$$RUV = \text{Sigma}(b, RUBCb * BRUbt)$$

System Regulation Down is greater than the sum of Block Regulation Down variables:

$$SRDt * timestepHRS \geq \text{Sigma}(b, BRDbt)$$

The Regulation Down Value equals the sum of the value of the Block Regulation Down variables:

$$RDV = \text{Sigma}(b, RDBCb * BRDbt)$$

